



JOI 2024/25 春 Day 1

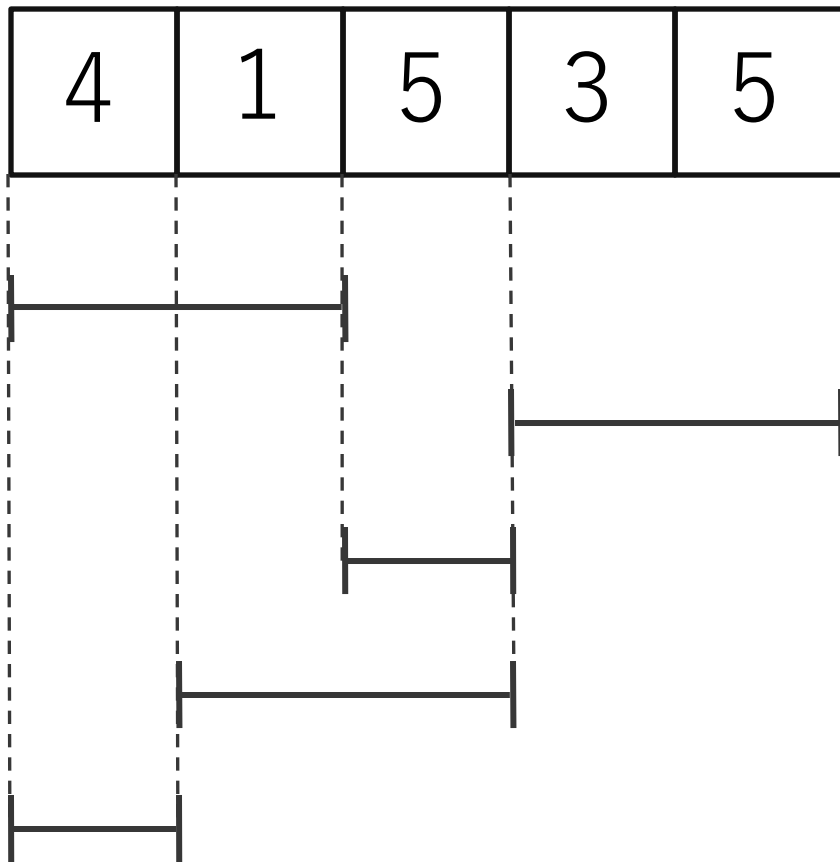
展覧会 3
(Exhibition 3)

解説：渡邊雄斗 (yuto1115)

問題概要

問題概要

- 長さ N の数列 $A = (A_1, A_2, \dots, A_N)$ が与えられる
- B_j ($1 \leq j \leq M$) $:= \max \{A_{L_j}, A_{L_j+1}, \dots, A_{R_j}\}$ と定義する
- A をうまく並び替えることで得られる、辞書順最大の数列 $B = (B_1, B_2, \dots, B_M)$ を出力せよ



B_j

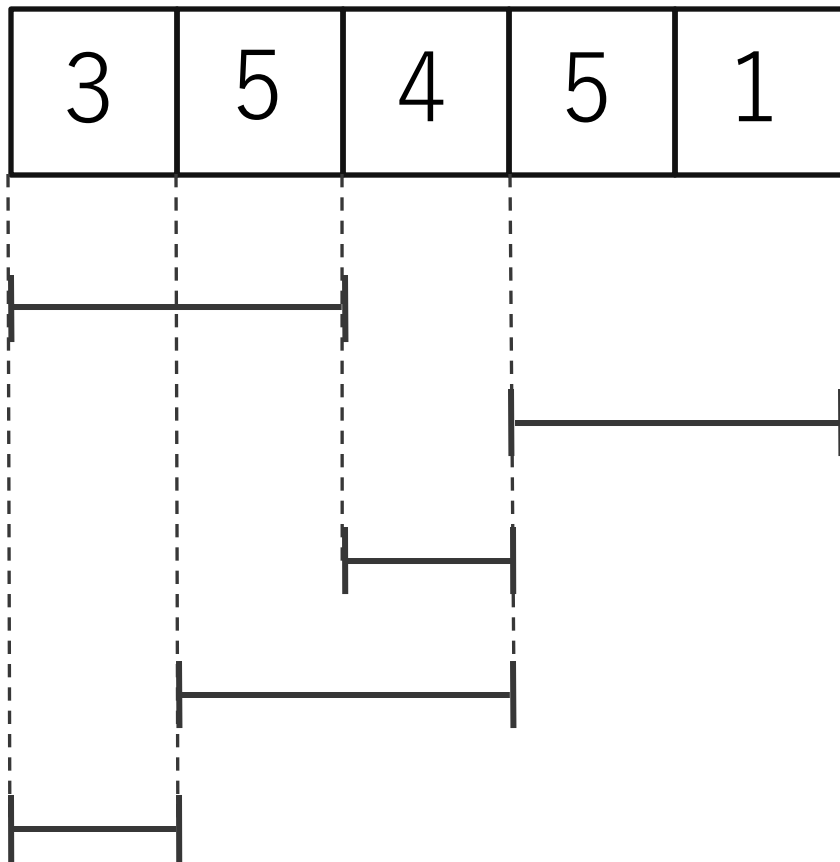
4

5

5

5

4



B_j

5

5

4

5

3

Step 1: 多項式時間で解く

小課題 1 $N \leq 400, M \leq 400$ (19 pts.)

Step 1: 多項式時間で解く

- 辞書順最大化なので、前から貪欲
 - こんな問題を解きたくなる：
 - $B_1, \dots, B_{j-1}$ の値が既に確定している（可能だと分かっている）。
- B_j の値を k ($1 \leq k \leq N$) とすることは可能だろうか？

Step 1: 多項式時間で解く

- B_j の値を k ($1 \leq k \leq N$) とすることは可能だろうか？
 - 値が既に k と確定した区間たちに区間 $j = [L_j, R_j]$ を追加した集合を考える
 - 各区間内に k が少なくとも一個は含まれなければならない
 - $c_k = (A$ に含まれる k の個数) として、 c_k 個以下の添字 ($1 \sim N$) をうまく選ぶことで、各区間内に選んだ添字が一つ以上含まれるようにできる必要がある
 - これは **区間スケジューリング問題の答えが c_k 以下** であることと同値
- これは十分条件（証明略）

Step 1: 多項式時間で解く

- いくつかの記法を定義する
 - S_i ($1 \leq i \leq N$) : $B_j = i$ となることが確定した区間 j の集合
 - $IS(S)$: 区間の集合 S に対し、 S から選べる、互いに交わりを持たない区間の個数の最大値（区間スケジューリング問題）

Step 1: 多項式時間で解く

- アルゴリズム：
 - $S_1, S_2, \dots, S_N$ を空集合で初期化する
 - $j = 1, 2, \dots, M$ の順に以下を行う
 - $IS(S_i \cup \{j\}) \leq c_i$ を満たす **最大** の i を求め、 B_j をその値にセット
 - S_{B_j} に区間 j を追加する
- 計算量： $O(MN + M \sum_{i=1..N} |S_i| \log |S_i|) = O(MN + M^2 \log M)$

Step 2: IS 高速化

小課題 2 $N \leq 400, M \leq 10^5$ (9 pts.)

小課題 3 $N, M \leq 10^5, A_i \leq 5$ (19 pts.)

Step 2: IS 高速化

- 小課題 2 ($N \leq 400, M \leq 10^5$) を解くには高速化が必要
 - 特に、 $M^2 \log M$ の項をどうにかしたい
 - 区間スケジューリング問題を、区間の数ではなく N に依存する計算量で解きたい

Step 2: IS 高速化

- 区間スケジューリング問題を異なる視点から捉える：
 - $N + 1$ 頂点 $(0, 1, \dots, N)$ からなる有向グラフがある
 - 各 i ($0 \leq i \leq N - 1$) に対し、 $i \rightarrow i + 1$, 長さ 0 の辺がある
 - 各区間 $[l, r)$ (0-indexed) に対し、 $l \rightarrow r$, 長さ 1 の辺がある
 - 頂点 0 から頂点 N への **最長経路** の長さは？

Step 2: IS 高速化

- $Len(i, s, t) :=$ 区間集合 S_i に対応するグラフにおいて、頂点 $s \rightarrow t$ への最長経路の長さ
- S_i に 区間 $[l, r)$ を新たに追加可能 $\Leftrightarrow Len(i, 0, l) + 1 + Len(i, r, N) \leq c_i$
- $Len(i, 0, k)$ および $Len(i, k, N)$ をすべての i, k ($1 \leq i \leq N, 0 \leq k \leq N$) について管理
 - ある区間 j に対し、
 - B_j の値を求めるのは $O(N)$
 - $Len(B_j, 0, k), Len(B_j, k, N)$ の更新は $O(N)$
 - よって、全体で $O(N^2 + MN)$

Step 2: IS 高速化

- $K = \max A_i$ として、 K が小さい場合の解法を考える
 - 現状だと $O(NK + MK + MN)$ (左から順に初期化、 S_j の求値、 Len の更新)
- 小課題 3 ($N, M \leq 10^5, K \leq 5$) を解くには高速化が必要
 - $Len(i, 0, k), Len(i, k, N)$ の更新に時間がかかりすぎる
 - 一方で、 S_j の求値には計算量的に余裕がある
 - $\Rightarrow$ 平方分割

Step 2: IS 高速化

- その前に、余分な区間を消しておきたい
 - 他の区間を完全に内包しているような区間はムダなので、消す
 - 残った区間の集合 $[l_0, r_0), [l_1, r_1), \dots, (l_i < l_{i+1}, r_i < r_{i+1})$ は set で管理できる
- すると、最長経路は貪欲で求まるようになる
 - 現在地を p として、 p を左端にもつ区間 (に対応する辺) があるならそれを使う、ないなら $p \rightarrow p + 1$ と進む

Step 2: IS 高速化

- D : バケットサイズ
 - $[0, N]$ を $[0, D), [D, 2D), [2D, 3D), \dots$ と N/D 個のバケットに分割
- $To(i, s) :=$ 区間集合 S_i に対応するグラフにおいて、頂点 s からスタートして、
 s と異なるバケットに到達するまで 貪欲に進み続けたときの、到達点
- $Len'(i, s) :=$ 頂点 s から $To(i, s)$ まで貪欲に進んだときの経路の長さ

Step 2: IS 高速化

- $Len(i, s, N)$ を求めるには？
 - $s \rightarrow To(i, s) \rightarrow To(i, To(i, s)) \rightarrow \dots$ と進み続ければ、 N/D 回以内に頂点 N に到達
 - この過程で通る辺の長さの総和は $Len'(i, s) + Len'(i, To(i, s)) + \dots$ で求まる

Step 2: IS 高速化

- B_j の値が求まった。 S_{B_j} に区間 j を追加する際に必要な更新操作は？
 - set をごにょごにょして、「余分な区間をすべて取り除いた後に残る区間集合」の差分を求める（グラフの問題における辺の追加・削除に相当）
 - ある辺 $l \rightarrow r$ の追加・削除は、 **l と異なるバケットに属する頂点 s に対する $To(i, s), Len'(i, s)$ の値には影響を及ぼさない**
 - l と同じバケット内の To, Len' は $O(D)$ かけて一から求め直す
 - 一度削除されたら追加されないので、更新回数は合計 $O(M)$

Step 2: IS 高速化

- 初期化 : $O(NK)$
- 各 j に対して、
 - S_j の求値 : $O(K \cdot \frac{N}{D})$
 - To, Len の更新 : ならし $O(\log N + D)$
- よって、 $D = \sqrt{N}$ のとき全体で $O(NK + MK\sqrt{N})$

$$D = \sqrt{NK} \text{ のとき全体で } O(NK + M\sqrt{NK})$$

Step 3: 大きい N への対応

小課題 4 $N, M \leq 10^5$, $A_i = i$ (12 pts.)

小課題 5 $N, M \leq 10^5$, $c_i \leq 5$ (17 pts.)

Step 3: 大きい N への対応

- $N \leq 10^5$ のとき、各 j に対して「 S_i に区間 j を追加できるか」を $i = N, N-1, \dots, 1$ と順にチェックしていくことはできない
- まずは $A_i = i$ (subtask 4) という特殊ケースについて考える
 - 特に、すべての i について $c_i = 1$

Step 3: 大きい N への対応

- $l_i := S_i$ に含まれる区間の**左端**の**最大**値

$r_i := S_i$ に含まれる区間の**右端**の**最小**値

- このとき、 $IS(S_i) \leq 1 \Leftrightarrow l_i < r_i$
- したがって、 S_i に区間 $[l', r')$ を追加可能

$$\Leftrightarrow \max\{l_i, l'\} < \min\{r_i, r'\}$$

$$\Leftrightarrow [l_i, r_i) \text{ と } [l', r') \text{ が共通部分を持つ}$$

Step 3: 大きい N への対応

- S_i に区間 $[l', r')$ を追加可能 $\Leftrightarrow [l_i, r_i)$ と $[l', r')$ が共通部分を持つ
- よって、 $B_j = [l_i, r_i)$ と区間 j が共通部分を持つ最大の i
- 以下を満たす配列 $x = (x_0, x_1, \dots, x_{N-1})$ を考える：
 - 各 i について、 $x_{l_i} = x_{l_i+1} = \dots = x_{r_i} = i$
 - それ以外の x の要素の値はすべて -1
- このとき、 B_j は区間 j 内に含まれる x の要素の最大値
ただし最大値が -1 のときは、 S_i が空である最大の i

Step 3: 大きい N への対応

- B_j が求まったあと、 $[l_{B_j}, r_{B_j})$ がどのように変化するかは簡単に分かり、
それに伴う配列 x の更新は区間代入操作に該当する
- よって、区間代入・区間 max が高速に処理できれば良い
- 遅延セグメント木を用いれば、全体で $O(N + M \log N)$
- なお、異なる i に対する $[l_i, r_i)$ が共通部分を持つことはない（証明略）

Step 3: 大きい N への対応

- $c_i = 1$ とは限らない、一般のケースについて考えたい

Step 3: 大きい N への対応

- 各 S_i について、以下を定める：
 - $k_i := IS(S_i)$
 - S_i に対する区間スケジューリング問題を解く際の貪欲法について考えたとき、
 - 左に詰めるように貪欲に取った区間の列（を左から右に並べたもの）
$$\left[\alpha_1^{(i)}, \beta_1^{(i)}\right), \left[\alpha_2^{(i)}, \beta_2^{(i)}\right), \dots, \left[\alpha_{k_i}^{(i)}, \beta_{k_i}^{(i)}\right)$$
 - 右に詰めるように貪欲に取った区間の列（を左から右に並べたもの）
$$\left[\gamma_1^{(i)}, \delta_1^{(i)}\right), \left[\gamma_2^{(i)}, \delta_2^{(i)}\right), \dots, \left[\gamma_{k_i}^{(i)}, \delta_{k_i}^{(i)}\right)$$
 - $\left[l_t^{(i)}, r_t^{(i)}\right) \ (1 \leq t \leq k_i) := \left[\gamma_t^{(i)}, \beta_t^{(i)}\right)$

Step 3: 大きい N への対応

- 各 S_i について、 $[l_t^{(i)}, r_t^{(i)})$ ($1 \leq t \leq k_i$) $:= [\gamma_1^{(i)}, \beta_1^{(i)})$
 - ここで、以下が成立する (証明略):
 - S_i に区間 $[l', r')$ を新たに追加しても $IS(S_i)$ の値が変化しない
 - $\Leftrightarrow$ ある t ($1 \leq t \leq k_i$) に対して、 $[l_t^{(i)}, r_t^{(i)})$ と $[l', r')$ が共通部分を持つ
 - $A_i = i$ のケースの一般化になっている
- $\Rightarrow$ 同様に遅延セグメント木を用いたアルゴリズムを設計できる！

Step 3: 大きい N への対応

- 説明のため、以下のように用語を定義：
 - ある i ($1 \leq i \leq N$) に対し、 $IS(S_i) = c_i$ であるとき S_i は **飽和している** といい、そうでないとき **未飽和である** という
 - 未飽和な S_i のうち、 i が最大であるものを **フロンティア** と呼ぶ
- フロンティアを S_f とすると、以下が成立する（証明略）：
 - すべての $i > f$ について、 S_i は飽和している
 - すべての $i < f$ について、 S_i は空である

Step 3: 大きい N への対応

- アルゴリズム：

1. 飽和した S_i たちに対する $[l_t^{(i)}, r_t^{(i)})$ を管理するための遅延セグメント木を用意する
2. $j = 1, 2, \dots, M$ の順に以下を行う：
 - I. 遅延セグメント木に対する区間 max クエリを用いて、飽和している S_i のうち区間 j を追加しても $IS(S_i)$ が変わらないものがあるか判定し、あるならばそのうち i が最大なものを求める
 - II. 飽和集合 S_i に追加できる場合： $[l_t^{(i)}, r_t^{(i)})$ を計算し直し、適切に遅延セグメント木への区間代入操作を行う
 - III. できない場合：フロンティア S_f に区間 j を追加する。 S_f が飽和したか確かめ、したならば S_{f-1} を新たにフロンティアとする

Step 3: 大きい N への対応

- 計算量を考える。ボトルネックは以下の通り：
 - 飽和集合 S_i に追加する場合の $[l_t^{(i)}, r_t^{(i)})$ の再計算
 - フロントィア S_f に追加する場合の $IS(S_f)$ の再計算
- 余分な区間 (他の区間を包含している区間) を逐次取り除いた上で、残った区間を set 等で適切に管理すれば、どちらも $O(\max c_i \cdot \log M)$ で可能
- $F = \max c_i$ として、全体で $O(N + M(F \log M + \log N))$

Step 4: 統合

小課題 6 $N, M \leq 10^5$ (18 pts.) (満点)

Step 4: 統合

- 小課題 5 解法のボトルネックは以下の通りであった：
 - 飽和集合 S_i に追加する場合の $[l_t^{(i)}, r_t^{(i)})$ の再計算 ... $O(F \log M)$
 - フロンティア S_f に追加する場合の $IS(S_f)$ の再計算 ... $O(F \log M)$

Step 4: 統合

- 小課題 5 解法のボトルネックは以下の通りであった：
 - 飽和集合 S_i に追加する場合の $[l_t^{(i)}, r_t^{(i)})$ の再計算 ... $O(F \log M)$
 - フロンティア S_f に追加する場合の $IS(S_f)$ の再計算 ... $O(F \log M)$
- 本当にすべてを再計算し直す必要はあるのか？ ない
 - 左に詰めて取った区間の列 $[\alpha_1^{(i)}, \beta_1^{(i)}), [\alpha_2^{(i)}, \beta_2^{(i)}), \dots, [\alpha_{k_i}^{(i)}, \beta_{k_i}^{(i)})$
の更新について考える
 - 新たに $[l, r)$ を追加するとする。 $\beta_t^{(i)} \leq l < \beta_{t+1}^{(i)}$ なる t をとる

Step 4: 統合

- 左に詰めて取った区間の列 $[\alpha_1^{(i)}, \beta_1^{(i)}), [\alpha_2^{(i)}, \beta_2^{(i)}), \dots, [\alpha_{k_i}^{(i)}, \beta_{k_i}^{(i)})$ の更新について考える
- 新たに $[l, r)$ を追加するとする。 $\beta_t^{(i)} \leq l < \beta_{t+1}^{(i)}$ なる t をとる
- $\alpha_u^{(i)}, \beta_u^{(i)}$ ($u \leq t$) の値は変化しないのでスルー
- 新たな $\alpha_u^{(i)}, \beta_u^{(i)}$ の値を $u = t + 1, t + 2, \dots$ の順に求める。
ある u に対して値が変化しなかったら、その後の処理は打ち切って良い
- 計算量は改善しているのか？

Step 4: 統合

- 改善していて、ならし $O(\log M)$ になっている
- 「左/右に詰めて取った区間の列」から一度削除された区間はもう二度とこの列に現れないことから従う
 - 列から削除された区間が再び列に現れるとき、 $IS(S_i)$ の値が 1 以上増加している。これは S_i が飽和していることに矛盾

Step 4: 統合

- 小課題 5 解法のボトルネックは以下の通りであった：
 - 飽和集合 S_i に追加する場合の $[l_t^{(i)}, r_t^{(i)})$ の再計算 ... ならし $O(\log M)$
 - フロンティア S_f に追加する場合の $IS(S_f)$ の再計算 ... $O(F \log M)$

Step 4: 統合

- 小課題 5 解法のボトルネックは以下の通りであった：
 - 飽和集合 S_i に追加する場合の $[l_t^{(i)}, r_t^{(i)})$ の再計算 ... ならし $O(\log M)$
 - フロンティア S_f に追加する場合の $IS(S_f)$ の再計算 ... $O(F \log M)$
- これも同様に「左/右に詰めて取った区間の列」を管理し、変化がなくなった時点で更新を打ち切ればならし $O(\log M)$ になるのでは？
 - ならない
 - 未飽和のとき、各区間は $O(c_i)$ 回この列から出し入れされうる

Step 4: 統合

- しかし、未飽和の区間集合のうち区間が追加されるのはフロンティア S_f ただ一つ
- 区間集合一つだけなら平方分割で管理できる
 - ただし、フロンティアが移るたびに $O(N)$ の初期化はできないことに注意
 - 変更が加わった箇所をすべて保持しておいて、その部分だけ元に戻せば良い
- 全体計算量： $O(N + M(\sqrt{N} + \log M))$

おわりに

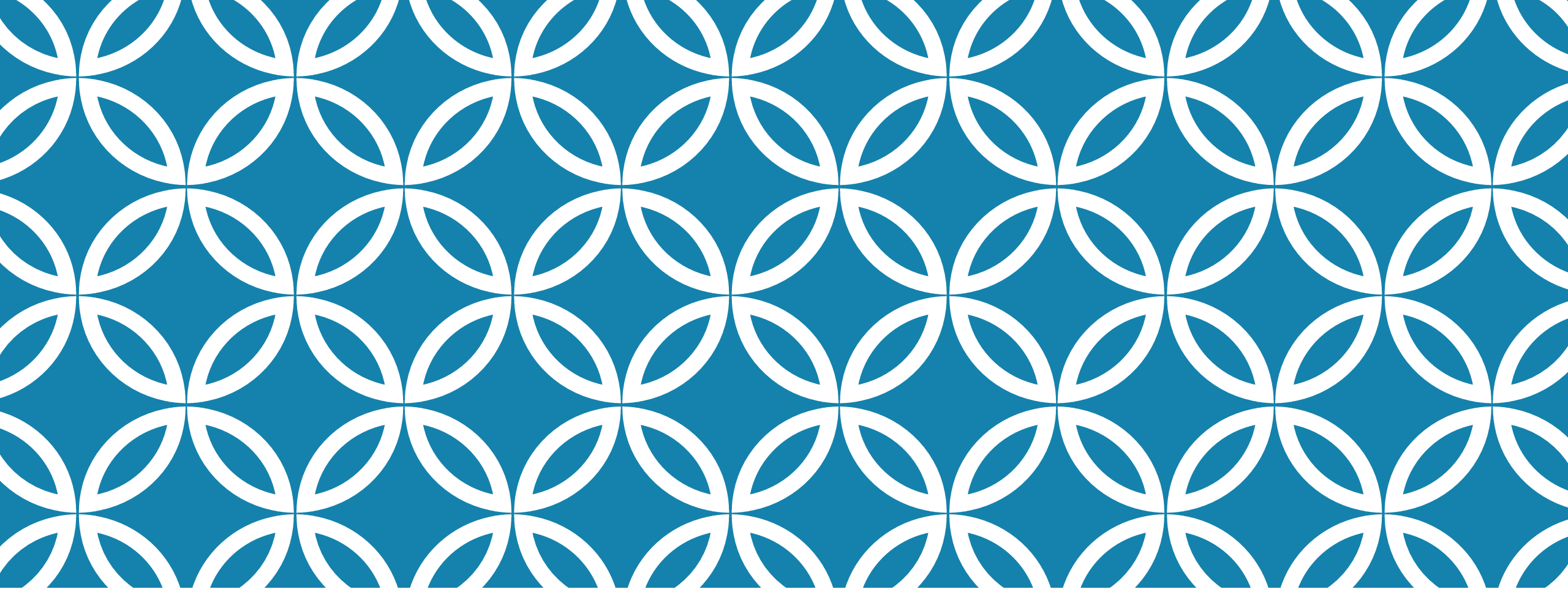
- 実装が大変
 - サンプルコードは 300 行ほど
 - 区間を扱う問題は頭が壊れがちですが、頑張りましょう
- 証明もちょっと大変
- 平方分割パートは Link-Cut Tree で代用可：全体 $O(N + M(\log M + \log N))$
 - 代用？

得点分布



得点	小課題	人数
40	1, 2, 4	2
31	1, 4	1
28	1, 2	2
19	1	8
12	4	1
0	∅	12





FORTUNE3

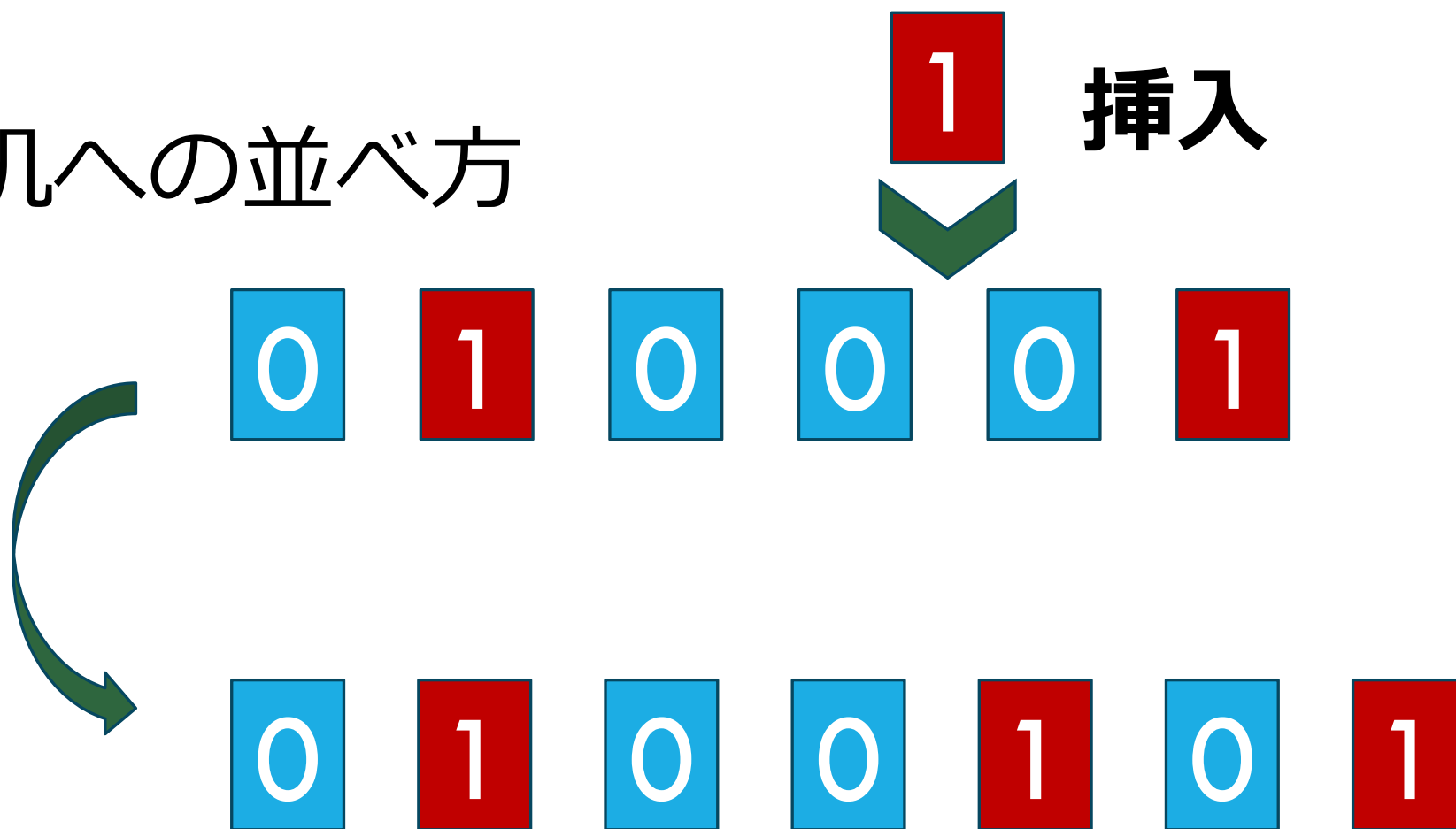
解説: 戸高空 (nxteru)

問題概要

- AnnaとBrunoが二人で占いをする
- Annaがカード( or )を**1枚ずつ**引く
⇒ 捨てる or 机に並べる
×900枚

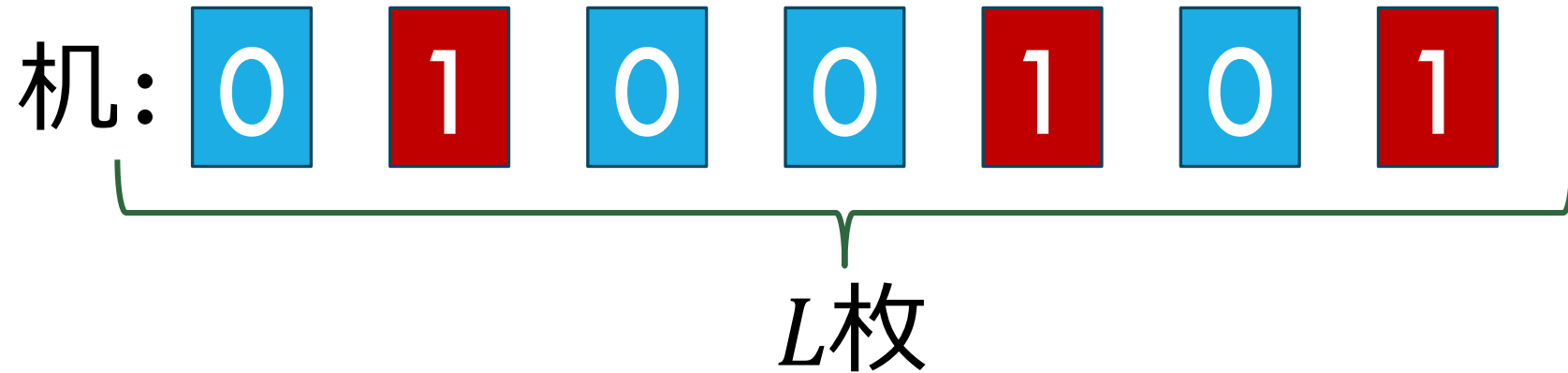
問題概要

- 机への並べ方



問題概要

- Brunoが机の**最終状態**を見る



- 引いた900枚のうち  が何枚あったかを答える
- 机に並べる枚数 L を少なくせよ

配点

$L = 900$ (3点)

$L = 451$ (16点)

$L = 63$ (34点)

$L = 30$ (49点)

$L = 18$ (71点)

$L = 16$ (81点)

$L = 14$ (100点)

配点

$L = 900$ (3点)

$L = 451$ (16点)

$L = 63$ (34点)

$L = 30$ (49点)

$L = 18$ (71点)

$L = 16$ (81点)

$L = 14$ (100点)

とりあえず

- Annaはすべてのカードを机に並べる

配点

$L = 900$ (3点) AC

$L = 451$ (16点)

$L = 63$ (34点)

$L = 30$ (49点)

$L = 18$ (71点)

$L = 16$ (81点)

$L = 14$ (100点)

配点

$L = 900$ (3点) AC

$L = 451$ (16点)

$L = 63$ (34点)

$L = 30$ (49点)

$L = 18$ (71点)

$L = 16$ (81点)

$L = 14$ (100点)

半分くらいにしよう

- 0の奇数枚目と1の奇数枚目を机に並べる

 この順に引く

0 0 1 0 0 1 1 0 1 0 0

机 : 0 1 0 1 0 0

- 机を2倍すると偶奇以外分かる

机の枚数で偶奇が分かる

0奇数

1	2	3	4	5	6	7
---	---	---	---	---	---	---

1奇数

1	2	3
---	---	---

机: $N/2 + 1$ 枚

0偶数

1	2	3	4	5	6
---	---	---	---	---	---

1偶数

1	2	3	4
---	---	---	---

机: $N/2$ 枚

配点

$L = 900$ (3点) AC

$L = 451$ (16点) AC

$L = 63$ (34点)

$L = 30$ (49点)

$L = 18$ (71点)

$L = 16$ (81点)

$L = 14$ (100点)

配点

$L = 900$ (3点) AC

$L = 451$ (16点) AC

$L = 63$ (34点)

$L = 30$ (49点)

$L = 18$ (71点)

$L = 16$ (81点)

$L = 14$ (100点)

考察

- 0と1は対称
 - 0の枚数が分かれば1の枚数も分かる
- 最後の1枚を引いたとき, 答えが確定する
⇒最後の1枚をうまく使って情報を伝えたい

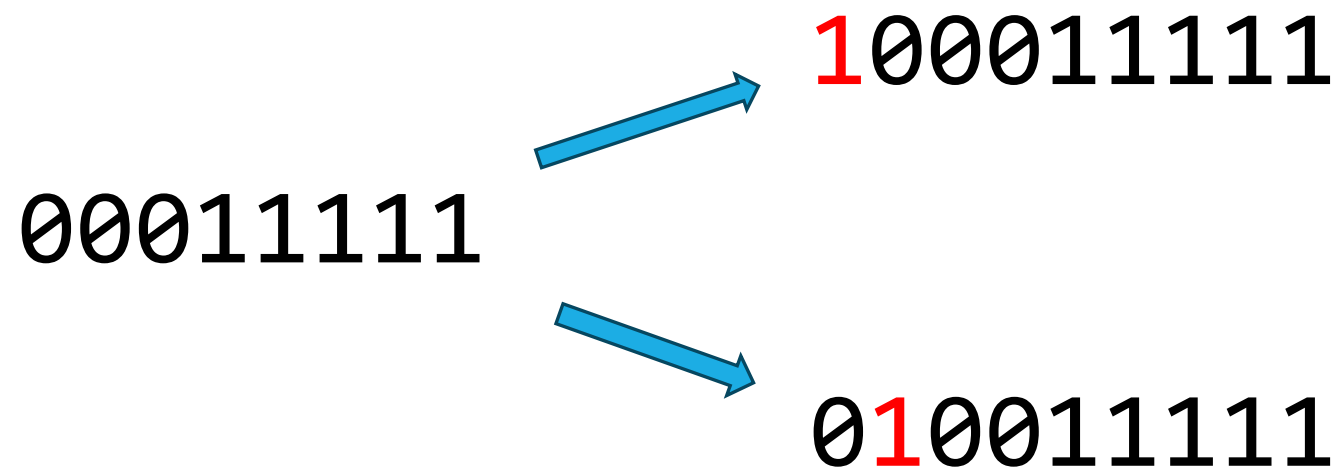
最後の1枚で伝える

- 最後の1枚までに机にこう並べておく

00011111

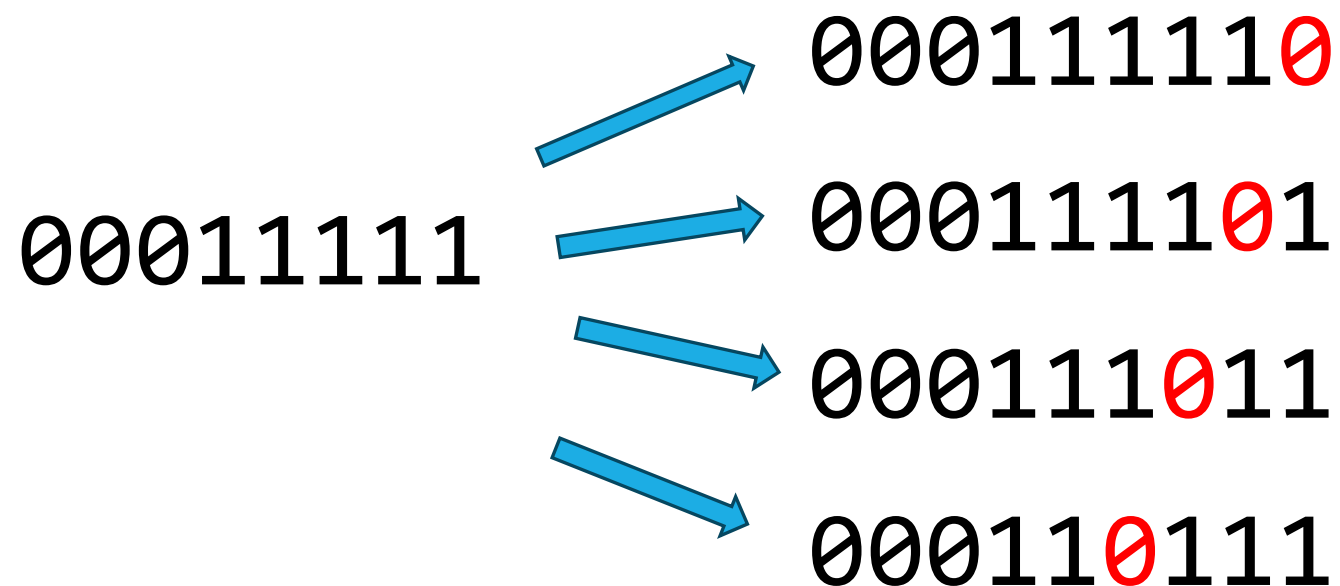
最後の1枚で伝える

- 最後の1枚が1のとき



最後の1枚で伝える

- 最後の1枚が0のとき



最後の1枚で伝える

- こんな風に0と1を分けて机に並べておくと

00011111

最後の1枚で

(机の0の枚数-1通り) or (机の1の枚数-1通り)

伝えられる

Anna

- 基本は机に0と1を分けながら並べていくが捨てる
れるときは捨てる
- 机に0が x 枚ならこれから0は $x - 2$ 枚は捨てる
- 机に1が y 枚ならこれから1は $y - 2$ 枚は捨てる

Anna



Anna 最後の1枚

机 : $\underbrace{0\dots 00}_x \underbrace{11\dots 1}_y$

- 最後の1枚が0なら
1を最後に何枚捨てたか($0 \sim y - 2$)伝える
- 最後の1枚が1なら
0を最後に何枚捨てたか($0 \sim x - 2$)伝える

Bruno

机 : 00010...0011...1



x y

- 0と1の枚数を見ると最後に捨てられた枚数以外のだいたいの枚数が分かる
- さらに0か1のどちらかは最後に捨てた枚数が伝えられている
- 0か1どちらかの枚数が正確に分かるので正しく答えられる

最悪ケース

- だいたい0と1が半々のときが多くなる
- このとき机の枚数 $L = 2\sqrt{N}$ くらい

配点

$L = 900$ (3点) AC

$L = 451$ (16点) AC

$L = 63$ (34点) AC

$L = 30$ (49点)

$L = 18$ (71点)

$L = 16$ (81点)

$L = 14$ (100点)

配点

$L = 900$ (3点) AC

$L = 451$ (16点) AC

$L = 63$ (34点) AC

$L = 30$ (49点)

$L = 18$ (71点)

$L = 16$ (81点)

$L = 14$ (100点)

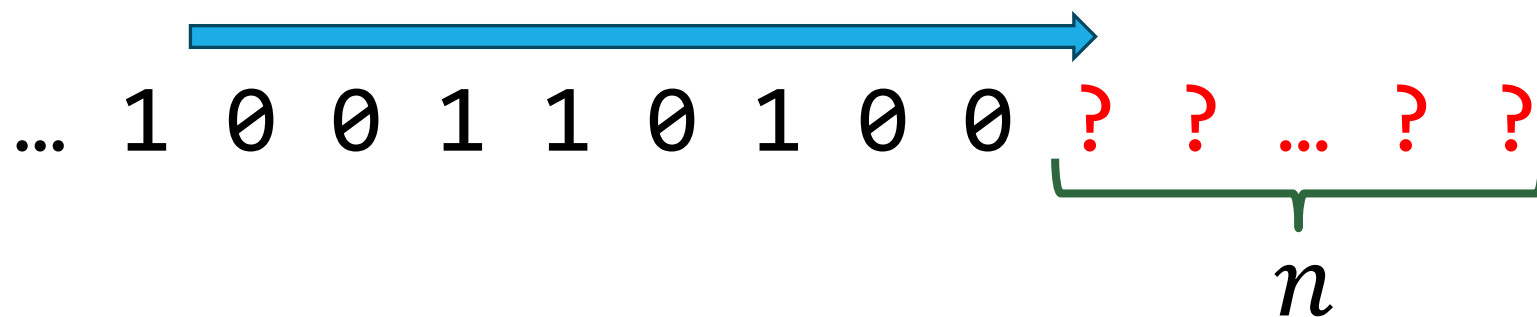
考察

- x 枚目を机に並べるときには、最大でも x 枚目までの情報しか考慮できない

⇒できるだけ最後の方で一気に情報を伝えたい

方針

- 最後の n 枚を机に並べそれまでの1の枚数を伝える

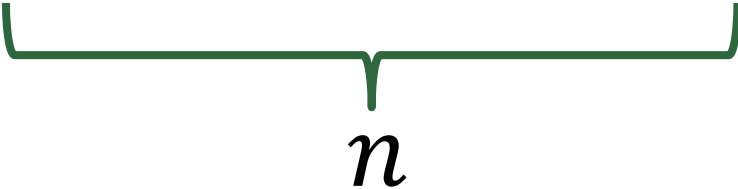


難しさ

- 最後の n 枚の0か1が未観測

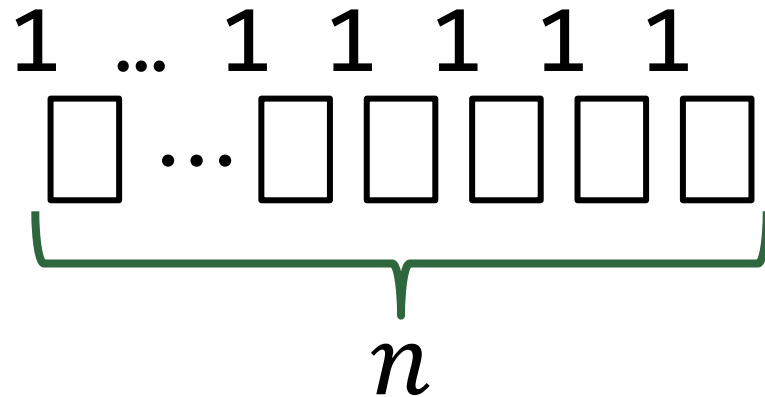
最後の n 枚がすべて $\emptyset$ のとき

- 机にこれを確保しておく

$1 \dots 1 \ 1 \ 1 \ 1 \ 1$

 n

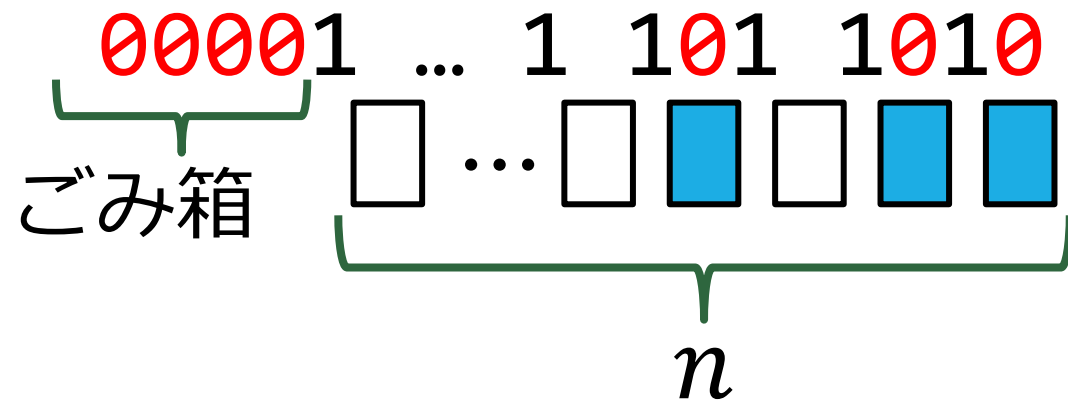
最後の n 枚がすべて $\emptyset$ のとき

- 机にこれを確保しておく



最後の n 枚がすべて0のとき

- 机にこれを確保しておく



- n 桁の2進数が伝えられる

最後の n 枚がすべて1のとき

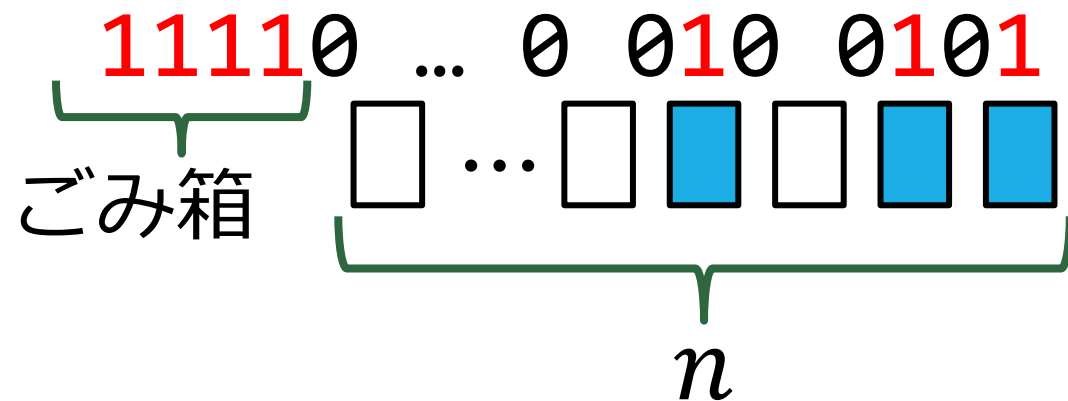
最後の n 枚がすべて1のとき

- 机にこれを確保しておく

$\underbrace{0 \dots 0 \ 0 \ 0 \ 0 \ 0 \ 0}_n$

最後の n 枚がすべて1のとき

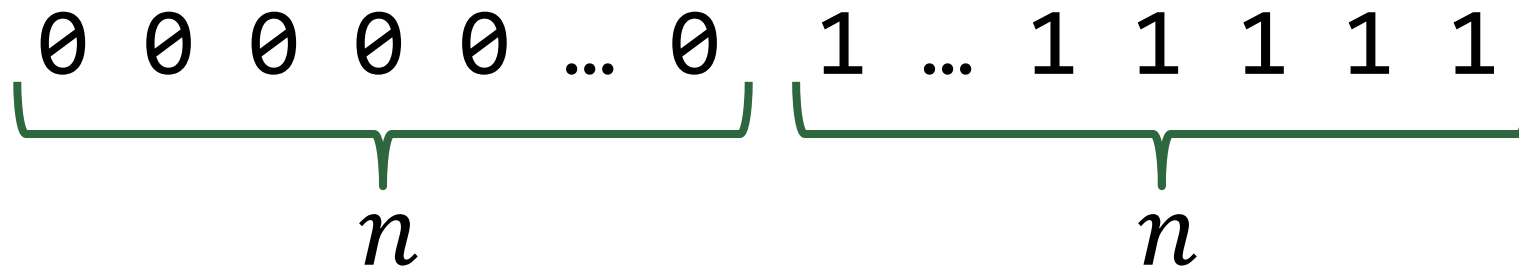
- 机にこれを確保しておく



- n 桁の2進数が伝えられる

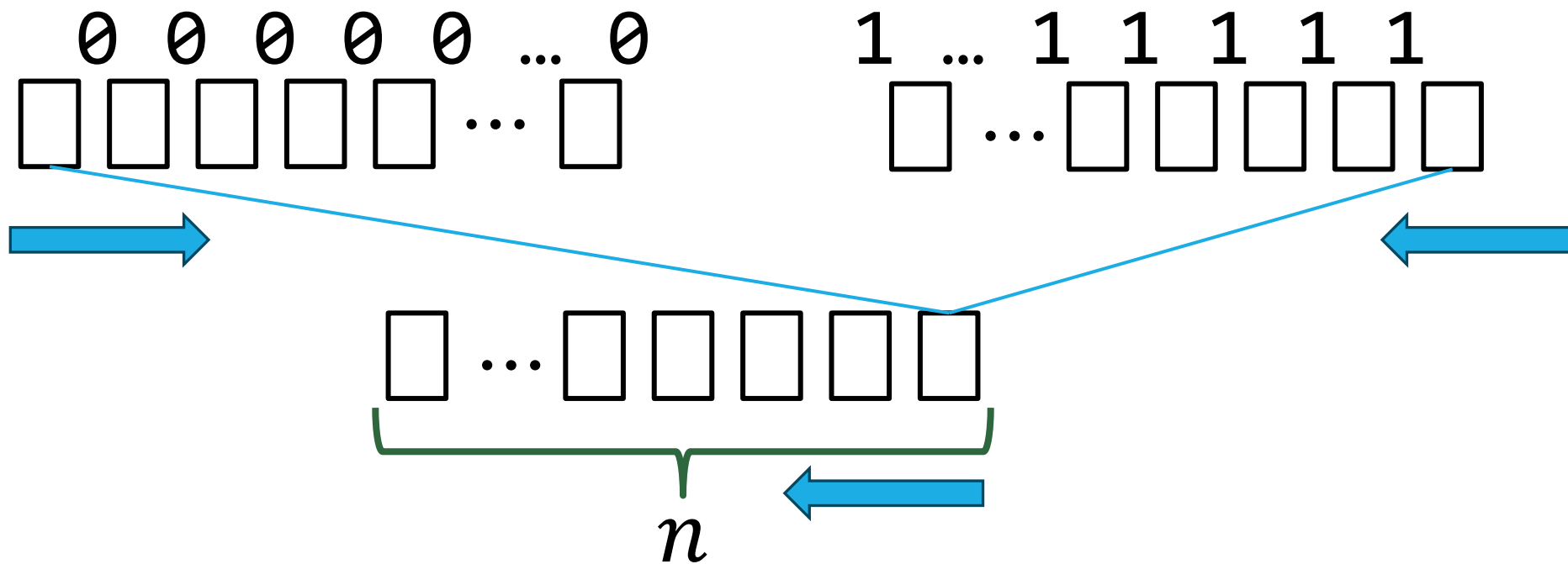
最後の n 枚が?のとき

- 机にこれを確保しておく



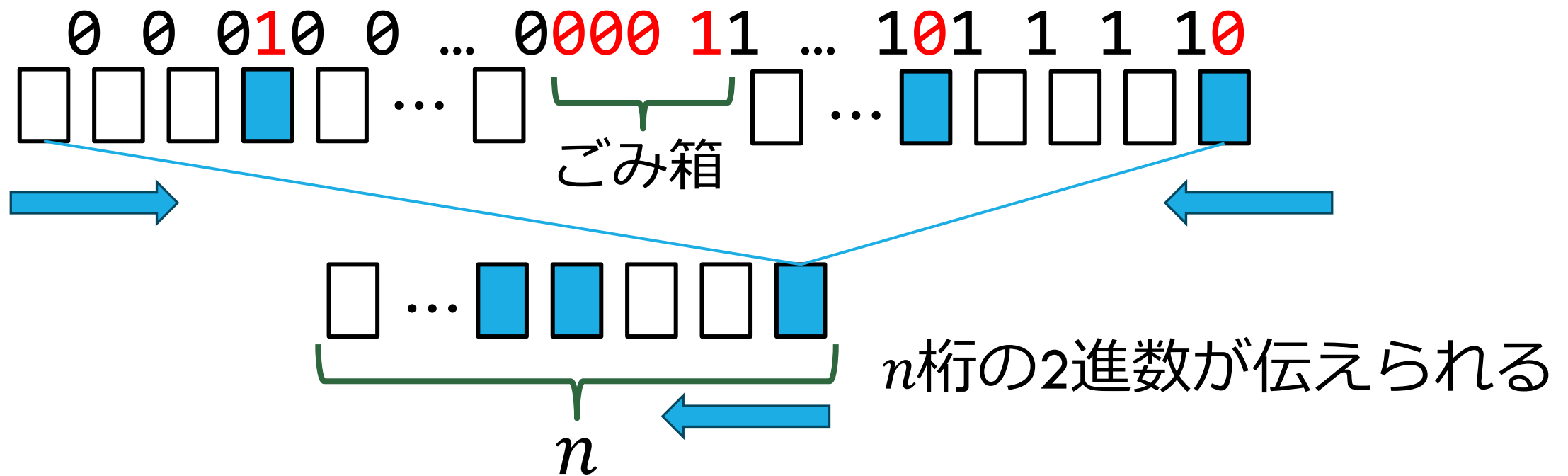
最後の n 枚が?のとき

- 机にこれを確保しておく



最後の n 枚が?のとき

- 机にこれを確保しておく



まとめ

- 900枚だから $n = 10$ で十分
- 机に0を10枚, 1を10枚確保しておく
 - 10枚確保できなかった場合は別処理
- 最後の10枚でそれまでの1の枚数を2進数で伝える
- $L = 30$

配点

$L = 900$ (3点) AC

$L = 451$ (16点) AC

$L = 63$ (34点) AC

$L = 30$ (49点) AC

$L = 18$ (71点)

$L = 16$ (81点)

$L = 14$ (100点)

配点

$L = 900$ (3点) AC

$L = 451$ (16点) AC

$L = 63$ (34点) AC

$L = 30$ (49点) AC

$L = 18$ (71点)

$L = 16$ (81点)

$L = 14$ (100点)

最後の n 枚がすべて $\emptyset$ のとき

- 机にこれを確保しておく

$$\underbrace{1 \dots 1 \ 1 \ 1 \ 1 \ 1}_p$$

最後の n 枚がすべて $\emptyset$ のとき

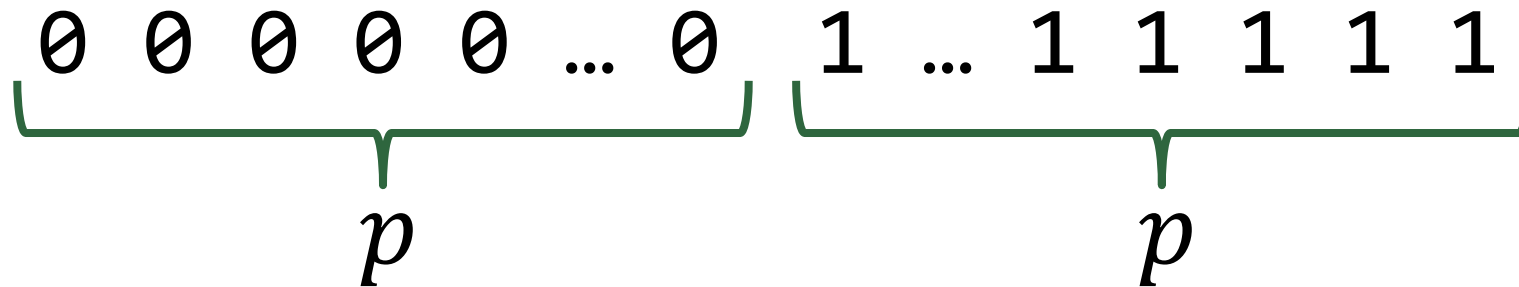
- 机にこれを確保しておく

$$\underbrace{1 \quad \emptyset \quad 1 \quad \emptyset \quad \emptyset \quad 1 \quad \dots \quad 1 \quad 1 \quad \emptyset \quad 1 \quad \emptyset}_{n+p}$$

- p 個の仕切りに n 個のボールを挿入する
 ${}_{n+p}C_n$ 通り伝えられる

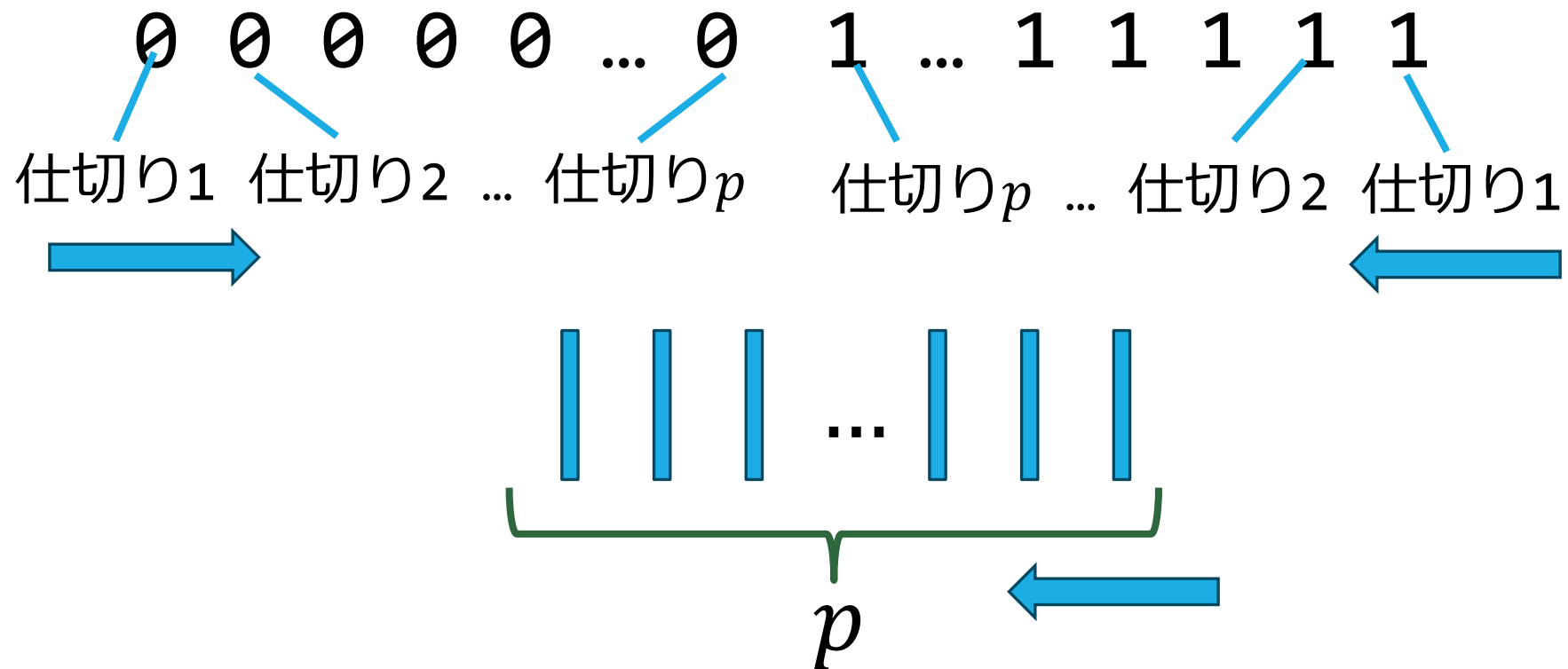
最後の n 枚が?のとき

- 机にこれを確保しておく



最後の n 枚が?のとき

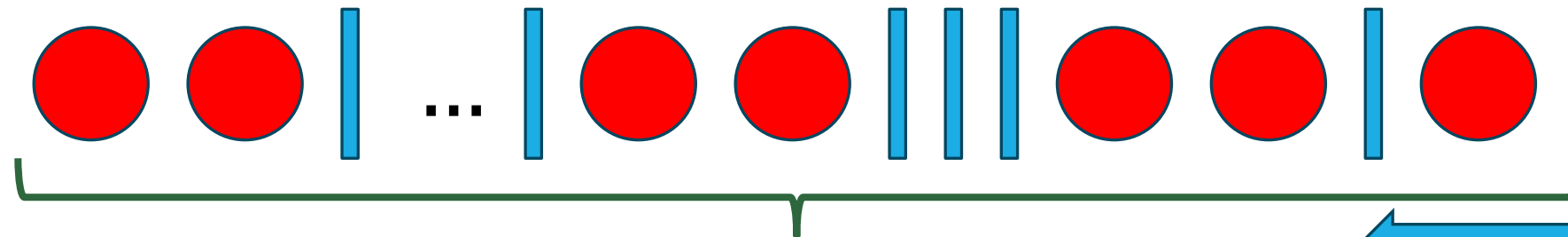
- 机にこれを確保しておく



最後の n 枚が?のとき

- p 個の仕切りに n 個のボールを挿入できる

$0 \overset{\text{blue arrow}}{\rightarrow} 1100010 \dots 01 \quad 01 \dots 1 \overset{\text{blue arrow}}{\leftarrow} 011110$

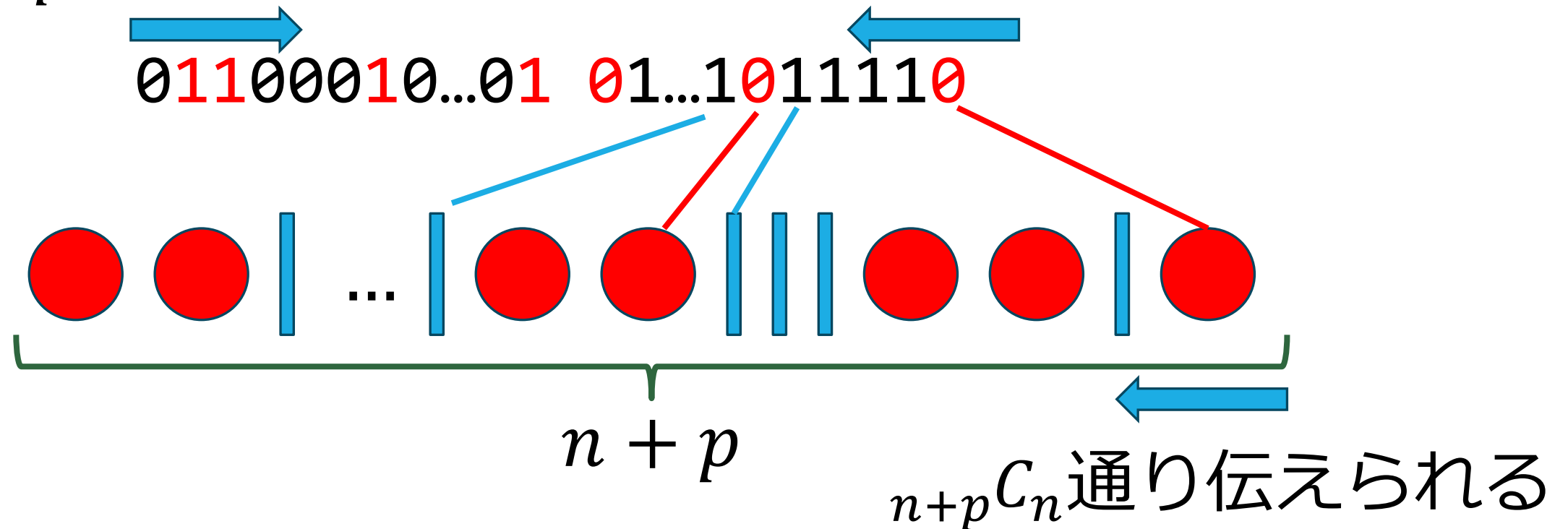


$n + p$

$\overset{\text{blue arrow}}{\leftarrow} n+pC_n$ 通り伝えられる

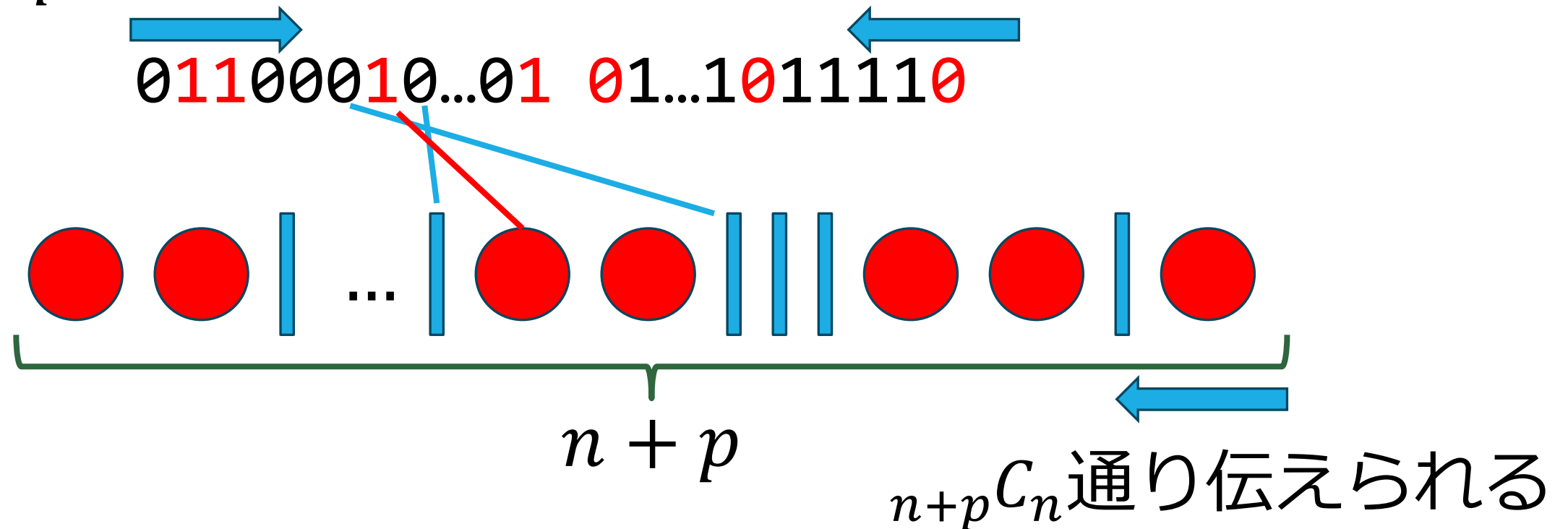
最後の n 枚が?のとき

- p 個の仕切りに n 個のボールを挿入できる



最後の n 枚が?のとき

- p 個の仕切りに n 個のボールを挿入できる



まとめ

- p 個の0と p 個の1を確保
- 最後の n 枚を挿入して ${}_{n+p}C_n$ 通り伝える
- $p = 5, n = 8$ で1287通り
- $L = 18$

配点

$L = 900$ (3点) AC

$L = 451$ (16点) AC

$L = 63$ (34点) AC

$L = 30$ (49点) AC

$L = 18$ (71点) AC

$L = 16$ (81点)

$L = 14$ (100点)

配点

$L = 900$ (3点) AC

$L = 451$ (16点) AC

$L = 63$ (34点) AC

$L = 30$ (49点) AC

$L = 18$ (71点) AC

$L = 16$ (81点)

$L = 14$ (100点)

ボールの個数を変えよう

- 仕切りとして p 個の0と p 個の1を確保
- 最後の n 枚を挿入して p 個の仕切りに n 個のボールを挿入する ${}_{n+p}C_n$ 通り伝える

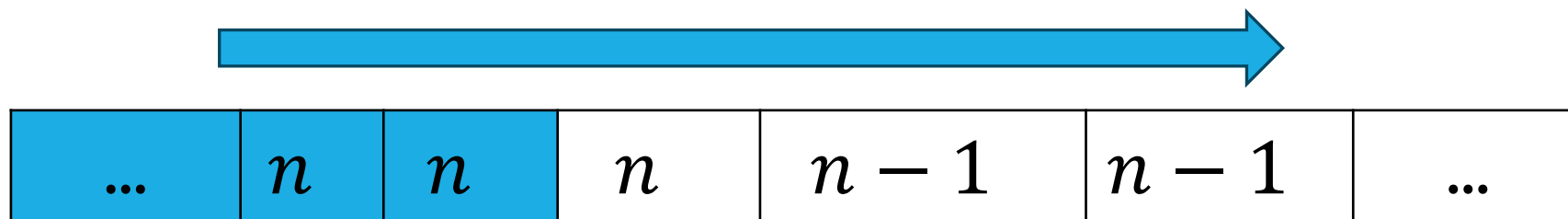
ボールの個数を変えよう

- 仕切りとして p 個の0と p 個の1を確保
- 最後の n 枚を挿入して p 個の仕切りに n 個のボールを挿入する ${}_{n+p}C_n$ 通り伝える
- **L を増やさずにパターンを増やしたい**
- **ボールの個数として $n-1, n-2 \dots$ も考える**
- **机に並べられた枚数でボールの個数を区別できる**

ボールの個数を切り替える

伝えたい1の枚数

ボール個数
の割り当て

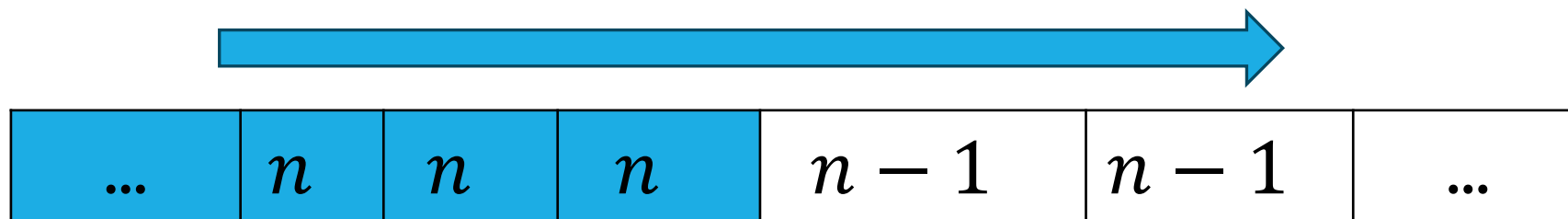


ボール個数 n

ボールの個数を切り替える

伝えたい1の枚数

ボール個数
の割り当て



ボール個数 n

ボールの個数を切り替える

伝えたい1の枚数

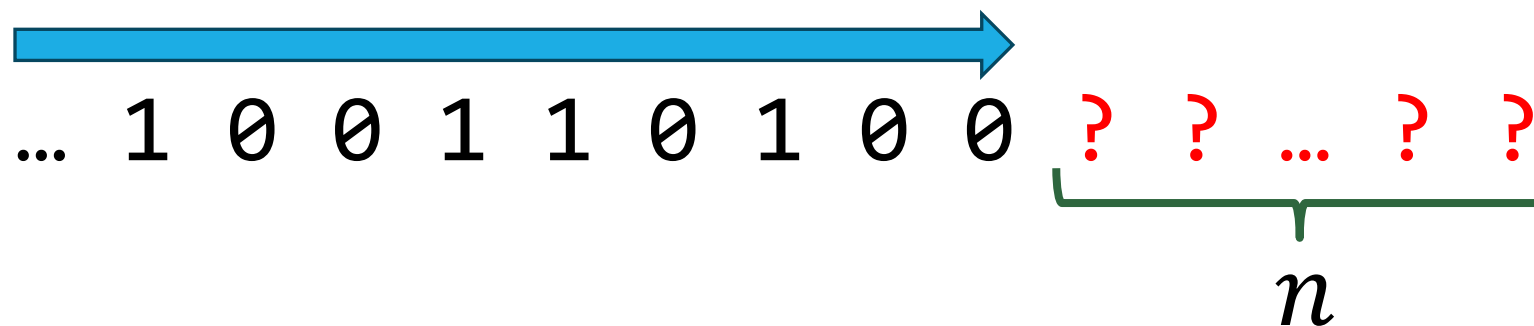


ボール個数
の割り当て

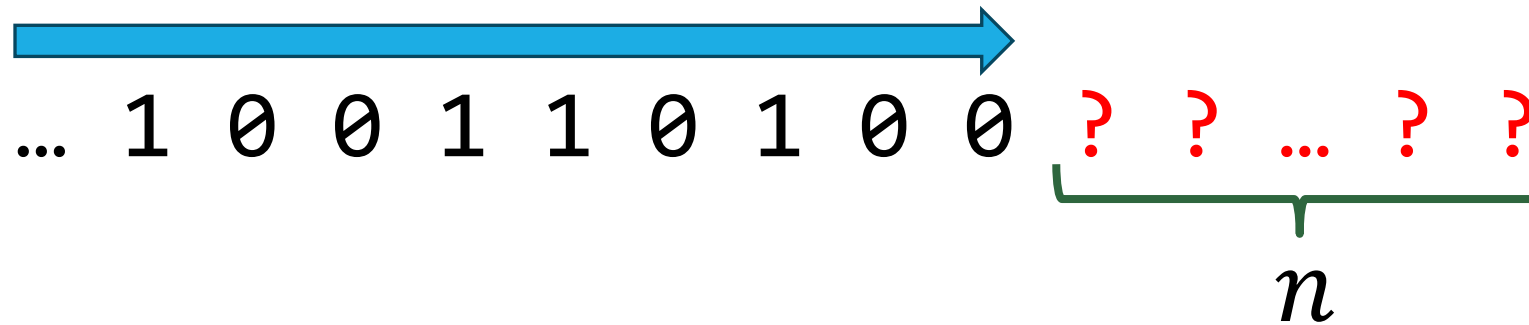
...	n	n	n	$n - 1$	$n - 1$	...
-----	-----	-----	-----	---------	---------	-----

ボール個数 $n - 1$

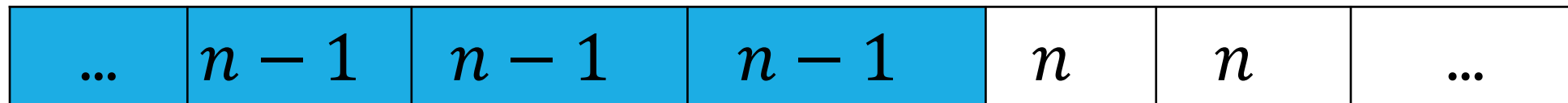
ボールの個数を切り替える



ボールの個数を切り替える



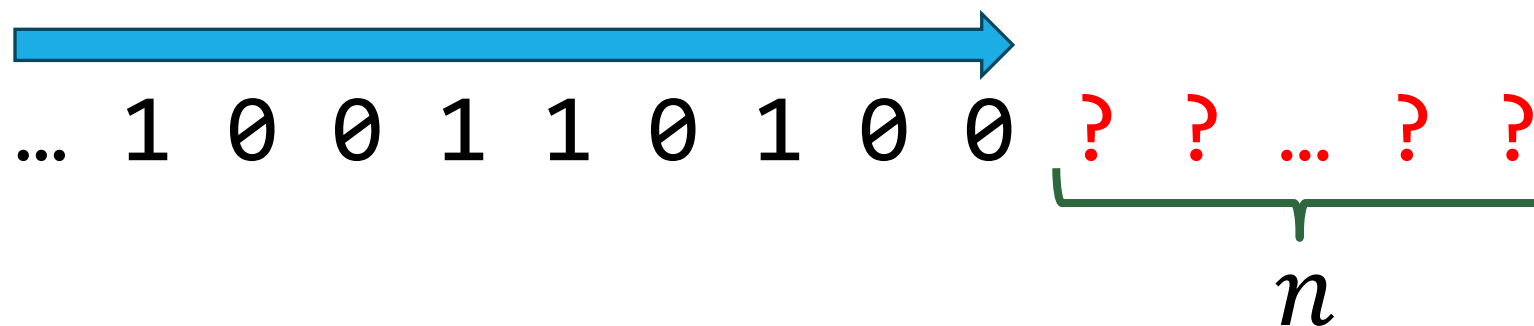
ボールの割り当て方を変なことにすると？



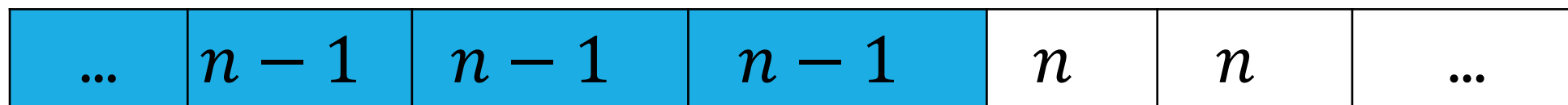
伝えたい1の枚数



ボールの個数を切り替える



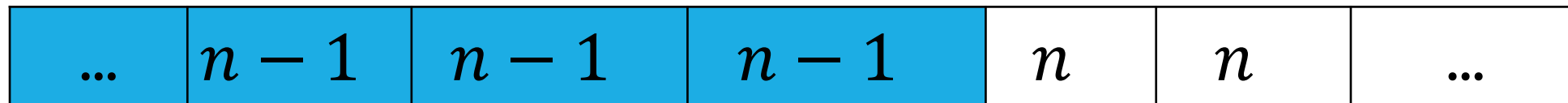
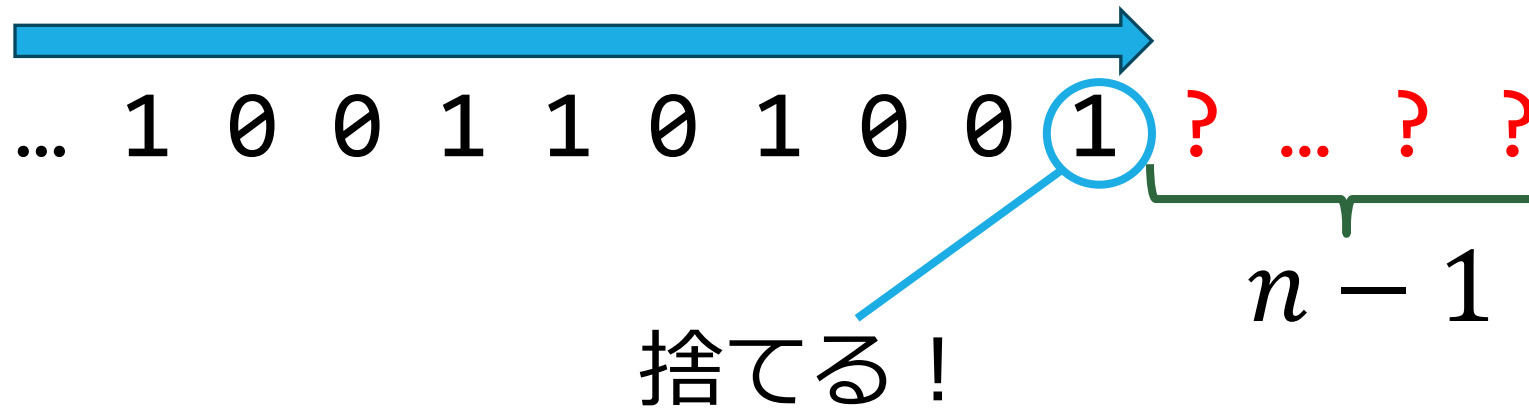
ボール $n - 1$ 個のパターンだ！



伝えたい1の枚数



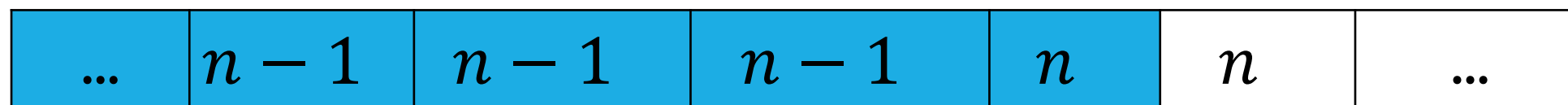
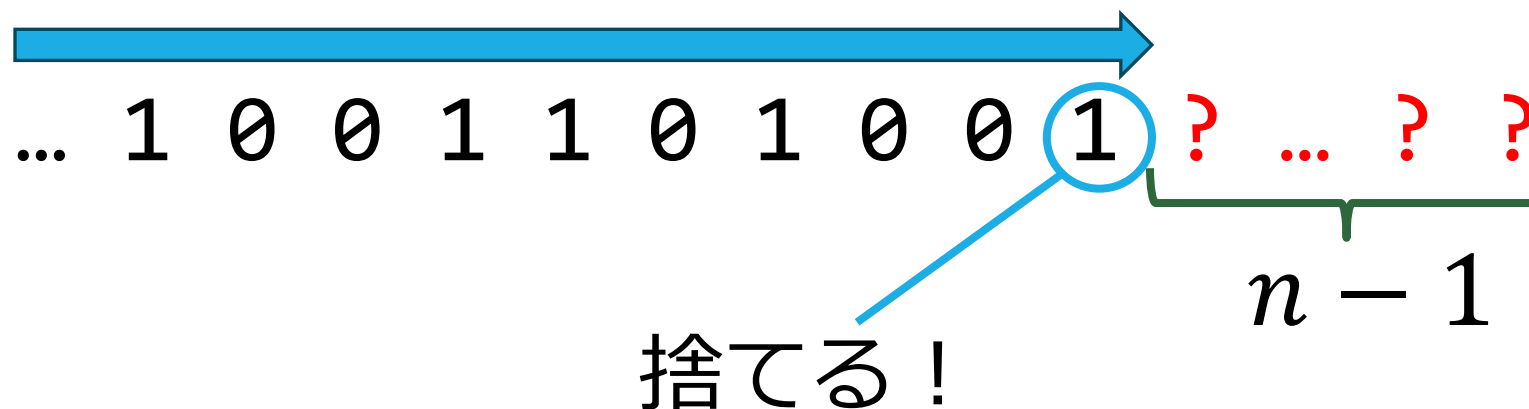
ボールの個数を切り替える



伝えたい1の枚数



ボールの個数を切り替える

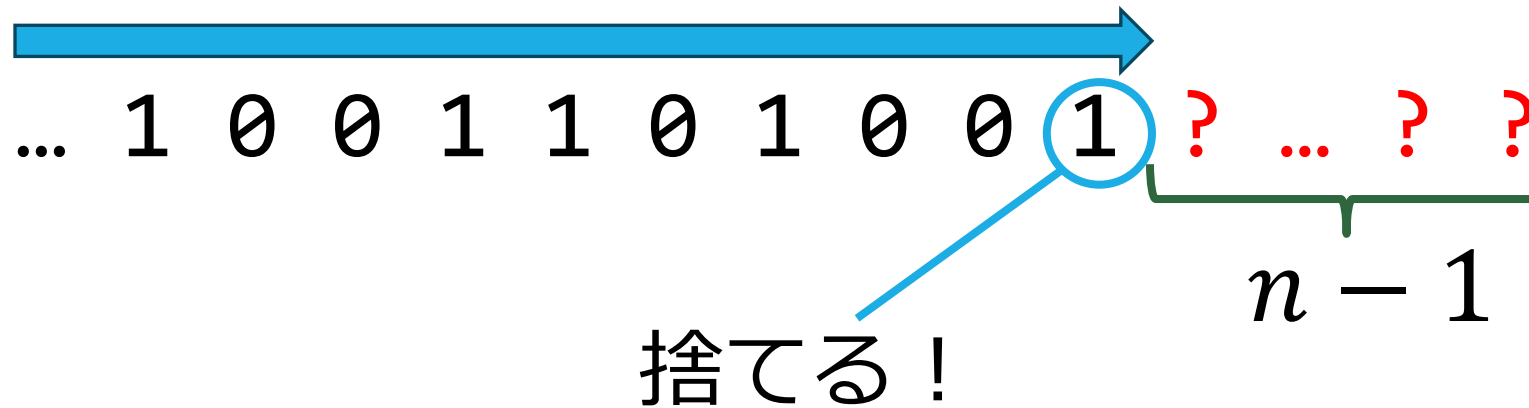


伝えたい1の枚数



マズイ！

ボールの個数を切り替える



...	$n - 1$	$n - 1$	$n - 1$	n	n	...
-----	---------	---------	---------	-----	-----	-----

伝えたい1の枚数が増えるにしたがって
ボールの個数が減るように割り当てる

マズイ！

まとめ

- p 個の0と p 個の1を確保
- 最後の n 枚を挿入して ${}_{n+p}C_n$ 通り伝える
- $n - 1$ 枚, $n - 2$ 枚...挿入するパターンも考える
- 机の枚数を見ればどのパターンか区別できる
- $p = 4, n = 8$ で ${}_{12}C_8 + {}_{11}C_7 + {}_{10}C_6 = 1035$ でOK
- $L = 16$

配点

$L = 900$ (3点) AC

$L = 451$ (16点) AC

$L = 63$ (34点) AC

$L = 30$ (49点) AC

$L = 18$ (71点) AC

$L = 16$ (81点) AC

$L = 14$ (100点)

配点

$L = 900$ (3点) AC

$L = 451$ (16点) AC

$L = 63$ (34点) AC

$L = 30$ (49点) AC

$L = 18$ (71点) AC

$L = 16$ (81点) AC

$L = 14$ (100点)

仕切りの個数も変えたい

- 仕切りは確保していくもの
- 伝えたい数が増えるほど仕切りの数も増えるとするのかよさそう
- しかし、仕切りを1個増やすと決めて0,1どちらも確保しようとしたとき、それ以降どちらかこなくて完全に確保できない場合がある

仕切りの個数も変えたい

アイデア：

- 仕切りとして0は p 個確保してしまう
- 伝えたい数(1の枚数)が増えるほど仕切りの数も増やして1を追加で確保していく
- 仕切りの数が決まったらボールの数もこれまで同様に変える
- $p = 4$ でやろう

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

${}_{10}C_1$ ${}_9C_1$...

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

4個 6個

机：

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机 : 0000

0の仕切りを4個確保

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個	1個	...	1個	1個
ボール9個	8個		2個	1個

2個	2個	...	2個	2個
8個	7個		2個	1個

机 : 0000

3個	3個	...	3個
7個	6個		2個

4個
6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個	1個	...	1個	1個
ボール9個	8個		2個	1個

2個	2個	...	2個	2個
8個	7個		2個	1個

3個	3個	...	3個
7個	6個		2個

机 : 00001

1の仕切りを1個確保

4個
6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

机 : 00001

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

4個 6個

机 : 00001

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机：00001

もしここで山札残り8枚を迎えたら？

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

${}_9C_1$

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机 : 00001

残り8枚????????を1個の仕切りに挿入して
 ${}_9C_1$ 通りを伝える

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机 : 00001
まだ続く

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机 : 00001
まだ続く

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机 : 00001

まだ続く

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机 : 00001

まだ続く

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机 : 000011

仕切り2個目を確保

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机 : 000011

仕切り2個目を確保

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

机 : 000011

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

机 : 000011

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

机 : 000011

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数

仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

机 : 000011

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机：000011

もしここで山札残り7枚を迎えたら？

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数

仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

${}_9C_2$

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

4個 6個

机 : 000011

残り7枚??????を2個の仕切りに挿入して
 ${}_9C_2$ 通りを伝える

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

4個 6個

机 : 000011
まだ続く

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

4個 6個

机 : 000011
まだ続く

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机 : 000011
まだ続く

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数

仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

机：000011

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

もしここで山札残り7枚を迎えたら？

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机 : 000011

ボール(残り山札)2個になるまで
捨てていくと

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数

仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机：000011

その過程で増えることがある

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

机 : 000011

これに対応できるように重なりがないといけない

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数

仕切り1個 ボール9個	1個 8個	...	1個 2個	1個 1個
----------------	----------	-----	----------	----------

2個 8個	2個 7個	...	2個 2個	2個 1個
----------	----------	-----	----------	----------

3個 7個	3個 6個	...	3個 2個
----------	----------	-----	----------

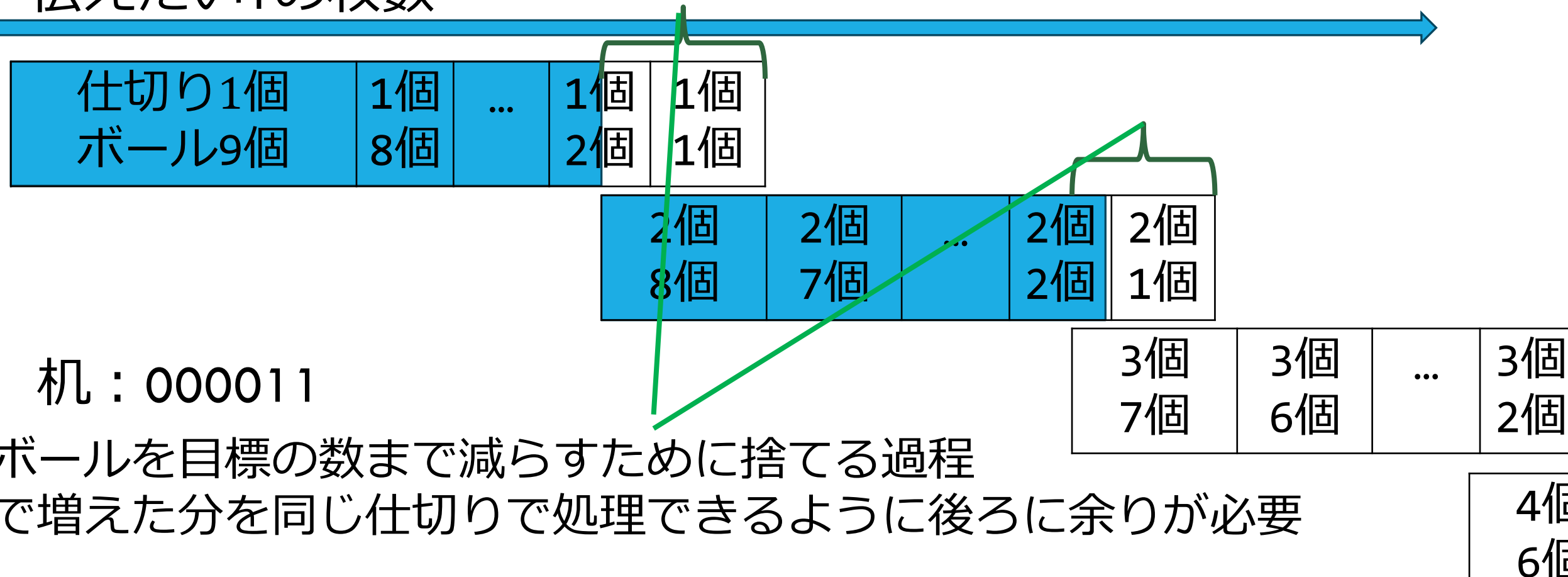
机 : 000011

今回捨てる過程で増えた分は
仕切り2個のままで対応する

4個 6個

仕切りの個数も変えたい

伝えたい1の枚数



まとめ

- 仕切りとして0は4個確保してしまう
- 送りたい数が増えるごとに仕切りの数を増やして1を追加で確保する
- ボールの個数も $L = 14$ に入るまですべてのパターンを考える

例：仕切り2個の場合,

0が4個+1が2個+ボールが8～1個

まとめ

- ${}_9C_1 + {}_8C_1 + \cdots + {}_2C_1$
 $+ {}_8C_2 + {}_7C_2 + \cdots + {}_3C_2$
 $+ {}_7C_3 + {}_6C_3 + \cdots + {}_4C_3$
 $+ {}_6C_4 + {}_5C_4 \quad - \text{重なりの分} \geq 900 \text{ ちょうど}$
- $L = 14$

配点

$L = 900$ (3点) AC

$L = 451$ (16点) AC

$L = 63$ (34点) AC

$L = 30$ (49点) AC

$L = 18$ (71点) AC

$L = 16$ (81点) AC

$L = 14$ (100点) AC

得点分布

(3点) × 17人

(16点) × 4人

(18点) × 1人

(45点) × 1人

(47点) × 1人

JOI 2024/2025 春季トレーニング Day 1

ビ太郎の旅 2

解説担当：兼下航輔(potato167)

問題概要

- $H \times W$ のマス目のマス (i, j) には整数 $T_{i,j}$ が書かれている。
- マス (a, b) からマス (c, d) に1回の**ハイジャンプ**で移動できることは以下と同値
 - マス (a, b) からマス (c, d) に、上下左右に隣り合う $L + T_{a,b}$ 以下の数字が書かれたマスのみを辿って移動できる。
- Q 個のクエリを処理する。
 - マス (A_k, B_k) からマス (C_k, D_k) に移動するのに何回の**ハイジャンプ**が必要か

小課題 1 : $H \times W \leq 300$

- 全てのマスの組 $((a, b), (c, d))$ について、マス (a, b) からマス (c, d) に 1 回のハイジャンプで移動できるかを求める。
 - 始点を固定すると BFS など全ての終点に対して 1 回のハイジャンプで移動か否かが $O(HW)$ で求まる
- その後、ワーシャルフロイド法をすると、全てのマスの組に対する答えが求まる。
 - 時間計算量は $O((HW)^3)$
- クエリは $O(1)$ で処理

小課題 2: $H \times W \leq 3000$

- マス (A_k, B_k) から 1 回目のハイジャンプで行くべきマスは以下のいずれかに限定して良い。
 - (C_k, D_k)
 - マス (A_k, B_k) から 1 回のハイジャンプで行けるマス (i, j) の中で、 $T_{i,j}$ が最も大きいマス。← (A_k, B_k) の**ベストな行き先**と呼びます。
- 最後の 1 回以外は行けるマスの中で $T_{i,j}$ が最も大きいマスに移動し続けるのが最適。

クエリ前計算

- 全てのマス (a, b) に対して、BFS 等で以下の 3 つを求めておく
 - (a, b) から 1 回のハイジャンプで移動可能なマス
 - 移動可能なマスの中で $T_{i,j}$ が最も大きいマス (i, j)
 - マス (a, b) から始めて、 $T_{i,j}$ が最も大きいマス (i, j) へハイジャンプを x 回 ($1 \leq x \leq HW$) 繰り返したときに辿りつくマス
- これらは $O((HW)^2)$ で求められる。

クエリ処理

- (A_k, B_k) から (C_k, D_k) まで x 回で辿り着けるかを判定問題を解く。
 - 前計算の結果を用いて $O(1)$ で求められる。
- x を二分探索すると、クエリごとに $O(\log(HW))$ で求まる。

小課題 3: $H \times W \leq 150000$ 始点固定

- 今まで通り全ての始点に対して行ける場所を列挙しては間に合わない。
- BFS していたところを priority_queue にすることで、全ての答えを $O(HW \log(HW))$ で求められる。

例

- マス (1, 1) から 1 回のハイジャンプでいけるマスと、それに隣接するマスを管理する。
- 次は $57 + L$ 以下のマスを隣接するマスから探す。
- 辿り着けるマスに隣接するマスを `priority_queue` で管理すれば良い。

53	55	51	49
56	60	89	45
54	57	92	43
96	99	95	92

小課題 4: $H \times W \leq 150000, Q \leq 150000$

以下の二つを高速に処理したい。

- 前計算で、マス (a, b) のベストな行き先を全ての (a, b) に対して求める。
- クエリを回数に対する二分探索で処理する。

前計算

- マス (a, b) から 1 回のハイジャンプで行けるマス (i, j) の中で、 $T_{i,j}$ が最も大きいマスを全ての (a, b) に対して求める。
- 全てのマスについて個別にやる余裕はないので、同時に求める。
- Union Find を用いると時間計算量が改善できる。

前計算

Q. マス (a, b) から 1 回のハイジャンプで行けるマス (i, j) は？

A. 書かれている数が $T_{a,b} + L$ 以下の隣り合うマスの組を全て辺で結んだときに、 (a, b) と連結であるもの

例

- T が左のマス目で表され、 $L = 3$ のときを考える。

3	6	8	5
7	5	7	4
7	9	1	2

例

- $(3, 3)$ の周りを見る。
- 変化なし。

3	6	8	5
7	5	7	4
7	9	1	2

例

- $(3, 4)$ の周りを見る。
- $(3, 3)$ とマージされる。

3	6	8	5
7	5	7	4
7	9	1	2

例

- $(1, 1)$ の周りを見る。
- 変化なし。

3	6	8	5
7	5	7	4
7	9	1	2

例

- $(2, 4)$ の周りを見る。
- $(3, 4)$ とマージされる。
- $T_{3,3} + L = 4$ であるため、 $(3, 3)$ のベストな行き先が決まる。
- この場合、 $(3, 3)$ のベストな行き先は $(2, 4)$

3	6	8	5
7	5	7	4
7	9	1	2

例

- $(2, 2), (1, 4)$ の周りを見る。
- $(2, 4)$ と $(1, 4)$ がマージされる。
- $(3, 4)$ のベストな行き先は $(1, 4)$

3	6	8	5
7	5	7	4
7	9	1	2

例

- $(1, 2)$ の周りを見る。
- $(1, 1), (2, 2)$ がマージされる。
- $(1, 1)$ のベストな行き先は $(1, 2)$

3	6	8	5
7	5	7	4
7	9	1	2

例

- 色々マージされる。
- $(2, 4)$ のベストな行き先は $(2, 1)$
- $(2, 3), (3, 1)$ としても良い。

3	6	8	5
7	5	7	4
7	9	1	2

前計算

- $T_{i,j}$ の小さい順に、自分の周りで、自分の値以下であるものと merge するという操作をする。
- $T_{i,j}$ の値が $T_{a,b} + L$ 以下であるものに対する操作が全て終わったタイミングで、 $T_{a,b}$ が含まれる連結成分の中にある $T_{i,j}$ が最も大きい (i, j) を確認する。
- 時間計算量は $O(HW \log(HW))$ や $O(HW \alpha(HW))$ で求められる。

クエリ処理

「 (A_k, B_k) から x 回のハイジャンプで (C_k, D_k) に辿り着けるか」という判定問題は、以下のように置き換えれる。

- (A_k, B_k) から今いるマスのベストな行き先に $(x - 1)$ 回移動したときのマスを (a, b) としたときに、 (a, b) から 1 回のハイジャンプで (C_k, D_k) に辿り着けるか。

クエリ処理

「 (A_k, B_k) から x 回のハイジャンプで (C_k, D_k) に辿り着けるか」という判定問題は、以下のように置き換えれる。

- (A_k, B_k) から今いるマスのベストな行き先に $(x - 1)$ 回移動したときのマスを (a, b) としたときに、 (a, b) から 1 回のハイジャンプで (C_k, D_k) に辿り着けるか。

クエリ処理

- (A_k, B_k) から今いるマスのベストな行き先に $(x - 1)$ 回移動したときのマスを高速に求めたい。
- 事前にダブリングしておくことで、 $O(\log(HW))$ で求められる。
 - (a, b) からベストの行き先に x 回移動したときのマスを、 $x = 1, 2, 4, \dots, 2^t$ に対して求めておく。

再掲

Q. マス (a, b) から 1 回のハイジャンプで行けるマス (i, j) は？

A. 書かれている数が $T_{a,b} + L$ 以下の隣り合うマスの組を全て辺で結んだときに、 (a, b) と連結であるもの

クエリ処理

- 前計算と同じように、Union Find を用いると、全てのクエリに対する判定問題を $O(HW\alpha(HW))$ で答えられる。
- 並列二分探索すると、全体で $O((Q + HW \log(HW)) \log(HW))$ で求められる。
- x 回ベストな行き先に行くパートを工夫すれば時間計算量は $O((Q + HW \log(HW))\alpha(HW))$ になる。

小課題 5 : $H \times W \leq 300000, Q \leq 300000$

- 小課題 4 の方法でも定数倍によっては満点が取れます
- 前述したものより、クエリ処理の部分の計算量が良い解を紹介します。

クエリ処理

- 今まで同じ Union Find のマージを $O(\log(HW))$ 回してきた
 - 流石に無駄じゃない...?
- Union Find のマージ過程の木を考える。

クエリ処理

(A_k, B_k) と (C_k, D_k) がマージされたタイミングで見ていたマスに書かれていた値を X とする。

- これは Union Find のマージ過程を表す木の LCA を求める問題に帰着できて、適切な前計算の元でクエリごとに $O(\log(HW))$ で求められる。

以下の判定問題をもう一度考える。

- (A_k, B_k) から今いるマスのベストな行き先に $(x - 1)$ 回移動したときのマスを (a, b) としたときに、 (a, b) から 1 回のハイジャンプで (C_k, D_k) 辿り着けるか。

クエリ処理

- (A_k, B_k) から今いるマスのベストな行き先に $(x - 1)$ 回移動したときのマスを (a, b) としたときに、 (a, b) から 1 回のハイジャンプで (C_k, D_k) 辿り着けるか。

この判定問題は以下のように解ける。

- $X \leq T_{a,b} + L$ ならば可能、そうでないなら不可能。

よって、判定問題を $O(1)$ で解けるため、二分探索することでクエリごとに $O(\log(HW))$ で答えが求まる。

統計情報

点数	人数	累計人数
100	13	13
80	3	16
50	1	17
30	1	18
10	5	23
0	3	26



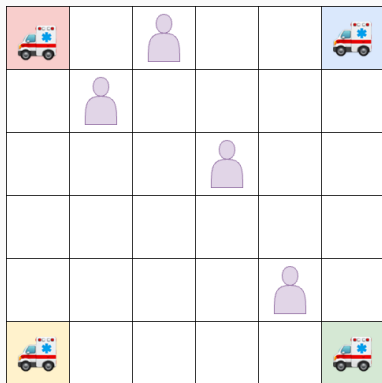
救急車 (Ambulance) 解説

林涼太郎 (Forested)

JOI 競技 2 日目 / JOIG 競技 1 日目

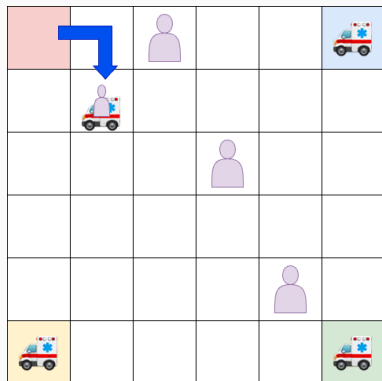
問題概要

問題概要



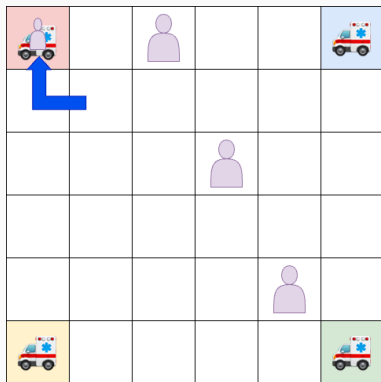
- ・ $L \times L$ のグリッドがあり、四隅に病院がある
- ・ 各病院には専属の救急車がある
- ・ N 人の人がグリッド上にいる

問題概要



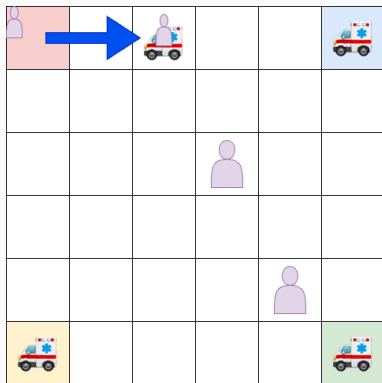
- ・ 救急車は 1 単位時間あたり 1 マス動ける
- ・ 各救急車には 1 人まで乗せられる
- ・ 専属の病院でのみ人を下ろせる

問題概要



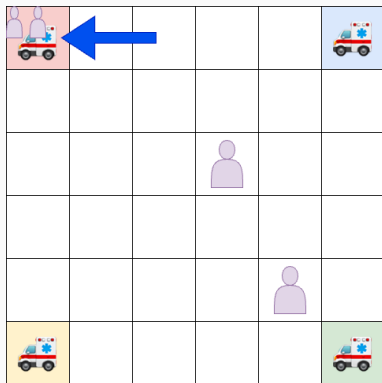
- ・ 救急車は 1 単位時間あたり 1 マス動ける
- ・ 各救急車には 1 人まで乗せられる
- ・ 専属の病院でのみ人を下ろせる

問題概要



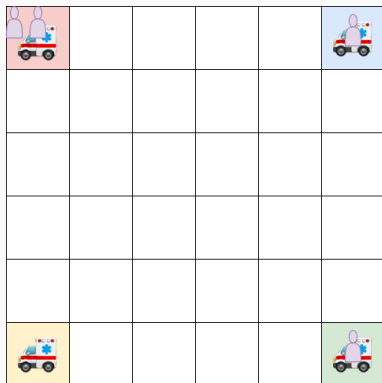
- ・ 救急車は 1 単位時間あたり 1 マス動ける
- ・ 各救急車には 1 人まで乗せられる
- ・ 専属の病院でのみ人を下ろせる

問題概要



- ・ 救急車は 1 単位時間あたり 1 マス動ける
- ・ 各救急車には 1 人まで乗せられる
- ・ 専属の病院でのみ人を下ろせる

問題概要



- ・ 4 台の救急車をうまく動かして、 T 単位時間以内に人を全て病院に移動させられるか？

制約

- $L \leq 10\,000$
- $N \leq 160$
- $T \leq 20\,000$

小課題 1, 2: T が小さい

小課題 3, 4, 5, 6: N が小さい (5 は追加制約)

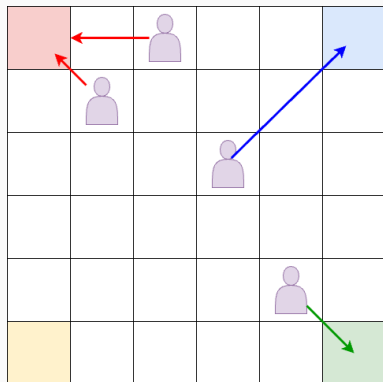
小課題 7: 満点

小課題 1 ($T \leq 50$)

救急車の動きを考える

- ・ ある人のもとへ向かう→病院に連れて帰る、の繰り返し

考察



それぞれの人について、4つの病院のうち1つを割り当てる
各病院について、往復の距離の和が T 以下になれば OK

解法

1					2
					
					
					
4					3

便宜上このように病院に番号を振っておく

DP をする

$DP[i][w][x][y][z]$: 人 $1, 2, \dots, i$ について病院の割り当てを決めて、現時点の病院 $1, 2, 3, 4$ の距離の総和がそれぞれ w, x, y, z になることはあるか？ (True / False)

状態 $\Theta(NT^4)$ 、遷移 $\Theta(1)$

$N = 160, T = 50$ のとき $NT^4 = 10^9$ で、やや厳しい

往復の距離の和が T 以下 $\iff$ 片道の距離の和が $T/2$ 以下だから、
 $T' := \lfloor T/2 \rfloor$ として、片道の距離の和を T' 以下にできるかを判定する
ように DP すると、大幅に高速化できる

- ・ 後の小課題でも使える、必須レベルの定数倍高速化

小課題 2 ($T \leq 160$)

小課題 1 の DP を速くする

よく言われていること：bool 値を持つ DP は無駄が多い

$DP[i][w][x][y]$: 人 $1, 2, \dots, i$ について病院の割り当てを決めて、現時点の病院 $1, 2, 3$ の距離の総和がそれぞれ w, x, y になったときの、病院 4 の距離の総和の最小値（値は整数）

状態 $\Theta(NT^3)$ 、遷移 $\Theta(1)$

小課題 3 ($N \leq 10$)

今度は N が小さい $\rightarrow$ 割り当て 4^N 通りを全て試せる

各割り当てについて条件のチェックは $\Theta(N)$ 時間でできるので、全体で $\Theta(4^N N)$

小課題 1, 2 より先に思いつく人もいるかも

小課題 4 ($N \leq 20$)

「人 $1, 2, \dots, N$ を 4 つの病院に割り当てる」ということを言い換える

→ 「人 $1, 2, \dots, N$ を好きな順番で並ばせて、さらに仕切りを 3 つ好きな位置に追加する」で同じことができる

解法 1

- ・ 人 $1, 2, \dots, N$ を適当な順番で並ばせる
- ・ 前から順に病院 1 との距離を計算していく
- ・ 合計が T' を超える直前で総和を 0 にリセットし、次からは病院 2 との距離を計算していく
- ・ リセット 3 回以内で済めば割り当て成功

これを bit DP にする

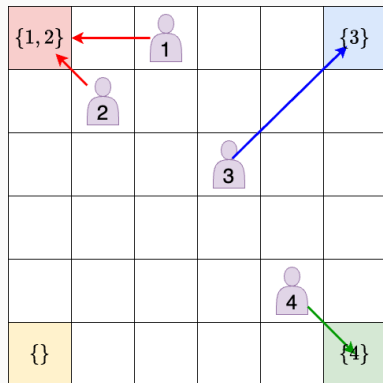
解法 1

列を前から決めていくことにする

$DP[i][S]$: これまでのリセット回数が i 回で、すでに列に並んだ人の集合が S のときの現在の総和の最小値

状態 $\Theta(2^N)$ で遷移 $\Theta(N)$ だから、全体で $\Theta(2^N N)$ 時間

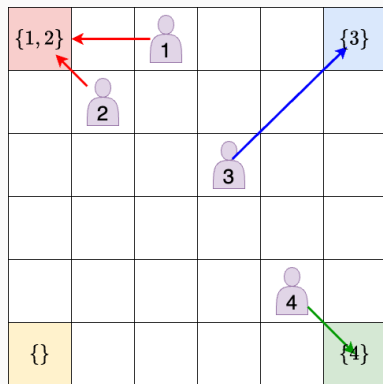
解法 2



4^N 通り全ては試せないので、高速化を考える

各病院について、担当する人の集合を $\{1, 2, \dots, N\}$ の部分集合から選んで定めると考える

解法 2



(病院, 人の集合) 全てに対して、距離の総和が T' 以下であるか確認：これは $\Theta(2^N N)$ 時間

病院 1, 2, 3, 4 に対応する集合 S_1, S_2, S_3, S_4 を選んで、 $S_1 \cup S_2 \cup S_3 \cup S_4 = \{1, 2, \dots, N\}$ となるようにしたい

Bitwise OR Convolution

列 $a = (a_0, a_1, \dots, a_{2^N-1})$ と $b = (b_0, b_1, \dots, b_{2^N-1})$ について、

$$c_k = \sum_{i \text{ OR } j = k} a_i b_j$$

で定まる $c = (c_0, c_1, \dots, c_{2^N-1})$ を計算することが実は $\Theta(2^N N)$ 時間
できる

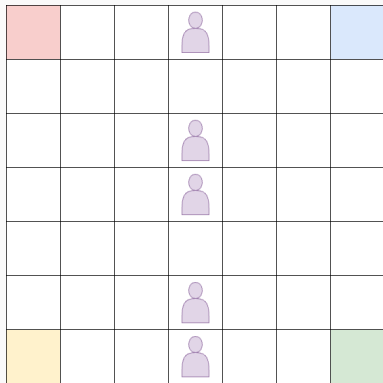
- ・ OR は bit ごとの論理和
- ・ 詳細は省略

これを 3 回行えば $S_1 \cup S_2 \cup S_3 \cup S_4 = \{1, 2, \dots, N\}$ となるような
 S_1, S_2, S_3, S_4 が取れるかわかる

全体で $\Theta(2^N N)$ 時間

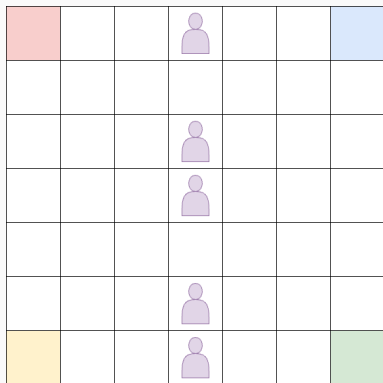
小課題 5 ($N \leq 45$, L は奇数,
 $Y_k = (L + 1)/2$)

制約



縦の中心の軸上に全ての人が乗っている

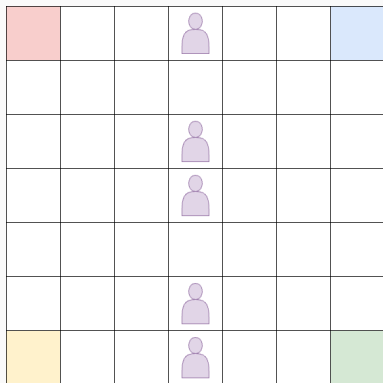
制約



縦の中心の軸上に全ての人が乗っている

病院 1 に割り当てるのと 2 に割り当てるので距離が変わらない！（3 と 4 でも同様）

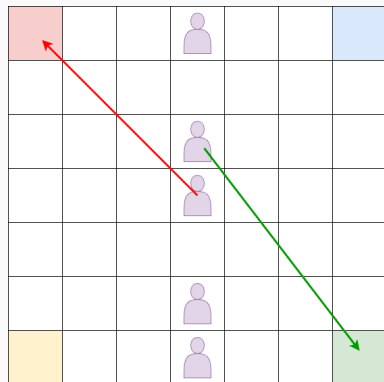
制約



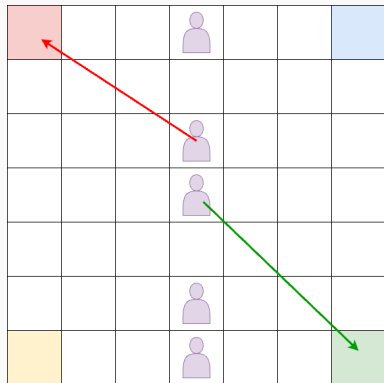
縦の中心の軸上に全ての人が乗っている

病院 1 に割り当てると 2 に割り当てるので距離が変わらない！（3 と 4 でも同様）

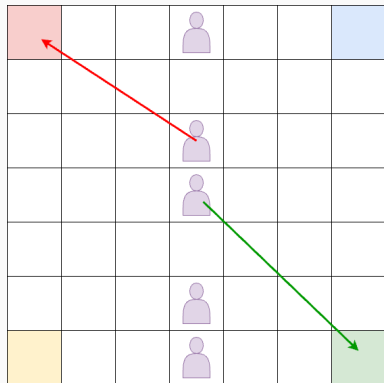
上と下のどちらに割り当てることが大事



こういう割り当てがあったとき、

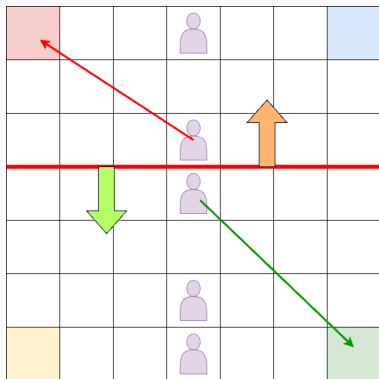


こうしてもどの病院も損しない



こうしてもどの病院も損しない

→ X_k が小さい人は上 2 つの病院に、大きい人は下 2 つの病院に割り当てるとして良い



ちゃんと言うと、 N 人に X の昇順に番号を付け直したとき

- ・ ある k が存在して、人 $1, 2, \dots, k$ は病院 1, 2 に、人 $k + 1, k + 2, \dots, N$ は病院 3, 4 に割り当てる

という形の割り当てだけ考えれば良い

k を 0 から N まで全て試す

それぞれの k について部分和問題を解く

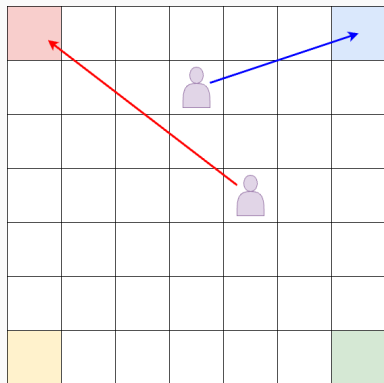
- ・ 人が k 人 / $N - k$ 人いて総和を T' 以下にするから $\Theta(NT)$ 時間

全体で $\Theta(N^2T)$ 時間

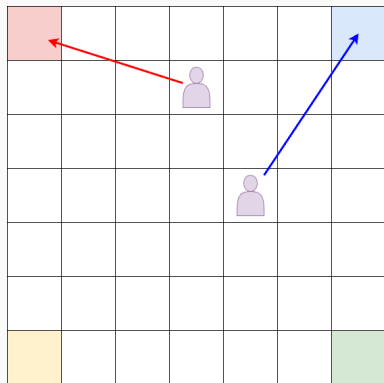
小課題 6 ($N \leq 45$)

小課題 5 がヒントっぽい感じがする

一般の場合でもうまく配置を絞り込めないか？

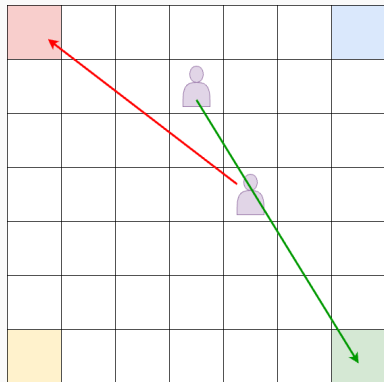


病院 1,2 についてうまく swap できないか？

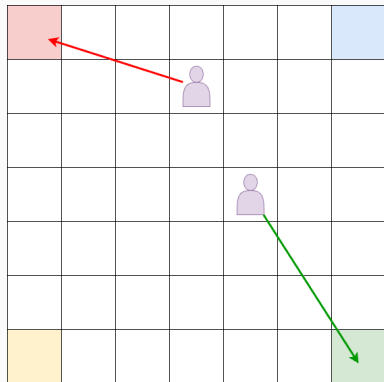


病院 1, 2 についてうまく swap できないか？

→病院 1 は得して 2 は損してしまった、失敗



実は病院 1, 3 についてならうまくいく

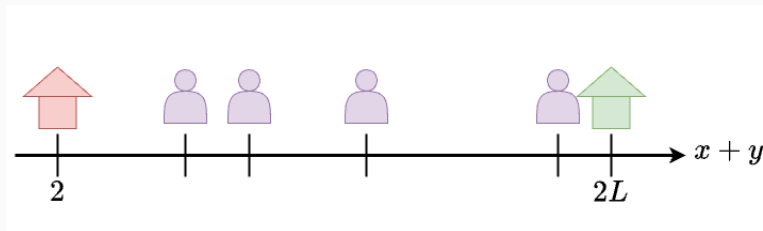


実は病院 1, 3 についてならうまくいく

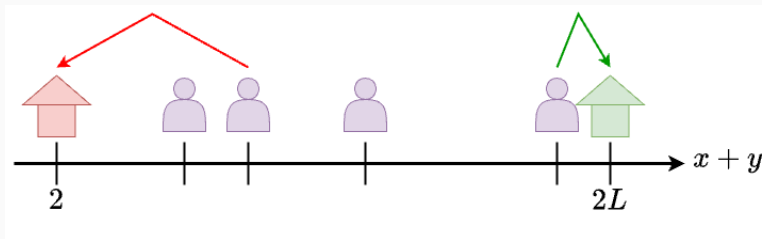
(x, y) にいる人について、

- ・ 病院 1 との距離： $(x + y) - 2$
- ・ 病院 3 との距離： $2L - (x + y)$

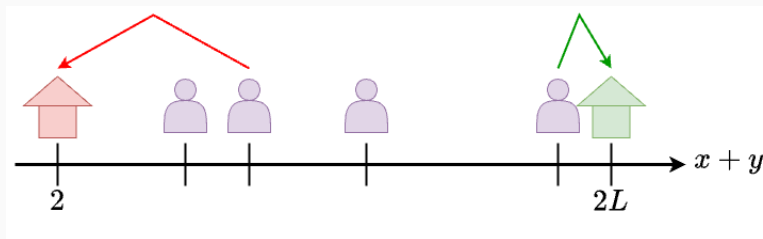
$x + y$ と 2 または $2L$ との差になっている！ →一次元で考えられそう



人 i について $X_i + Y_i$ の値を数直線に乗せてみる



左の人は病院 1 に割り当てたい、右の人は病院 3 に割り当てたい

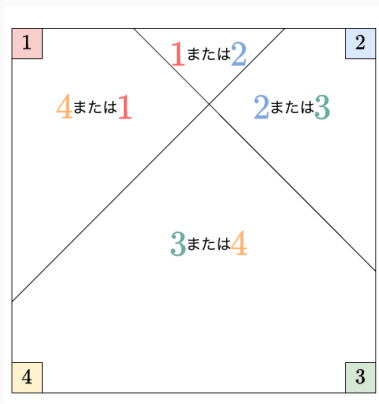


ちゃんという、ある整数 k が存在して、この数直線について

- ・ 左 k 人以外病院 1 を使わない
- ・ 右 $N - k$ 人以外病院 3 を使わない

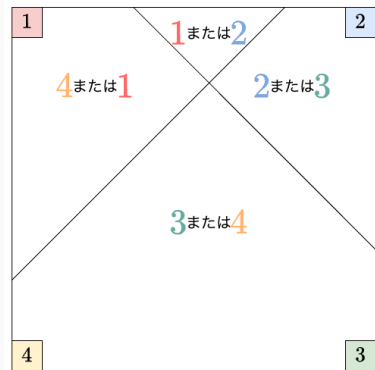
という形の割り当てのみ考えればよい

病院 2 と 4 についても、 $X_i - Y_i$ の数直線を考えると同じことが言えるので…？



こうなる

解法

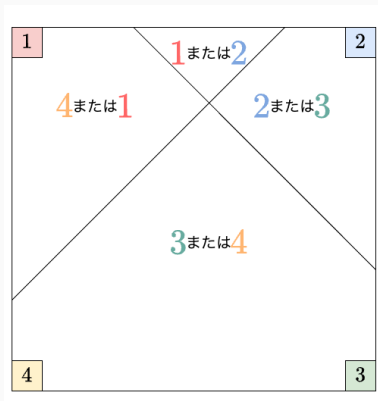


領域の区切り方 $(N + 1)^2$ 通りをとりあえず全て試す

各領域について、片方の病院の負担を減らすと、もう片方の病院の負担が増えるかもしれない

→ DP しよう！

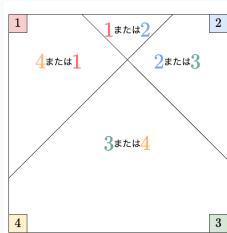
解法



DP[i]: 「 x または y 」の領域内において、病院 x の距離の総和を i 以下にするときの、病院 y の距離の総和の最小値

これを $(x, y) = (1, 2), (2, 3), (3, 4), (4, 1)$ について計算 ($\Theta(NT)$ 時間)

解法



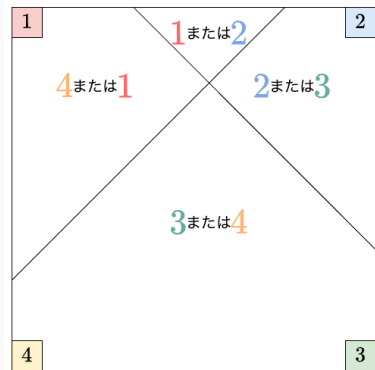
1. 「1 または 2」の領域における病院 1 の距離の総和を決め打つ
2. 「1 または 2」の領域における病院 2 の距離の総和がわかる
3. 「2 または 3」の領域における病院 2 の距離の総和がわかる
4. 「2 または 3」の領域における病院 3 の距離の総和がわかる
5. ...

最後までやって、整合性を確認すればよい ($\Theta(T)$ 時間)

$\Theta(NT) + \Theta(T)$ 時間が $(N + 1)^2$ 回あるので、全体で $\Theta(N^3T)$ 時間

小課題 7 (追加制約なし)

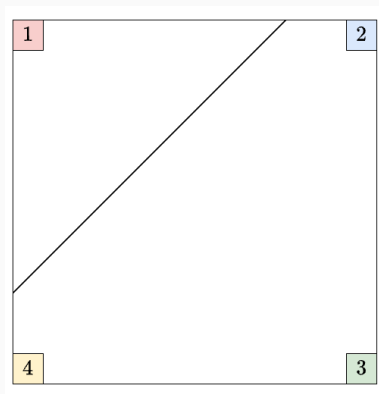
復習



小課題 6 の解法を復習

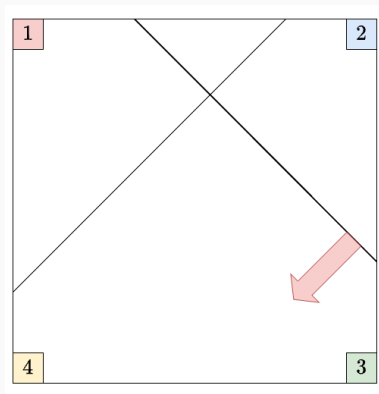
- ・ DP テーブルの計算： $\Theta(N^3T)$ 時間
- ・ 決め打って整合性確認： $\Theta(N^2T)$ 時間

DP テーブルを $\Theta(N^2T)$ 時間で計算できればよさそう



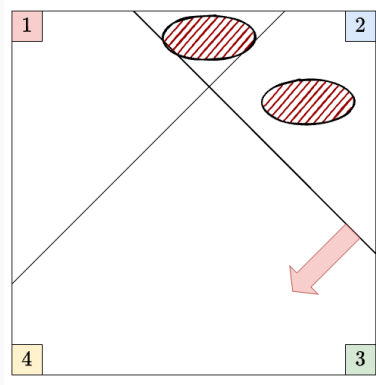
とりあえず $X_i + Y_i$ の切れ目を固定

解法



$X_i - Y_i$ の切れ目をこの向きに動かしてみる

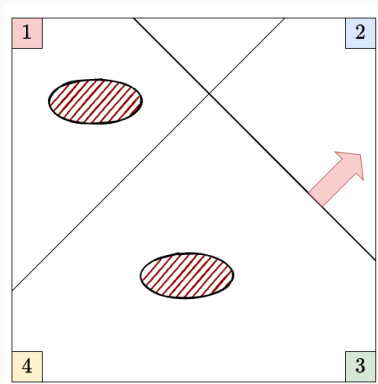
解法



$X_i - Y_i$ の切れ目をこの向きに動かしてみる

この部分の DP テーブルは切れ目を 1 人分動かすたびに $\Theta(T)$ 時間で更新できる！

解法



逆向きにもやれば残り 2 つの DP テーブルも計算できる

全ての DP テーブルが合わせて $\Theta(N^2T)$ 時間で計算できた！

TL に余裕はないかも

- ・ 使い回せる配列を使い回す
- ・ in-place に更新できる部分はそうする

などで高速化しましょう

A large school of small, silvery fish swimming in clear blue water. The fish are densely packed in some areas and more spread out in others, creating a dynamic and lively scene. The water is a vibrant blue, and the lighting suggests a bright, sunny day.

Stamps 4

Editorial : blackyuki

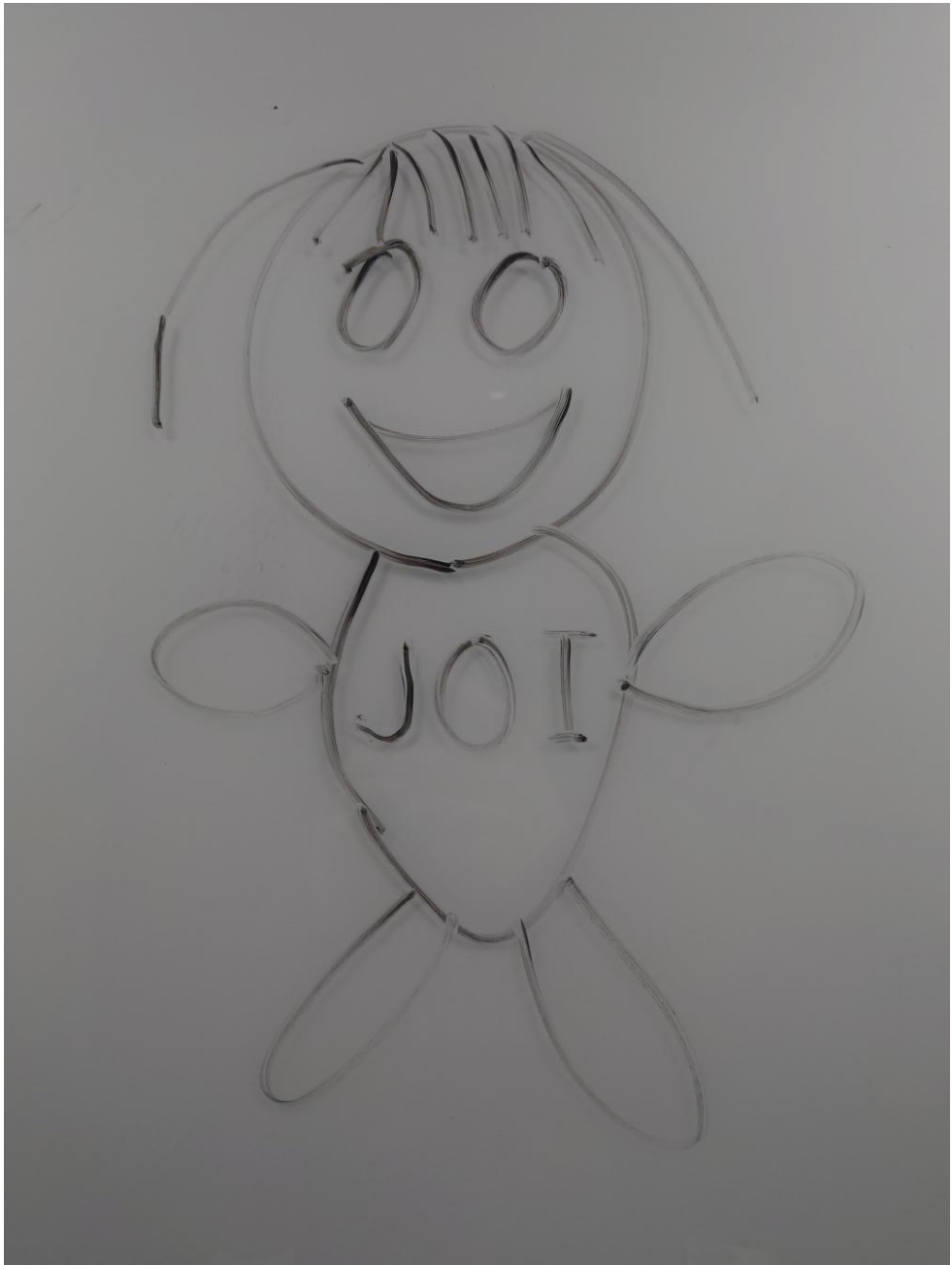
問題概要

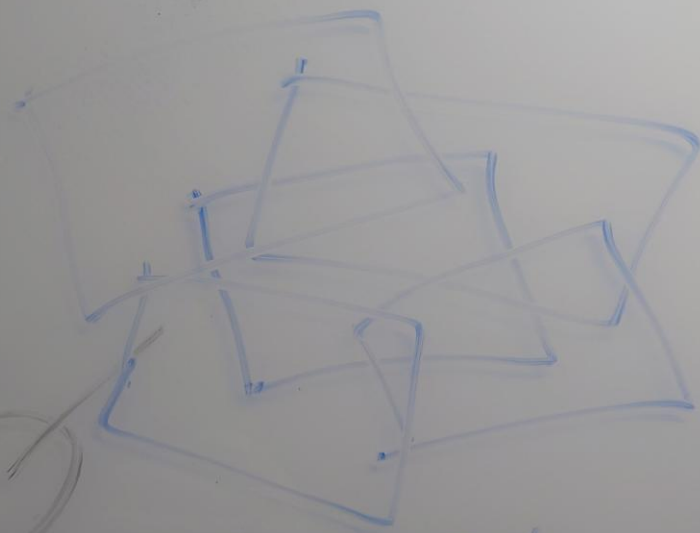
JOI 君がたくさんのスタンプカードを持ってスタンプラリーに参加する



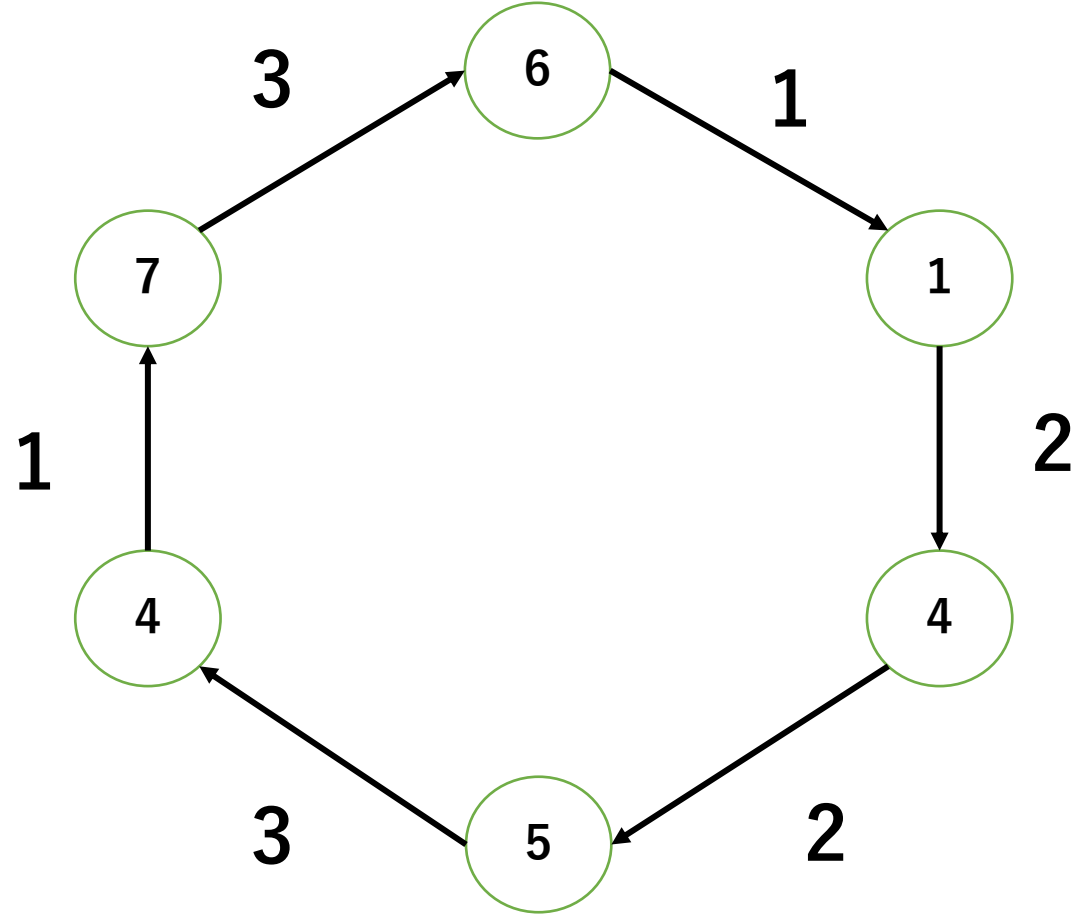
勇者ビ太郎

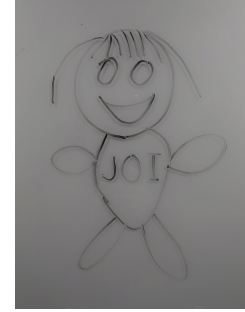
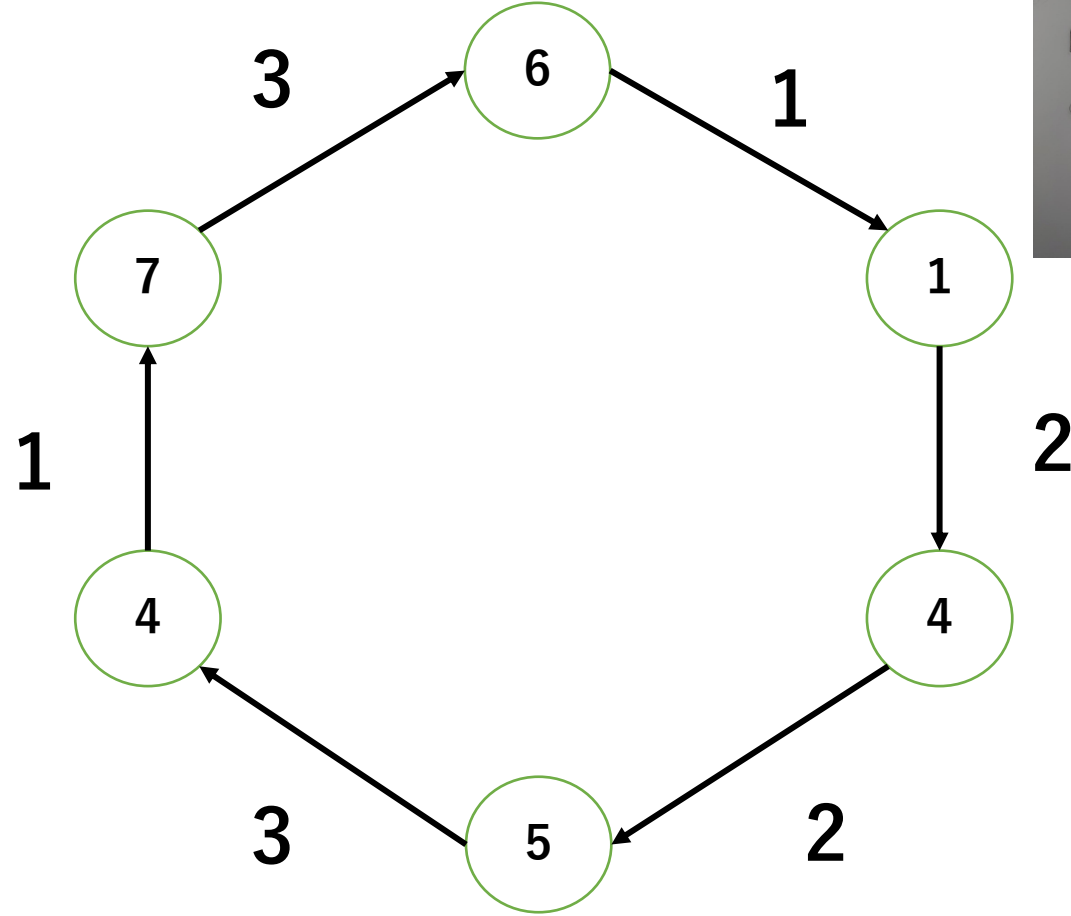
絵: yuto1115

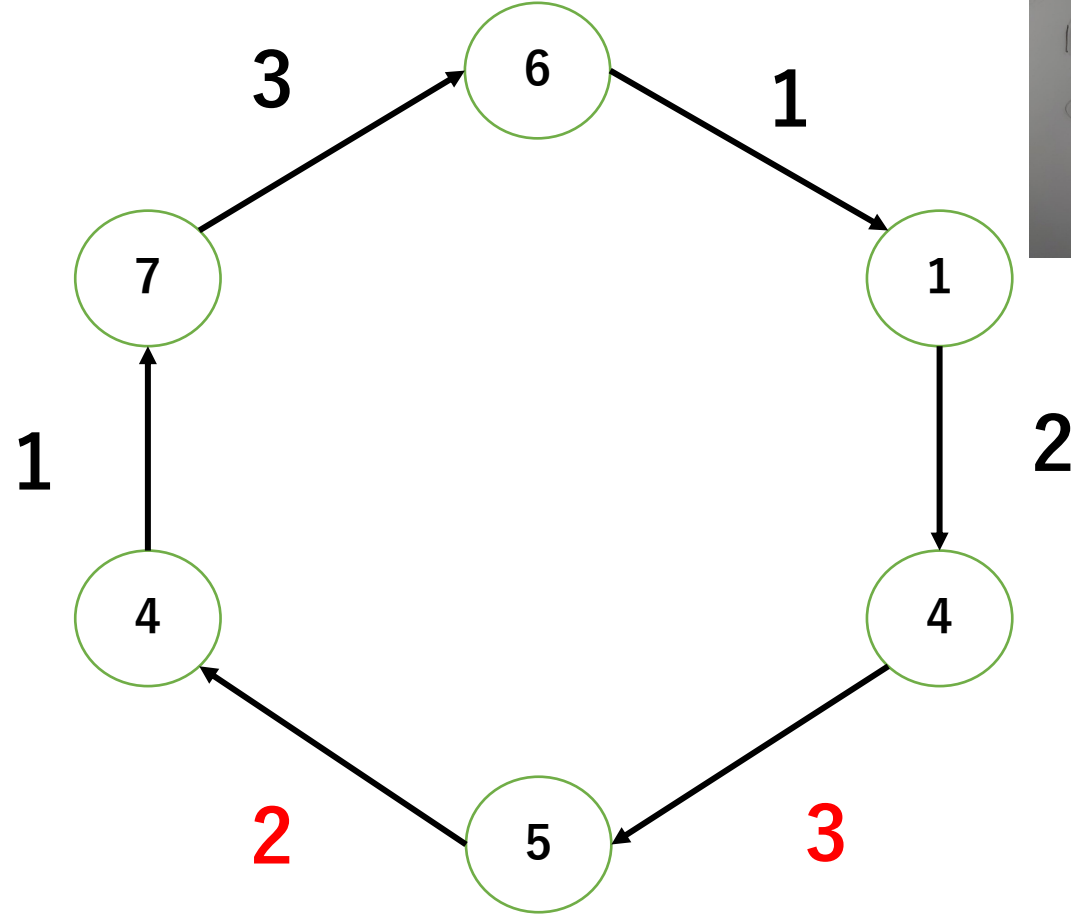


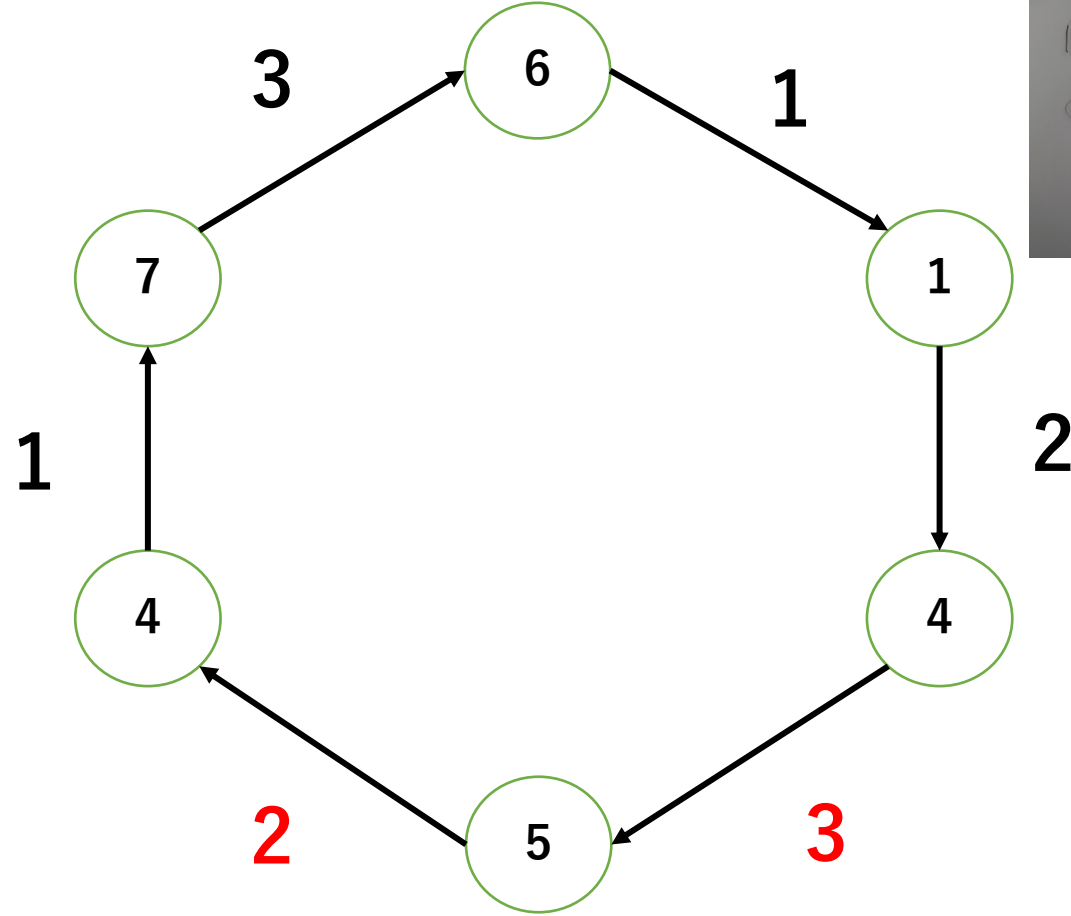


スナップカード

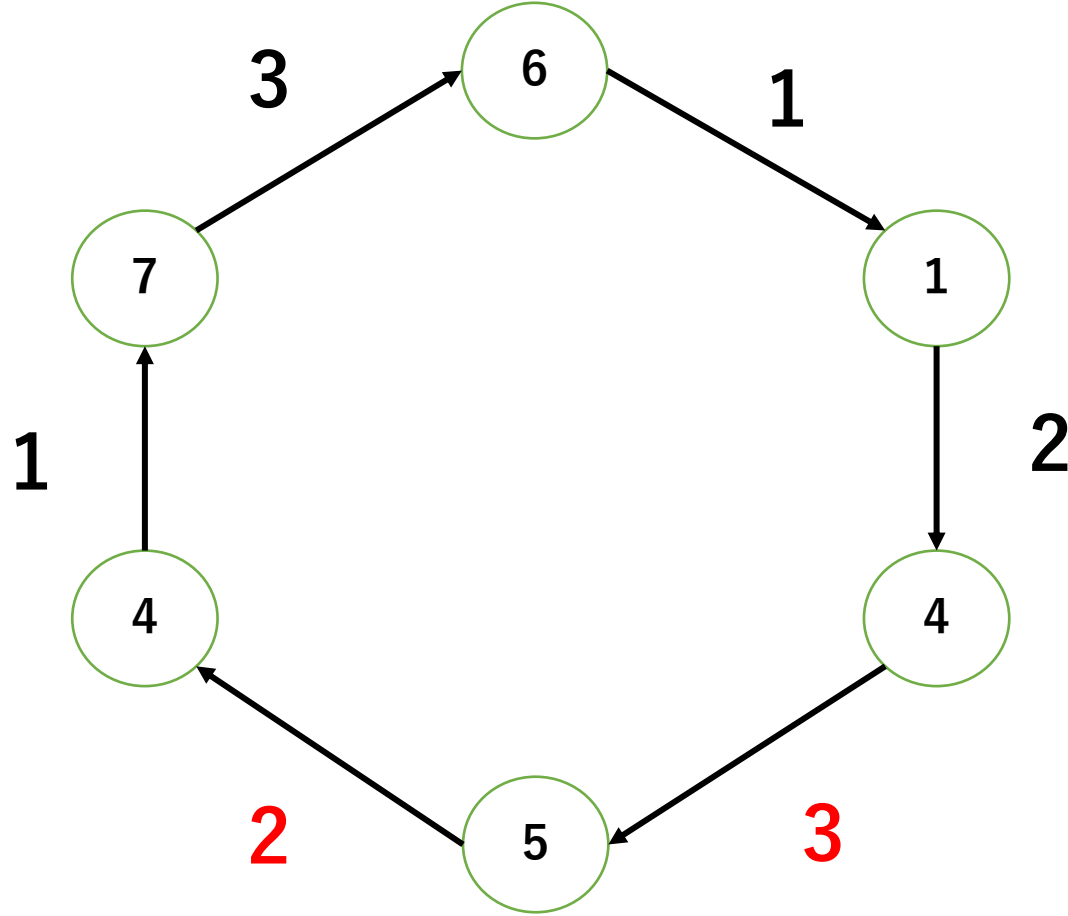


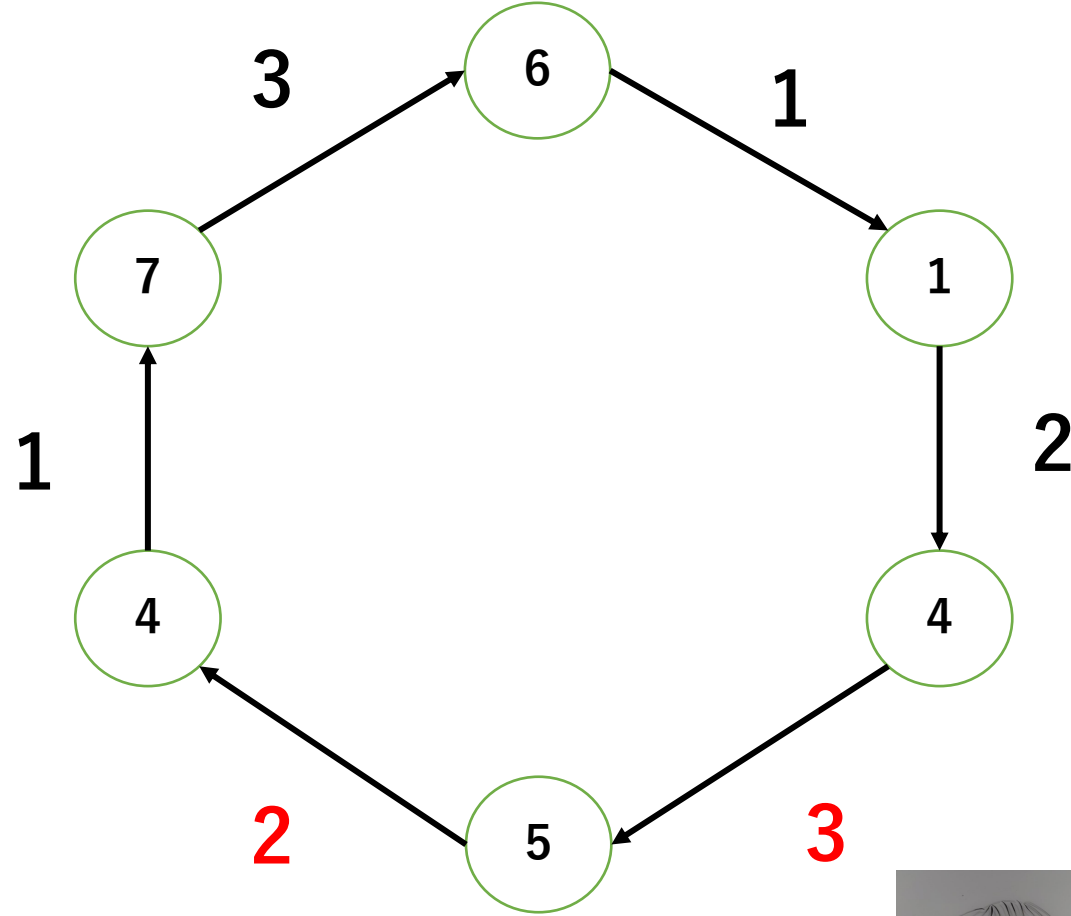


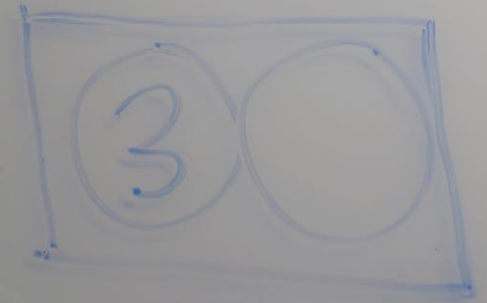
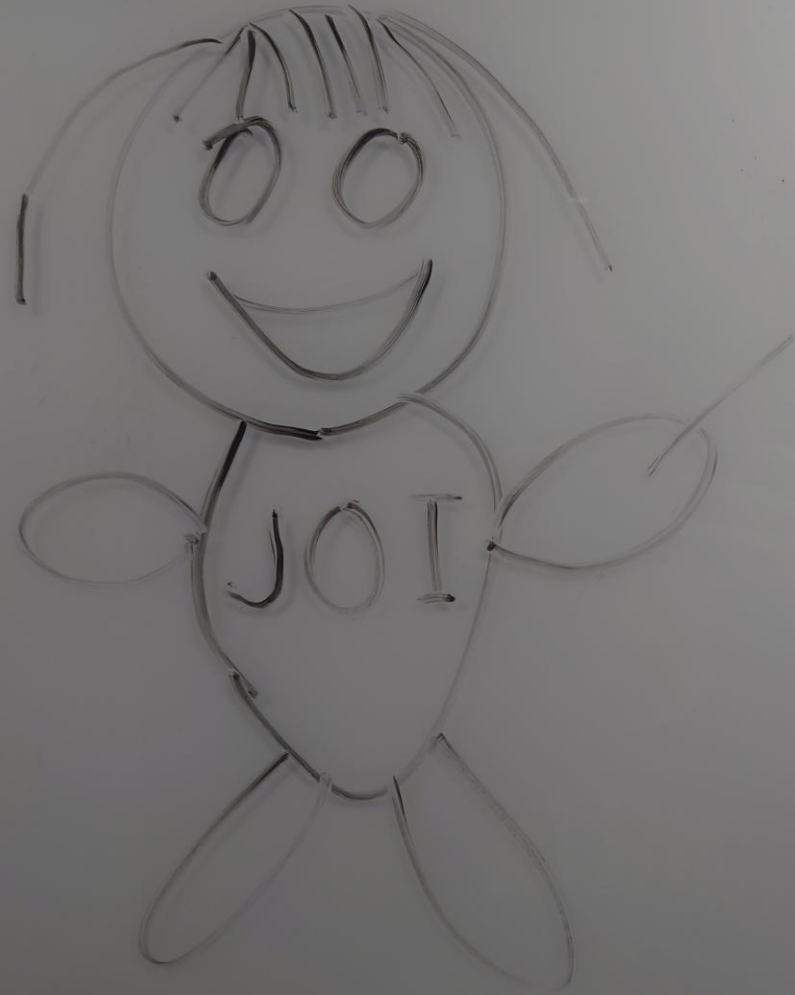


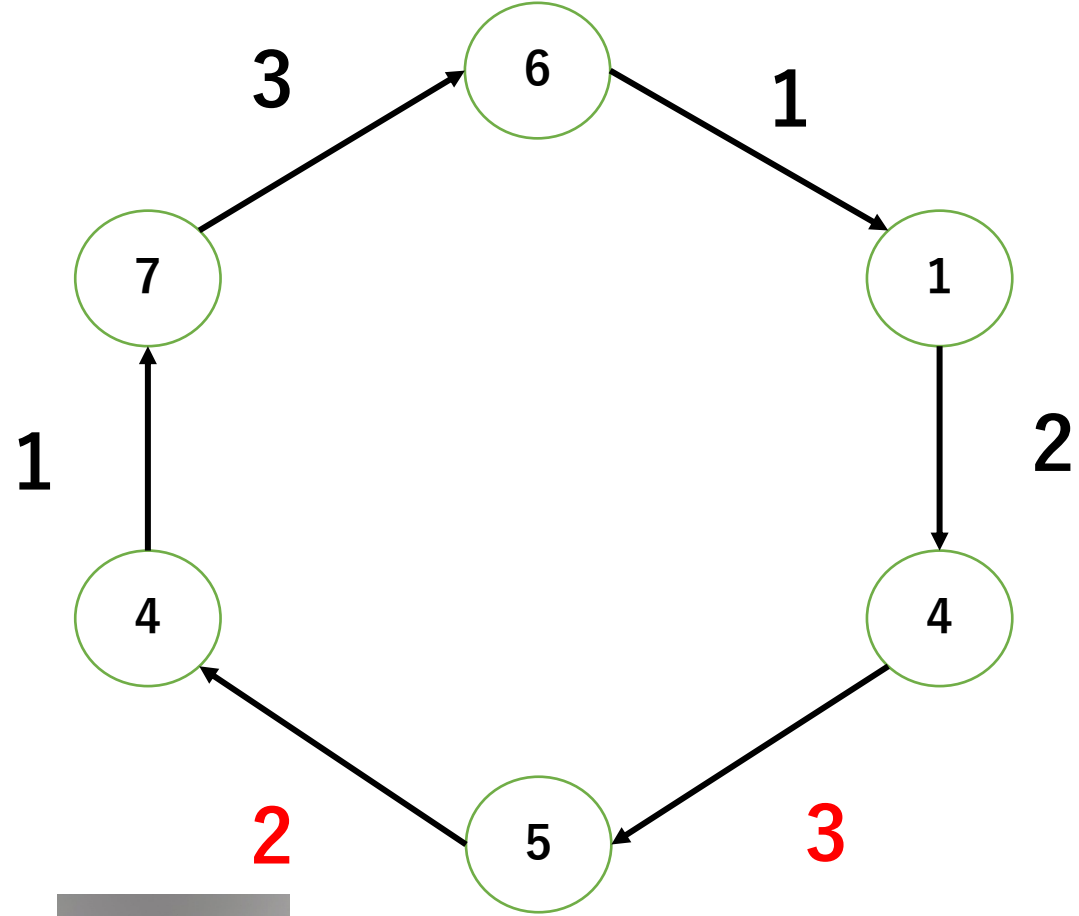


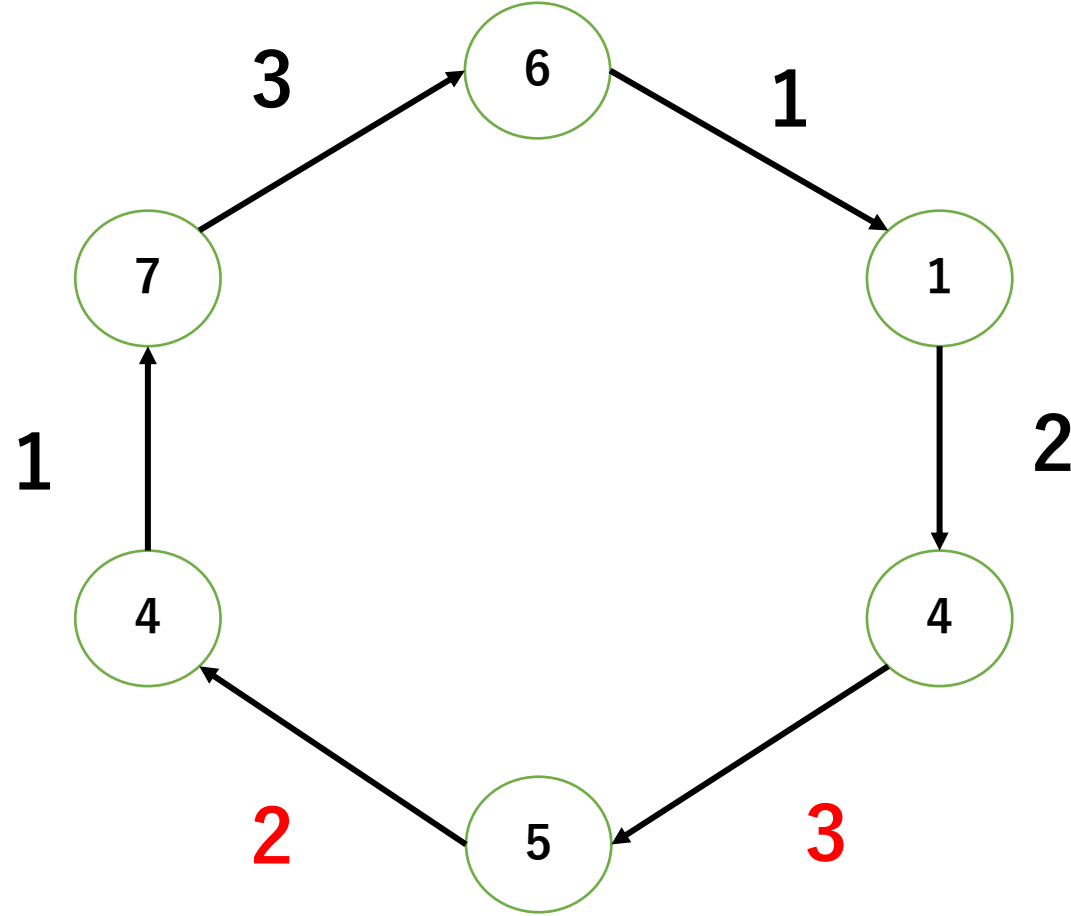
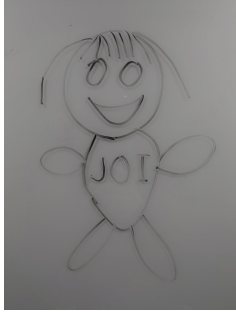
欲しいカード : (3, 1)

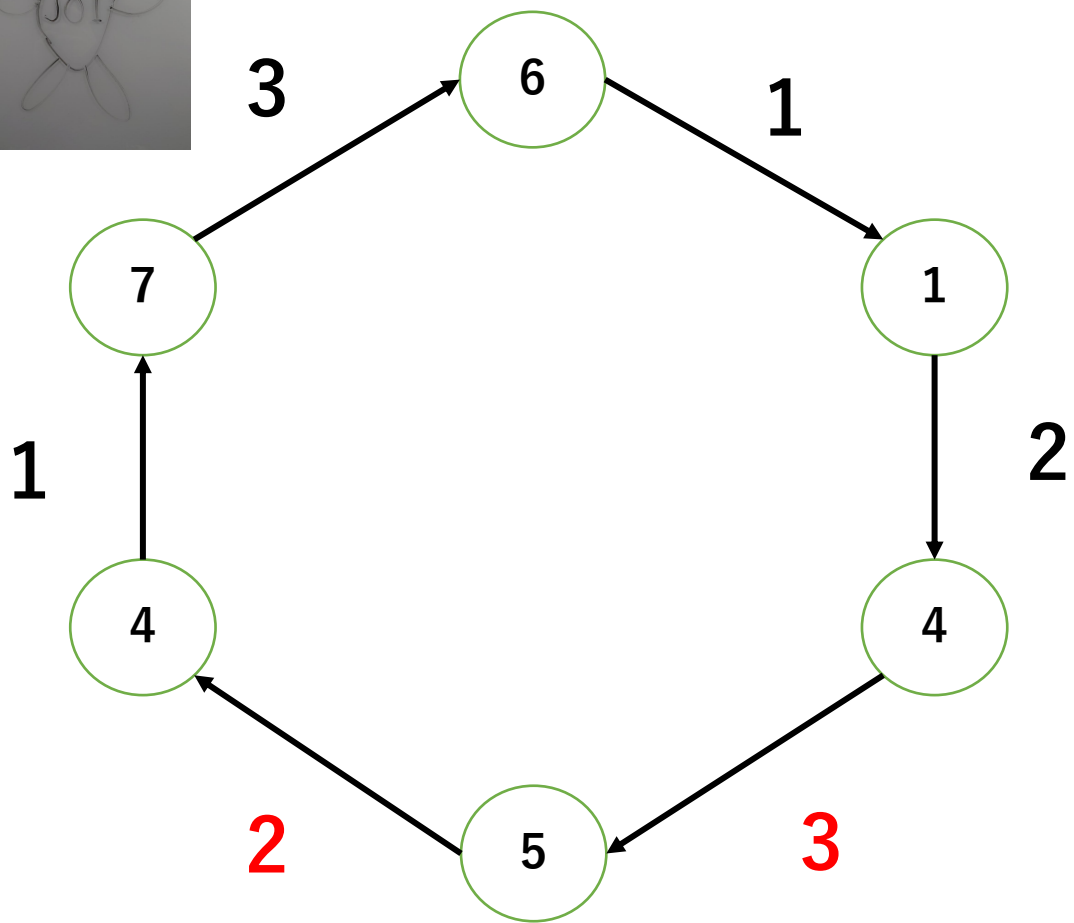
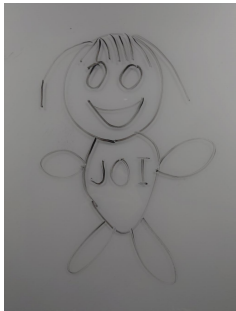


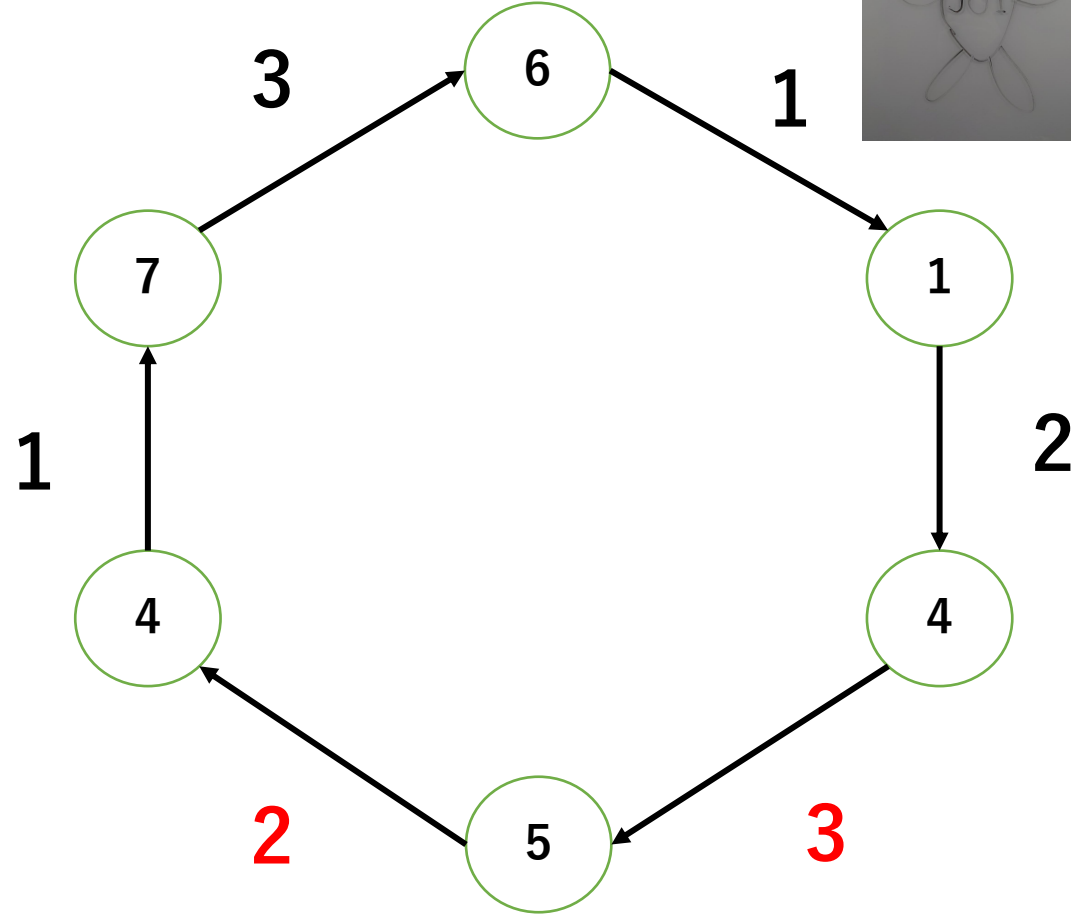
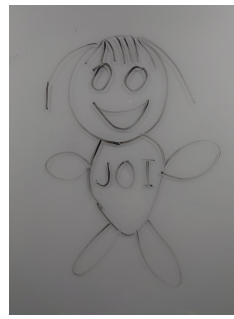


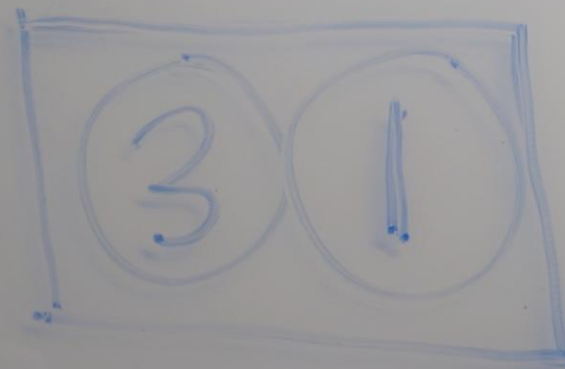
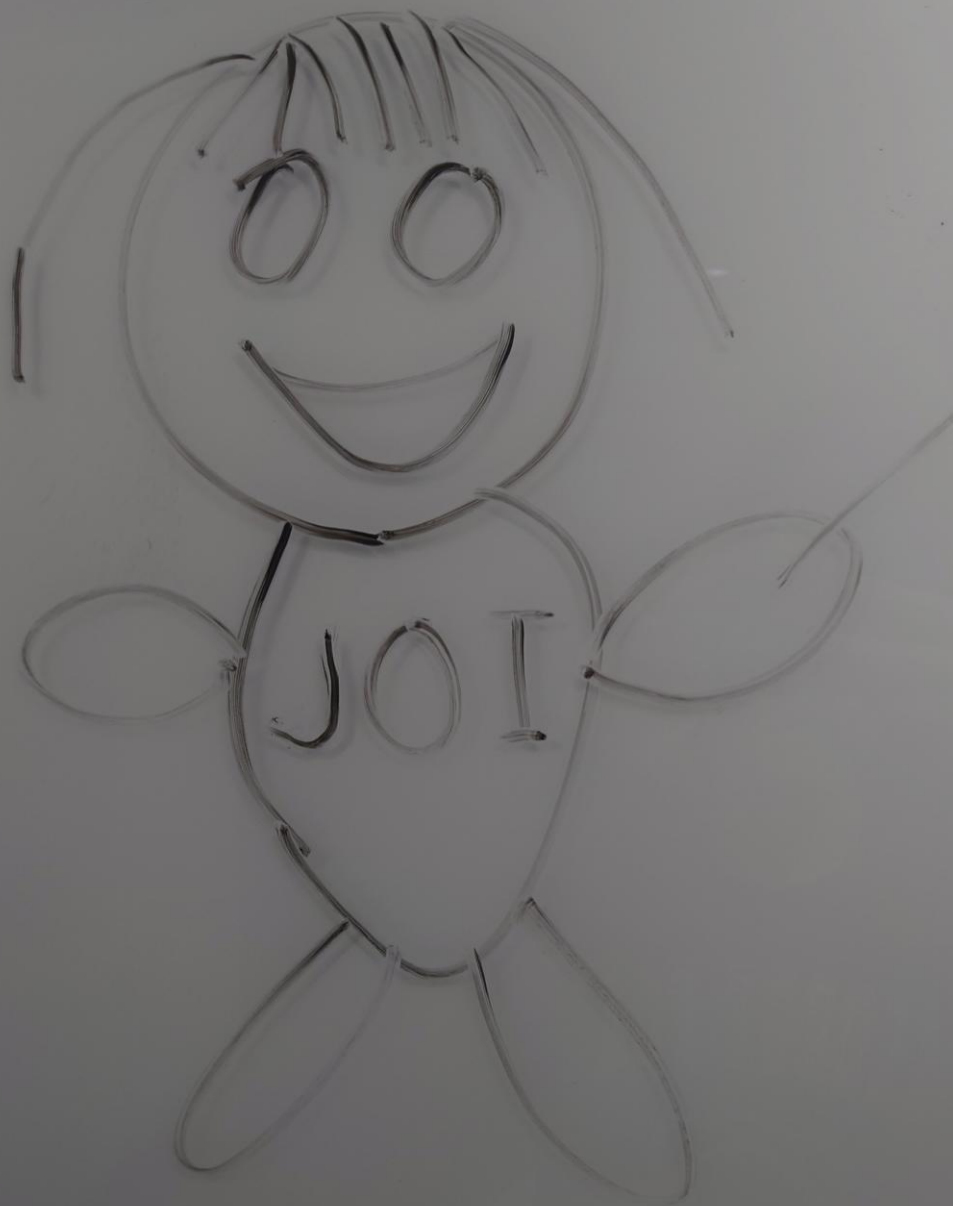


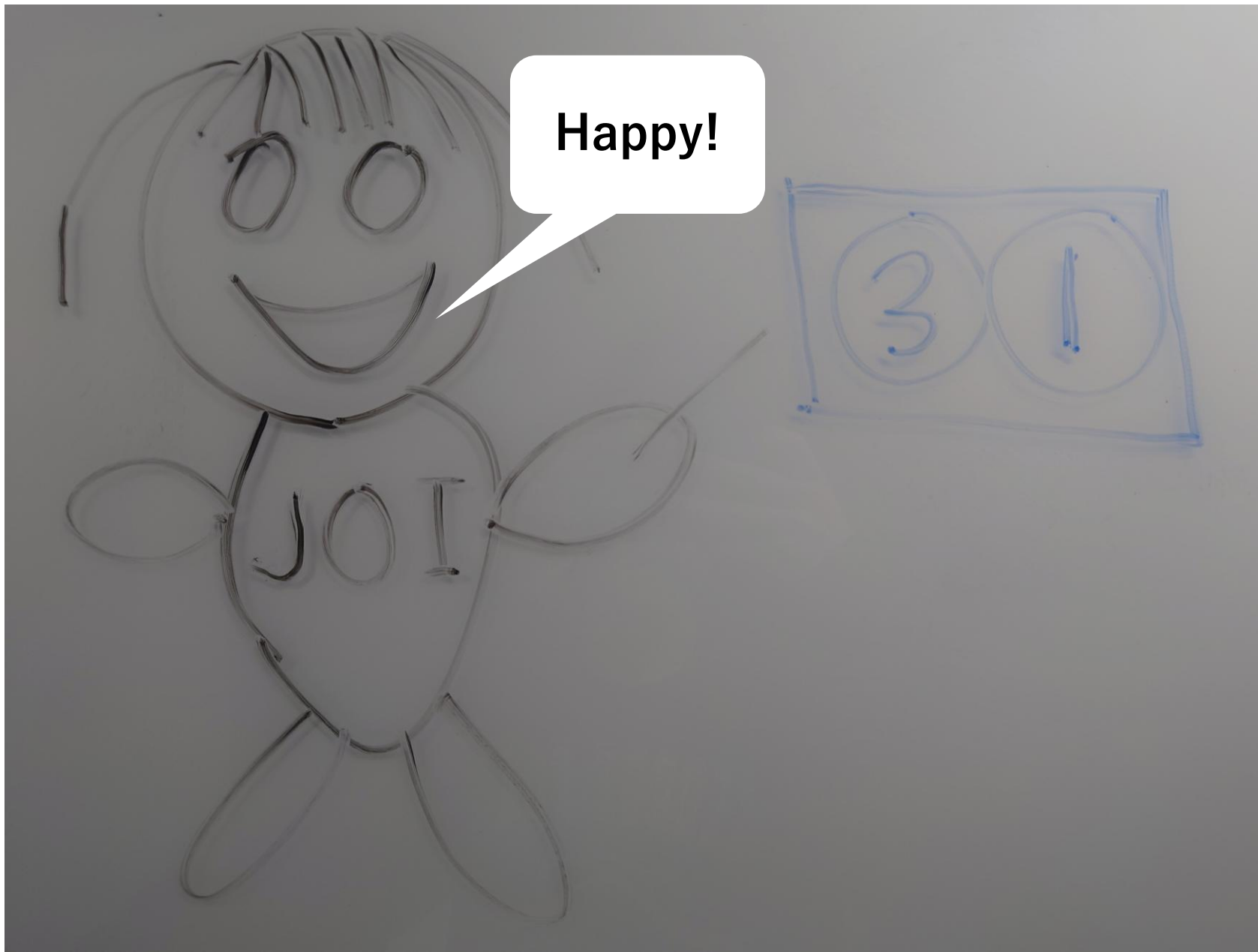












問題概要

- $(1,1,2,2,\dots,N,N)$ の並び替え A が与えられる。
- A を何度か cyclic shift させる。
- 次に A の隣接する 2 要素を何度かスワップする。
- A の長さ 2 の部分列の種類数を K 以上にするのに必要なコストは？
- Q 個のクエリに答えよ（クエリごとに K が与えられる）

小課題 1

$N \leq 4$

全探索！

小課題 2

$N \leq 5000$, $Q=1$, $K=N^2$

気づき

- cyclic shift & swap 後の文字列を考える
- スタンプカード (a,b) と (b,a) が
両方得られるのはどんな時？
- ‘abab’, ‘abba’, ‘baab’, ‘baba’ の順に登場するとき
- つまり、 N^2 通り全ての部分列が得られるような
文字列というのは…？

気づき

$[(1, 2, \dots, N) \text{ の並び替え}]$ $[(1, 2, \dots, N) \text{ の並び替え}]$

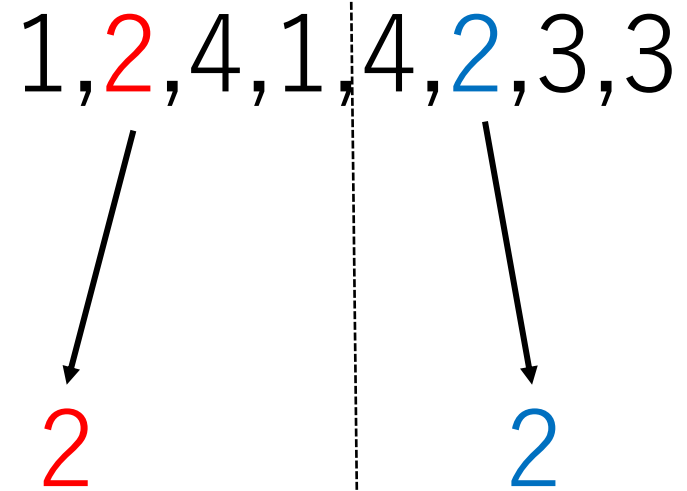
気づき

- ・ スワップの最小回数は？

1,2,4,1,4,2,3,3

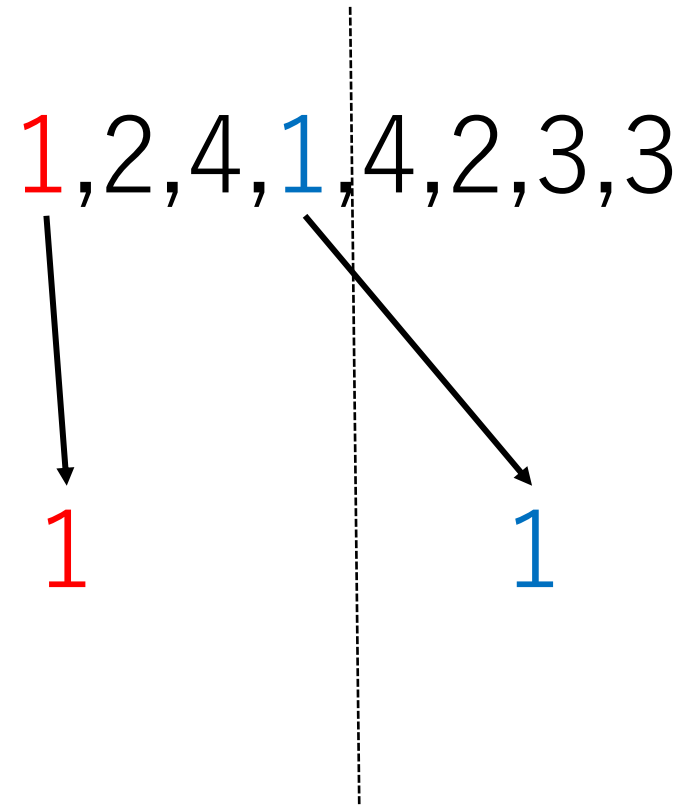
気づき

- ・ スワップの最小回数は？



気づき

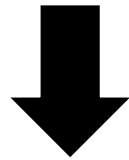
- ・ スワップの最小回数は？



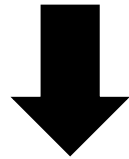
気づき

- 各文字について、最初の登場をX、最後の登場をYとする

1,2,4,1,4,2,3,3



XXXXYYXY



XXXXXYYYY

01 転倒数！

小課題 2

- ・ 円環の始点を決めると $O(N)$ で解ける

→ 全体 $O(N^2)$ で小課題 2 AC

小課題 3

$N \leq 5000, Q = 1$

小課題 3

- ・ 先ほど同様、円環の始点を決め打つ

1,2,4,1,4,2,3,3

- ・ 集めれないカードはどれ？

(1,4),(4,1) -> '1414'なので両方 OK

(1,3),(3,1) -> '1133'なので片方 NG

小課題 3

- いけるパターン：XXYY
- ダメなパターン：XYXY

小課題 3

つまり・・・

部分列の種類数は「 $N^2 - (\text{XY列の転倒数})$ 」である

1 回のスワップでXY列の転倒数をちょうど 1 だけ減らせる

必要なスワップ回数は「 $\max(0, K - (N^2 - (\text{XY列の転倒数})))$ 」

小課題 3

- 前処理 $O(N^2)$ + クエリあたり $O(N)$

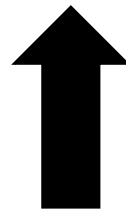
→ 全体 $O(N^2 + QN)$ で小課題 3 AC

小課題 4

$N \leq 5000$

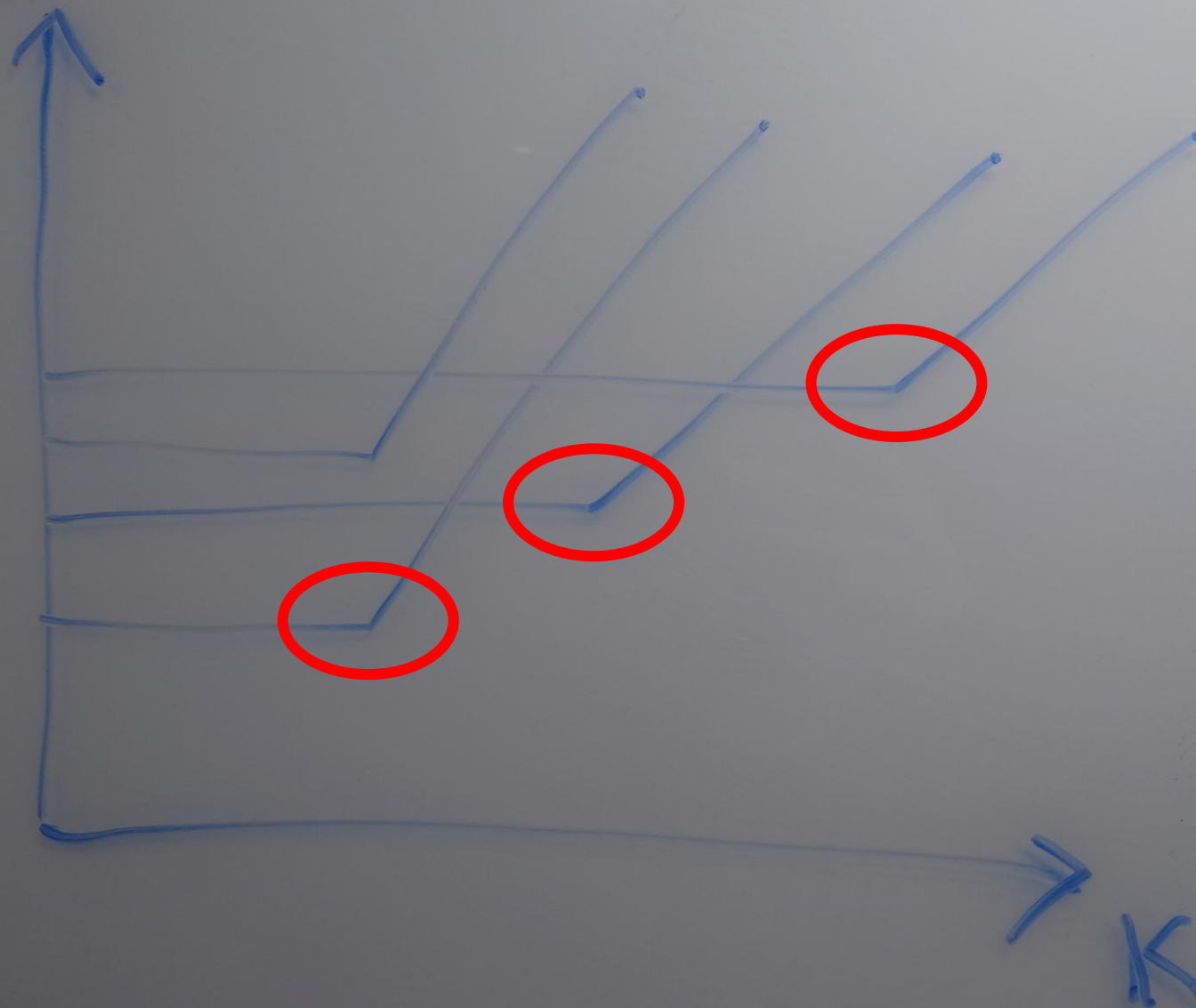
小課題 3 (再掲)

- ・ 前処理 $O(N^2)$ + クエリあたり $O(N)$



ここをなんとかしたい！

コスト



小課題 4

- 最小値をとりうる折れ線だけを管理、座標でソート
- 各クエリで自分に合った折れ線を探す（二分探索）
- あるいはクエリもソート→尺取り

小課題 4

- ・ 前処理 $O(N^2)$ + クエリあたり $O(\log N)$

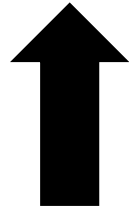
→ 全体 $O(N^2 + Q \log N)$ で小課題 4 AC

小課題 5

Q=1

小課題 3 (再掲)

- ・ 前処理 $O(N^2)$ + クエリあたり $O(N)$



ここをなんとかしたい！

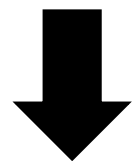
小課題 5

- ・今までは始点ごとに独立に XY 転倒数を求めている

→いい感じに使い回せないか？

小課題 5

1, 2, 4, 1, 4, 2, 3, 3



niteru

2, 4, 1, 4, 2, 3, 3, 1

小課題 5

- ・差分が区間和で表せる

→BIT の一点更新区間和取得で十分

小課題 5

- 前処理パートが $O(N\log N)$

→ 全体 $O(N\log N + QN)$ で小課題 5 AC !

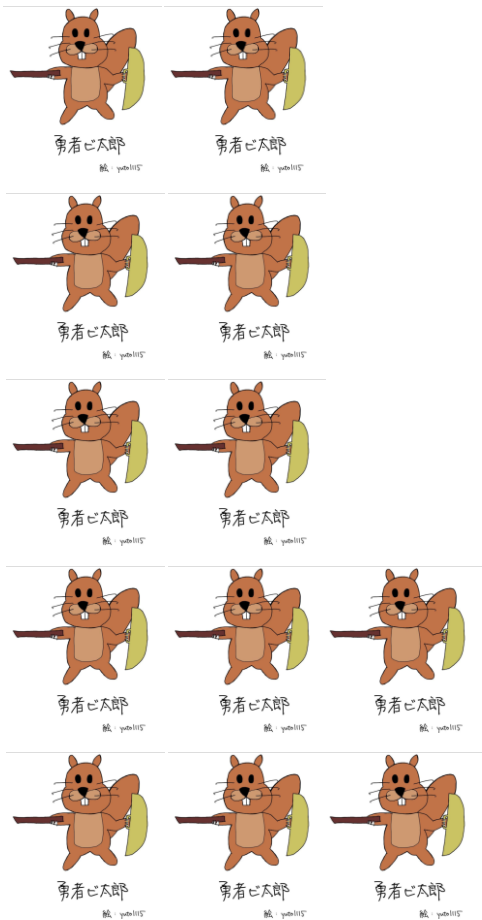
小課題 6

追加の制約なし

小課題4と小課題5の解法を組み合わせます

AC !

得点分布



100点



85点



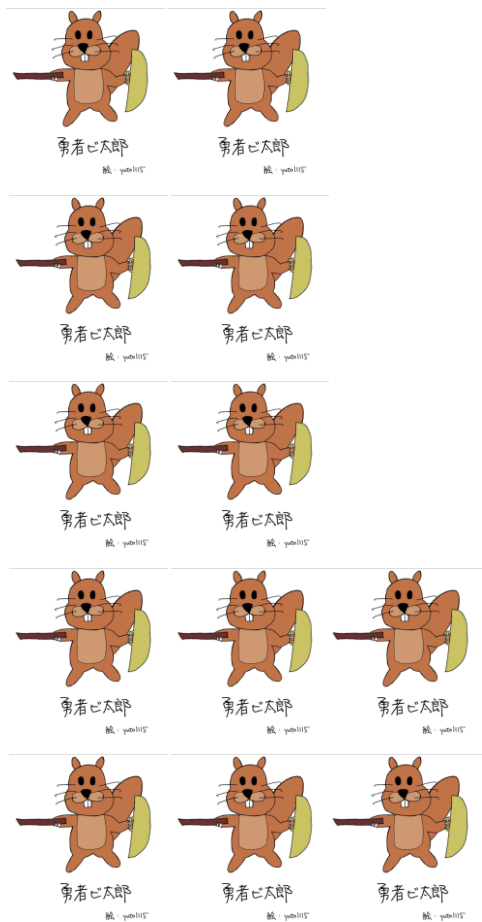
64点



25点



20点



100点



85点



64点



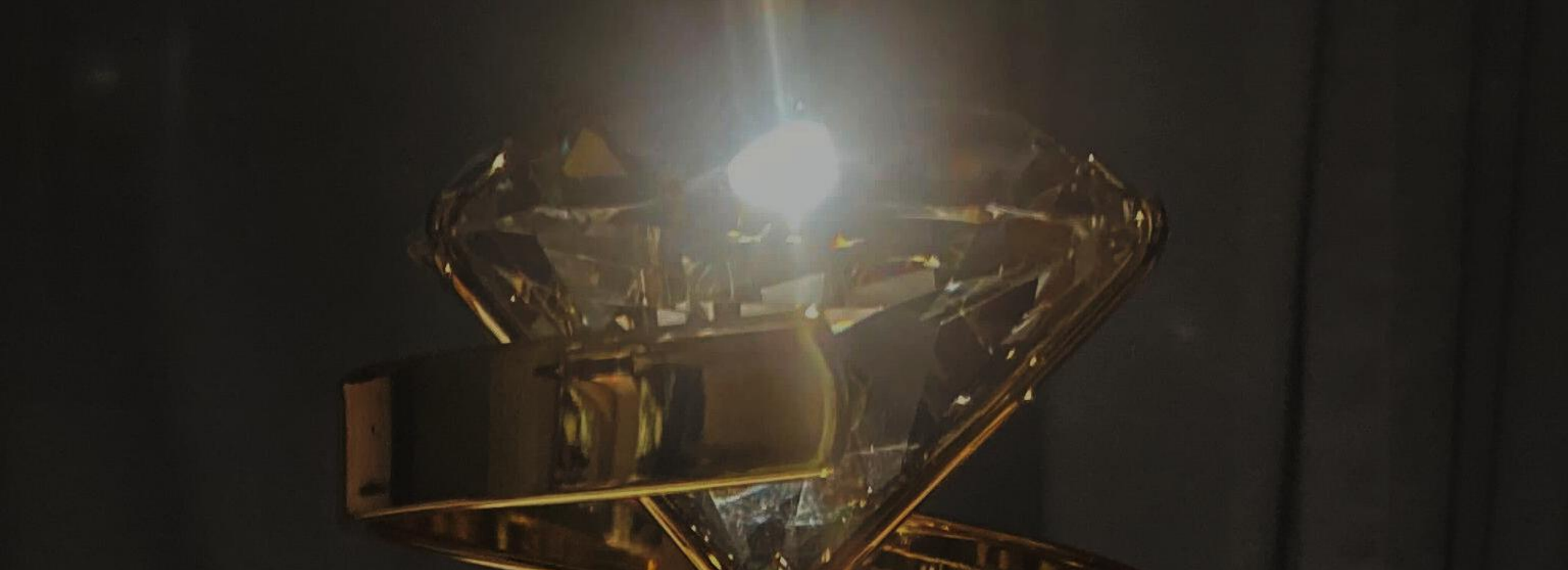
25点



20点



0点



宇宙怪盗 (Space Thief) 解説

解説者: 米田 寛峻 (**square1001**)

2025 年 3 月 22 日

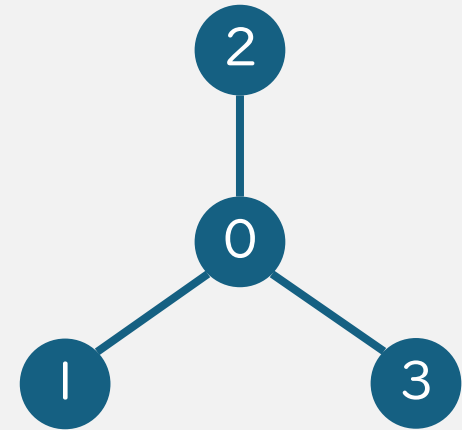
問題概要

N 頂点の木が与えられる

以下の質問を繰り返して、秘密の頂点 A, B を当てたい ($A \neq B$)

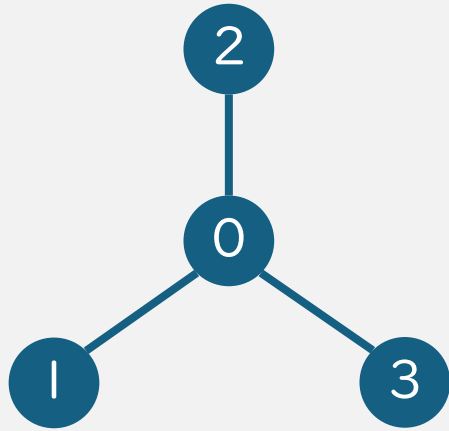
質問 木のすべての辺に自由に向きを付けて
A $\rightarrow$ B に到達できるかどうか質問する

300 回以内の質問で A, B を特定せよ



問題概要

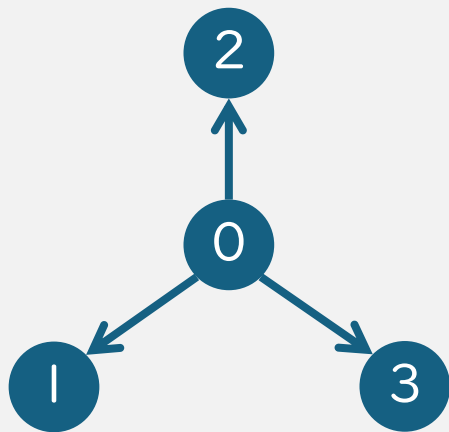
例: 4 頂点で辺 0-1, 0-2, 0-3 からなる木の場合



問題概要

例: 4 頂点で辺 $0-1$, $0-2$, $0-3$ からなる木の場合

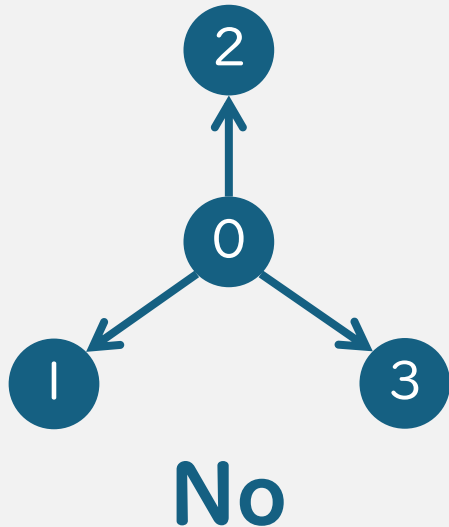
質問 1: この向き付けで $A \rightarrow B$ に到達できるか?



問題概要

例: 4 頂点で辺 $0-1$, $0-2$, $0-3$ からなる木の場合

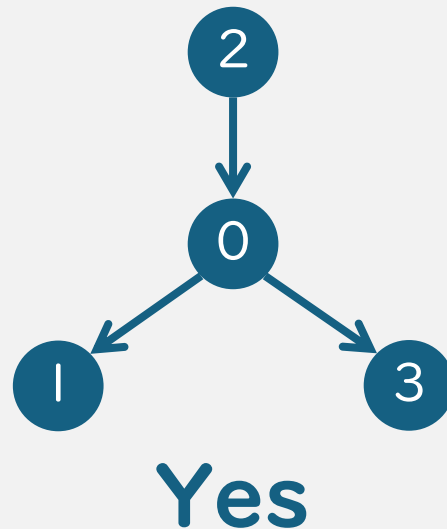
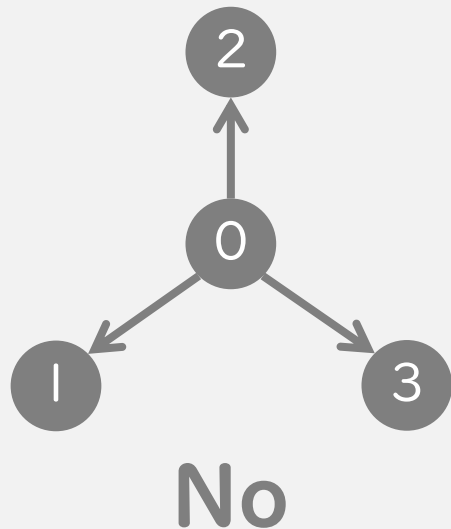
質問 1: この向き付けで $A \rightarrow B$ に到達できるか? (NO)



問題概要

例: 4 頂点で辺 $0-1$, $0-2$, $0-3$ からなる木の場合

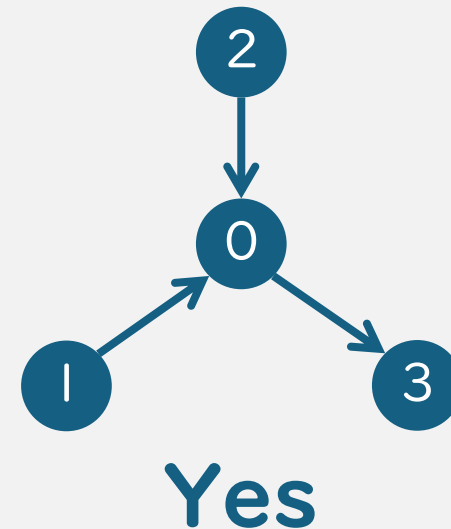
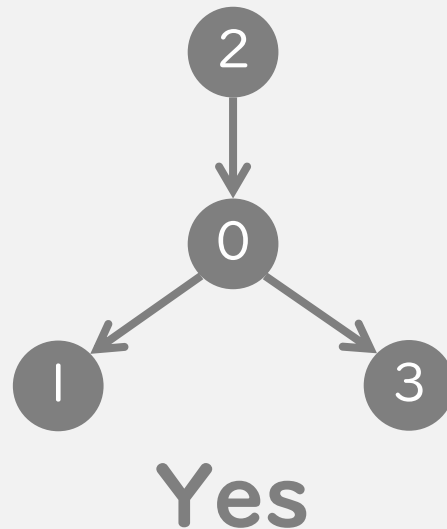
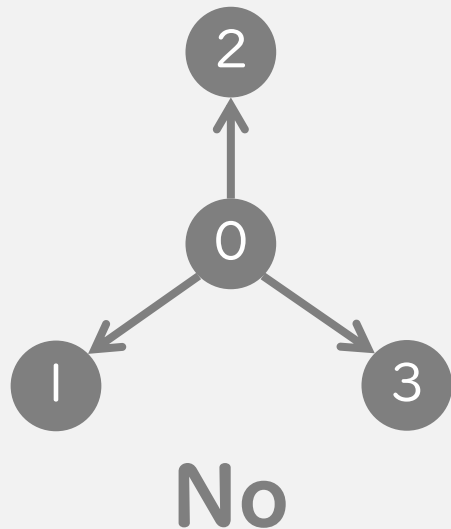
質問 2: この向き付けで $A \rightarrow B$ に到達できるか? (YES)



問題概要

例: 4 頂点で辺 $0-1$, $0-2$, $0-3$ からなる木の場合

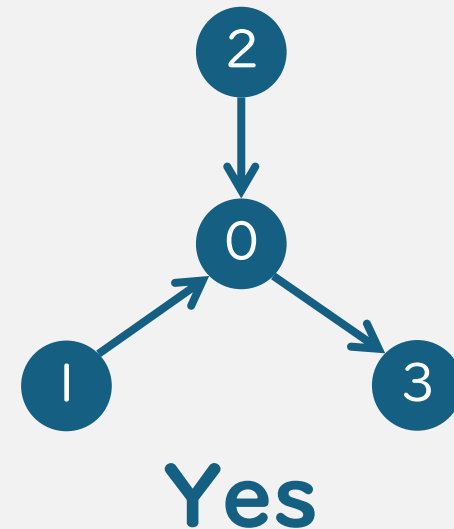
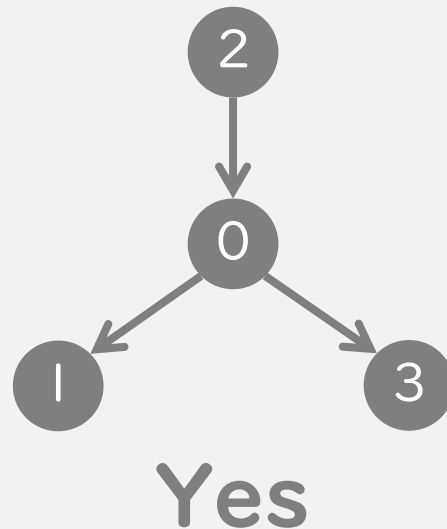
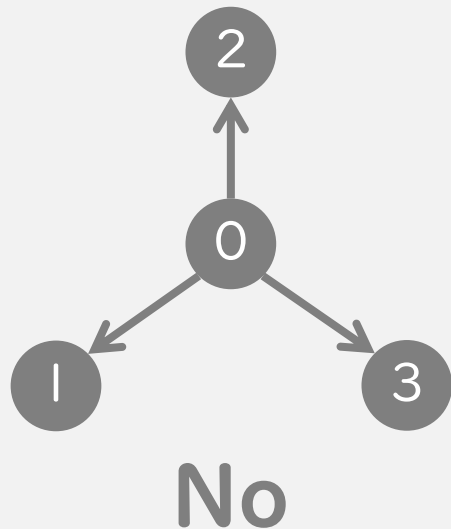
質問 3: この向き付けで $A \rightarrow B$ に到達できるか? (YES)



問題概要

例: 4 頂点で辺 0-1, 0-2, 0-3 からなる木の場合

この 3 つの情報から $(A, B) = (2, 3)$ と特定できる



問題概要

より一般の木で、少ない質問回数で解ければ、高得点が取れる

#1	7 点	$N \leq 10000$, 木はパスグラフ
#2	13 点	$N \leq 10000$, 木はスターグラフ
#3	2 点	$N \leq 8$
#4	8 点	$N \leq 50$
#5	5 点	$N \leq 150$
#6	5 点	$N \leq 250$
#7	40 点	$N \leq 10000$ (質問回数 70 以下で満点)
#8	20 点	木ではなく一般のグラフでも解く (質問回数 70 以下で満点)

小課題 1: パスグラフ

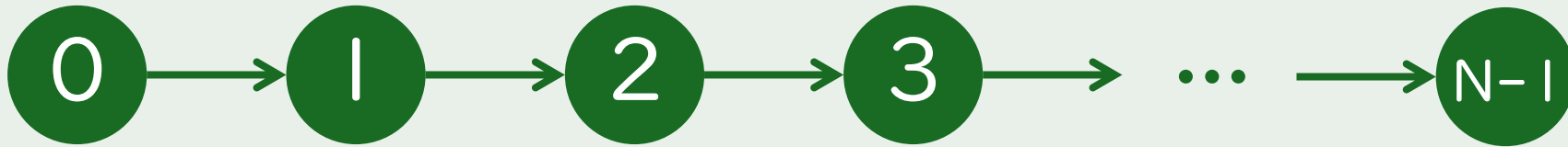
木がパスグラフの場合に解きたい



小課題 1: パスグラフ

最初に以下のような質問をする

Yes なら $A < B$ と、No なら $A > B$ と分かる



$A < B$ かどうか判別できる!

小課題 1: パスグラフ

最初に以下のような質問をする

Yes なら $A < B$ と、No なら $A > B$ と分かる

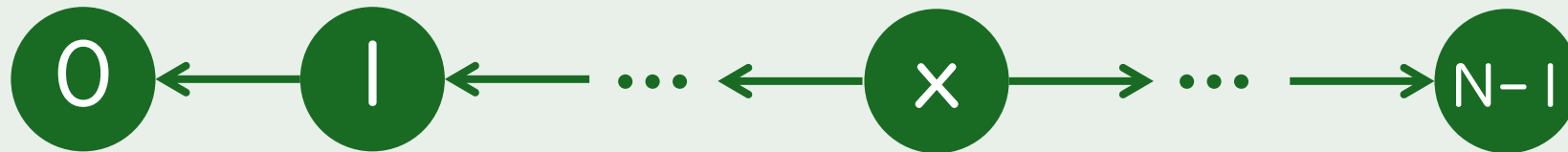
残りは A, B の値を
二分探索で求めよう!

$A < B$ かどうか判別できる!

小課題 1: パスグラフ

ここでは $A < B$ の場合を考える

“ A が x 以上か?” の判定は以下のようにできる!

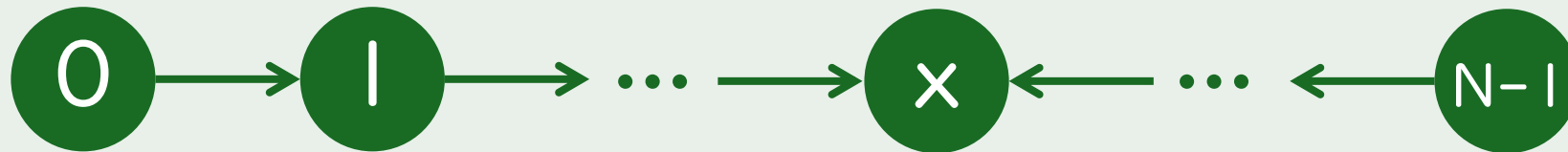


Yes なら $A \geq x$ で、No なら $A < x$

小課題 1: パスグラフ

ここでは $A < B$ の場合を考える

“B が x 以下か?” の判定も以下のようにできる!



Yes なら $B \leq x$ で、No なら $B > x$

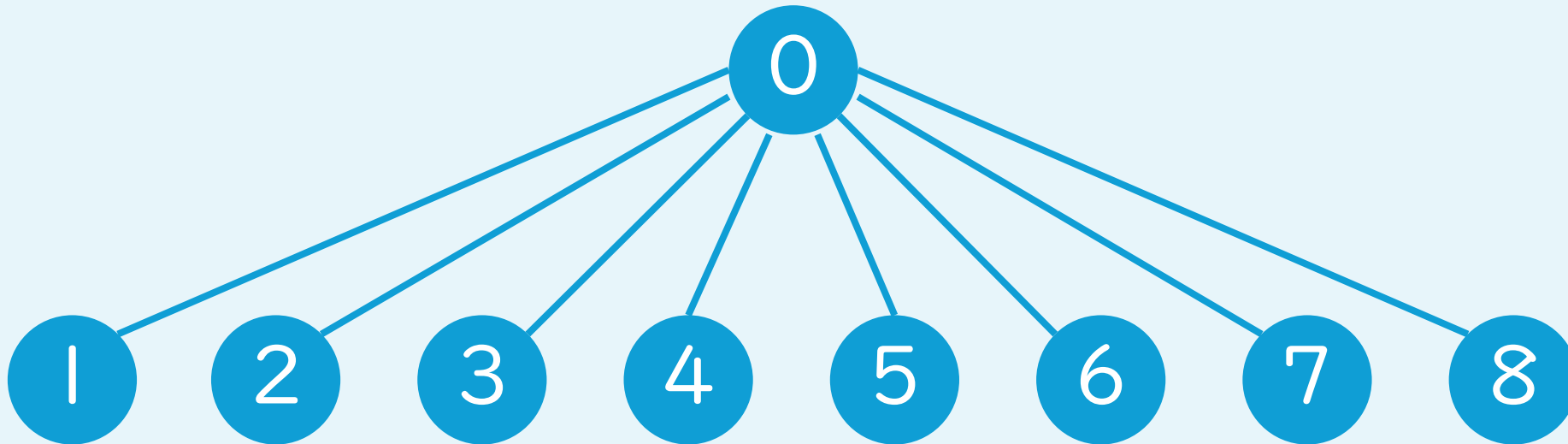
小課題 1: パスグラフ

まとめると... $2 \log N + 1 = 29$ 回の質問で解ける!

- 1 $A < B$ かどうか判定する (1 クエリ)
- 2 A を二分探索で求める ($\log N$ クエリ)
- 3 B を二分探索で求める ($\log N$ クエリ)

小課題 2: スターグラフ

木がスターグラフの場合を解きたい

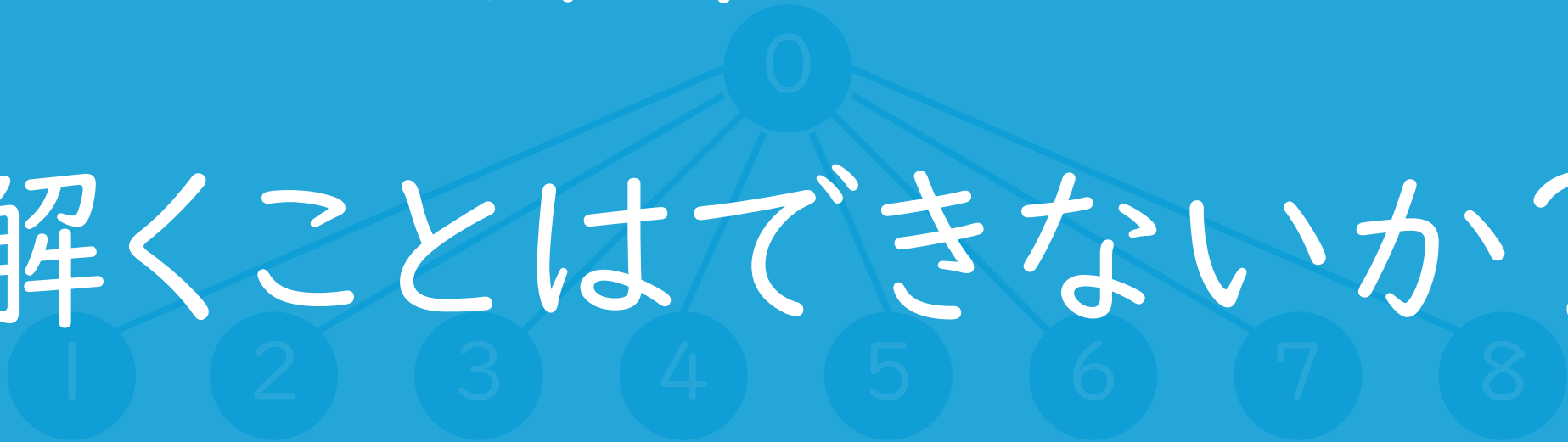


小課題 2: スターグラフ

木がスターグラフの場合を解きたい

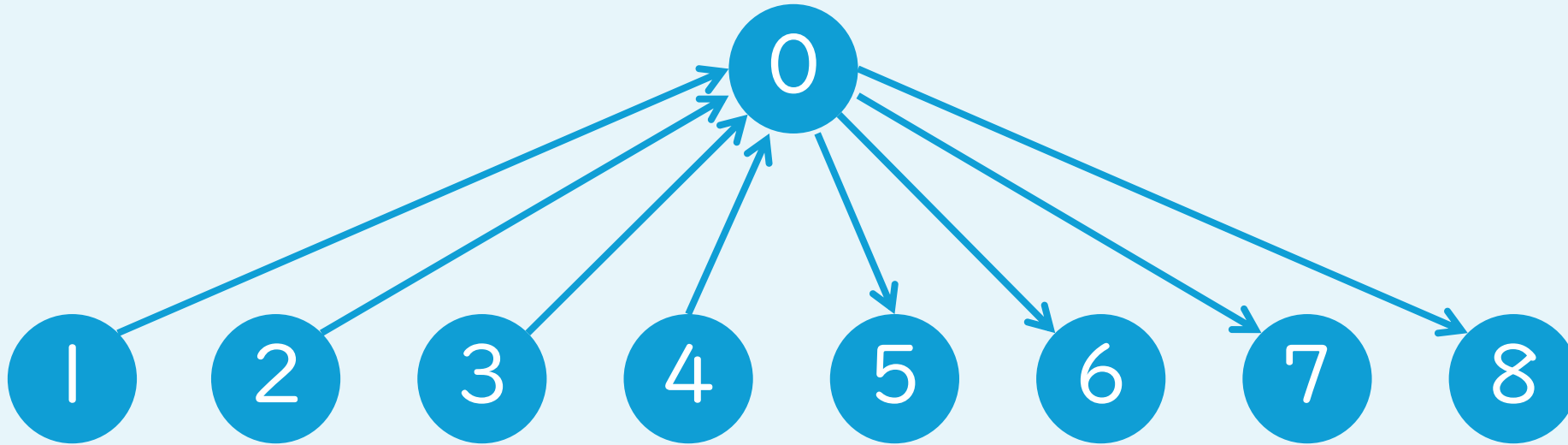
二分探索みたいに

解くことはできないか？



小課題 2: スターグラフ

とりあえず 1-4 で「0 に入る辺」、5-8 で「0 から出る辺」にする



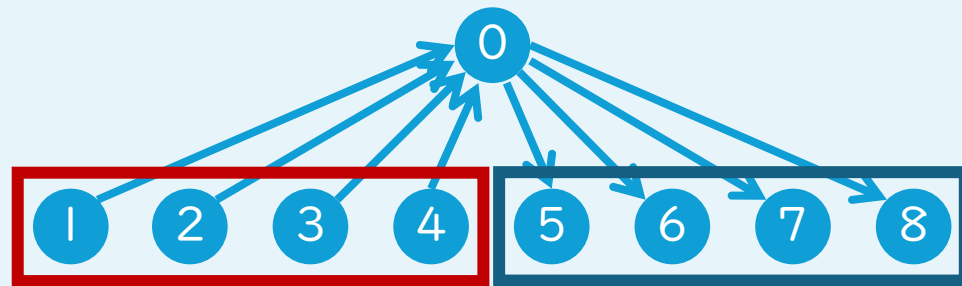
Yes なら「A が 1-4 かつ B が 5-8」が分かる

※ ただし A, B のどちらかが 0 になるケースは除く

小課題 2: スターグラフ

もし Yes であれば?

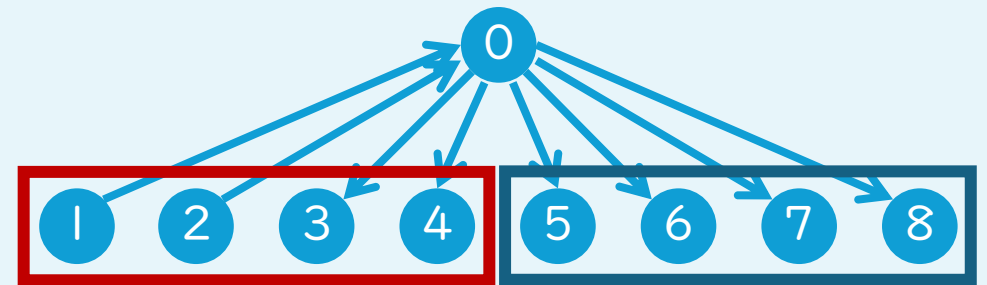
→ 以下のようにして、A (および B) を二分探索で求められる!



A がこの範囲

B がこの範囲

Yes

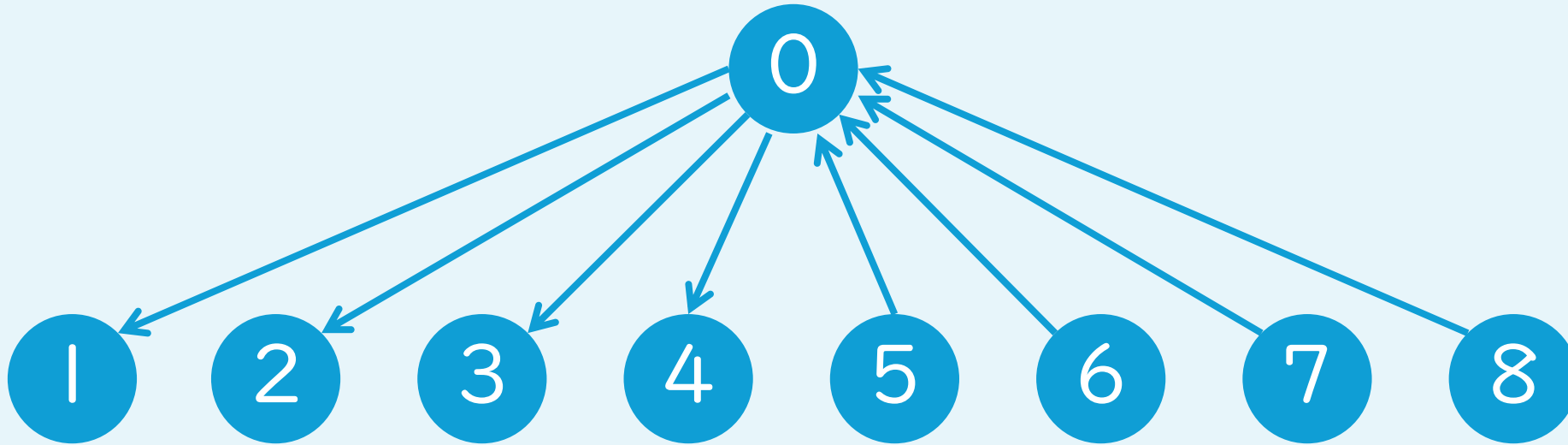


3, 4 の辺を逆転させる

$A \leq 2$ ならば Yes

小課題 2: スターグラフ

今度は逆に 5-8 で「0 に入る辺」、1-4 で「0 から出る辺」にする



これで Yes の場合も先ほどと同様に二分探索で (A, B) が求まる

※ ただし A, B のどちらかが 0 になるケースは除く

小課題 2: スターグラフ

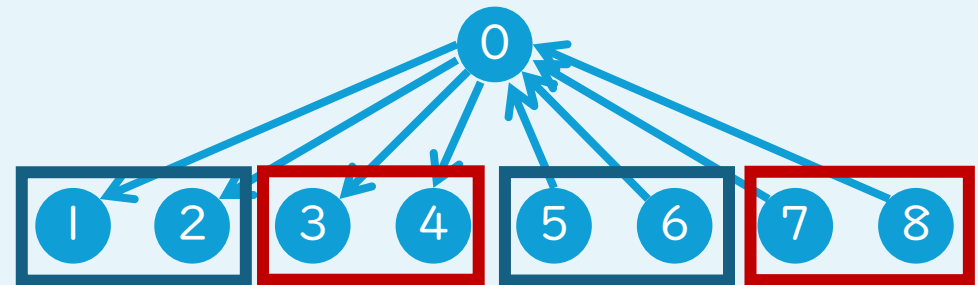
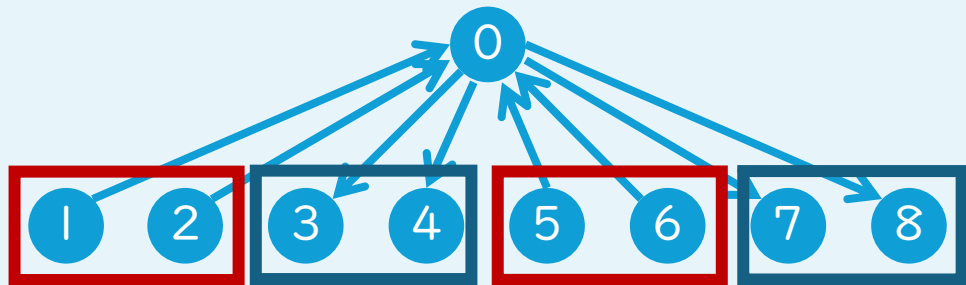
では、どちらも No の場合はどうなるか？



A, B が同じグループ (1-4 or 5-8) に入っていることが分かる

小課題 2: スターグラフ

今度は各グループを 2 つに分ける

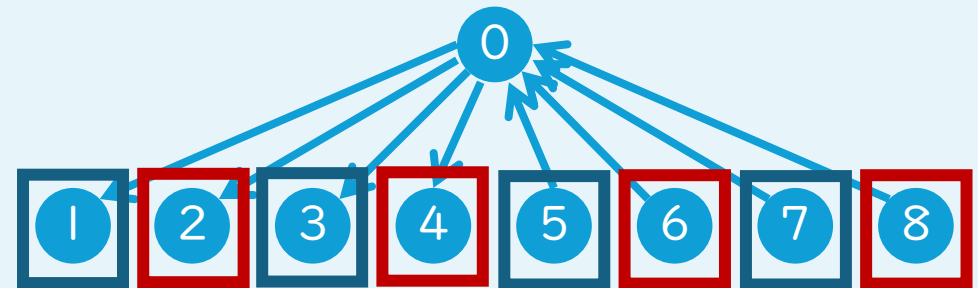
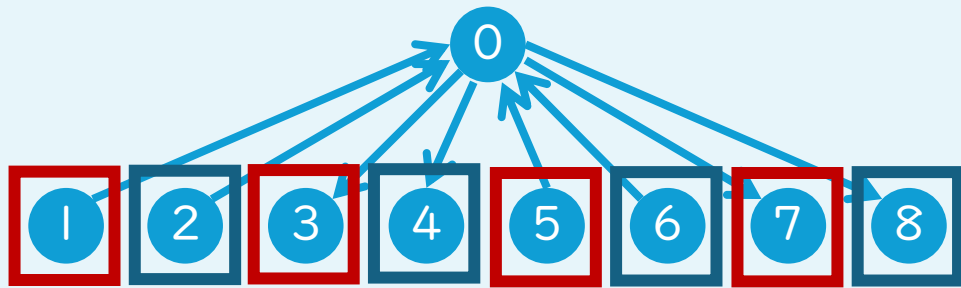


片方 Yes の場合 → 先ほどと同様に二分探索で (A, B) が求まる

両方 No の場合 → A, B が同じグループ (1-2, 3-4, 5-6, 7-8)

小課題 2: スターグラフ

今度は各グループをさらに 2 つに分ける



片方 Yes の場合 → 先ほどと同様に二分探索で (A, B) が求まる

両方 No の場合 → **$A \neq B$ なのであり得ない!**

小課題 2: スターグラフ

今度は各グループをさらに2つに分ける
グループ分けに $2 \log N$ 回

二分探索に $2 \log N$ 回

→ 合計 $4 \log N$ 回の質問で十分!

片方 Yes の場合 → 先ほどと同様に二分探索で (A, B) が求まる

両方 No の場合 → $A \neq B$ なのであり得ない!

一般の木を解くにあたって

小課題 1, 2 の解法を振り返ると...

全部同じ向きにして
 $A < B$ か判定



A, B をそれぞれ
二分探索で求める

小課題 1: パスグラフ

グループを 2 倍ずつに
分けていく



A, B をそれぞれ
二分探索で求める

小課題 2: スターグラフ

一般の木を解くにあたって

解法の共通部分①: 最後は A, B を二分探索で求める

全部同じ向きにして
 $A < B$ か判定



A, B をそれぞれ
二分探索で求める

小課題 1: パスグラフ

グループを 2 倍ずつに
分けていく



A, B をそれぞれ
二分探索で求める

小課題 2: スターグラフ

一般の木を解くにあたって

解法の共通部分②: 何らかの方法で質問に“Yes”と答えさせる

小課題 1 では「全部同じ向き」「全部逆向き」の 2 パターン試して“Yes”を得る

全部同じ向きにして
 $A < B$ か判定



A, B をそれぞれ
二分探索で求める

小課題 1: パスグラフ

グループを 2 倍ずつに
分けていく



A, B をそれぞれ
二分探索で求める

小課題 2: スターグラフ

A, B を二分探索で求める

まずは "Yes" が返ってきた後に A, B を求める方法を説明します

全部同じ向きにして
 $A < B$ か判定



A, B をそれぞれ
二分探索で求める

小課題 1: パスグラフ

グループを 2 倍ずつに
分けていく

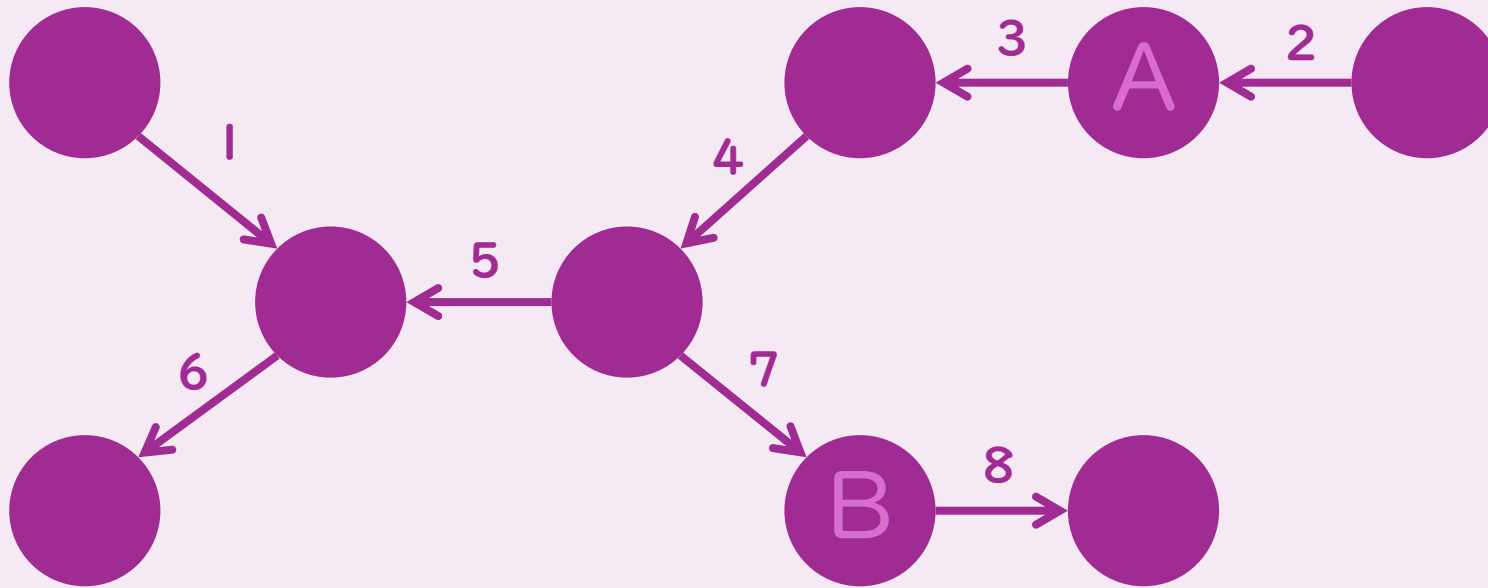


A, B をそれぞれ
二分探索で求める

小課題 2: スターグラフ

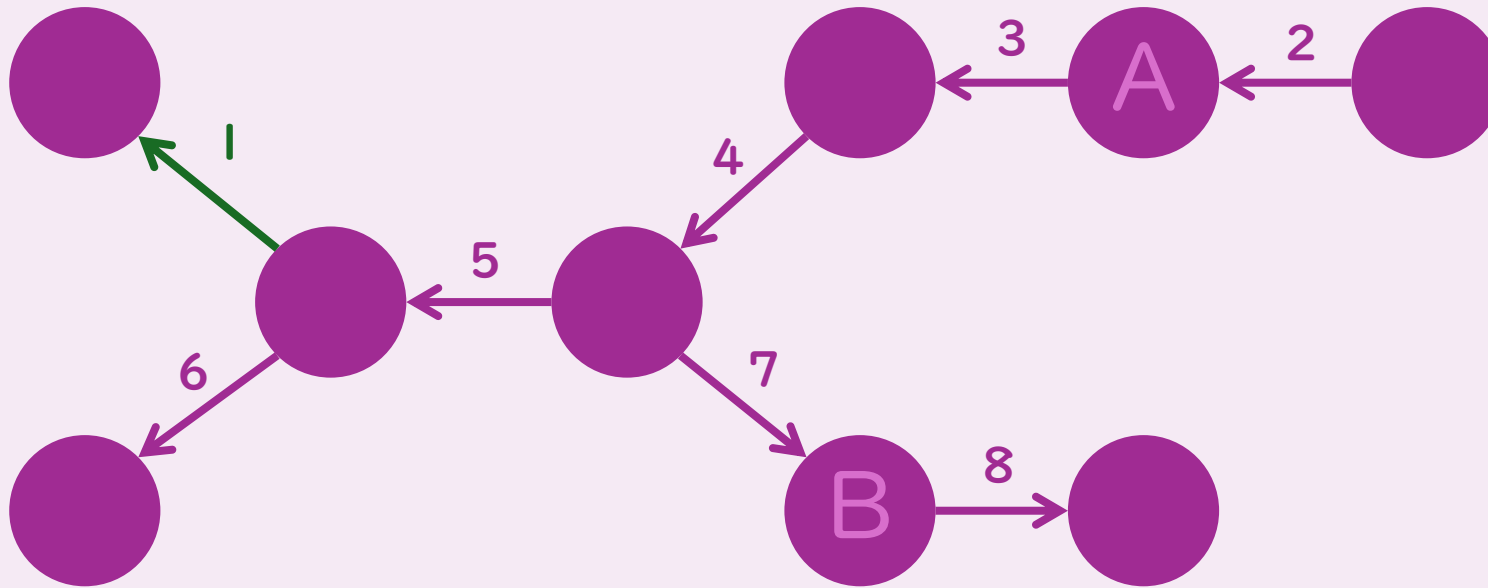
A, B を二分探索で求める

ある辺の向き付けで “Yes” が返ってきたとする



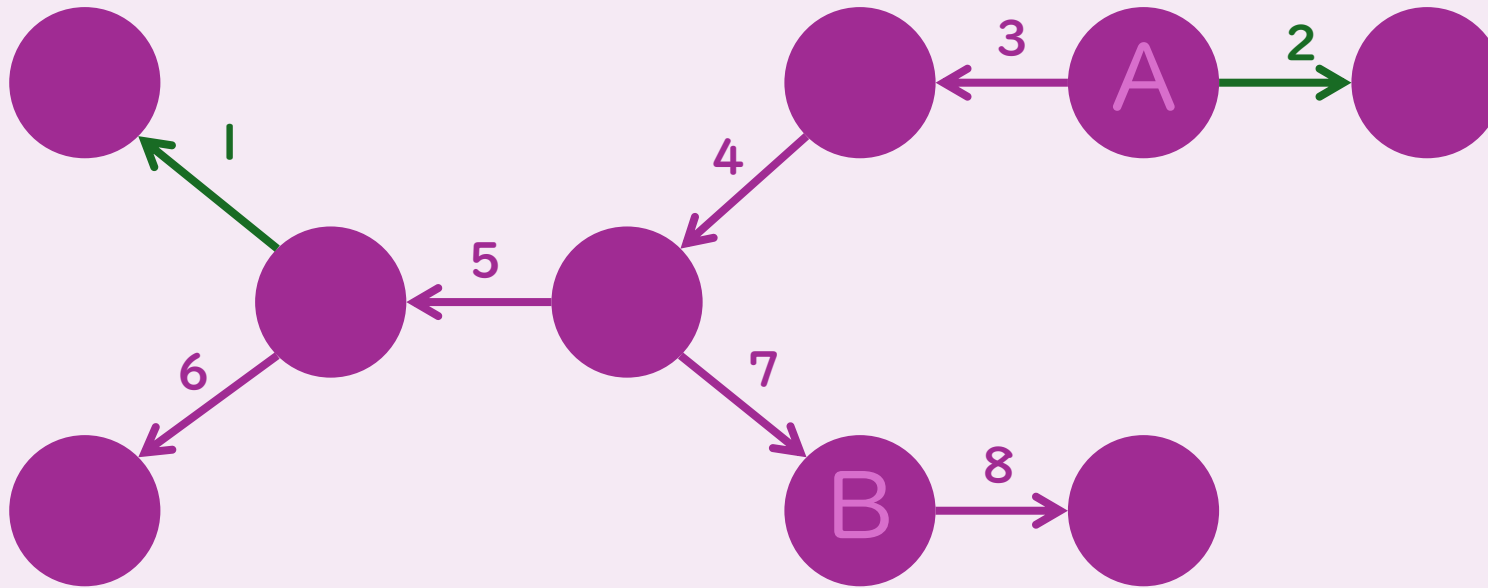
A, B を二分探索で求める

ひとつずつ反転させていくことを考える



A, B を二分探索で求める

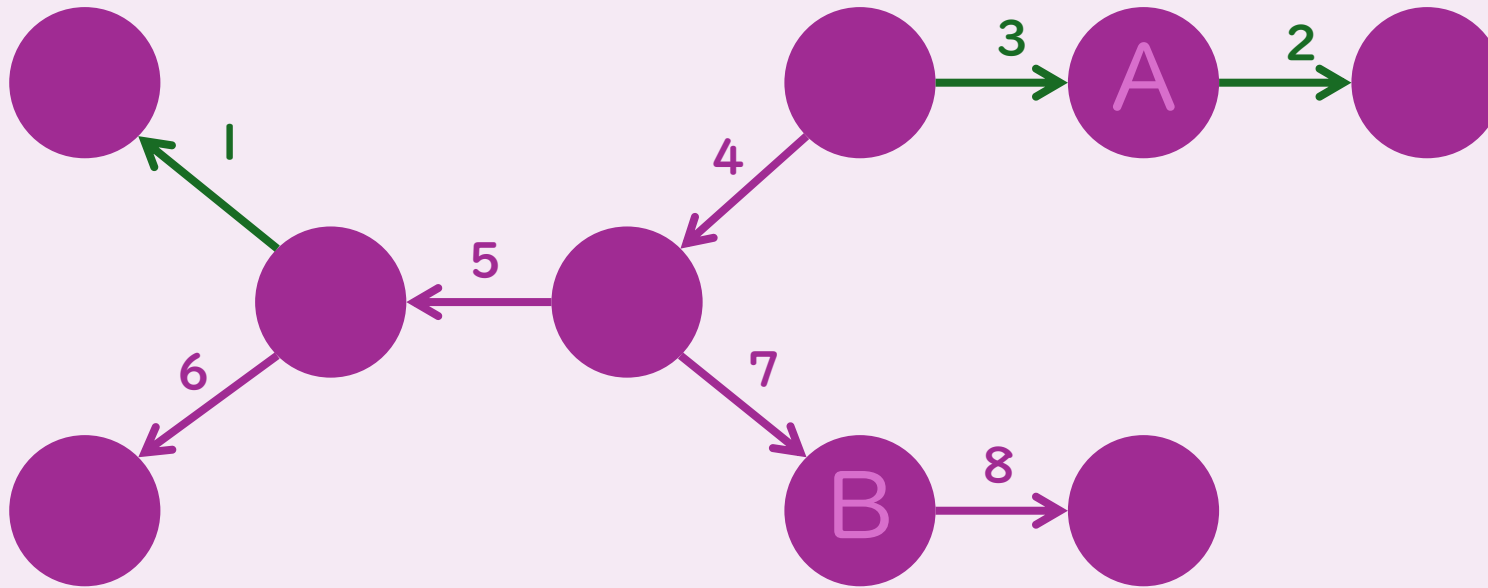
ひとつずつ反転させていくことを考える



A, B を二分探索で求める

どこかのタイミングで $A \rightarrow B$ に到達できなく (“No” に) なる!

番号 3, 4, 7, 8 の辺のいずれかが反転されれば “No”



A, B を二分探索で求める

どこかのタイミングで $A \rightarrow B$ に到達できなく (“No” に) なる!

番号 3, 4, 7, 8 の辺のいずれかが反転すれば “No”
“Yes” が “No” になる境界は

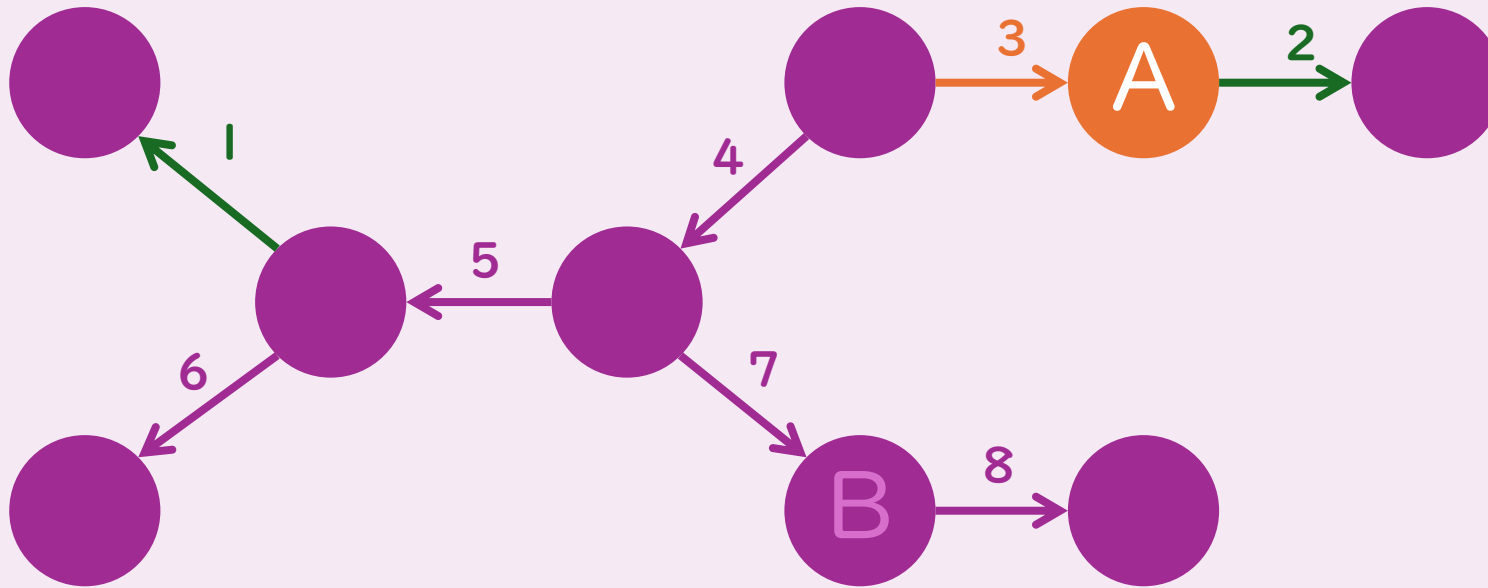
二分探索で求まる!



A, B を二分探索で求める

特に、辺の番号をトポロジカル順でソートされている場合…

※ トポロジカル順: 辺の流れの順番で番号が並んでいること



「境界の辺」の端点が A となる!

A, B を二分探索で求める

こうして A, B が二分探索で求まる

※ トポロジカル順: 辺の流れの順番で番号が並んでいること

つまり ”Yes” の返答から

$2 \log N$ 回の質問で解ける!

「境界の辺」の端点が A となる!

Yes の返答を一度得る

次に、一度でも“Yes”の返答を得る方法を考える

全部同じ向きにして
 $A < B$ か判定



A, B をそれぞれ
二分探索で求める

小課題 1: パスグラフ

グループを 2 倍ずつに
分けていく

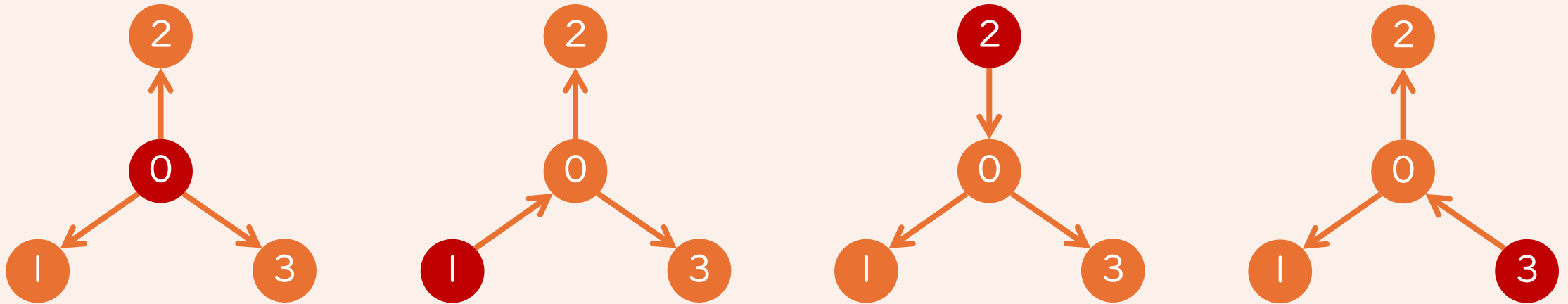


A, B をそれぞれ
二分探索で求める

小課題 2: スターグラフ

Yes の返答を一度得る

各頂点 v を根としてそこから広がるように辺を向き付けると、必ずいずれかで “Yes” になる (例えば $v = A$ の場合)



→ $N + 2 \log N$ 回の質問で解ける! (小課題 5 に正解)

小課題 5: $N \leq 250$

各頂点 v を根としてそこから広がるように辺を向き付けると、必ずいずれかで “Yes” になる (例えば $v = A$ の場合)



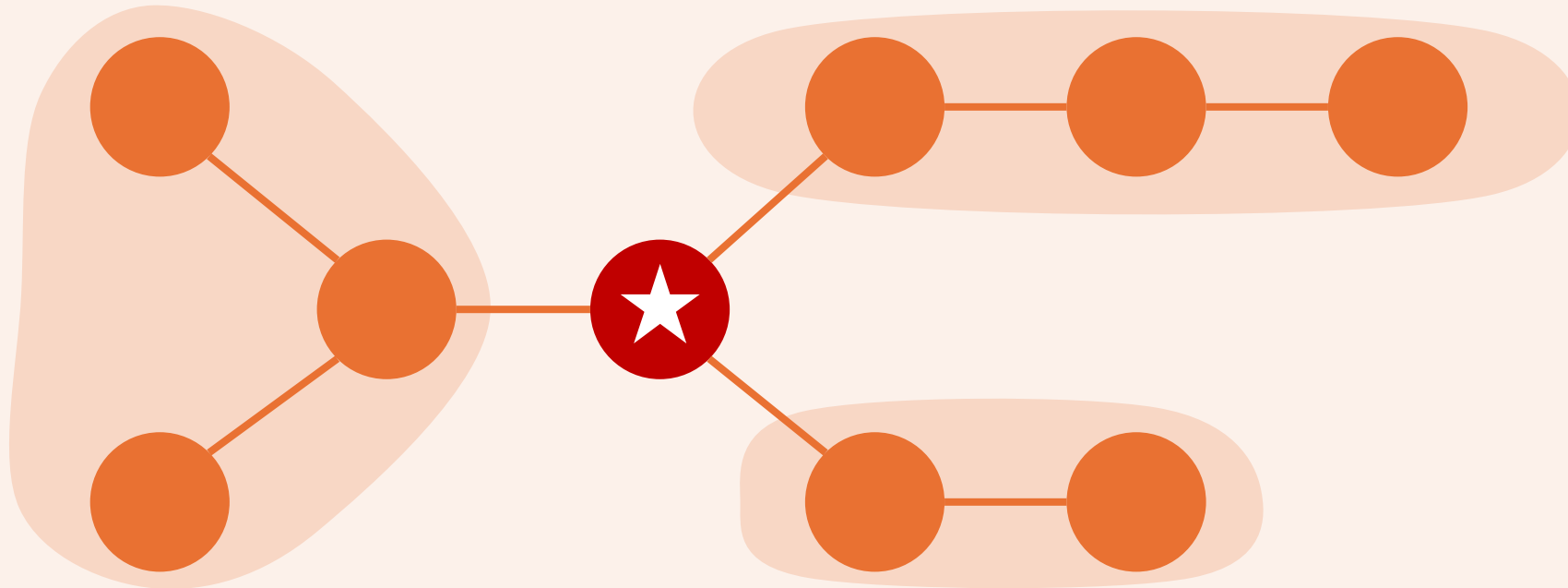
効率化しよう!

→ $N + 2 \log N$ 回の質問で解ける!

木の重心

木には **重心 (centroid)** とよばれる頂点が存在する

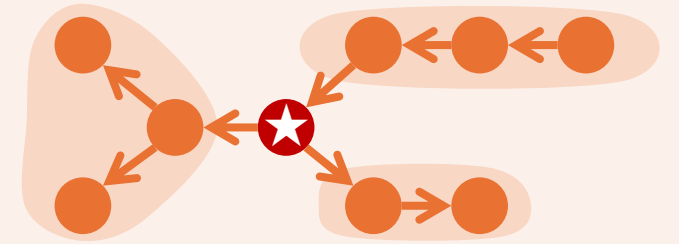
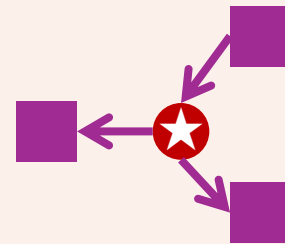
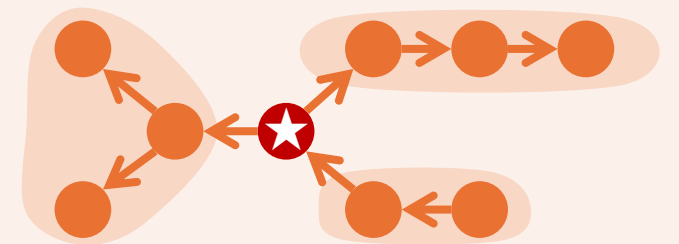
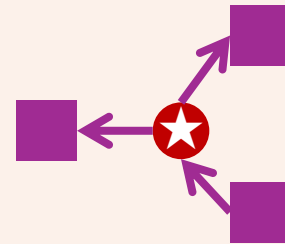
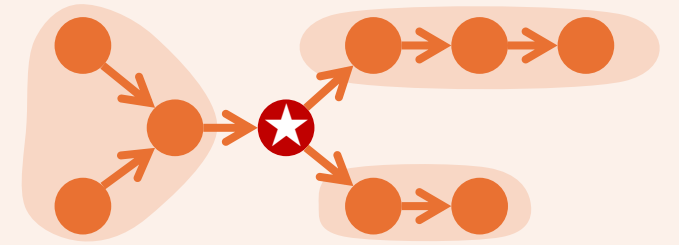
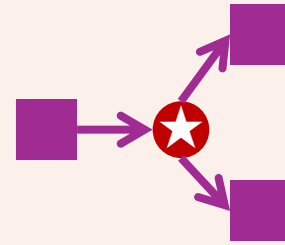
- この頂点を削除すると、各連結成分が $N/2$ 頂点以下になるもの



木の重心で分ける

木の重心を中心としたスターグラフ
だと思って問題を解く

→ もし No しか得られなかった場
合 A, B は同じ“連結成分”

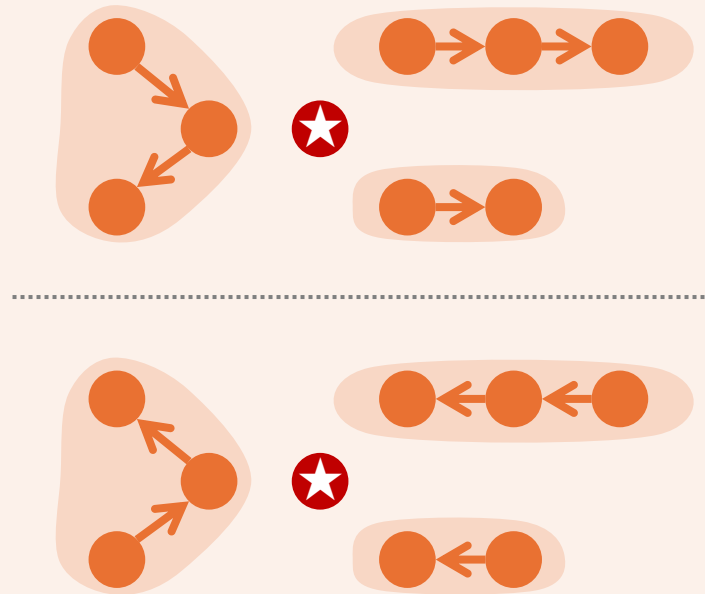


木の重心で分ける

木の重心を中心としたスターグラフ
だと思って問題を解く

→ もし No しか得られなかった場
合 A, B は同じ“連結成分”

アイデア: このとき各連結成分に対して同じ問題を並列に解く!

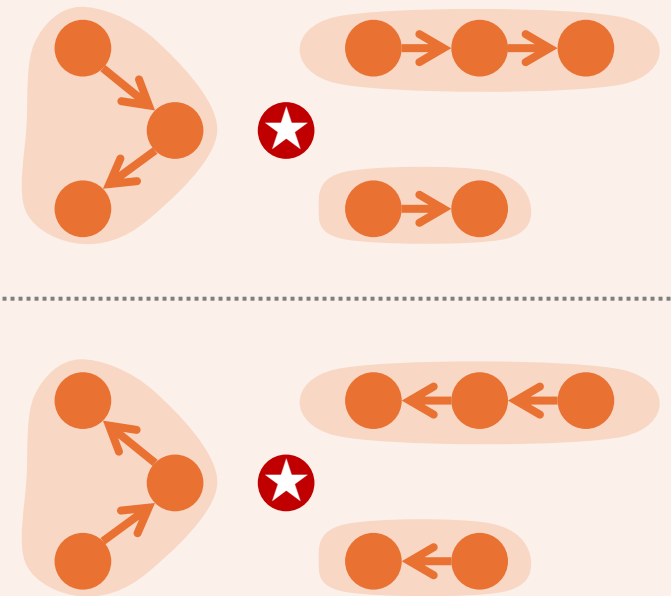


木の重心で分ける

アイデア: このとき各連結成分に対して同じ問題を並列に解く!

計算量解析

- 連結成分のサイズは毎回半分以下になるので $\log N$ ステップしか起こらない
- スターグラフを解くのに $2 \log N$ 回質問
→ $2 \log N + 2 \log(N/2) + \dots = \log^2 N$ 回程度の質問で “Yes” が得られ、小課題 7 で 20/40 点獲得



木の重心で分ける

アイデア: このとき各連結成分に対して同じ問題を並列に解く!

改善点: 定数回の質問で

計算量解析

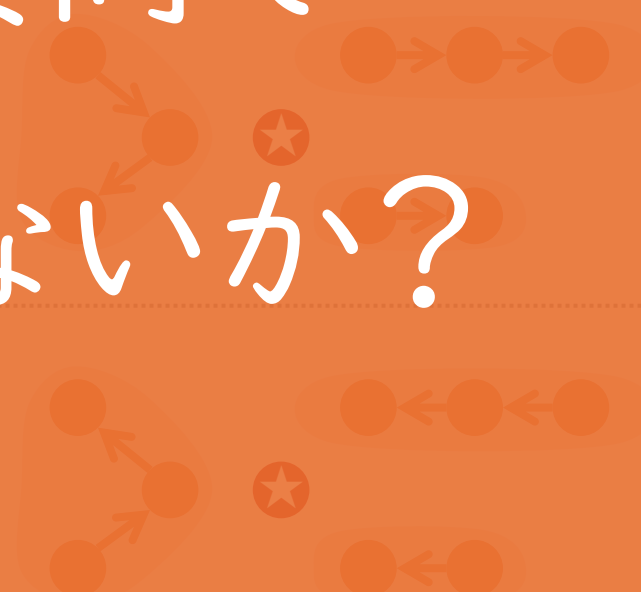
- 連結成分のサイズは毎回半分以下になるので $\log$

頂点数を半分にできないか?

- スターグラフを解くのに $2 \log N$ 回質問

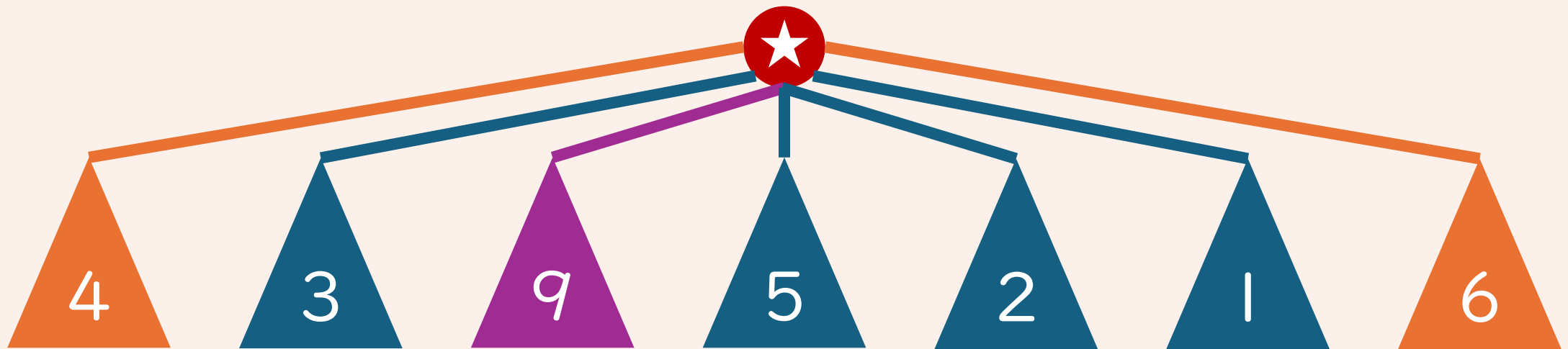
→ $2 \log N + 2 \log(N/2) + \dots = \log^2 N$ 回程度の

質問で “Yes” が得られ、小課題 7 で 20/40 点獲得



“3 分割定理”の利用

アイデアは、重心でたくさんの連結成分ができた場合、全部別々のグループとは考えずに適切にまとめることである



この例では 3 グループに分けるだけで、各グループの頂点数を元の半分以下にしている（数字は連結成分のサイズ）

“3 分割定理”の利用

Q. こんなことは常に可能なのか？

定理

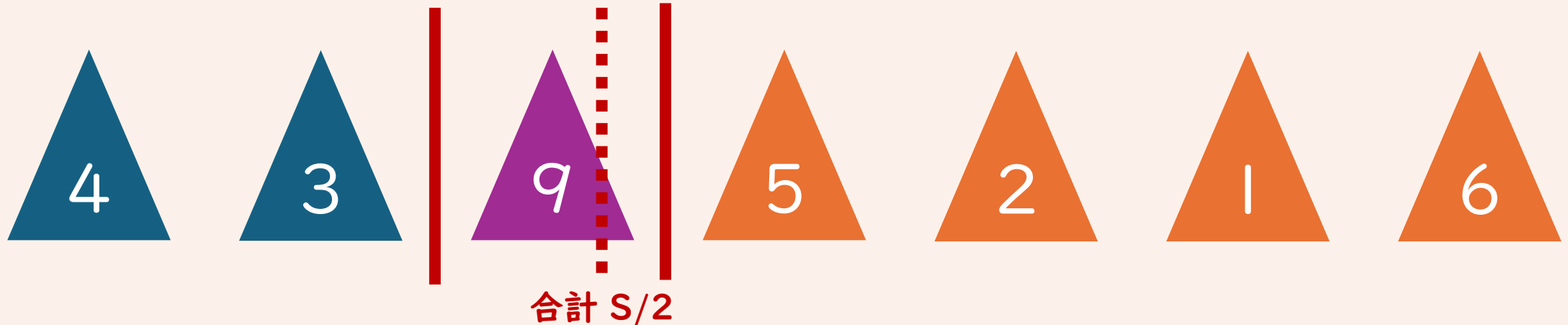
自然数 $a_1, \dots, a_n$ が $\max(a_1, \dots, a_n) \leq S/2$ を満たすとき、どのグループの総和も $S/2$ 以下になるようにこれらを 3 分割することが可能である。ここで $S = a_1 + \dots + a_n$ とする。

“3 分割定理”の利用

“3 分割定理”の証明

左からの合計が $S/2$ になる直前と直後で分割すれば OK

→ 1, 3 番目はもちろん $S/2$ 以下、2 番目も 1 つの要素なので $S/2$ 以下



“3分割定理”の利用

“3分割定理”の証明
3回の質問で頂点数を半分に!

左からの合計が $S/2$ になる直前と直後で分割すれば OK

質問回数は $3 \log N + 2 \log N$ 回

→ 小課題 7 で満点獲得!



そして、一般グラフへ

実は一般グラフでも、同じように解くことができる

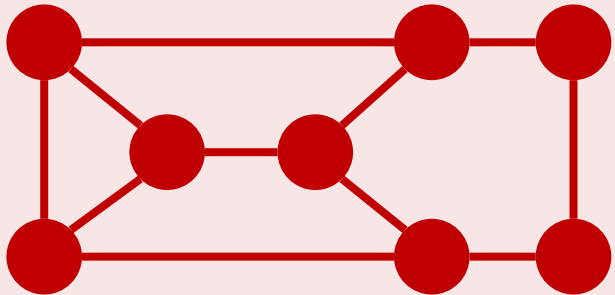
- 1 一度でも “Yes” の返答を得る
- 2 最後は A, B を二分探索で求める

そして、一般グラフへ

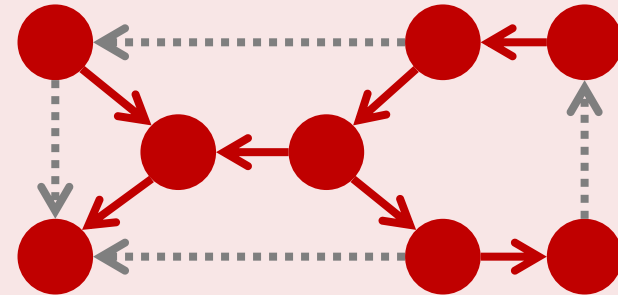
I 一度でも“Yes”の返答を得る

一般グラフで“Yes”を得るのは木の場合より簡単

- 全域木は「木の場合」と同様にして、それ以外の辺は適当でいい



全域木のみ考える



そして、一般グラフへ

2 最後は A, B を二分探索で求める

もし辺の向き付けが **DAG (閉路のないグラフ)** になっていれば、木の場合と同様に “Yes” から $2 \log N$ 回の質問で A, B が求まる

注意点

- ① 全域木に含まれない辺は DAG になるように向き付ける
- ② 辺ではなく頂点のトポロジカル順を使って二分探索する

そして、一般グラフへ

2 最後は A, B を二分探索で求める

一般グラフでも $5 \log N$ 回を実現

もし辺の向き付けが DAG (閉路のないグラフ) になっていれば、木の場合と同様に “Yes” から $2 \log N$ 回の質問で A, B が求まる
→ 小課題 8 で満点獲得!

注意点

- ① 全域木に含まれない辺は DAG になるように向き付ける
- ② 辺ではなく頂点のトポロジカル順を使って二分探索する

勇者ビ太郎 3 解説

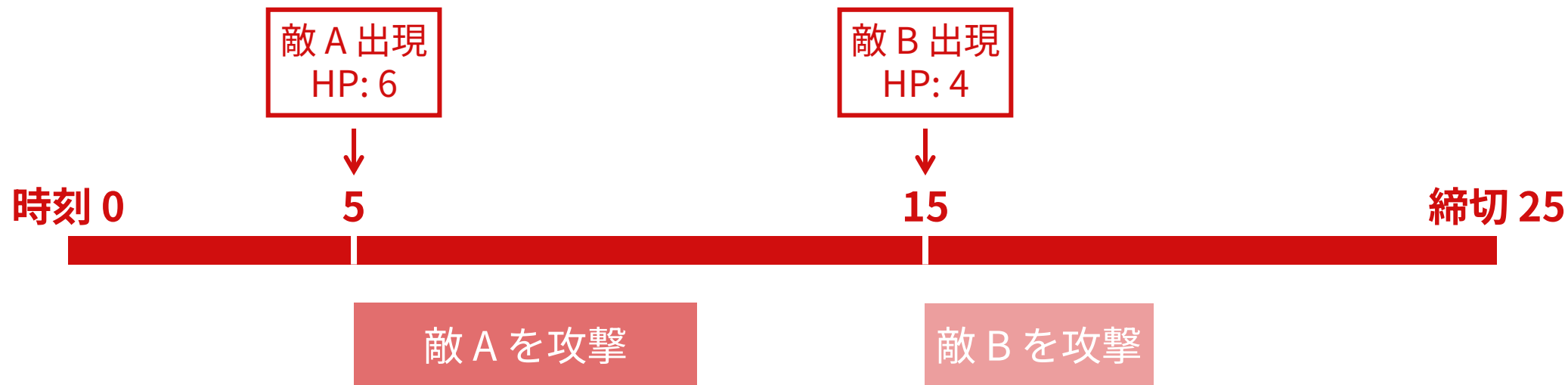
2025/3/23

米田優峻 (E869120)

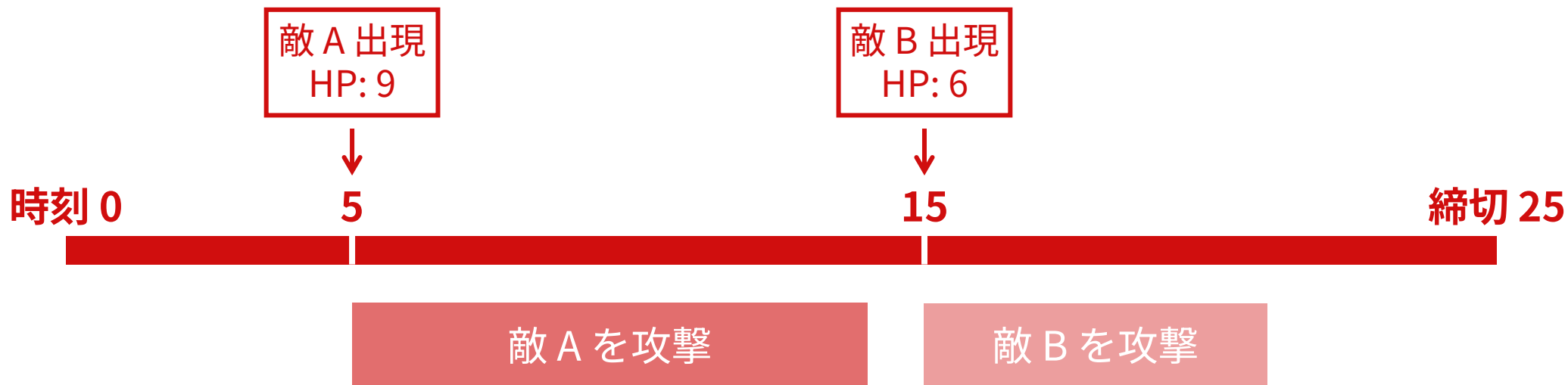
まず、次の問題を考えてみよう



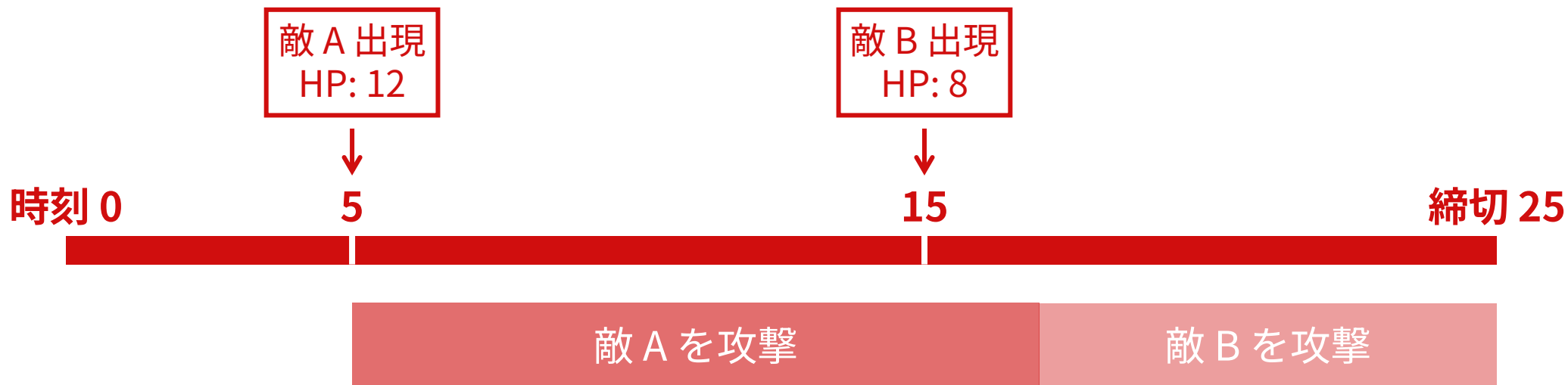
1 秒につきどれかの敵の HP を 1 減らすことができる
難易度いくつまでクリア (全部の敵を倒すこと) できるか？



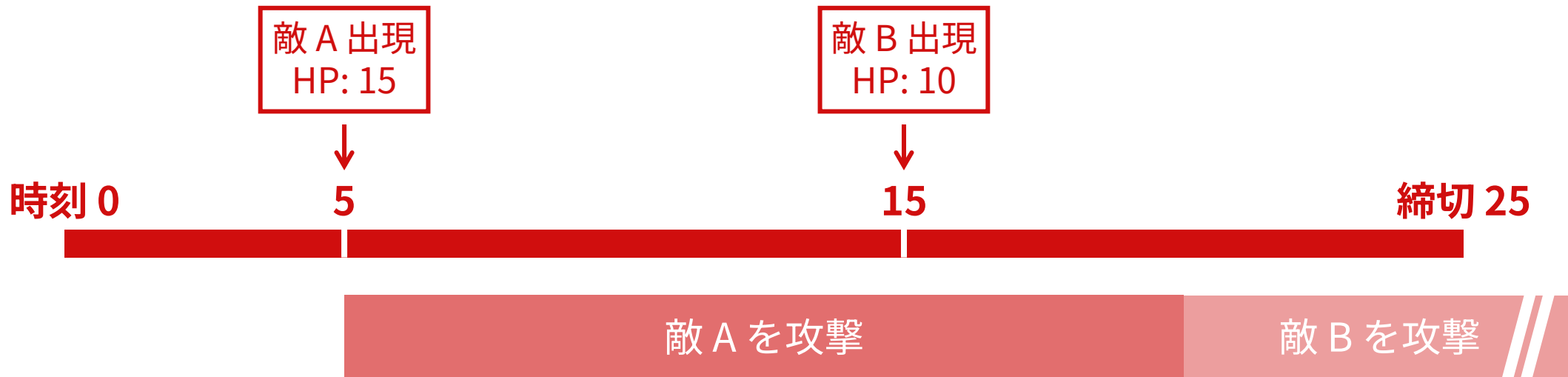
難易度 2 であれば、上記の戦略でクリア可能



難易度 3 であれば、上記の戦略でクリア可能



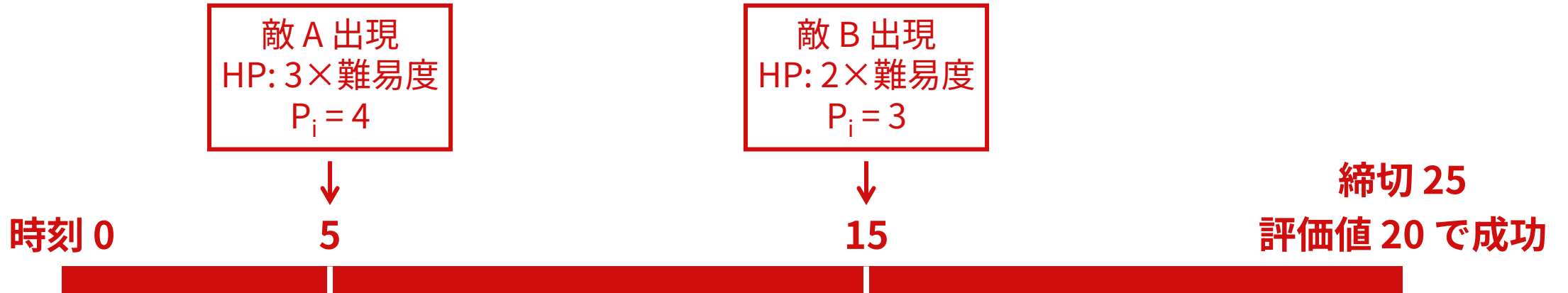
難易度 4 であれば、上記の戦略でクリア可能



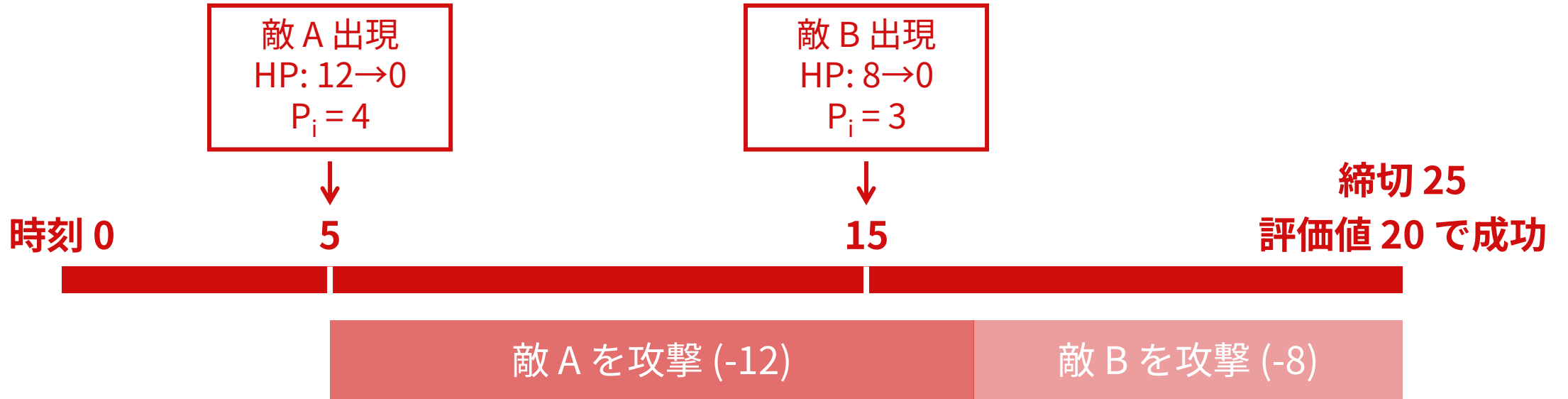
しかし、難易度 5 はクリアすることができない
よって答えは 4

しかし、実際の問題では
敵を全部倒す必要はない

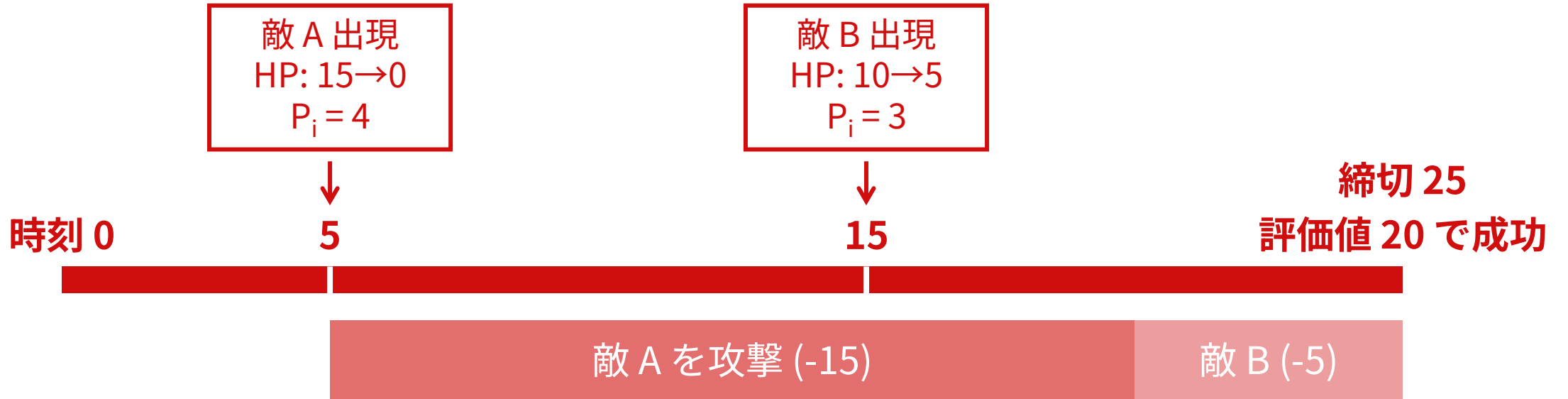
各敵について強さ P_i が付けられていて
最後の $HP \times P_i$ の合計 (評価値という) が
一定の基準値以下であれば成功



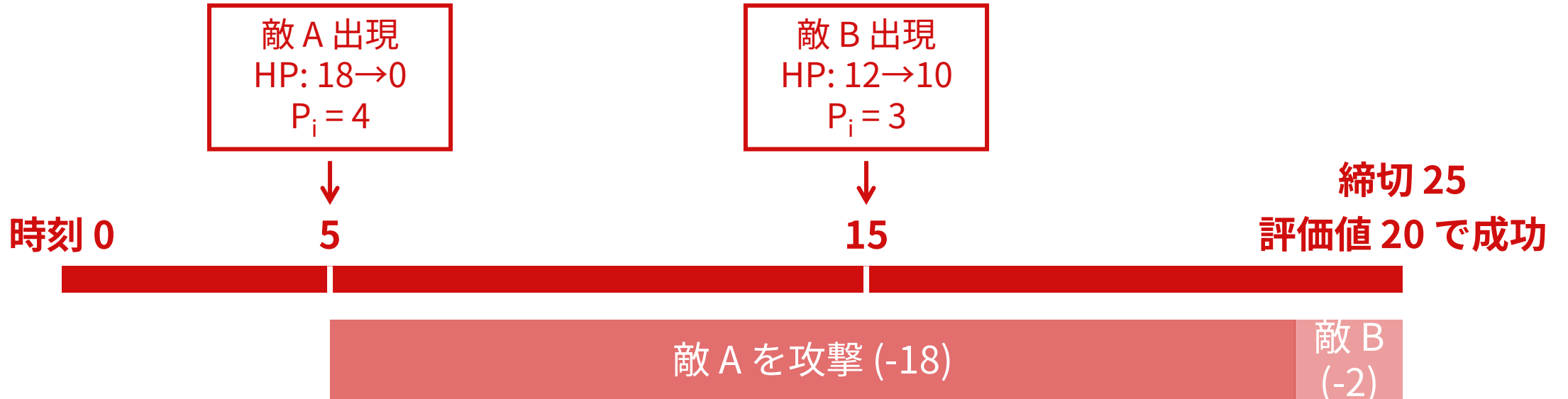
たとえば上のようなケースの場合 (評価値 20 で成功)
難易度いくつまですることがクリアできるか？



まず、難易度 4 はクリア可能 (評価値 $0 \times 4 + 0 \times 3 = 0$)



また、難易度 5 もクリア可能 (評価値 $0 \times 4 + 5 \times 3 = 15$)



しかし、難易度 6 は不可能 (評価値 $0 \times 4 + 10 \times 3 = 30$) → 答えは 6

今回は基準値が「評価値 20」でしたが
このような問題を $Q (\leq 10^6)$ 個の基準値について解けますか？
というのが今回の問題です

- モンスターの数 $N \leq 6000$
- クエリ (基準値) の数 $Q \leq 1000000$
- 締切時刻 $T \leq 10^{18}$

小課題 1

小課題 1

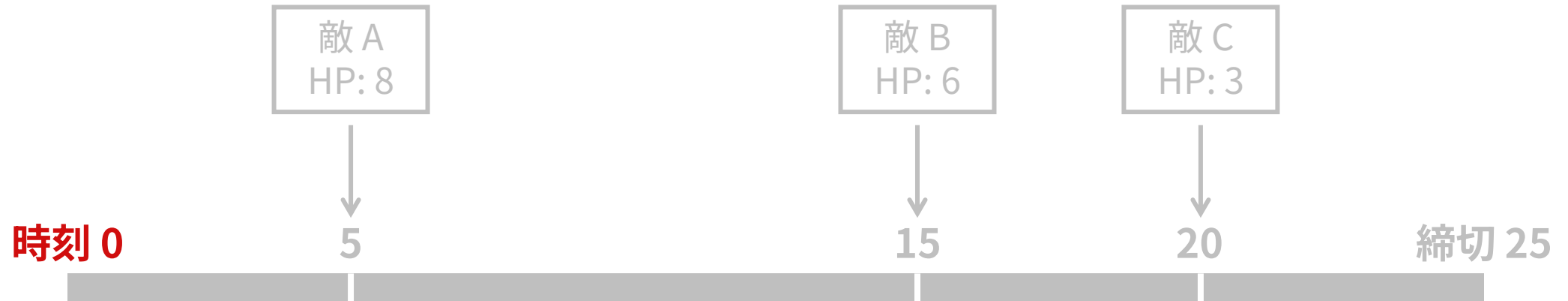
- $Q = 1, M_1 = 1, L = 0$
- つまり、難易度 1 のダンジョンで全部の敵を倒せるかを判定できれば良い

小課題 1: 解法

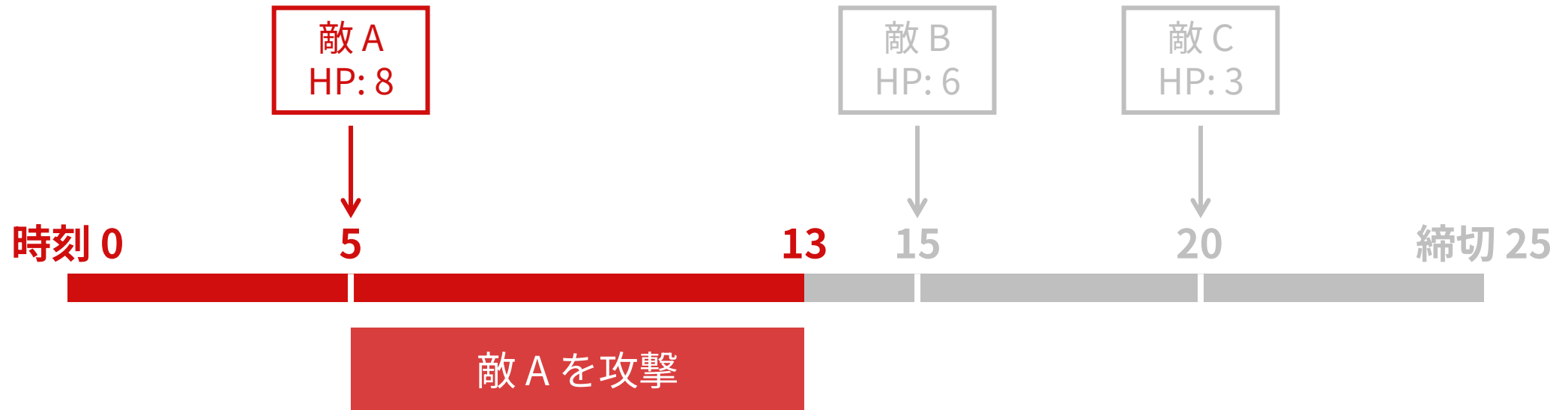
- 難易度が決まっているので、単にモンスターの出現時刻 s_i が小さい順に殺せばよい
- モンスターを s_i の小さい順にソートすると解ける

1 点

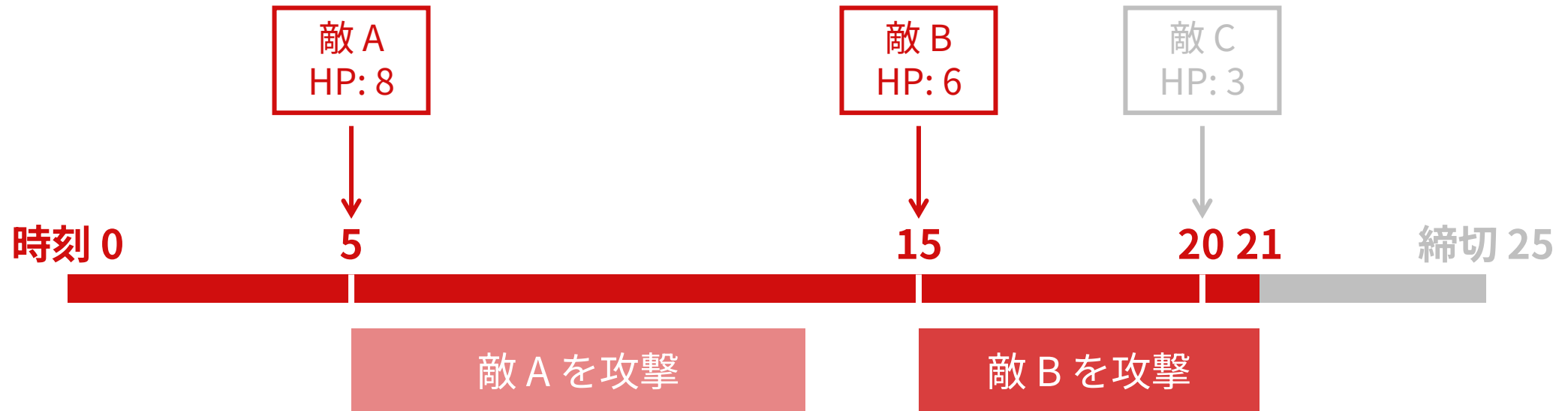
小課題 1: 例 1



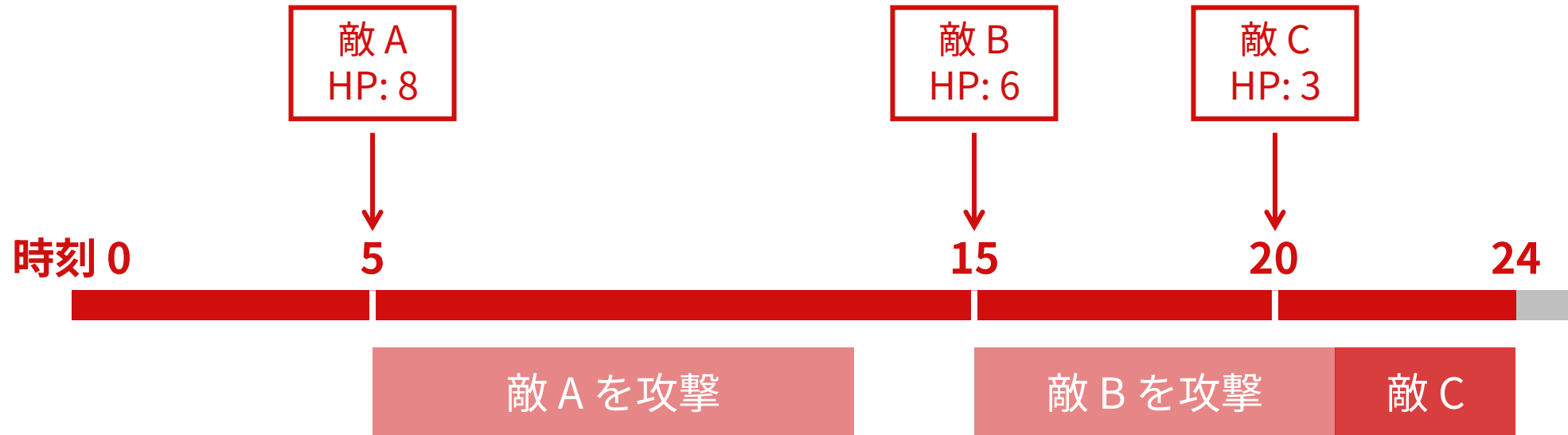
小課題 1: 例 1



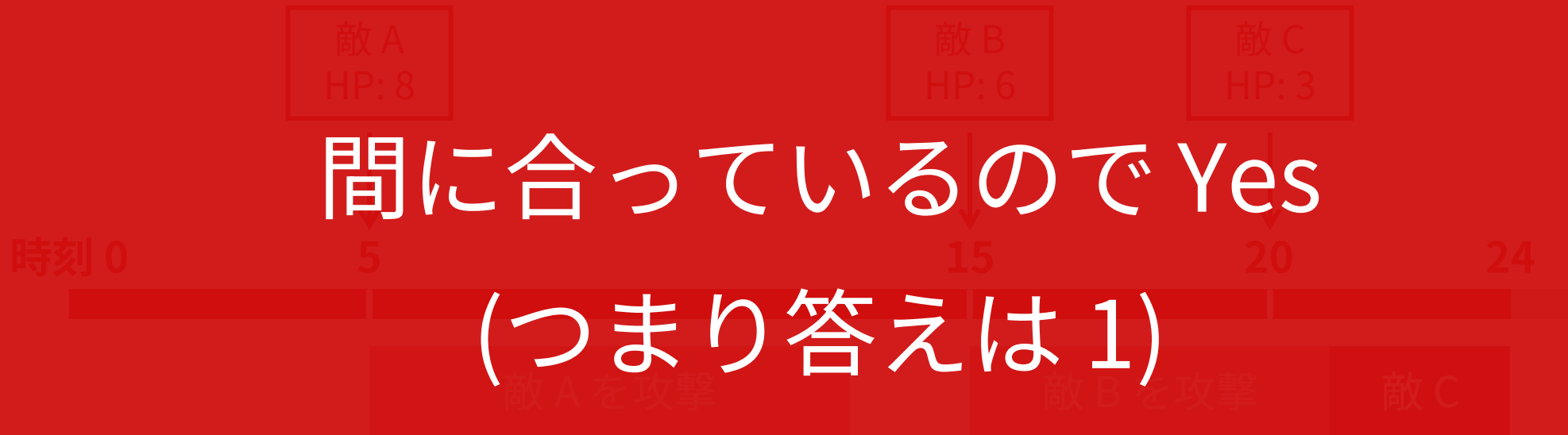
小課題 1: 例 1



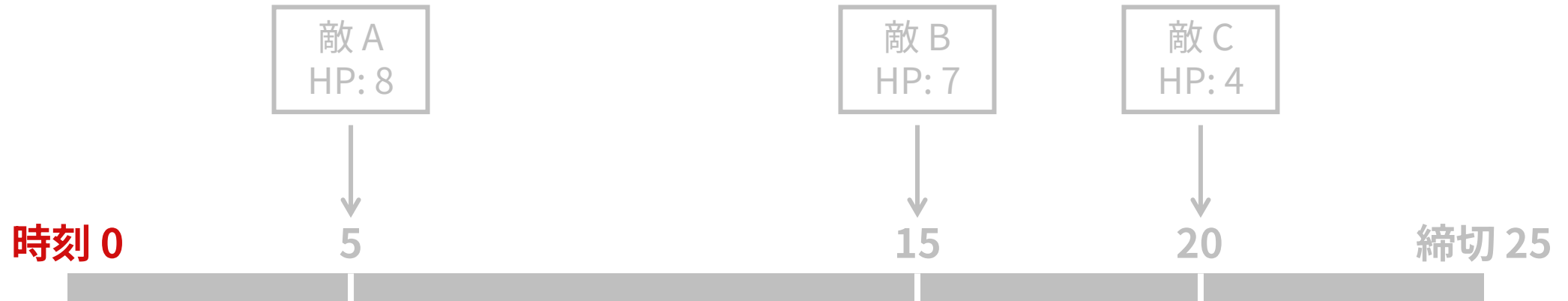
小課題 1: 例 1



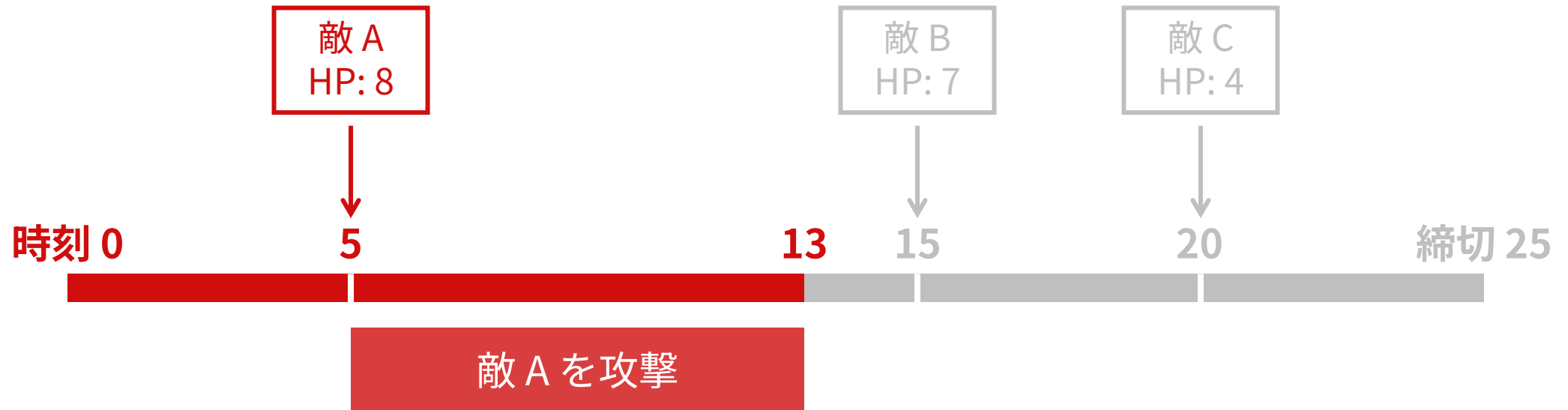
小課題 1: 例 1



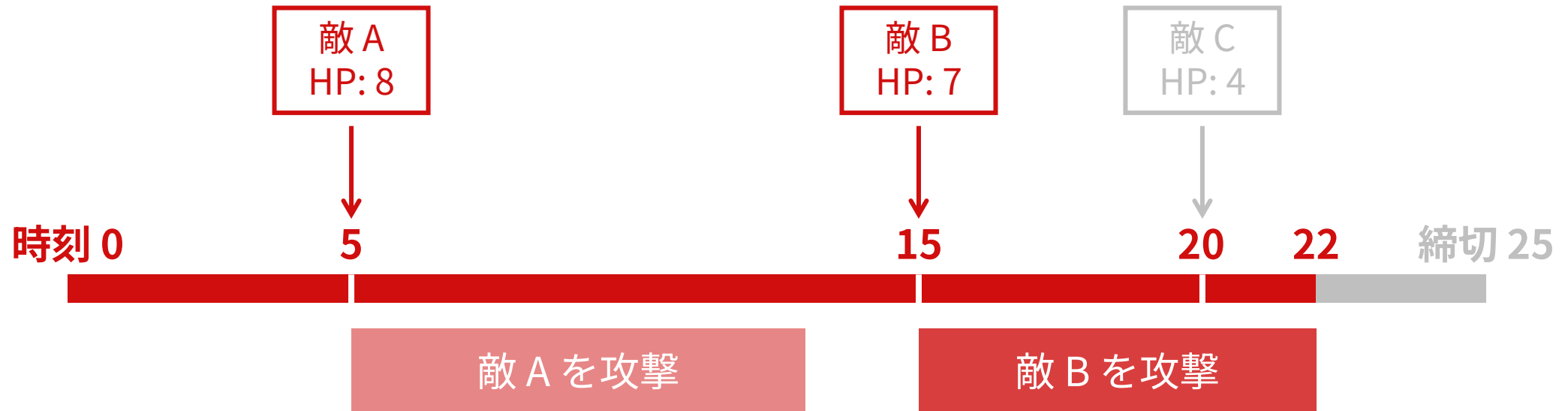
小課題 1: 例 2



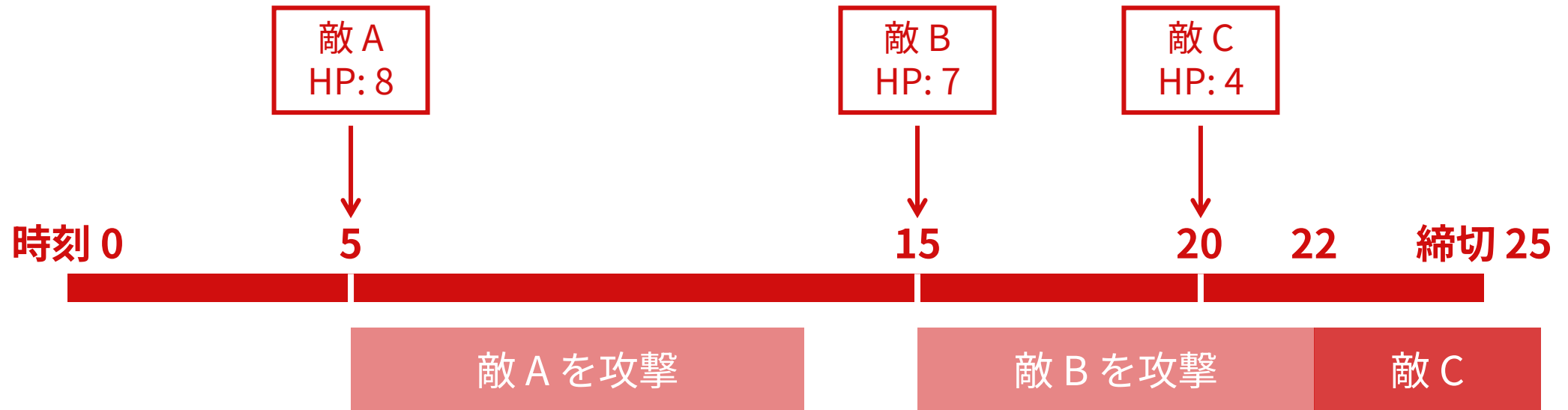
小課題 1: 例 2



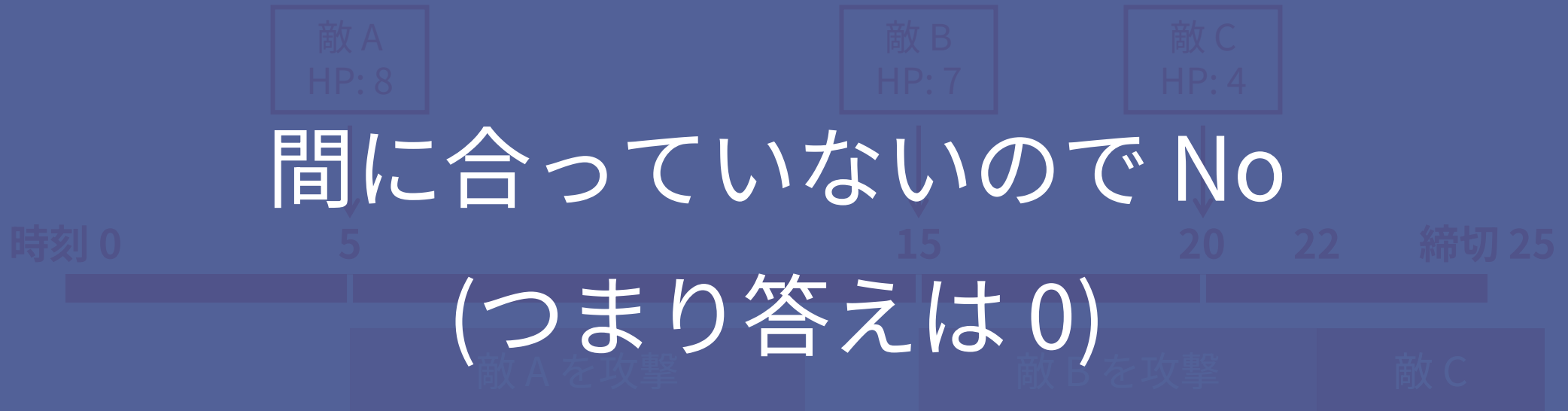
小課題 1: 例 2



小課題 1: 例 2



小課題 1: 例 2



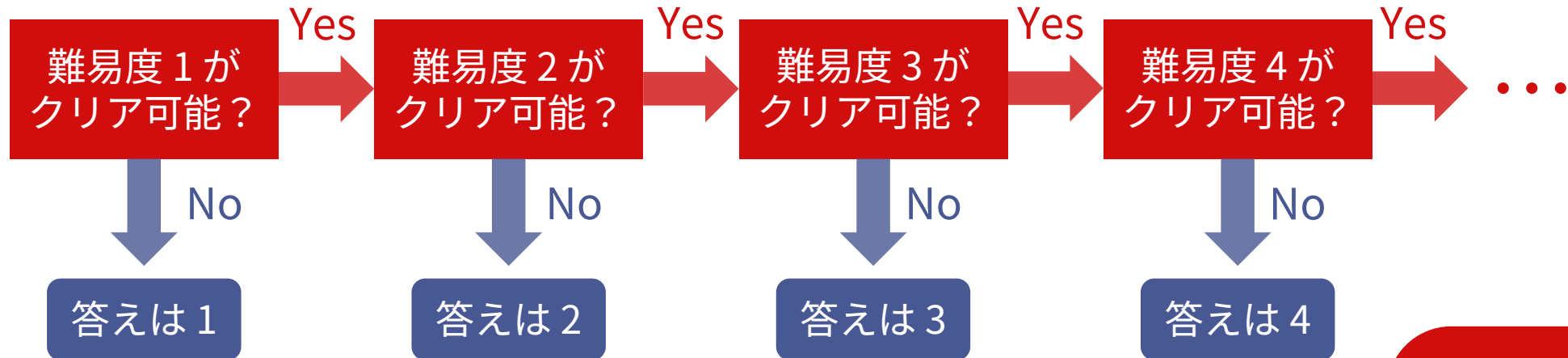
小課題 2

小課題 2

- $N \leq 30, Q = 1, M_1 = 0$
- つまり全敵の HP をゼロにできる最大の難易度を求めればよい
- 難易度 $L \leq 10^7$

解法 1: 全探索

以下のように全探索をすると、計算量が $O(NL)$ になる。制約が $N \leq 30, L = 10^7$ なので通る

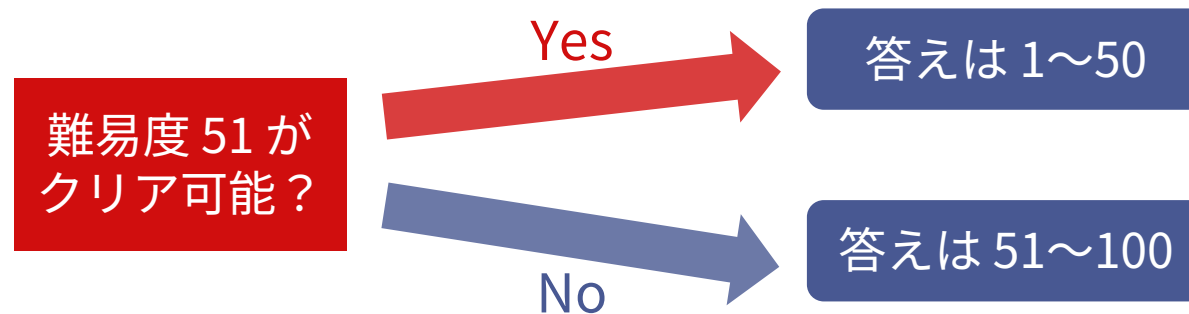


4 点

解法 2: 二分探索

しかし、二分探索をすると $O(N \log L)$ に改善される

二分探索のアイデアとしては、たとえば $L = 100$ として、最初に難易度 51 がクリア可能かどうかを聞く（それで選択肢が半分になるので、それを繰り返す）



4 点

小課題 3

小課題 3

- $N \leq 30, Q \leq 3$
- これまであった $M_i = 0$ の制約が消えているので、全部の敵を必ずしも倒す必要はない

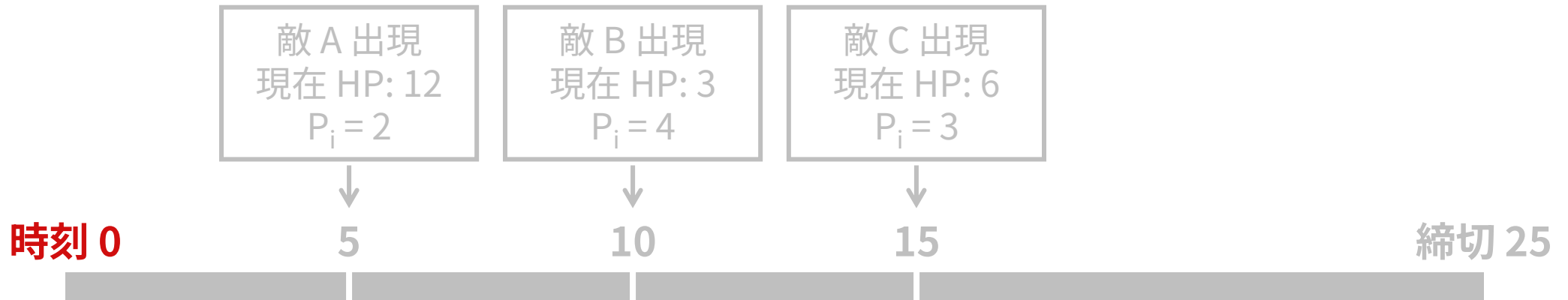
ある難易度 l をクリアするための
最適な戦略を考えてみよう

最小化したいのは (最終 HP) $\times P_i$ の合計

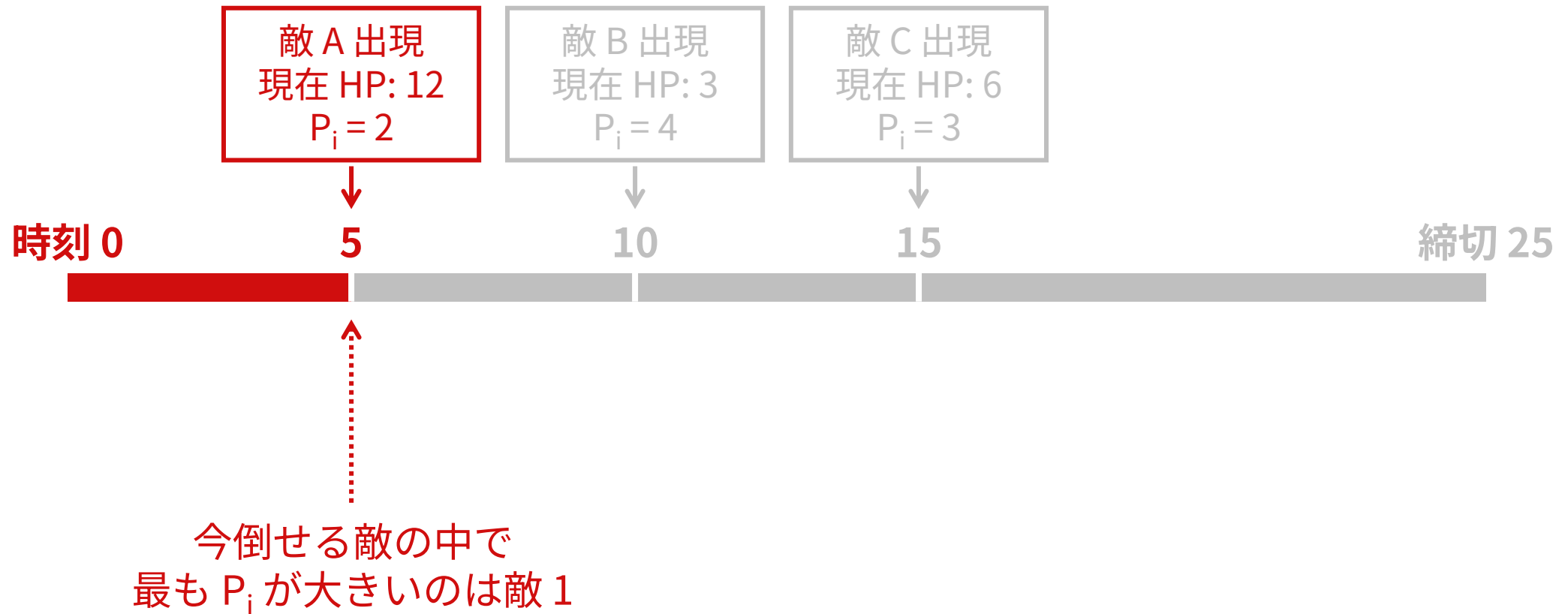


今攻撃できる敵のうち最も P_i が大きいものを
攻撃すればよいのではないか？

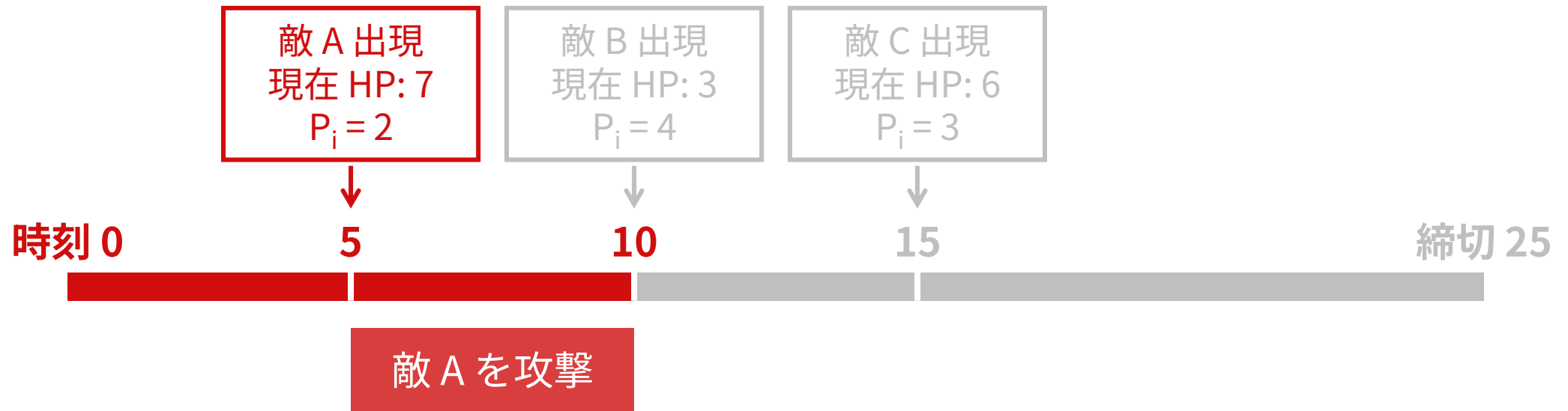
最適戦略: 例



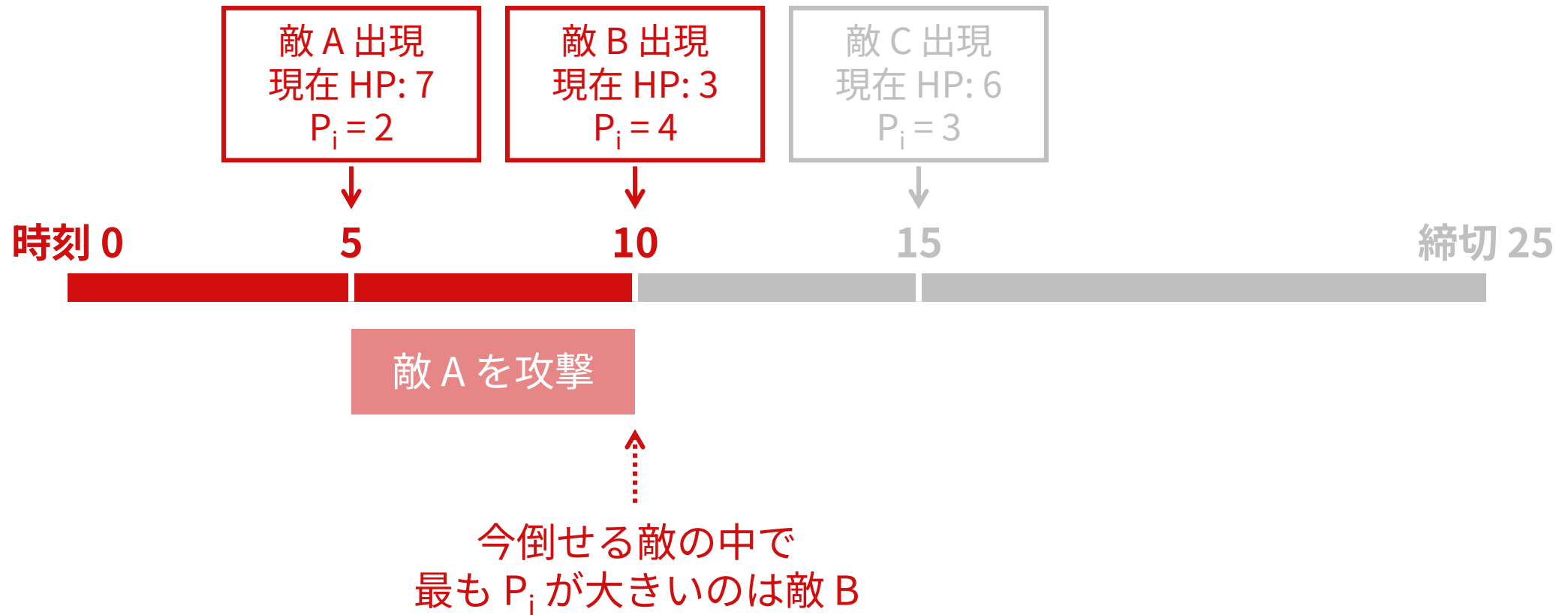
最適戦略: 例



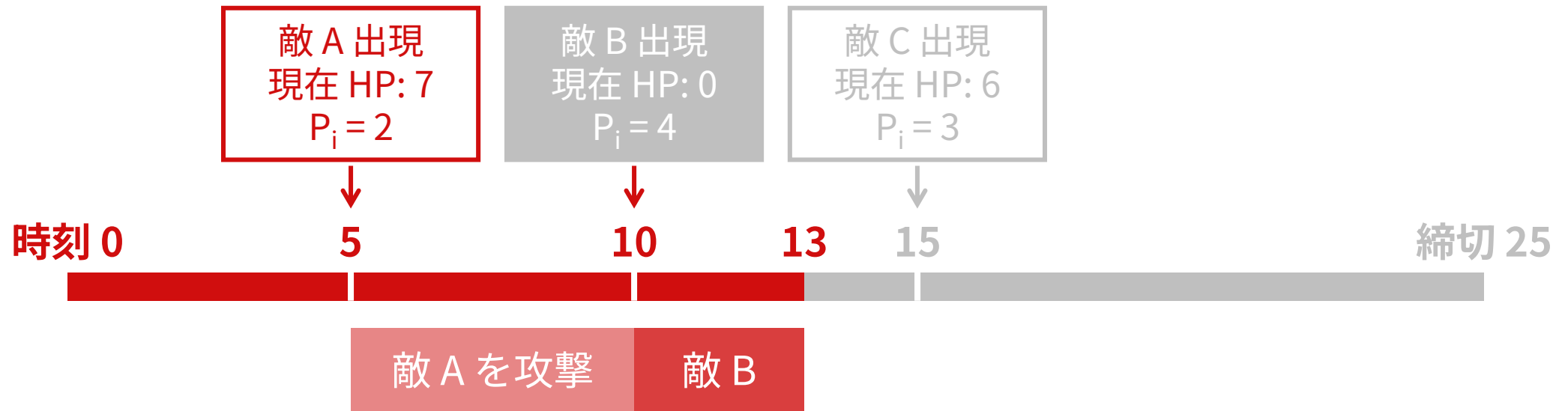
最適戦略: 例



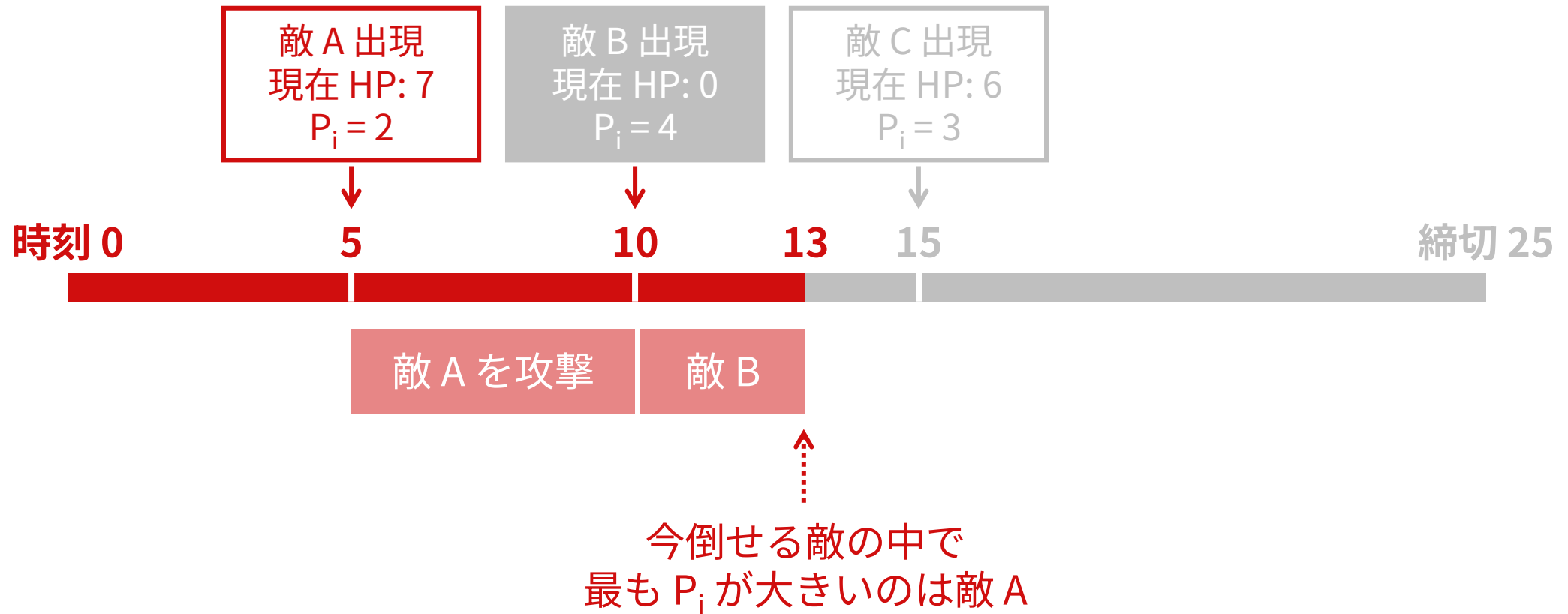
最適戦略: 例



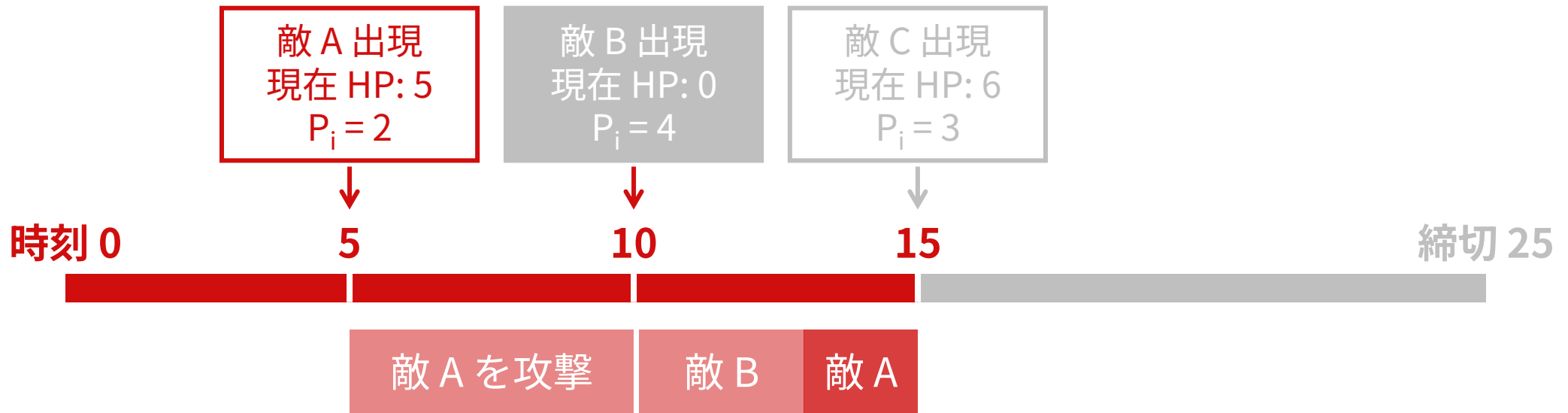
最適戦略: 例



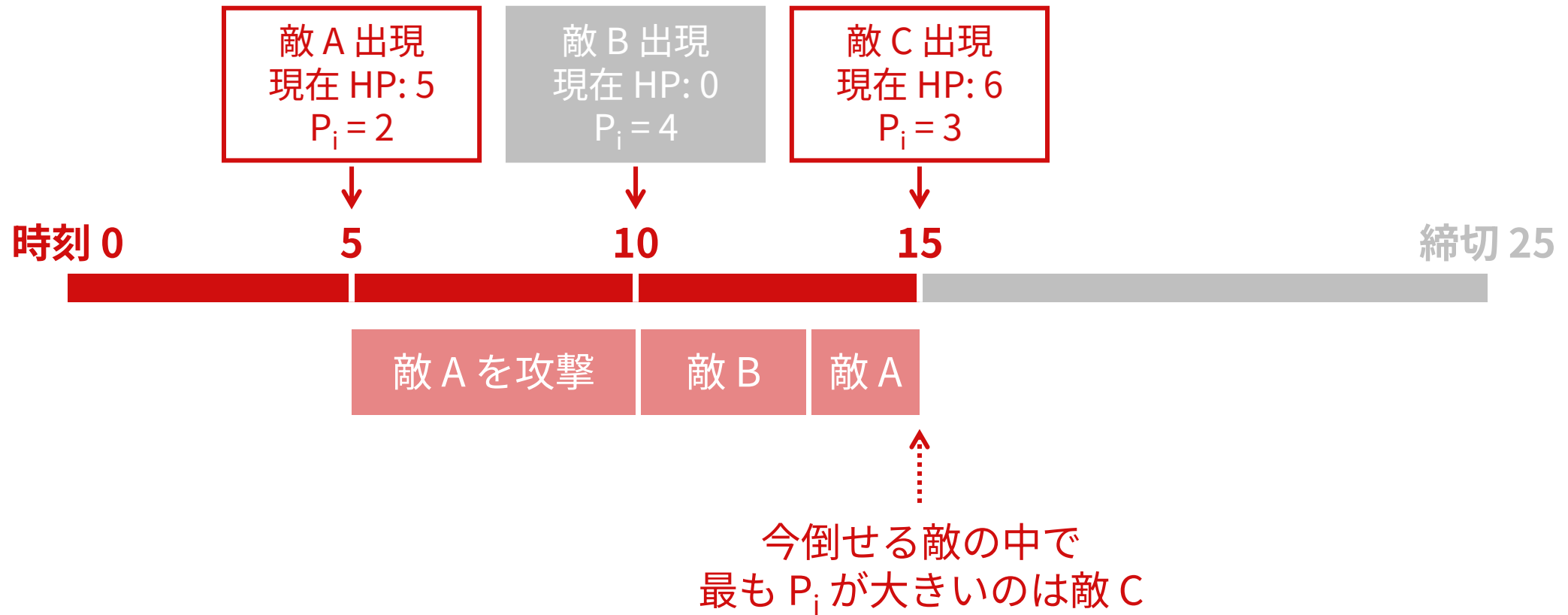
最適戦略: 例



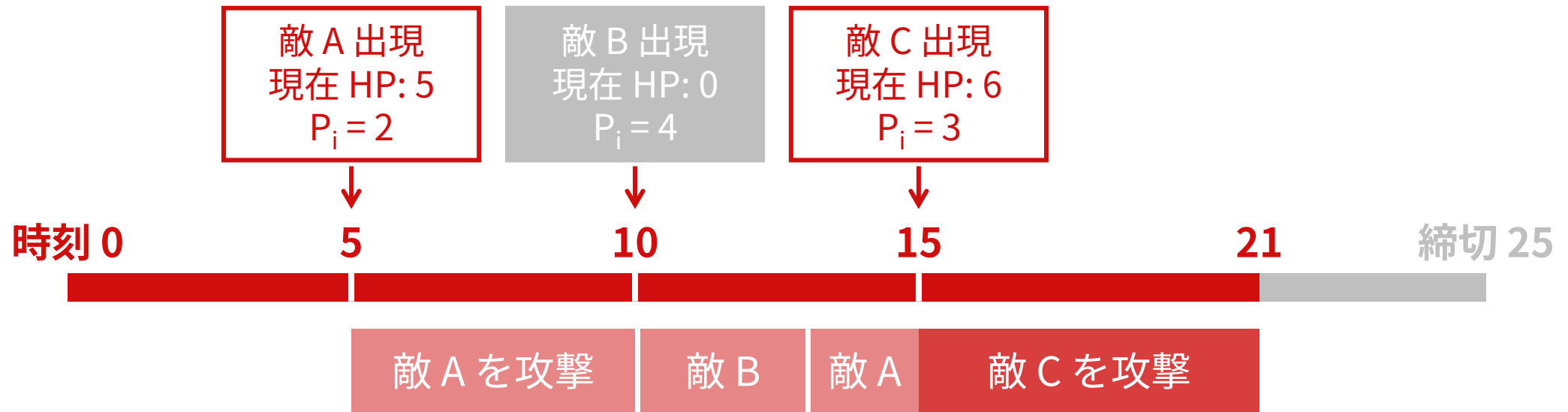
最適戦略: 例



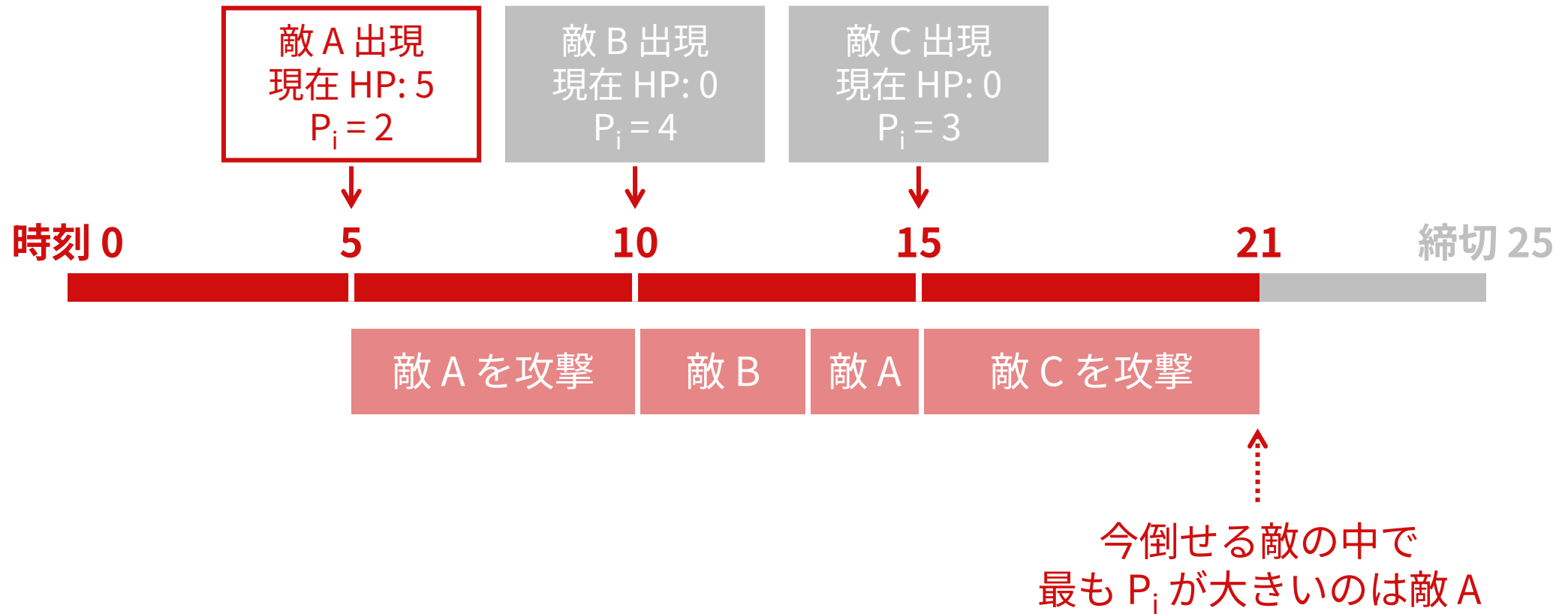
最適戦略: 例



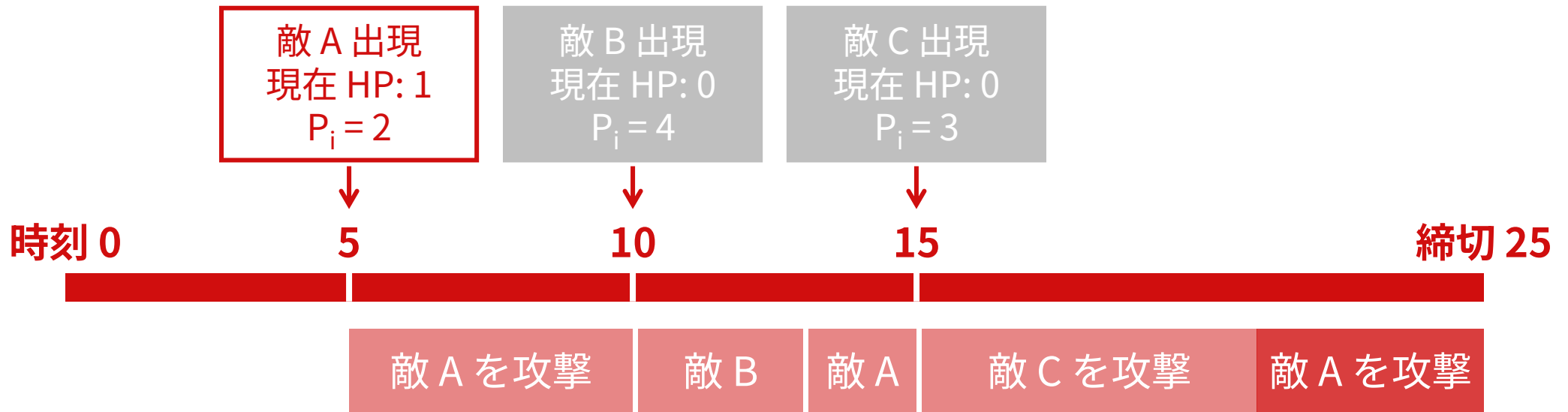
最適戦略: 例



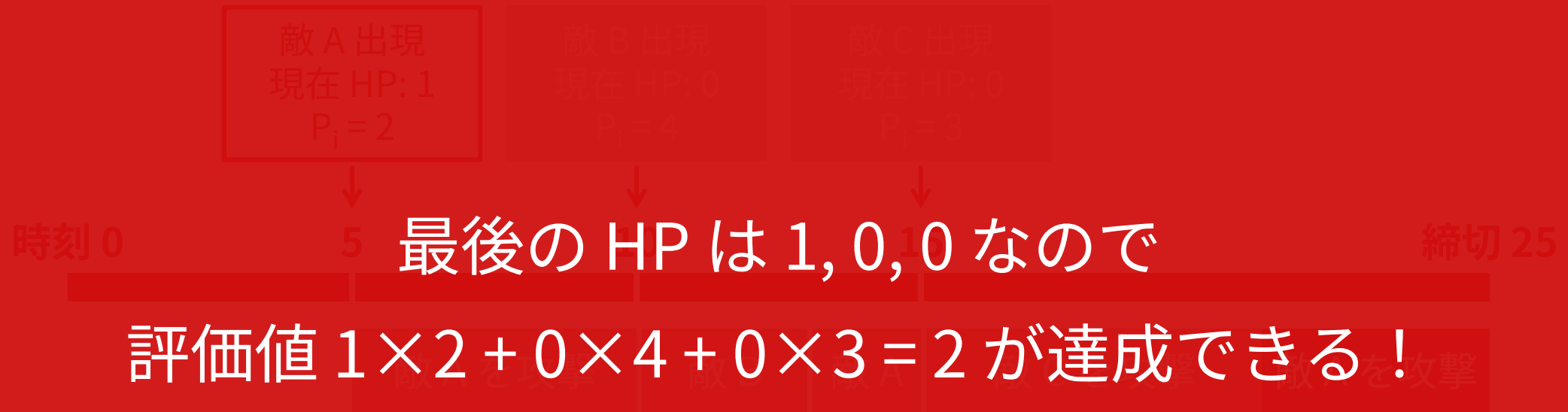
最適戦略: 例



最適戦略: 例



最適戦略: 例



小課題 3

攻撃する敵が変わる可能性のあるタイミングは以下

- 敵が出現したタイミング
- 敵の HP がゼロになったタイミング

合計 $2N$ 回しかなく、 P_i の最大値を求めるのに $O(N)$ かかるので各難易度について計算量は $O(QN^2)$

答えて二分探索をして $O(QN^2 \log L)$ → **小課題 3 に AC**

14 点

小課題 4

小課題 4

- $N \leq 6000, Q = 3$
- N の制約が 30 から 6000 に上がったので、小課題 3 の解法 (計算量 $O(QN^2 \log L)$) が通らない

小課題 3 の見直し

小課題 3 では P_i の最大値を求めるのに $O(N)$ かった



優先度付きキューで高速化できないか？

小課題 3 の見直し

- 優先度付きキューを使うと、計算量が $O(QN \log N \log L)$ になり
小課題 4 まで通る
- ただし、実装を考えるのがそこまで簡単ではない (※実装量自体は少ない)

24 点

小課題 5, 6

小課題 5, 6

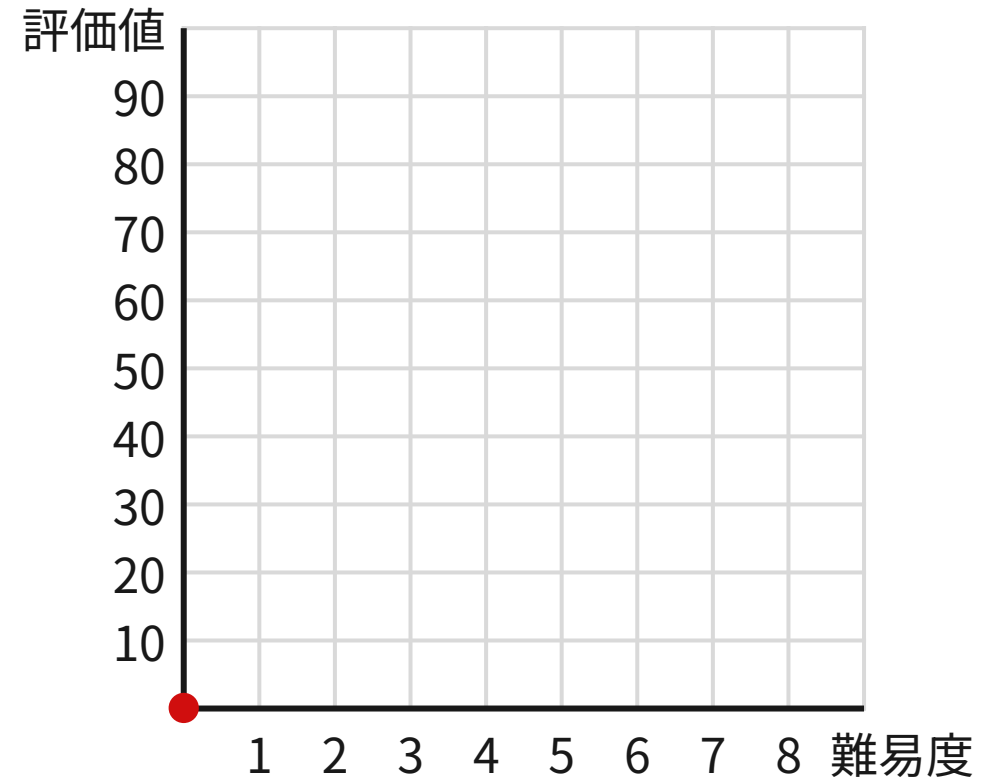
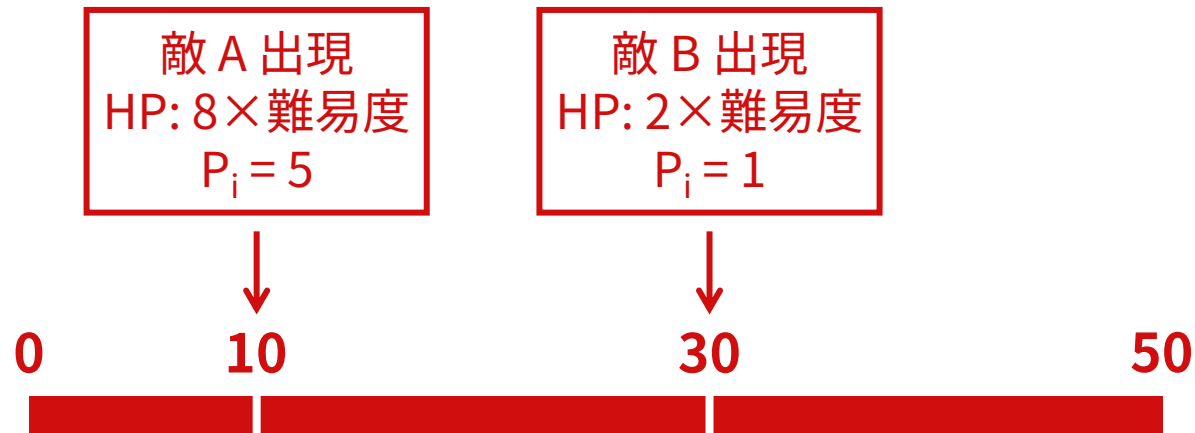
- $N \leq 30, Q \leq 1000000$
- Q に対して N が非常に小さい

重要な考察

難易度 l が増えた時に可能な評価値の最大値 $f(l)$ が
どう変化するのかを観察してみよう！

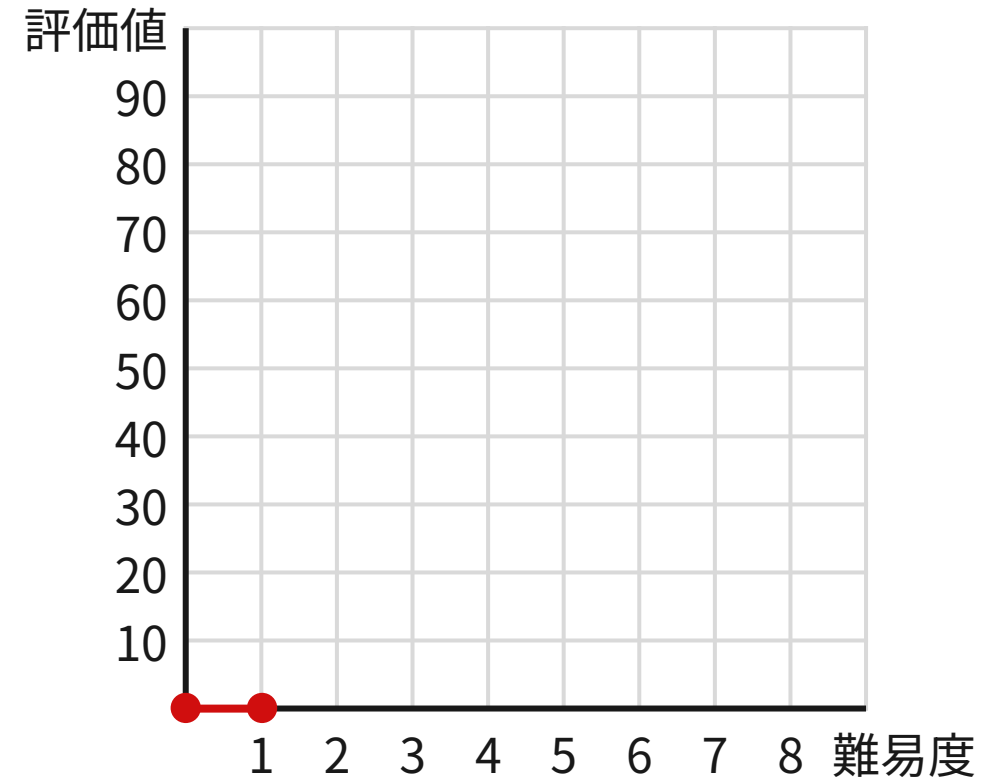
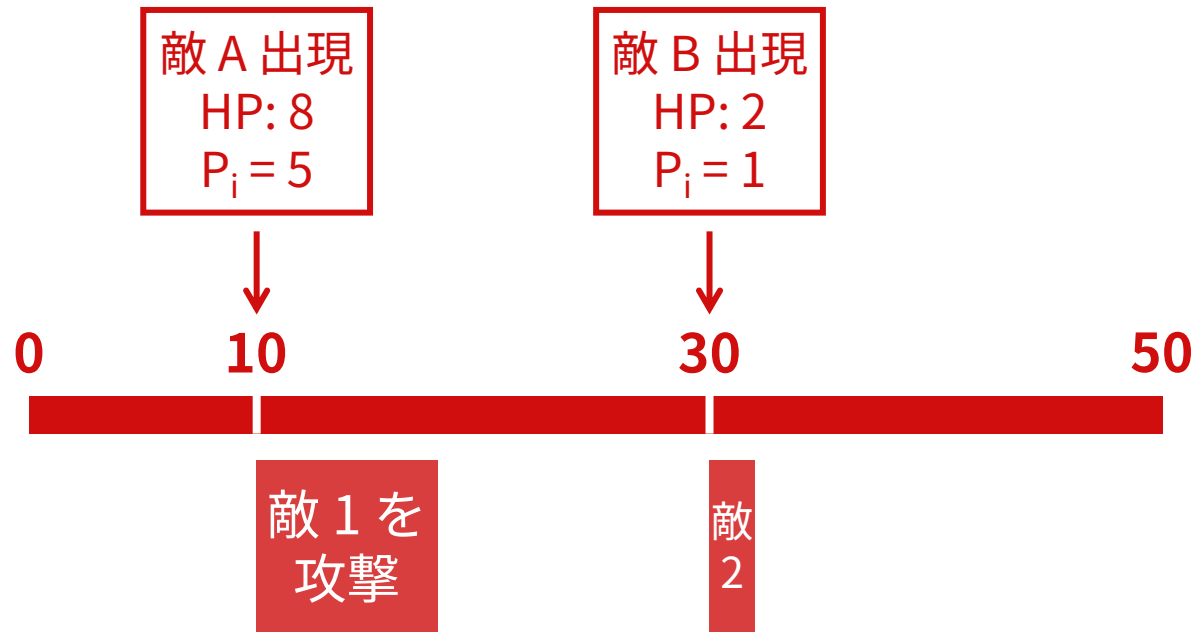
重要な考察: 例

まずは以下の簡単なケースを考える



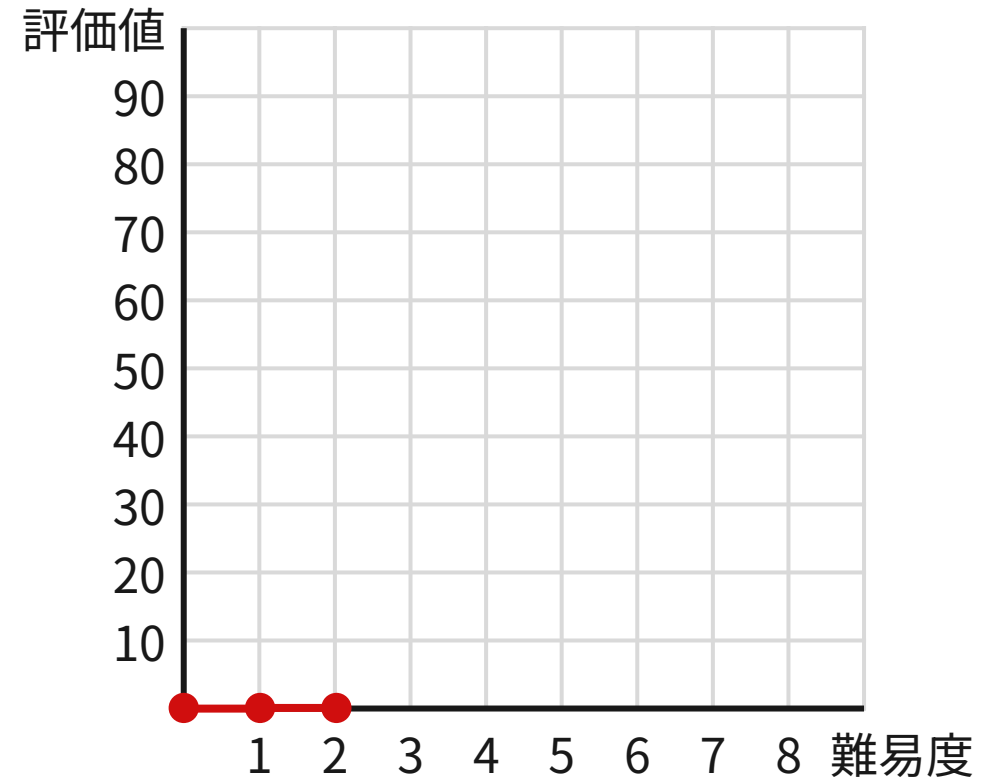
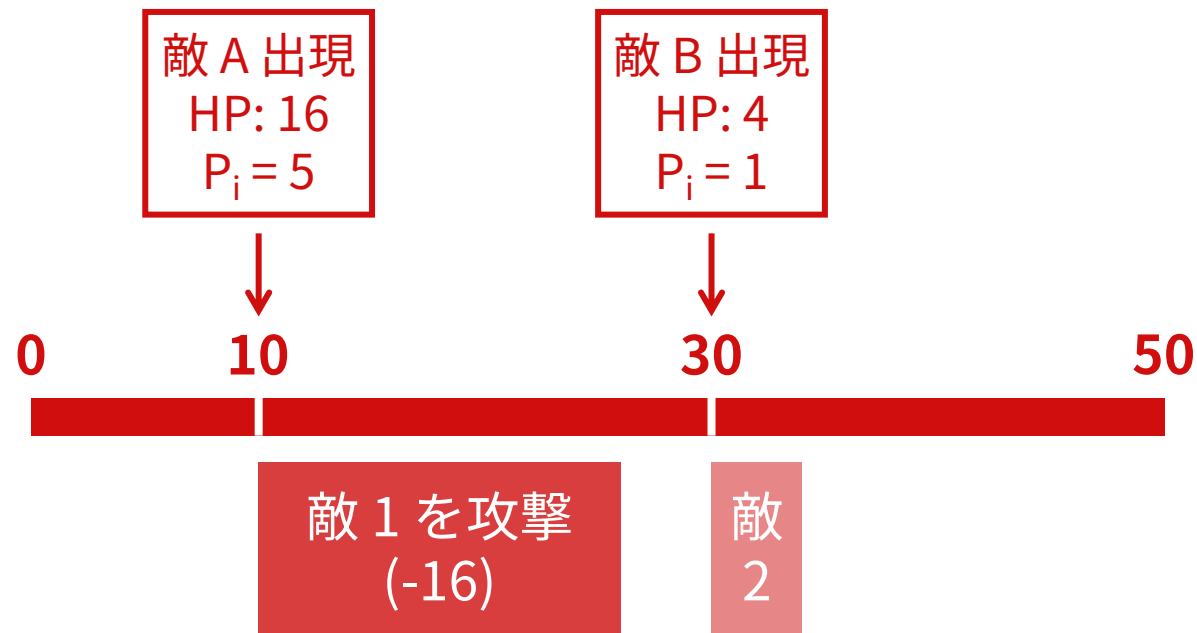
重要な考察: 例

難易度 1 の場合、評価値 0 を達成することができる



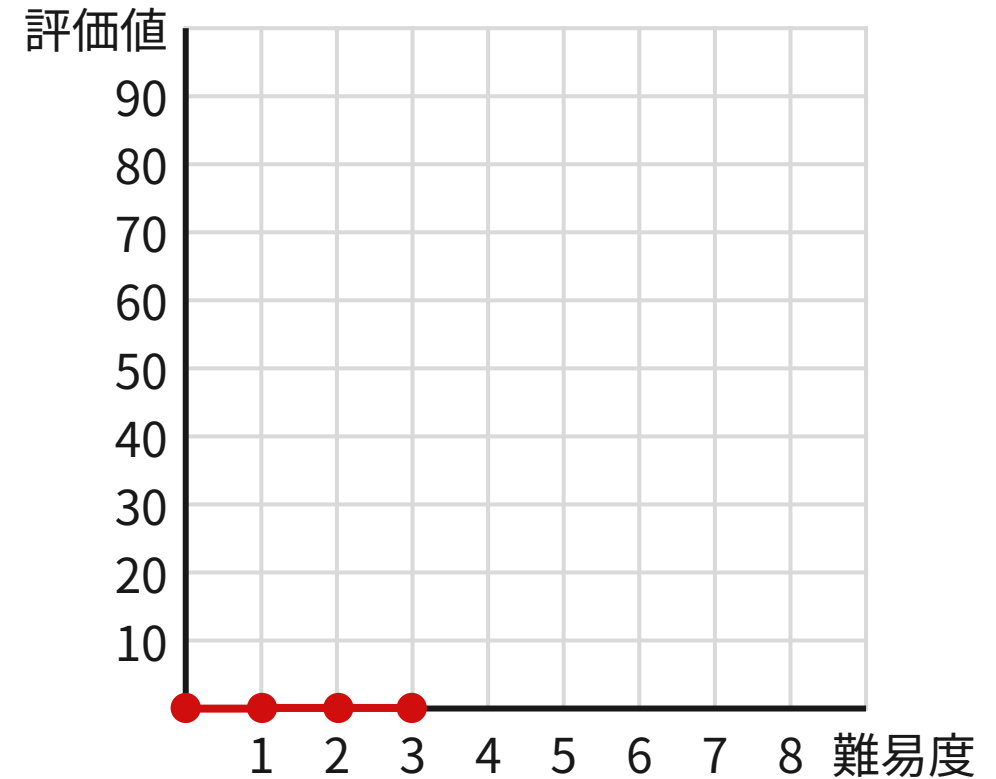
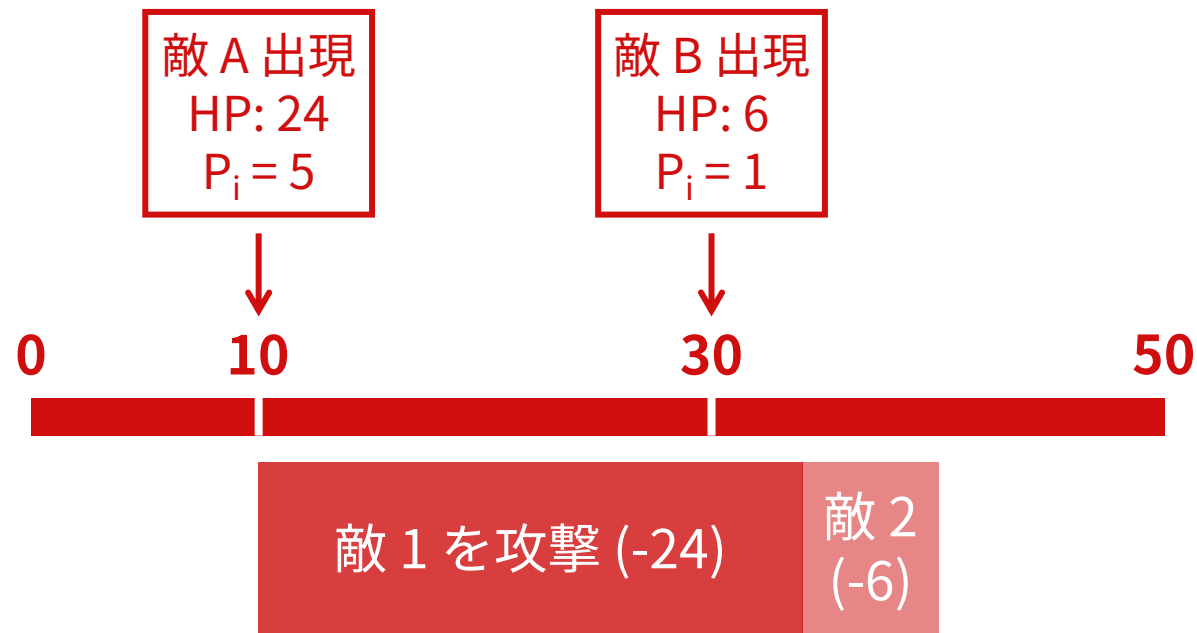
重要な考察: 例

難易度 2 の場合も、評価値 0 を達成することができる



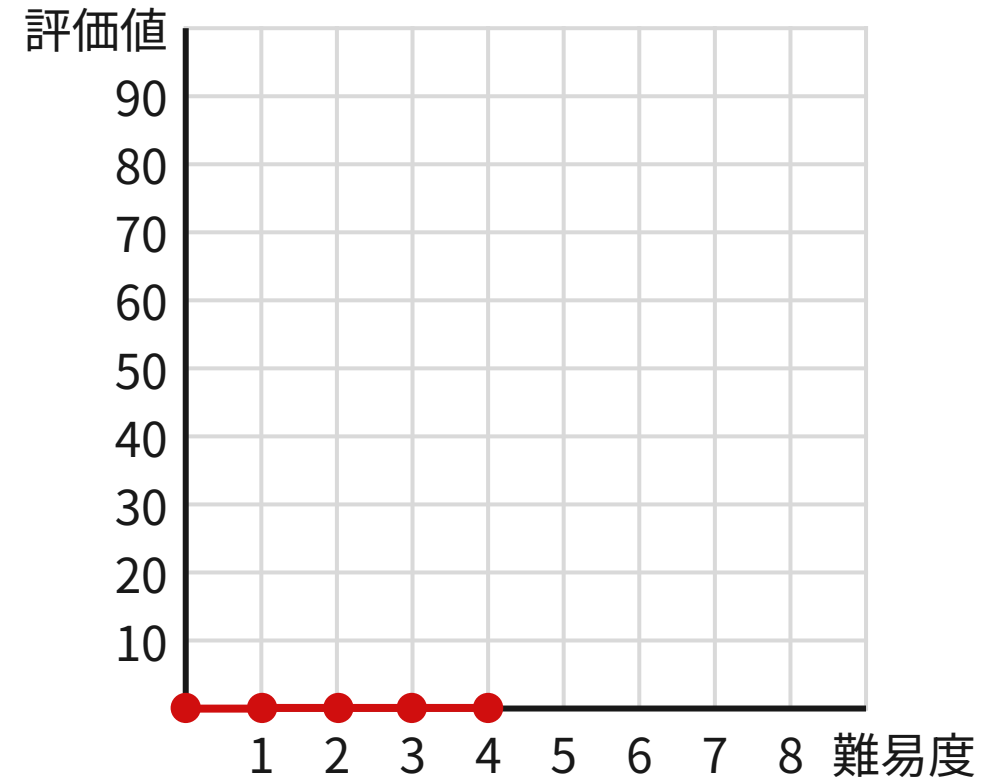
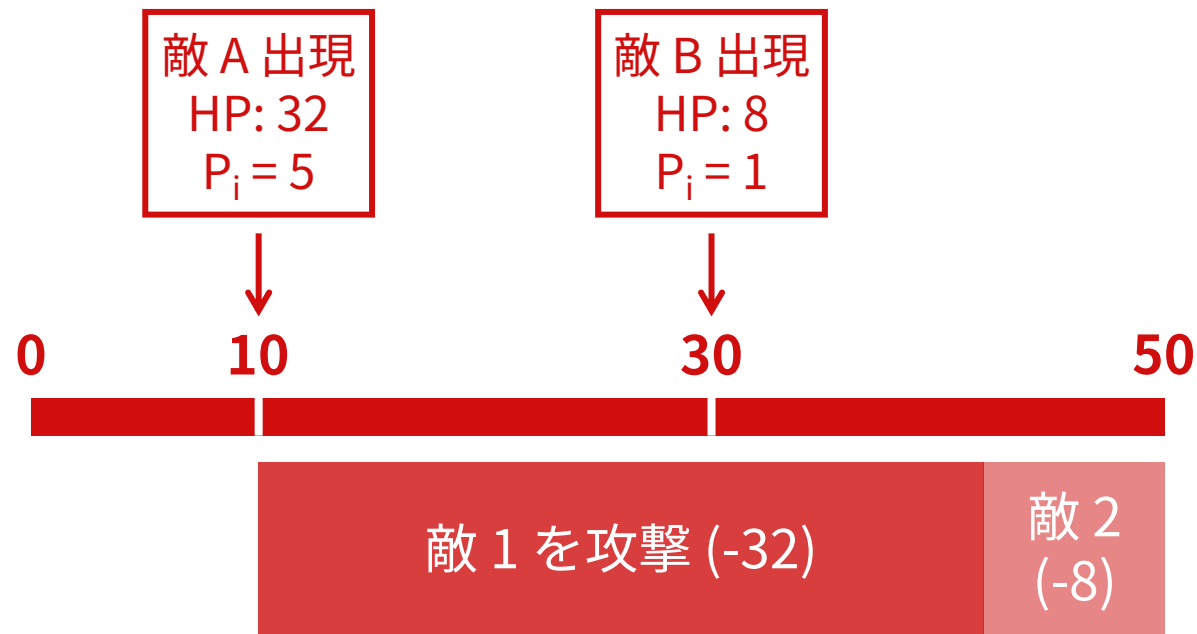
重要な考察: 例

難易度 3 の場合も、評価値 0 を達成することができる



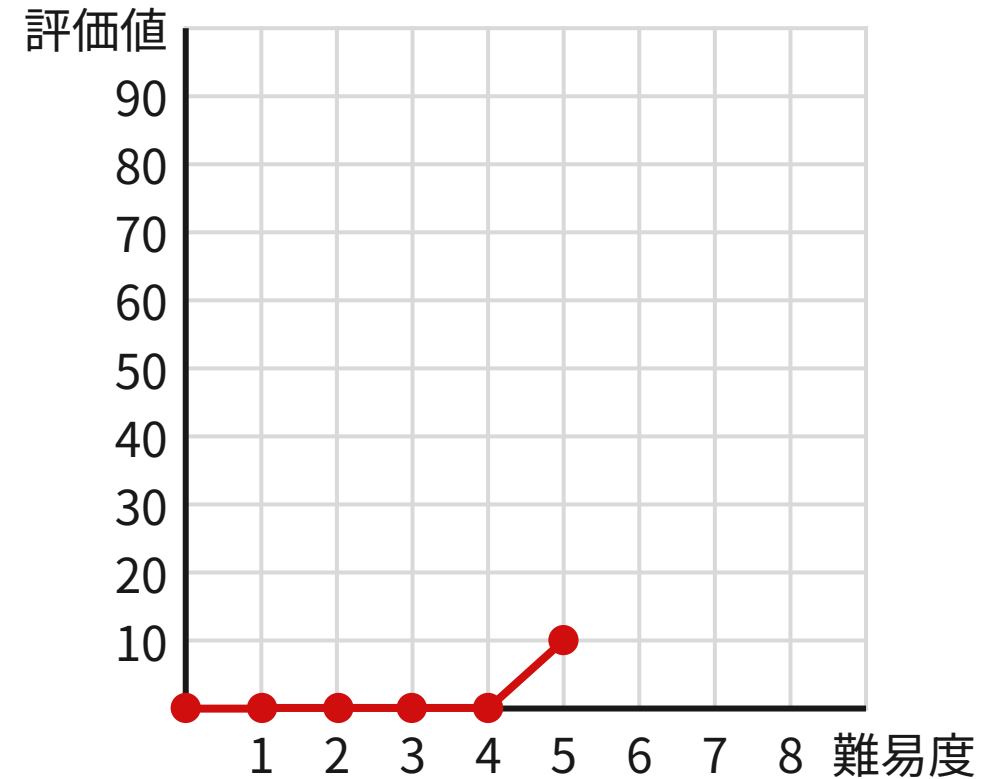
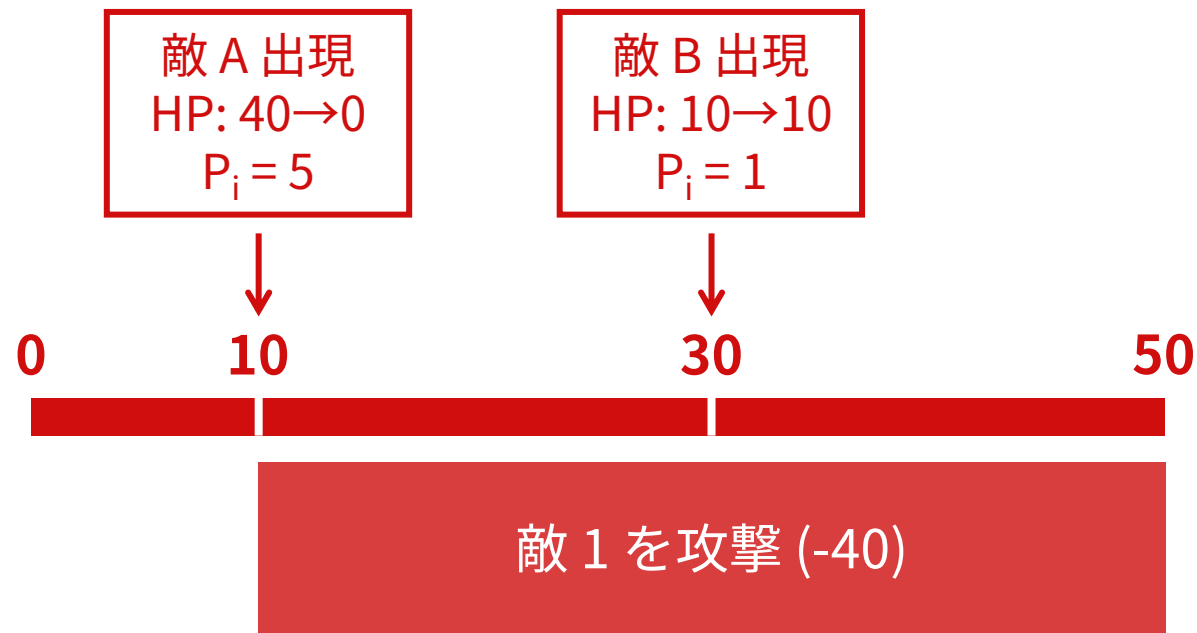
重要な考察: 例

難易度 4 の場合も、評価値 0 を達成することができる



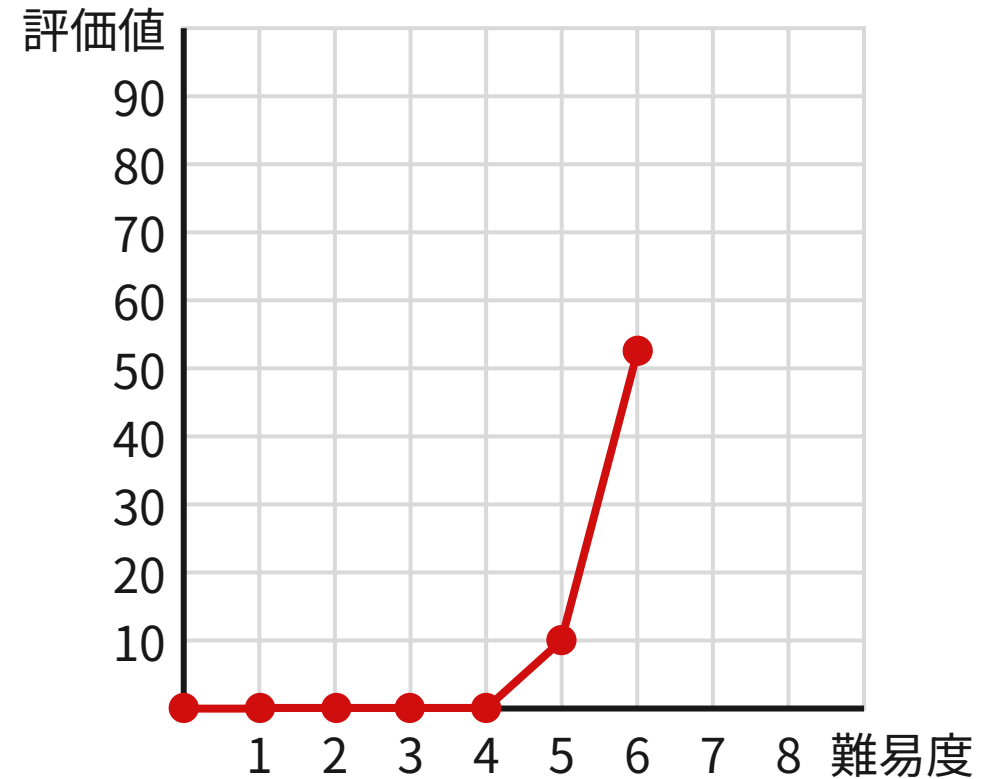
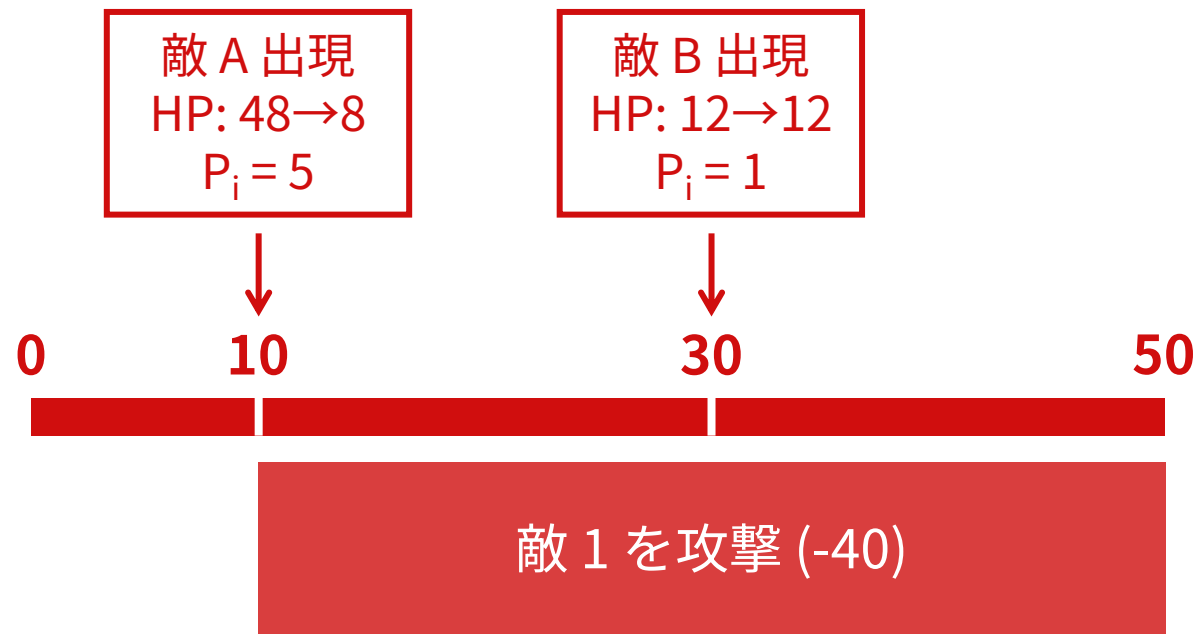
重要な考察: 例

難易度 5 の場合、評価値 10 を達成することができる



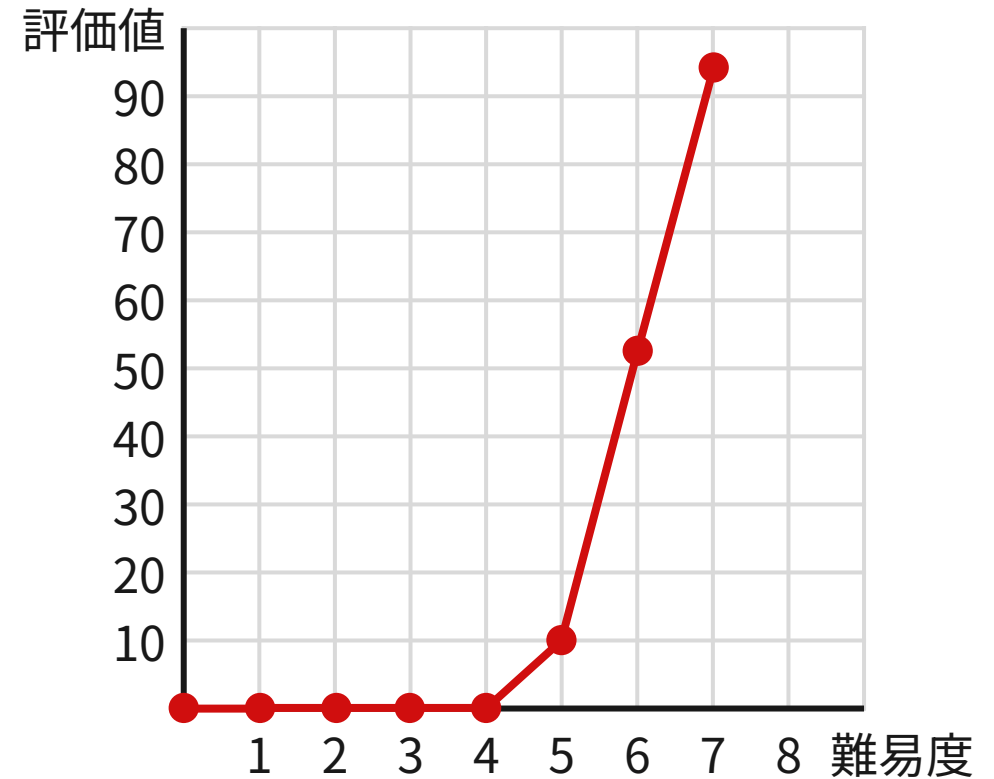
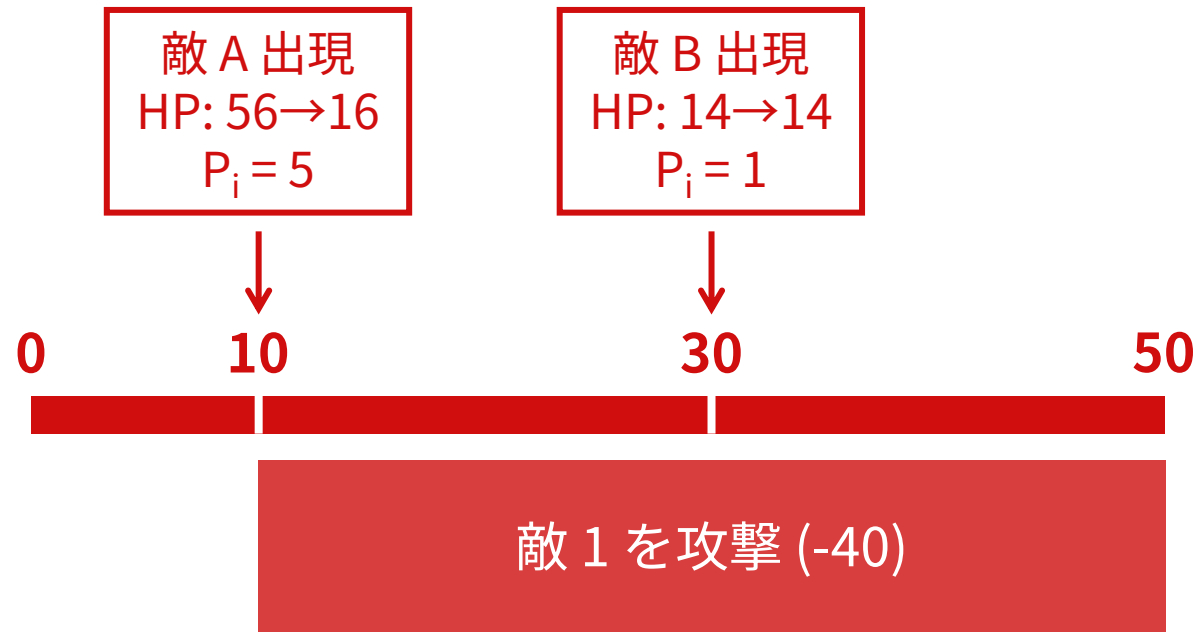
重要な考察: 例

難易度 6 の場合、評価値 $8 \times 5 + 1 \times 12 = 52$ を達成可能



重要な考察: 例

難易度 7 の場合、評価値 $16 \times 5 + 1 \times 14 = 94$ を達成可能



重要な考察: 例

難易度 7 の場合、評価値 $16 \times 5 + 1 \times 14 = 94$ を達成可能

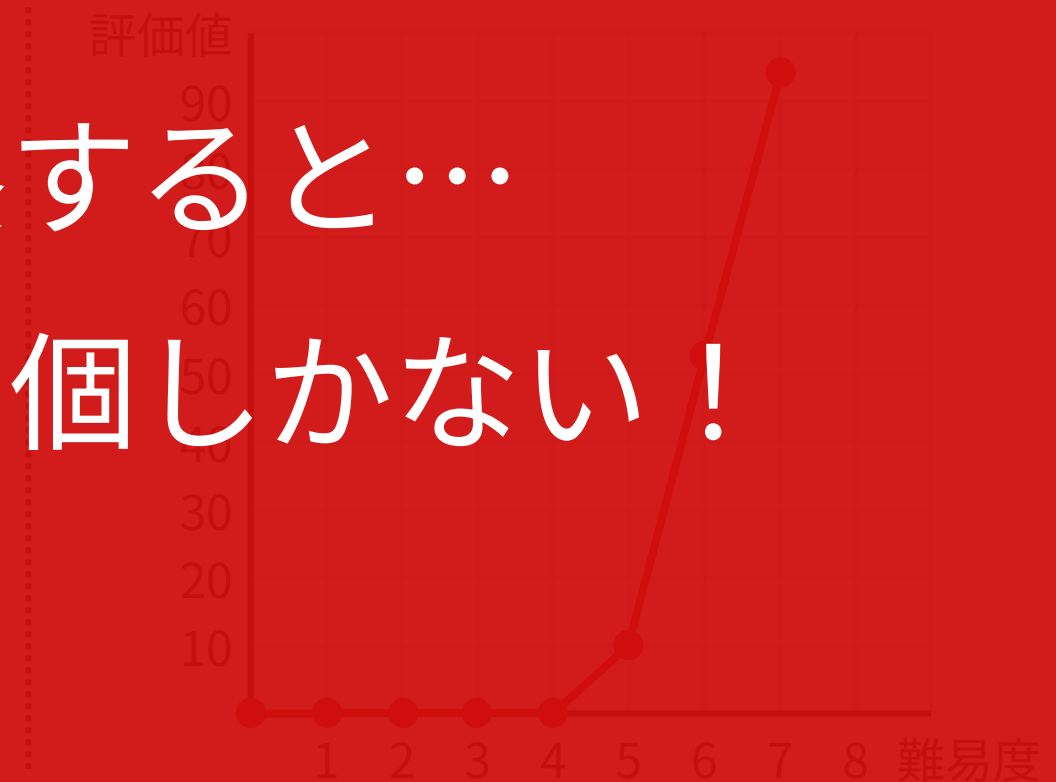
敵 A 出現
HP: 56 → 16
 $P_i = 5$

敵 B 出現
HP: 56 → 14
 $P_i = 1$

直線をよく観察すると…

傾きが変わる点が 2 個しかない！

敵 1 を攻撃 (-40)



実はどんなケースでも傾きが変わる点が多くないのでは？

重要な考察

- 実は、傾きが変わる回数が $O(N^2)$ 以下であることが証明できる (証明は満点解法の考察を参照)
- つまり、傾きが変わる点を上手く求めれば $N \leq 30$ に正解することができそう

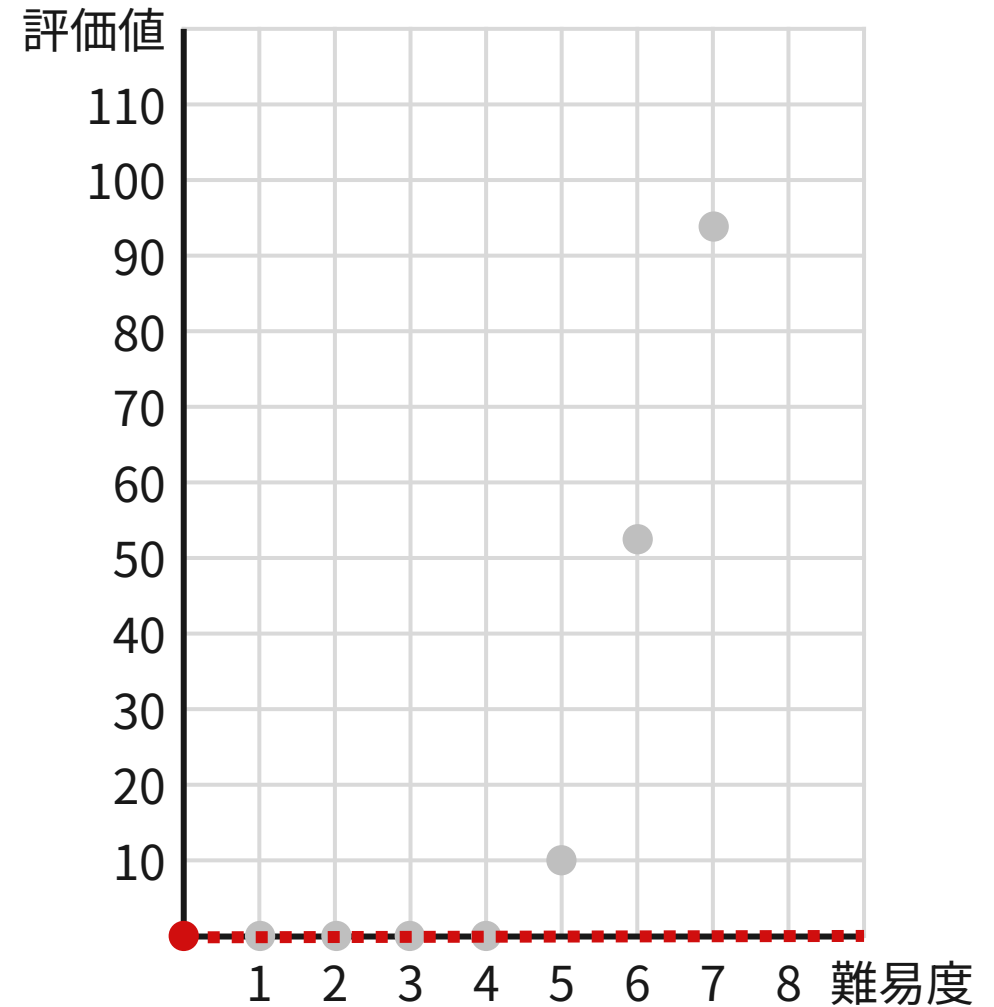
 **傾きが変わる点をどう求めるか？**

重要な考察

実は、グラフは凸※なので
二分探索できる

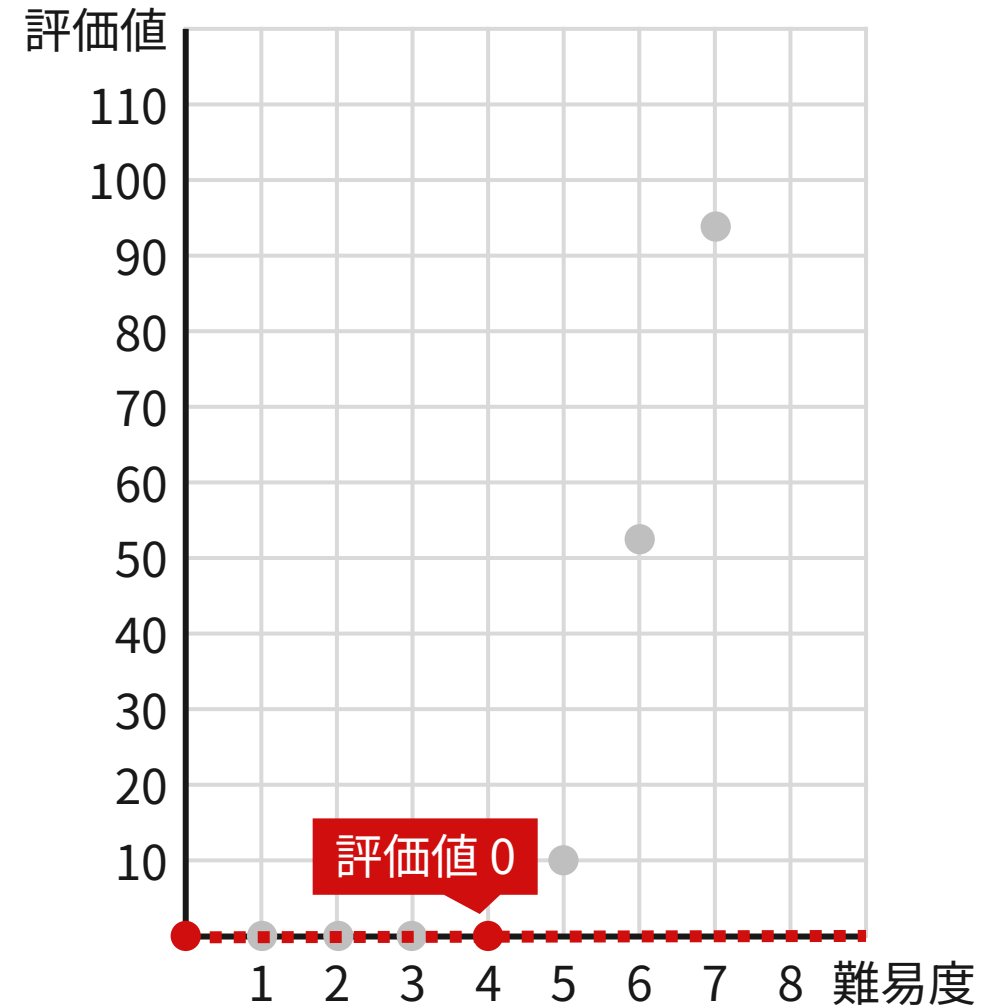
二分探索の例

- 最初の傾きは 0
- 次に傾きが変わる点はいつか？



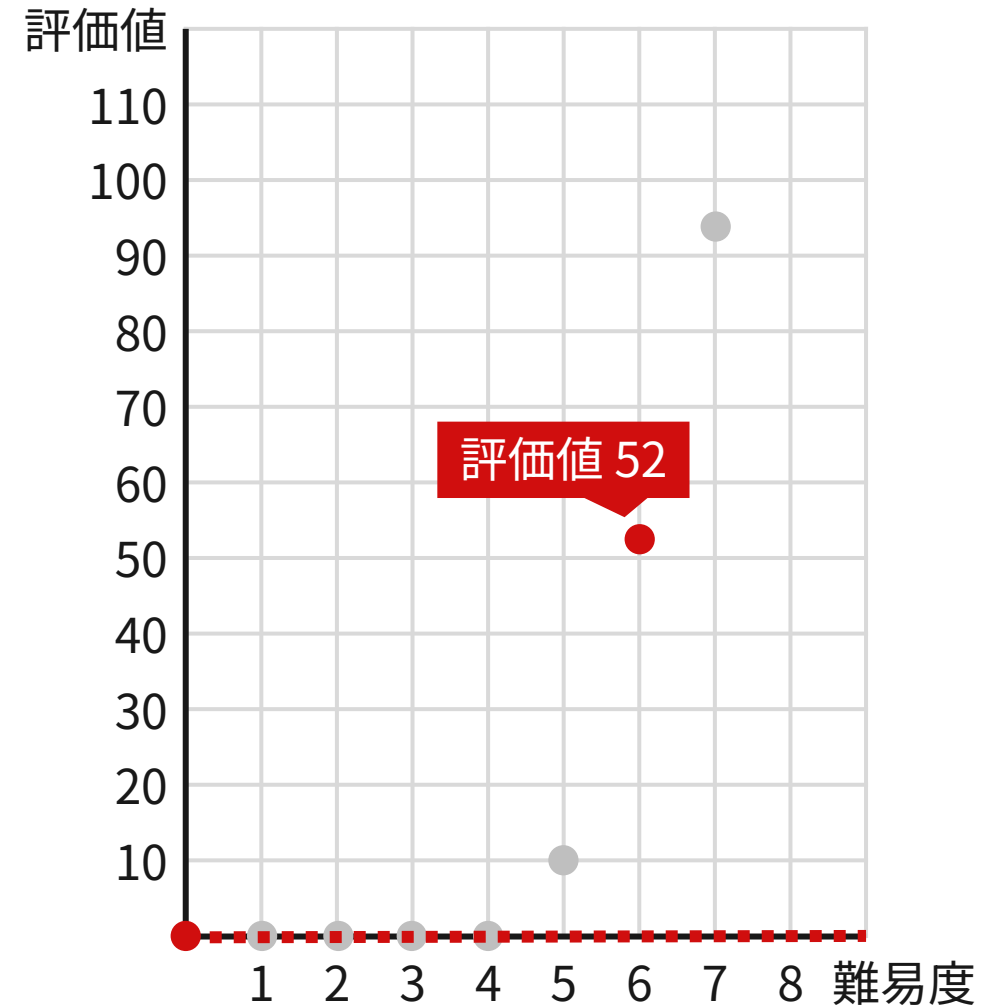
二分探索の例

- 最初の傾きは 0
- 次に傾きが変わる点はあるか？
 - 難易度 4 では変わらない



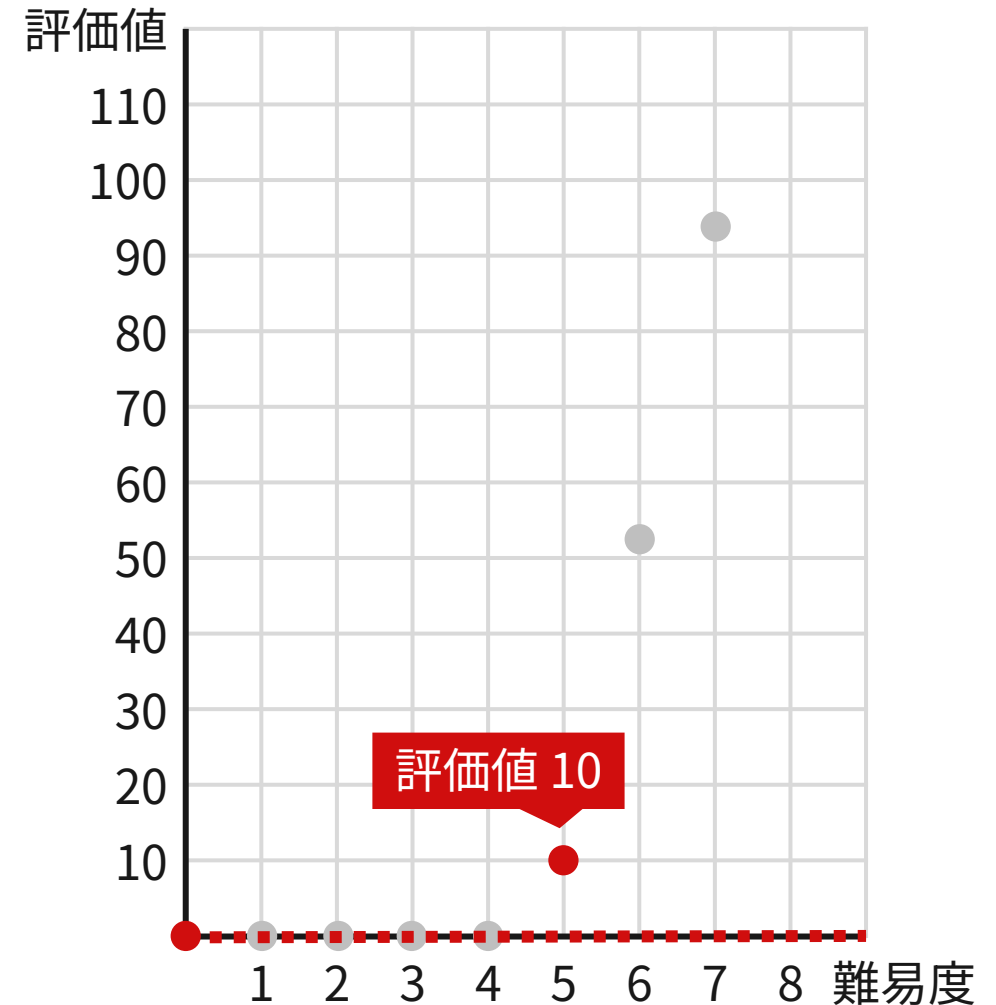
二分探索の例

- 最初の傾きは 0
- 次に傾きが変わる点はいつか？
 - 難易度 4 では変わらない
 - 難易度 6 では変わる



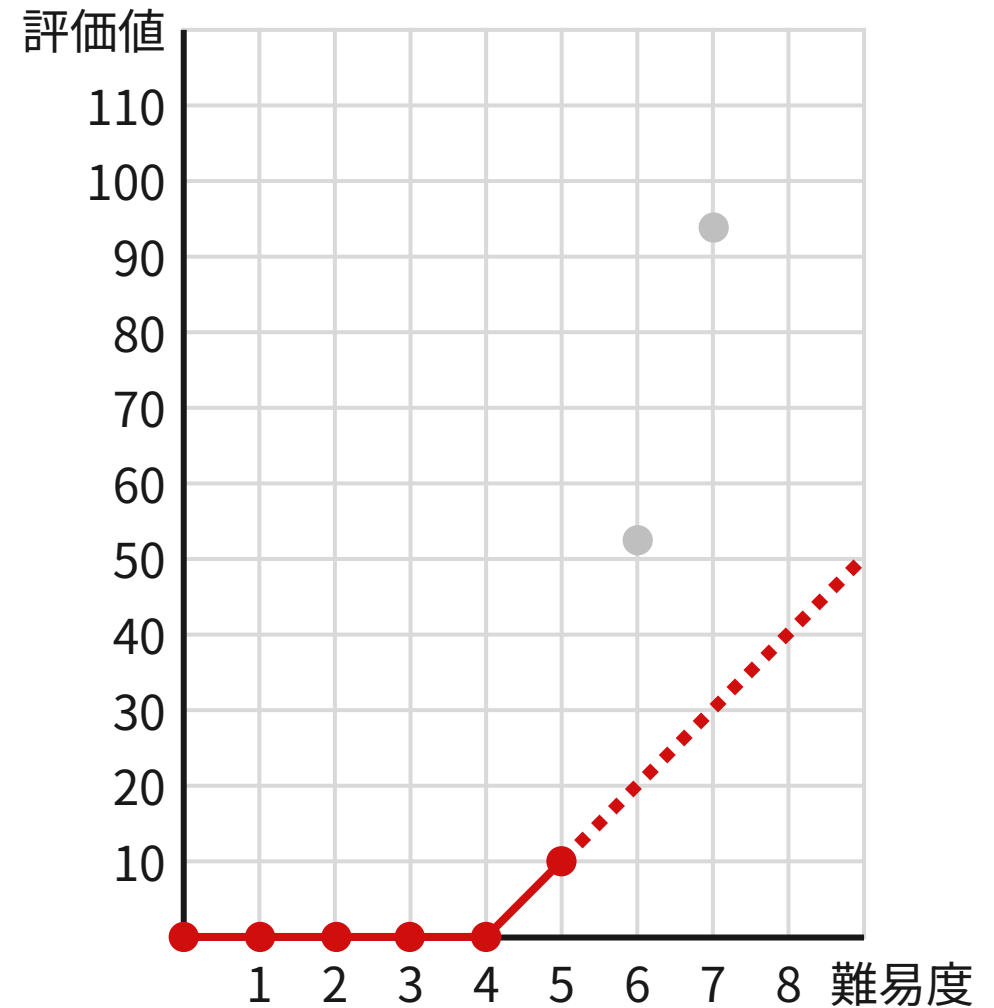
二分探索の例

- 最初の傾きは 0
- 次に傾きが変わる点はいつか？
 - 難易度 4 では変わらない
 - 難易度 6 では変わる
 - 難易度 5 でも変わる
 - → 次に傾きが変わるのは難易度 5



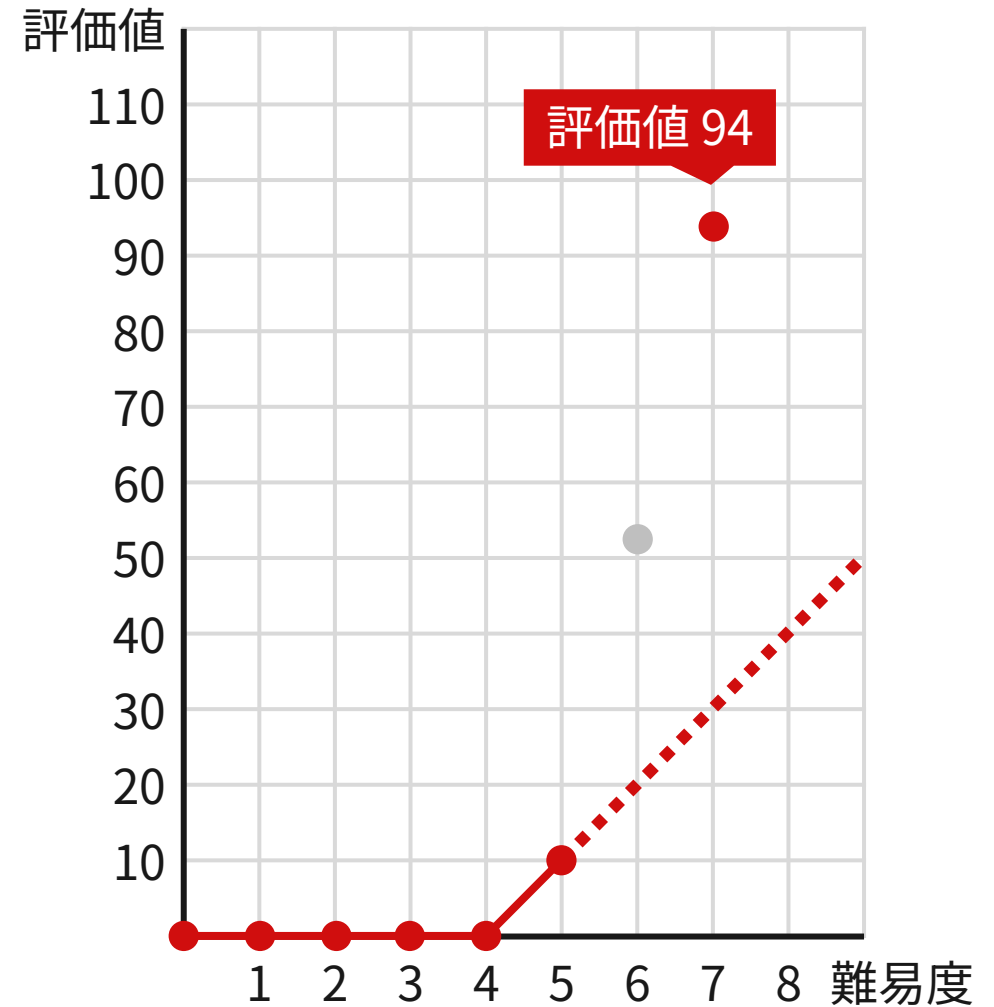
二分探索の例

- 現在の傾きは 10
- 次に傾きが変わる点はあるか？



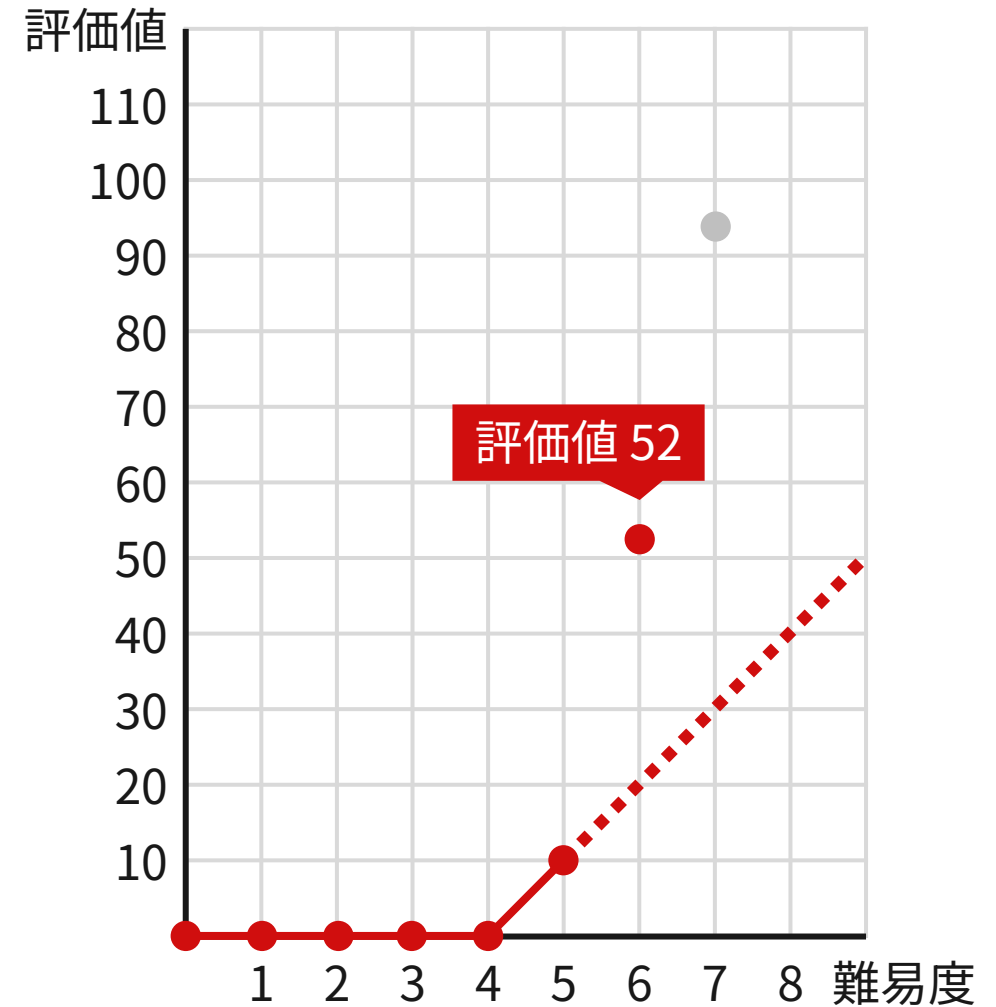
二分探索の例

- 現在の傾きは 10
- 次に傾きが変わる点はあるか？
 - 難易度 7 では変わる



二分探索の例

- 現在の傾きは 10
- 次に傾きが変わる点はいつか？
 - 難易度 7 では変わる
 - 難易度 6 でも変わる
 - → 次に傾きが変わるのは難易度 6



小課題 5

- このように二分探索をすると、合計で $O(N^2 \log L)$ 個の難易度について※ 最善評価値を求めればよい
- 最善評価値の計算に $O(N \log N)$ かかるので、全体で $O(N^3 \log N \log L)$ かかる
- $N \leq 30$ では間に合う

59 点

※傾きが変わる点が $O(N^2)$ 個であるため

- 実は傾きが変わる点は $O(N)$ 個なのではないか？
- 証明はできていないが、解説担当者はそうだと思っている
- 定数倍によっては小課題 6 に通るかもしれない

67 点

小課題 7

小課題 7

- 難易度 l のときの最善評価値 $f(l)$ の関数を高速に求めたい
- もし以下の値がわかれば、関数 $f(l)$ もわかる
 - 最適な戦略で動いた時の、 P_i が 1 番目に大きいものの攻撃時間
 - 最適な戦略で動いた時の、 P_i が 2 番目に大きいものの攻撃時間
 - 最適な戦略で動いた時の、 P_i が 3 番目に大きいものの攻撃時間
 - :
- 一体どうすればこれらの値が求まるのか？

小課題 7

- 難易度 l のときの最善評価値 $f(l)$ の関数を高速に求めたい
- もし以下の値がわかれば、関数 $f(l)$ もわかる
 - 最適な戦略で動いた時の、 P_i が上位 1 個のものの合計攻撃時間
 - 最適な戦略で動いた時の、 P_i が上位 2 個のものの合計攻撃時間
 - 最適な戦略で動いた時の、 P_i が上位 3 個のものの合計攻撃時間
 - :
- 一体どうすればこれらの値が求まるのか？

重要な言い換え

最適な戦略における
 P_i が上位 k 個のものの
合計攻撃時間

=

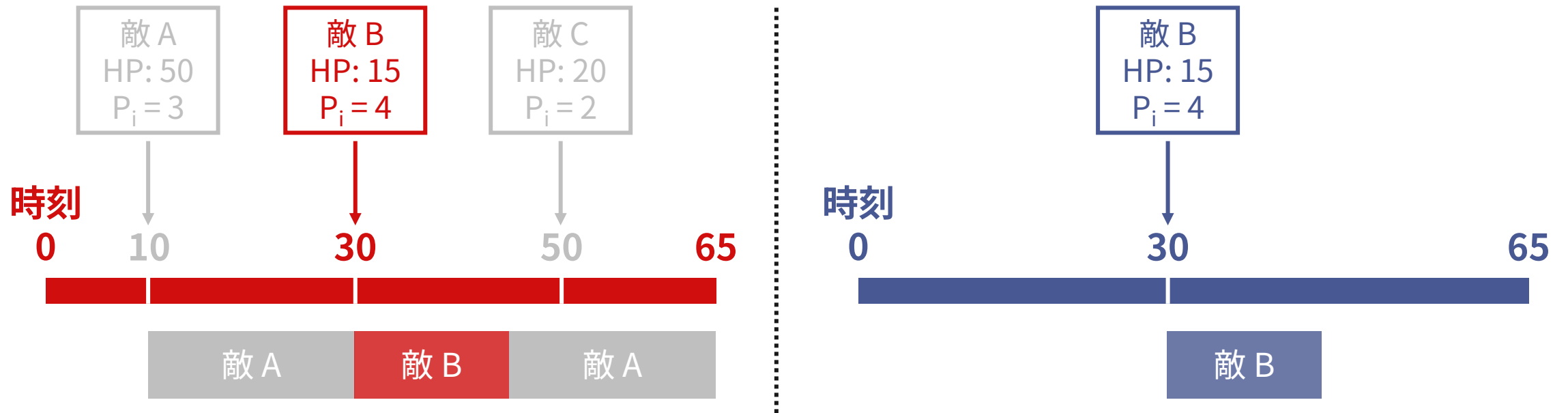
その k 体の敵だけがいるときの
(つまり $N = k$ のときの)
合計攻撃時間

重要な言い換え

よくわからないと思うので

例を説明します

重要な言い換え: 例 ($k=1$)



$N = 3$ のときの敵 B の合計攻撃時間 (15) が
敵 B だけを残したとき ($N = 1$) の合計攻撃時間 (15) と同じ

重要な言い換え

なぜそんなことが起こるのか？

重要な言い換え

なぜそんなことが起こるのか？

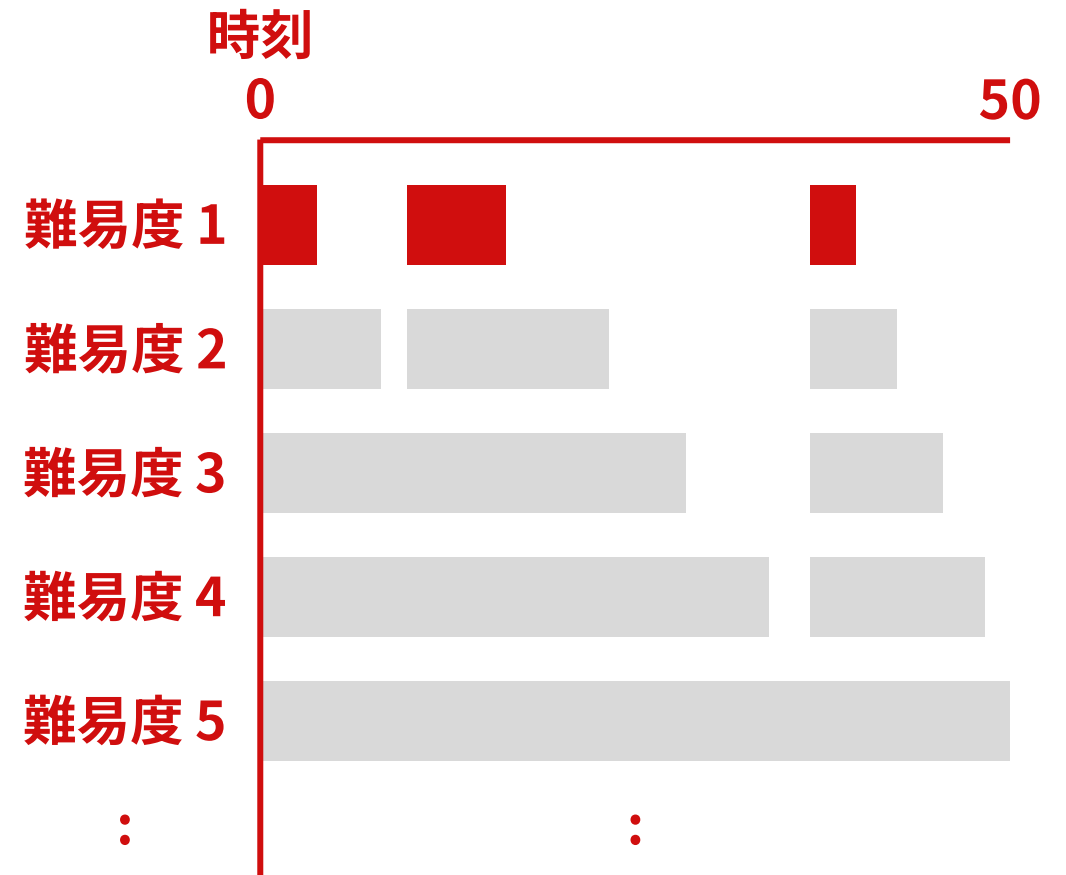


小課題 4 の貪欲法において、 P_i が上位 k 個の敵への攻撃は
他のどの敵よりも優先されるため

では、どのようにして
合計攻撃時間の関数を計算するか？

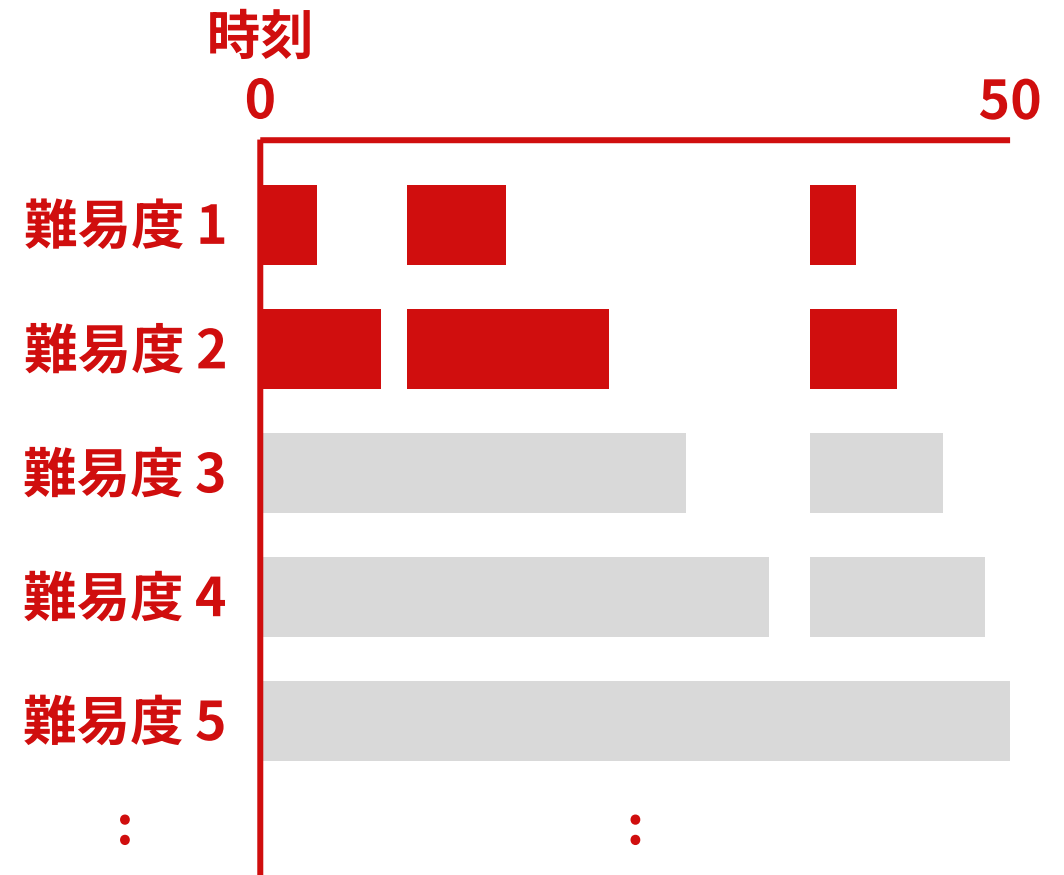
小課題 7

- これは単に、優先度付きキューで（難易度を増やしていった時の）敵の攻撃時間の衝突を右図のようにシミュレーションすればよい
- 計算量は $O(N^2 \log N)$ となり小課題 7 に通る



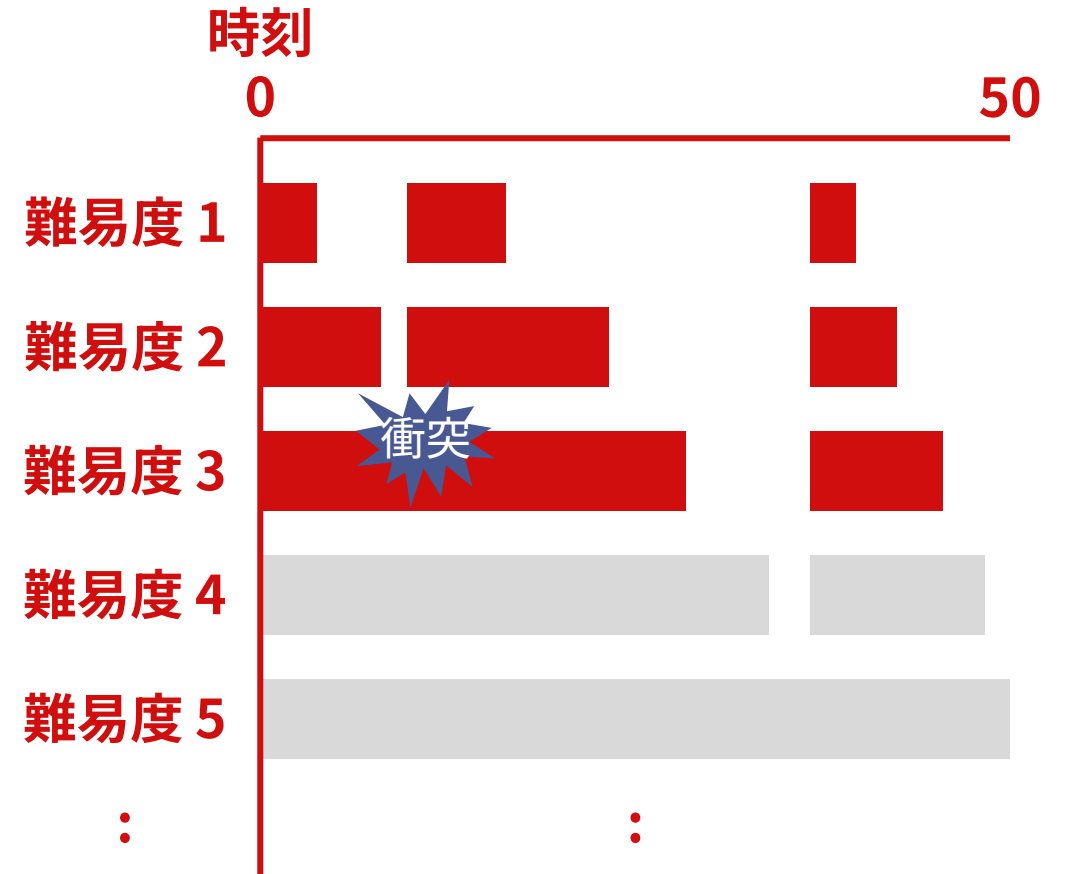
小課題 7

- これは単に、優先度付きキューで（難易度を増やしていった時の）敵の攻撃時間の衝突を右図のようにシミュレーションすればよい
- 計算量は $O(N^2 \log N)$ となり小課題 7 に通る



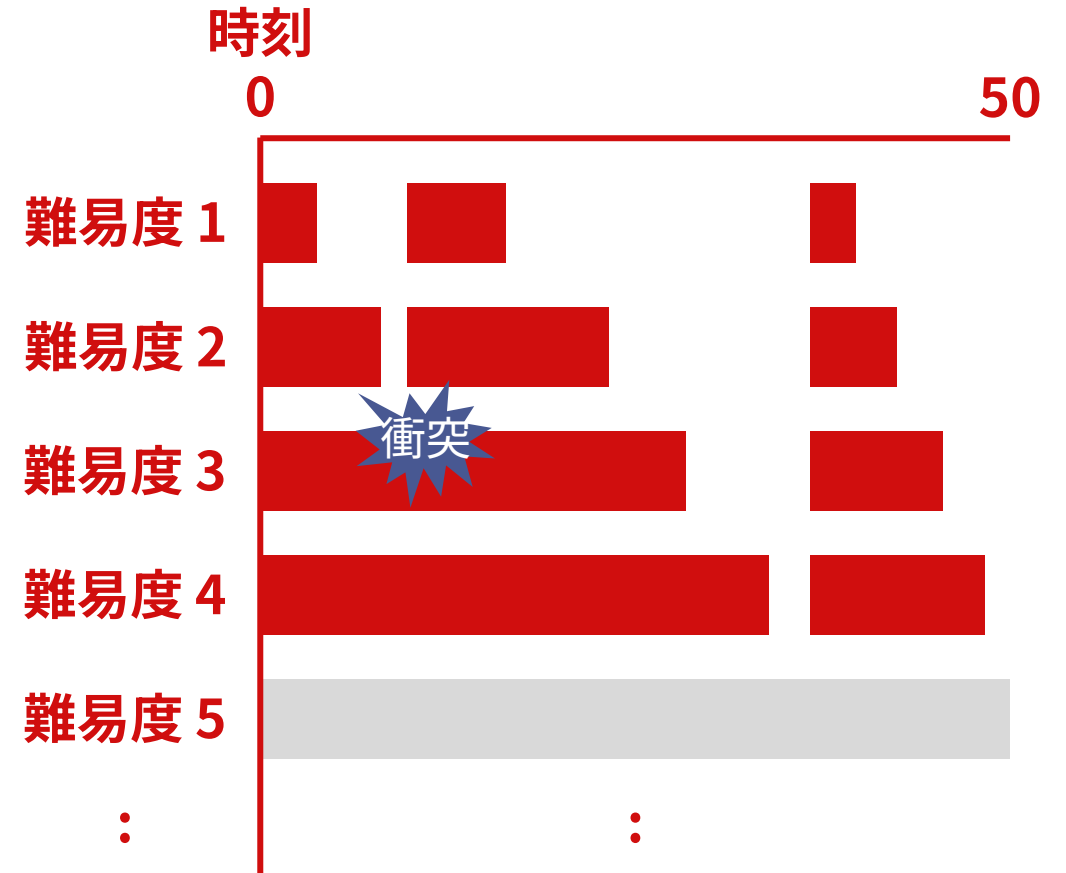
小課題 7

- これは単に、優先度付きキューで（難易度を増やしていった時の）敵の攻撃時間の衝突を右図のようにシミュレーションすればよい
- 計算量は $O(N^2 \log N)$ となり小課題 7 に通る



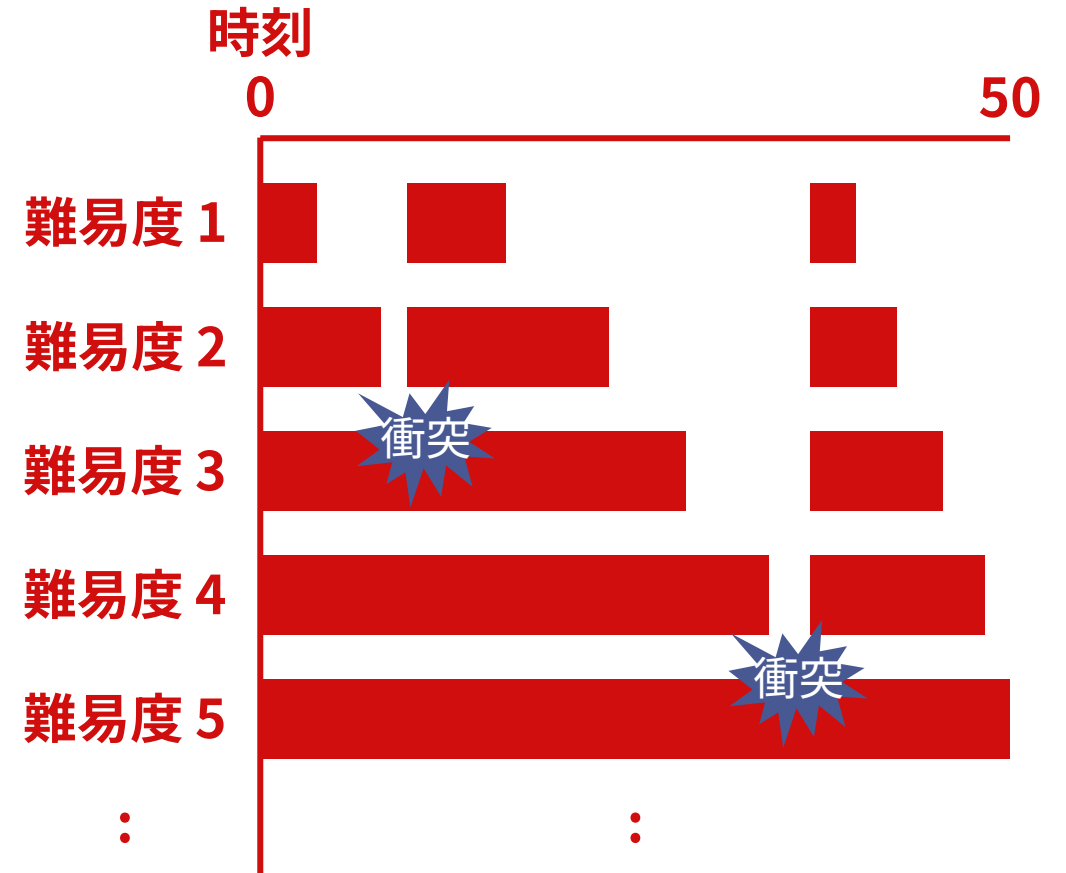
小課題 7

- これは単に、優先度付きキューで（難易度を増やしていった時の）敵の攻撃時間の衝突を右図のようにシミュレーションすればよい
- 計算量は $O(N^2 \log N)$ となり小課題 7 に通る



小課題 7

- これは単に、優先度付きキューで（難易度を増やしていった時の）敵の攻撃時間の衝突を右図のようにシミュレーションすればよい
- 計算量は $O(N^2 \log N)$ となり小課題 7 に通る



満点

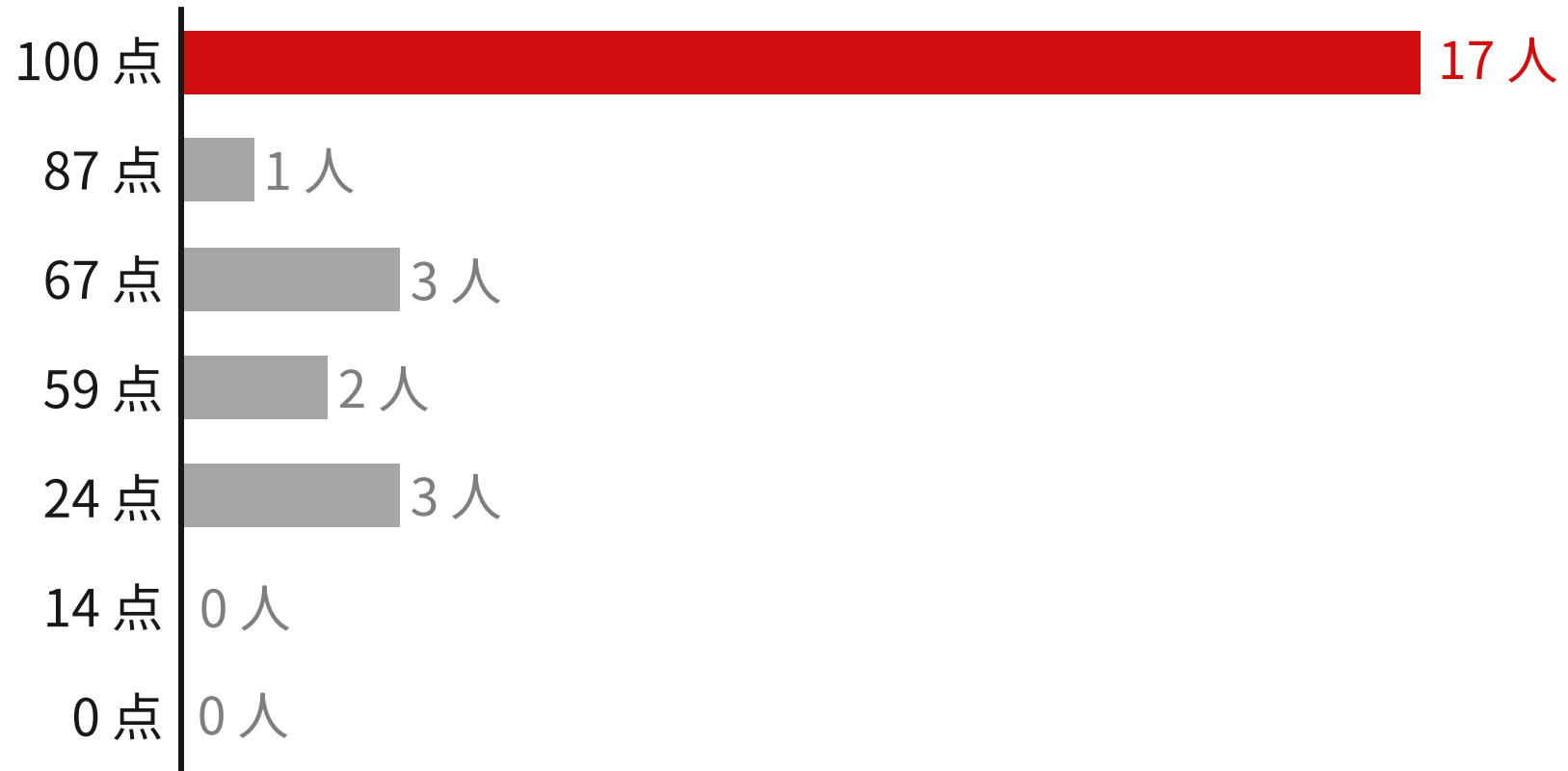
満点解法

- 実は、後ろから（終了時刻ギリギリの敵から）考えてスタックを使うと、 $O(N \log N)$ でシミュレーションしていたところが $O(N)$ になる
- 全体の計算量が $O(N^2)$ になり、満点まで通る

100 点

得点分布

得点分布 (理想)





JOI 2024/2025 春季トレーニング Day3 会議 (Conference) 解説

解説: 蜂矢 倫久 (Mitsubachi)

Step 0

問題概要

0

問題概要

- 長さ N の文字列 S があります
- S は $A, B, C, ?$ からなります
- $?$ に A, B, C を入れて文字が変わる回数を少なくしたいです
- $AABBACA$ なら $AA \mid BB \mid A \mid C \mid A$ で 4 回
- Q クエリ解きたいです
- 各クエリでは $?$ に A, B, C をそれぞれ X, Y, Z 個割り当てる場合に上の問題を解いてください

- $N \leq 300\,000$
- S の先頭と末尾は A
- $Q \leq 200\,000$
- $X + Y + Z$ は S の ? の個数

小課題番号	N	S	Q	Z	配点
1	50	? は 13 個以下			4 点
2	500				7 点
3	5000		10		13 点
4	5000				18 点
5			10		12 点
6		C がない		$Z = 0$	8 点
7				$Z = 0$	13 点
8					25 点

0

今後について

- 解説では文字が変わる回数のことを**コスト**と言います
- AABBC なら 2 で、ACCBBA なら 3 みたいな感じ
- 答えはコストの最小値です
- ネタバレ：
- とても大変な証明をすると問題が 2 パターンの形になって、とても大変な考察(と実装)をすると高速化できて解けます
- スライドもとても大変で **181** ページもあります

Subtask 1

$N \leq 50$, S の ? は 13 個以下

1

全探索

- ? が 13 個以下
- つまり ? への入れ方は高々 $3^{13} = 1594323$ 通り
- ? への入れ方を決めたらコストの計算は $O(N)$
- 普通に for 文を回せば OK
- 前もって全入れ方におけるコストを計算しておく
- ? の個数を M として $O(N 3^M + Q)$ など解ける
- 流石に各クエリで全部試す $O(N 3^M Q)$ は通らないかも

1

高速に解く

- 差分更新のイメージで？に入れる時にその前後だけ考えると $O(M \cdot 3^M + Q)$ になる
- $AB??C$ なら 1 つめの？に入れたとき重要なのは $B?$ と $??$ のところで、2 つめなら $??$ と $?C$ のところだけといった感じ
- 遅くても間に合いそうなときはシンプルな方を優先するとバグったときのタイムロスが少なくなります

Subtask 2

$$N \leq 500$$

2 考察

- 小課題 1 の解法だと $3^{500} \doteq 3.6 * 10^{238}$ ぐらいの計算が必要
- 何をどう考えても厳しい
- 差分更新の話を出す
- 差分更新は結局決めたところの近傍だけ大事でした
- このアイデアを使えないか

2 DP

- 先頭から決めてくことを考える
- 新たに決めたときコストの変化は自分とその 1 つ前のところだけ
- AABB から AABBC にしたとき大事なのは BC のところ
- 状態として大事なものは？に入れた A, B, C の個数と末尾
- DP を考える

2 DP

- $DP[i][j][k][l][m]$:= 先頭から i 文字目まで見て ? に A, B, C を j, k, l 個入れて、末尾が m ($= A, B, C$ のどれか) となるとき先頭から i 文字目までについてのコスト
- 初期化は $DP[0][0][0][0][A,B,C] = 0$ で他は $+\infty$
- 遷移は $\min$ の形で書ける
- 1 遷移 $O(1)$
- 状態数 $O(N^4)$ とかで遷移は $O(N^4)$
- TLE や MLE になりそう

2 高速化

- $DP[i][j][k][l][m]$:= 先頭から i 文字目まで見て？に A, B, C を j, k, l 個入れて、末尾が m ($= A, B, C$ のどれか) となるとき先頭から i 文字目までについてのコスト
- まず次元を減らす l は i, j, k から分かるので減らせる
- というわけで $O(N^3)$ で解けました
- 出してみると不正解と返ってくる
- ジャッジが壊れていそう

2

省メモリ化

- 自分のコードを読んでみる
- $500^3 * 3$ の int 配列は重たい
- 1.5GB ぐらい
- 実際は多次元なのでもっと重たい
- ① inplace にする
- ② short 配列を使う

2 省メモリ化

- ① inplace にする
- 昔の JOI 過去問を AtCoder で解くと ML: 64MB になってるがその頃のを解くのに使われたテクニック
- $DP[i]$ が $DP[i - 1]$ だけから反映されるとき 2 つの配列 A, B を用意して $(A, B) = (DP[0], DP[1]), (DP[2], DP[1]), \dots$ と更新していく
- 時間は変わらないがメモリは 1 次元削減

2

省メモリ化

- ② short 配列を使う
- $[-2^{15}, 2^{16} - 1] = [-32768, 32767]$ まで使える数値型
- int の半分のデータ量
- ① でもそうだが配列をグローバルに宣言したり vector ではなく `int dp[501][501][3]` みたいに宣言するとより良い

- CMS のテスト機能を使えばメモリを使うか確認できる
- プラクティスでこの仕様があることを確認しましたか？

Subtask 3

$N \leq 5\,000$, $Q \leq 10$

3 その前に

- 後ろの方の小課題の方が簡単に感じることもある
- 今回は小課題 6 がそれにあたる
- 一旦小課題 6 の話をしよう

Subtask 6

S に C が無い, $Z = 0$

6

制約の意味

- S に C がいない $\rightarrow S$ は $A, B, ?$ のみ
- $Z = 0 \rightarrow ?$ には A, B だけ割り当てる
- つまり文字が A, B の 2 種類に減った
- 文字が 1 種類ならどうなるか $\rightarrow$ 全部 A だからコスト 0 でした

6

重要なアイデア

- ? への入れ方は A を入れる個数で一意
- ここで重要なアイデア「**文字列を分割**」を使う
- $S = A???B?A?ABBA$ とかで考えてみる

6

文字列を分割

- ここで重要なアイデア「**文字列を分割**」を使う
- $S = A???B?A?ABBA$ とかで考えてみる
- このときコストは $A???B + B?A + A?A + AB + BB + BA$ についてのコストの総和になる
- 文字と文字の間に着目して分けた感じ
- 各文字列は (A か B) + (? が何個か) + (A か B) の形
- このような分割は一意になる
- また長さの総和は $O(N)$ になる(各文字の寄与を考えれば OK)

6

各文字列ごとに考える

- 各文字列は (A か B) + (? が何個か) + (A か B) の形
- ? に入れる A と B の個数を決めたときコストはどうなるか
- ① 両端が同じケース
- A???A で考えてみよう
- A が 0 個、B が 3 個 → ABBBA コストは 2
- A が 1 個、B が 2 個 → AABBA や ABABA や ABBAA がある
- コストは 2 か 4 なので最小コストは 2
- A が 2 個、B が 1 個 → AABAA とすると最小コストは 2
- A が 3 個、B が 0 個 → AAAAA コストは 0

6

下界を考える

- ① 両端が同じケース
- $A???A$ で考えてみよう
- 全部 A ならコスト 0 でそうでないならコスト 2 でした
- **予想**「全部 A なら 0 で B があるなら 2」を立ててみる
- **証明**をする
- 全部 A なら 0 は明らか
- 下界を考える B があるとする
- 左から見てくことを考えると 左 A 右 B と 左 B 右 A の 2 回は起こる つまり 2 以上

6

下界を達成する

- ① 両端が同じケース
- $A???A$ で考えてみよう
- **予想** 「全部 A なら 0 で B があるなら 2」
- **証明**
 - B があるなら下界は 2 以上だった あとは 2 を作れば OK
 - ? の左側に B を詰めて ? の右側に A を詰めれば達成可能
 - 同じ文字を 1 つに集めるイメージ
 - 例えば A が 1 個で B が 2 個なら AABBA とすればできる
- よって証明できた

6

予想を立てる

- ② 両端が違うケース
- $A???B$ で考えてみよう
- A が 0 個、B が 3 個 $\rightarrow$ ABBBBB コストは 1
- A が 1 個、B が 2 個 $\rightarrow$ AABBBB や ABABBB や ABBBAB がある
- コストは 1 か 3 なので最小コストは 1
- A が 2 個、B が 1 個 $\rightarrow$ AAABBB とすると最小コストは 1
- A が 3 個、B が 0 個 $\rightarrow$ AAAABB コストは 1
- **予想** 「常に 1」を立ててみる
- **証明** をする

6

証明する

- ② 両端が違うケース
- $A???B$ で考えてみよう
- **予想** 「常に 1」
- **証明**
- 下界を考える B があるとする
- 左から見ていくと 左 A 右 B の 1 回は起こる つまり 1 以上
- 逆に左に A を入れて右に B を入れれば達成可能
- よって証明できた

6

ここまでのまとめ

- 各文字列について
 - ① 両端が同じケース
 - 全部端の文字なら 0 で、そうでないなら 2
 - ② 両端が違うケース
 - 常に 1
- 本質的な気付き「**どの文字を入れるか入れないかだけが重要**」
 - 1 個以上なら何個でも同じということ

6

貪欲戦略

- つまり A を何個か入れて残りに B を入れるなら
- 両端が同じ文字列についてはその間を端の文字で全部埋めたい
- $A???A$ なら $AAAAA$ にしたいということ
- よって以下の戦略が最適
- $A + (? \text{ が何個か}) + A$ の形について短い順に貪欲に A を入れる
- $B + (? \text{ が何個か}) + B$ の形について短い順に貪欲に B を入れる
- あとは適当に入れる

6

コストの計算

- よって以下の戦略が最適
- $A + (? \text{ が何個か}) + A$ の形について短い順に貪欲に A を入れる
- $B + (? \text{ が何個か}) + B$ の形について短い順に貪欲に B を入れる
- あとは適当に入れる
- コストは $[(A + (? \text{ が何個か}) + A \text{ の形で } B \text{ が入っている個数}) + (B + (? \text{ が何個か}) + B \text{ の形で } A \text{ が入っている個数})] * 2 + (A + (? \text{ が何個か}) + B \text{ の形の個数}) * 1$ と計算できる
- $(A + (? \text{ が何個か}) + B \text{ の形の個数})$ の計算は簡単

6

コストの計算

- $(A + (? \text{ が何個か}) + A \text{ の形で } B \text{ が入っている個数})$ を数える
- $(A + (? \text{ が何個か}) + A \text{ の形で } A \text{ だけが入っている個数})$ を数えられれば良い
- これは ? の個数を並べて sort した数列を考えれば数列の先頭から足していってある数を超えるのはいつですか という形
- 例えば $A??A, A???A, A?A, A?A$ で A を 5 個入れるなら
- $(1, 1, 2, 3)$ の先頭から足していって 5 をを超えるのはいつですか
- これは累積和などをとって二分探索すれば良い

6

答え

- よってクエリあたり $O(\log N)$ など解ける
- 全体では $O(N + Q \log N)$

6

別方針

- 別方針
- あらかじめ $\text{ans}[i] := ?$ に A を i 個入れる時の答え を用意する
- $\text{ans}[0]$ は簡単 全部に B を入れる
- $\text{ans}[i]$ から $\text{ans}[i + 1]$ を作りたい
- これはさっきの貪欲戦略にしたがって B を A に入れ替えることをすると $O(N)$ で $\text{ans}[i]$ が列挙できる
- 全体では $O(N + Q)$

Subtask 3

$N \leq 5\,000$, $Q \leq 10$

3

文字列の分割

- 小課題 6 のように分割すると以下のような問題になる
- $(A, B, C \text{ のどれか } l) + [? \text{ が何個か}] + (A, B, C \text{ のどれか } r)$ のような形の文字列が何個か与えられる ($C[???]B$ みたいな感じ)
- $?$ に A, B, C を決められた個数入れる
- それぞれにおけるコストを足し合わせた値を最小化したい
- 各文字列について $[?]$ のところに入れた文字で、 l, r のどちらでもない文字の種類数を**ペナルティ**とする ($A[BBC]A$ なら 2)
- A, B のときはコストは両端とペナルティだけに依存していた

3

文字列の分割

- 各文字列について $[?]$ のところに入れた文字で、 l, r のどちらでもない文字の種類数を **ペナルティ** とする ($A[BBC]A$ なら 2)
- **予想** 「A, B, C でもコストは両端とペナルティだけに依存する」
- なんと成り立つ
- **証明** をする

3 証明

- **予想** 「A, B, C でもコストは両端とペナルティだけに依存する」
- **証明**
 - ① 両端が同じケース ($l = r$)
 - ペナルティが 0, 1, 2 のとき下界として 0, 2, 3 が取れる
 - これは達成できる
 - 先ほどと同様に同じ文字を 1 つに集めれば OK
 - A???A に B を 2 個と C を 1 個なら ABBCA にするイメージ

3 証明

- **予想** 「A, B, C でもコストは両端とペナルティだけに依存する」
- **証明**
 - ② 両端が異なるケース ($l \neq r$ でない)
 - ペナルティが 0, 1 のとき下界として 1, 2 が取れる
 - これも達成できる
 - 先ほどと同様に同じ文字を 1 つに集めれば OK
 - A???B に B を 2 個と C を 1 個なら ACBBB にするイメージ
- よって証明できた

3

言い換え

- **性質** 「A, B, C でもコストは両端とペナルティだけに依存する」
- これを用いて問題文を書き直す
- (A, B, C のどれか l) + [? が何個か] + (A, B, C のどれか r) のような形の文字列が何個か与えられる (C[???]B みたいな感じ)
- ? に A, B, C を決められた個数入れる
- それぞれにおけるコストを足し合わせた値を最小化したい
- 各文字列について [?] のところに入れた文字で、l, r のどちらでもない文字の種類数を**ペナルティ**とする (A[BBC]A なら 2)

3

言い換え

- 各文字列について [?] のところに入れた文字で、l, r のどちらでもない文字の種類数を **ペナルティ** とする (A[BBC]A なら 2)
- 両端が同じとき ペナルティが 0, 1, 2 ならコストは 0, 2, 3
- 両端が異なるとき ペナルティが 0, 1 ならコストは 1, 2
- 今回は $N \leq 5\,000$ で $Q \leq 10$ で解けばいい
- クエリあたり $O(N^2)$ とか $O(N^2 \log N)$ とかで解きたい

3

用語の準備

- 小課題 6 では貪欲な戦略を考えた
- 今回も同様にできないか？
- 実はある
- まず色々用語の準備をしよう
- $A[?]A$ の形における $?$ の個数の合計を p とする
- $B[?]B, C[?]C, A[?]B, B[?]C, C[?]A$ についても同様に q, r, s, t, u と定める

3

用語の準備

- $A[?]A$ の形における ? の個数の合計を p とする
- $B[?]B, C[?]C, A[?]B, B[?]C, C[?]A$ についても同様に q, r, s, t, u と定める
- $S = A???AB??B??C??B$ なら
- $A[???]A + A[]B + B[??]B + B[??]C + C[??]B$ となって
- $(p, q, r, s, t, u) = (3, 2, 0, 0, 3, 0)$ ということ
- p, q, r, s, t, u は明らかに 0 以上
- 証明で使うことがあるのでここで明記

3

Large の定義

- $A[?]A$ の形における $?$ の個数の合計を p とする
- $B[?]B, C[?]C, A[?]B, B[?]C, C[?]A$ についても同様に q, r, s, t, u と定める
- 各文字がたくさんあるか全然ないかを定義したい
- A, B, C について **Large** と **Small** という概念を定義する
- **Large:** 個数が自分の文字が端となる文字列における $?$ の個数の総和より多い
- 例えば $X > p + s + u$ なら A を Large と呼ぶ
- B, C については $Y > q + s + t, Z > r + t + u$

3

Small の定義

- **Large:** 個数が自分の文字が端となる文字列における ? の個数の総和より多い
- 例えば $X > p + s + u$ なら A を Large と呼ぶ
- B, C については $Y > q + s + t, Z > r + t + u$
- **Small:** 個数が自分の文字が両端となる文字列における ? の個数の総和より少ない
- 例えば $X < p$ なら A を Small と呼ぶ
- B, C については $Y < q, Z < r$

3

用語のイメージ

- **Large:** 文字がたくさんあってペナルティが増えるような文字列にはみ出る
- **Small:** 文字があまりなくて自分の文字を両端とする文字列に他の文字が入ってくる
- ? の個数の議論より $X + Y + Z = p + q + r + s + t + u$ が成立
- Large や Small の個数で場合分けをしていく

3

怒涛の証明

- ここから場合分けしつつ問題の性質を証明します
- なんと 17 個もあります
- 基本的なアイデアは「**このように操作しても損しないので、最適解はこれが成り立っているとして良い**」というもの
- 以降示した性質は成り立っているとします

3

怒涛の証明の前に

- ここから場合分けしつつ問題の性質を証明します
- なんと 17 個もあります
- 基本的なアイデアは「**このように操作しても損しないので、最適解はこれが成り立っているとして良い**」というもの
- 多分もう話し疲れているので一旦休憩
- 今日の駒場の写真を流す

3 休憩タイム

- この場所から南に行くとあります
- 馬神と書いてある



3 休憩タイム

- きれいだ
- 証明をしよう



- **性質 01:**

- ペナルティが 2 となる文字列の数 d は 1 以下

- **証明 01:**

- $d \geq 2$ とする ペナルティが 2 の文字列を 2 つ取る
- ① $A [A * p1, B * q1, C * r1]$ A と $A [A * p2, B * q2, C * r2]$ A のように両端の文字が同じとき
- 前者の B と後者の C を $\min(q1, r2)$ 回 swap すると d を 1 か 2 減らしてコストも 1 減らせる

3

証明 01

- **性質 01:**

- ペナルティが 2 となる文字列の数 d は 1 以下

- **証明 01:**

- ② $A [A * p1, B * q1, C * r1]$ A と $B [A * p2, B * q2, C * r2]$ B のように両端の文字が違うとき
- 前者の B と後者の A を $\min(q1, p2)$ 回 swap すると同様に示せる

3

場合分け

- Large の個数で場合分けをする
- ① Large が 3 つあるとき
- $X > p + s + u, Y > q + s + t, Z > r + t + u$ ということ
- これは $X + Y + Z = p + q + r + s + t + u$ に矛盾
- ② Large が 2 つあるとき
- A と B が Large としていい
- $X > p + s + u, Y > q + s + t$ より $Z < r - s$
- 特に $Z < r$ すなわち C は Small

3

場合分け

- ② Large が 2 つあるとき
- A と B が Large としていい
- $X > p + s + u$, $Y > q + s + t$ より $Z < r - s$
- 特に $Z < r$ すなわち C は Small
- このとき以下の性質が成り立つ
- **性質 02:** $A[?]A$, $B[?]B$ の中に C は入らない
- **性質 03:** $A[?]B$ の中に C は入らない
- **性質 04:** $A[?]A$, $B[?]B$ の中はそれぞれ A, B のみ入る
- **性質 05:** $B[?]C$, $C[?]A$ の中にそれぞれ A, B は入らない
- **性質 06:** $B[?]C$, $C[?]A$ の中に C は入らない

3

記法の定義

- 証明の中の記法として $A[BC]A$ のようなものを使います
- これは両端が A で？に入れた端でない文字は B と C の 2 種類の文字列ですよという意味 (A は入れて良い)
- 他には $A[C]B$ なら両端が A と B で？に入れた端でない文字は C の 1 種類の文字列ですよという意味 (A, B は入れて良い)
- 同じものを 1 つにしたよと思えば良い
- **性質** 「 A, B, C でもコストは両端とペナルティだけに依存する」を抑えると自然な記法

- **性質 02:**

- $A[?]A$, $B[?]B$ の中に C は入らない

- **証明 02:**

- 背理法を使う ある $A[?]A$ の中に C が入っているとしよう
- $Z < r$ より $C[?]C$ の中の A, B と swap することで $A[?]A$ の中の C を全て $C[?]C$ の中に持ってくる事が可能
- $A[?]A$ は $A[BC]A$ か $A[C]A$ の形

- **性質 02:**

- $A[?]A, B[?]B$ の中に C は入らない

- **証明 02:**

- ① $A[BC]A$ の場合

- **性質 01** 「ペナルティが 2 となる文字列の数 d は 1 以下」より
 $C[?]C$ の中に A, B があるものは $CAA...AC$ か $CBB...BC$ の形

- $A[BC]A$ と $C[A]C$ で swap $\rightarrow$ 最悪でも $A[B]A$ と $C[AB]C$ でコストは増えない
- $A[BC]A$ と $C[B]C$ で swap $\rightarrow A[B]A$ と $C[B]C$ でコストは 1 減少

- **性質 02:**

- $A[?]A, B[?]B$ の中に C は入らない

- **証明 02:**

- ② $A[C]A$ の場合
- $A[C]A$ と $C[A]C$ で swap $\rightarrow A[]A$ と $C[A]C$ でコストは 2 減少
- $A[C]A$ と $C[B]C$ で swap $\rightarrow$ 最悪でも $A[B]A$ と $C[B]C$ でコストは増えない
- $A[C]A$ と $C[AB]C$ で swap $\rightarrow$ 最悪でも $A[B]A$ と $C[AB]C$ でコストは増えない

- **性質 02:**

- $A[?]A$, $B[?]B$ の中に C は入らない

- **証明 02:**

- よってコストを増やさないまま $A[?]A$ の中の C を $C[?]C$ に移すことができた
- $B[?]B$ の中に C は入らないはさっきの証明の A を B に変えるだけ

- **性質 03:**

- $A[?]B$ の中に C は入らない

- **証明 03:**

- $A[?]B$ の中に C は入らないも同様にできる
- むしろ $A[?]B$ の中に A, B を入れる分にはペナルティが増えないので損をすることがない

- **性質 04:**

- $A[?]A$, $B[?]B$ の中はそれぞれ A , B のみ入る

- **証明 04:**

- $A[?]A$ の中は A のみ入るが示せば A と B を変えるだけで良い
- まず**性質 02** 「 $A[?]A$ の中に C は入らない」より B があるなら $A[B]A$ の形
- 同様に $B[?]B$ の中に A があるなら $B[A]B$ の形
- $A[B]A$ と $B[A]B$ が両方あるならこの 2 つで swap すれば良い
- よって $B[A]B$ はない

- **性質 04:**

- $A[?]A, B[?]B$ の中はそれぞれ A, B のみ入る

- **証明 04:**

- $A[B]A$ があると仮定
- 同様に考えると $A[?]B, B[?]C$ の中に A があればコストが増えない swap が可能
- $A[?]B, B[?]C$ の中に A はない
- つまり A は $A[?]A, A[?]C, C[?]C$ のどれかに入る

- **性質 04:**

- $A[?]A$, $B[?]B$ の中はそれぞれ A , B のみ入る

- **証明 04:**

- $A[B]A$ があると仮定
- A は $A[?]A$, $A[?]C$, $C[?]C$ のどれかに入る
- A が Large より $X > p + s + u \geq p + u$ である つまり $A[?]A$ の ? の個数と $A[?]C$ の ? の個数は埋められる
- よって $A[?]A$ の中にある B の個数以上 $C[?]C$ の中に A がある

- **性質 04:**

- $A[?]A$, $B[?]B$ の中はそれぞれ A , B のみ入る

- **証明 04:**

- $A[B]A$ があると仮定
- $A[?]A$ の中にある B の個数以上 $C[?]C$ の中に A がある
- $A[?]A$ の中にある B を全て $C[?]C$ の中の A と swap すれば良い
- $A[?]A$ によるコストの寄与の差分は -2 以下で、 $C[?]C$ によるコストの寄与は $+1$ 以下で抑えられる
- $C[A]C \rightarrow C[AB]C$ となることは 1 個で抑えられるため

- **性質 04:**
- $A[?]A$, $B[?]B$ の中はそれぞれ A , B のみ入る
- **証明 04:**
- $A[?]A$ の中は A のみ入るが示された
- A と B を変えるだけで $B[?]B$ の中は B のみ入るが示せる
- よって示されました

- **性質 05:**

- $B[?]C, C[?]A$ の中にそれぞれ A, B は入らない

- **証明 05:**

- $B[A]C$ があるとする
- $C[B]C, A[C]B, C[B]A$ があるならこれと swap すればいい
- もしないなら $Y \leq q + t$ となり Y が Large に矛盾
- よって $B[A]C$ はない 同様に $C[B]A$ もない
- 以上より示されました

- **性質 06:**

- $B[?]C, C[?]A$ の中に C は入らない

- **証明 06:**

- B が Large と $A[?]A$ と $A[?]C$ に B がない(**性質 02, 05** より)

- よって $B[?]C$ の中にある C の分 $C[?]C$ の中に B がある

- これを swap すれば良い

- $C[?]A$ についても同様

- 以上より示されました

3

まとめ

- ② Large が 2 つあるとき
- A と B が Large とする このとき C は Small となる
- **性質 02:** $A[?]A, B[?]B$ の中に C は入らない
- **性質 03:** $A[?]B$ の中に C は入らない
- **性質 04:** $A[?]A, B[?]B$ の中はそれぞれ A, B のみ入る
- **性質 05:** $B[?]C, C[?]A$ の中にそれぞれ A, B は入らない
- **性質 06:** $B[?]C, C[?]A$ の中に C は入らない
- つまり $A[?]A, C[?]A$ の中は A が入り $B[?]B, B[?]C$ の中は B が入る また C は $C[?]C$ の中のみに入る

3

最適解の構造

- ② Large が 2 つあるとき
- A と B が Large とする このとき C は Small となる
- $A[?]A$, $A[?]C$ は A のみ入る
- $B[?]B$, $B[?]C$ は B のみ入る
- C は $C[?]C$ の中のみに入る
- 決まってないのは $A[?]B$ に A, B を何個入れるか
- $A[?]B$ の A の個数を固定してみる $C[?]C$ に A を a 個入れるとする
- すると次の **Subproblem1** の形に帰着できる

3 Subproblem1

- **Subproblem1**

- 数列 c が与えられる ($C[?]C$ の ? の個数に対応)
- c の正の要素を 1 つ選んで減らす操作を Z 回行う ($C[?]C$ の ? に C を入れること)
- この後の c について **罰金** を以下のように定める 最小化せよ
- $c_i \geq 1$ なる i の個数を W とする (C 以外が入る $C[?]C$ の数)
- c の部分和で a が作れるなら $2 * W$
- 作れないなら $2 * W + 1$ (ペナルティが 2 となる分を足す)
- $a \leq \text{sum}(c_i) - Z$ は成り立っています

3 Subproblem1 の解法

- **Subproblem1** を考える
- まず W は最小値としていい
- W の最小値として W' が達成できるなら罰金は $2 * W' + 1$ 以下にできる
- 逆に $W \geq W' + 1$ なら罰金は $2 * W' + 2$ 以上になり不適
- W の最小化は常に最小の c_i を 1 減らせば良い
- 操作の残り回数的にもう $c_i = 0$ となる i を増やせなくなったとする

3 Subproblem1 の解法

- **Subproblem1** を考える
- W の最小化は常に最小の c_i を 1 減らせば良い
- 操作の残り回数的にもう $c_i = 0$ となる i を増やせなくなったとする
- $c = (3, 4, 6, 7)$ で $Z = 8$ なら 7 回の操作後には
- $c = (0, 0, 6, 7)$ で後 1 回減らせるタイミングに来たという感じ
- a (部分和で作りたい数) が 5 なら 6 を減らせばいいし、6 なら 7 を減らせばいい
- どっちを減らせばいいんだ

3 Subproblem1 の解法

- **Subproblem1** を考える
- 実は解ける
- もう $c_i = 0$ となる i を増やせなくなったタイミングにおける残り操作回数を z とする
- すると z 回操作した後の部分和で a が作れることの必要十分条件はそのタイミングでの部分和で $[a, a + z]$ のどれかが作れること
- 十分性は $a + k$ が作れるならその部分和に採用されたのについて k 回操作してそうでないのから $z - k$ 回操作すれば示せる
- 必要性は z 回後に a が作れるなら a を作るために選ばれた要素の z 回前の総和を考えると示せる

3 Subproblem1 の解法

- **Subproblem1** を考える
- z 回操作した後の部分和で a が作れることの必要十分条件はそのタイミングでの部分和で $[a, a + z]$ のどれかが作れること
- $c = (3, 4, 6, 7)$ で $Z = 8$ なら 7 回の操作後には
- $c = (0, 0, 6, 7)$ で後 $z = 1$ 回減らせるタイミングに来たという感じ
- この c の部分和は 6, 7, 13 が取れる
- $a = 5$: $[5, 5 + 1] = [5, 6]$ で 6 が作れるので OK
- $a = 6$: $[6, 6 + 1] = [6, 7]$ で 6 と 7 が作れるので OK

3 Subproblem1 の解法

- **Subproblem1** を考える
- z 回操作した後の部分和で a が作れることの必要十分条件はそのタイミングでの部分和で $[a, a + z]$ のどれかが作れること
- 十分性は $a + k$ が作れるならその部分和に採用されたのについて k 回操作してそうでないのから $z - k$ 回操作すれば示せる
- 必要性は z 回後に a が作れるなら a を作るために選ばれた要素の z 回前の総和を考えると示せる

3 Subproblem1 の解法

- **Subproblem1** を考える
- 部分和問題は $DP[i][j] := i$ 番目まで見て総和が j を作れるかとする DP で $O(N^2)$ で解ける
- a の値は $A[?]B$ の ? の個数を固定すると一意に定まるのだった
- だが $A[?]B$ の ? の個数は色々ありうる
- しかし、 $A[?]B$ の ? の個数を 1 変化すると a は 1 変化する
- よって a のとりうる値は範囲となる

3 Subproblem1 の解法

- **Subproblem1** を考える
- a のとりうる値は範囲となる
- そして知りたいものは a がある範囲を動く ($= A[?]B$ に使う A の個数を動かす) 中での **Subproblem1** の答え ($= C[?]C$ の部分のコストへの寄与) の最小値
- また Z の値は a によらず一緒 $\rightarrow$ 同じ部分和问题の DP !
- よって **Subproblem1** の $[a, a + z]$ を $[\min(a), \max(a) + z]$ に置き換えたものを解けば良い
- 同様に $O(N^2)$ で解ける

3

場合分けに戻ってくる

- ③ Large が 1 つあるとき
- A が Large としていい
- $X > p + s + u$ より $Y + Z < q + r + t$
- [#1] $Y > q + t$ かつ $Z > r + t$ のとき
- $Y + Z$ の条件に矛盾
- [#2] $Y > q + t$ のとき
- $Z < r$ である
- ② と同様の性質が成立するので **Subproblem1** に帰着できる
- B が Large でないため $Y \leq q + s + t$ となる

3 証明

- ③ Large が 1 つあるとき
- [#2] $Y > q + t$ のとき
- $q + t < Y \leq q + s + t$ より $B[?]B$, $B[?]C$ に B を入れた後の残りは全部 $A[?]B$ に入れば良い
- よって $A[?]B$ に入れる A の個数は一意なので **Subproblem1** が少し簡単に書ける
- やることは変わらないが範囲の端の計算が簡潔
- [#3] $Z > r + t$ のとき
- [#2] と同様に解ける

3 証明

- ③ Large が 1 つあるとき
- [#4] $Y \leq q + t$ かつ $Z \leq r + t$ のとき
- このとき以下の性質が成り立つ
- **性質 07:** $B[?]B, C[?]C$ の中にそれぞれ C, B は入らない
- **性質 08:** $A[?]B, C[?]A$ の中にそれぞれ C, B は入らない
- **性質 09:** $A[?]A$ の中に B, C は入らない
- **性質 10:** $A[?]B, C[?]A$ の中にそれぞれ B, C は入らない

- **性質 07:**

- $B[?]B, C[?]C$ の中にそれぞれ C, B は入らない

- **証明 07:**

- $B[?]B$ の中に C が入らないを証明したい
- $Z \leq r + t$ より $B[?]B$ の中にある C の個数分の A か B が $C[?]C$ と $B[?]C$ の中にある
- **証明 02** と同様の議論を考えると $B[?]B$ の中にある C を全部 $C[?]C$ や $B[?]C$ の中の A と B で swap してコストは増えないので損しない

- **性質 07:**

- $B[?]B, C[?]C$ の中にそれぞれ C, B は入らない

- **証明 07:**

- $B[?]B$ の中に C が入らないを証明した
- B と C を入れ替えれば $C[?]C$ の中に B が入らないも証明できる
- よって示されました

- **性質 08:**

- $A[?]B, C[?]A$ の中にそれぞれ C, B は入らない

- **証明 08:**

- $A[?]B$ の中に C は入らないを証明する

- **証明 07** と同様の議論をする $Y \leq q + t$ より $A[?]B$ 中にある C の個数分の A か B が $B[?]B$ と $B[?]C$ 中にある

- swap してもコストは増えないため損しない

- よって示されました

- **性質 08:**

- $A[?]B, C[?]A$ の中にそれぞれ C, B は入らない

- **証明 08:**

- $A[?]B$ の中に C は入らないを証明する
- B, C を入れ替えれば $C[?]A$ の中に B は入らないも証明できる
- よって示されました

- **性質 09:**
- $A[?]A$ の中に B, C は入らない
- **証明 09:**
- **証明 07** と同様の議論をする
- $A[?]A$ の B を $B[?]B$ や $B[?]C$ の A か C と swap するイメージ

- **性質 10:**
- $A[?]B, C[?]A$ の中にそれぞれ B, C は入らない
- **証明 10:**
- **証明 07** と同様の議論をする
- $A[?]B$ の B を $B[?]B$ の A と swap するイメージ

3 まとめ

- ③ Large が 1 つあるとき
- [#4] $Y \leq q + t$ かつ $Z \leq r + t$ のとき
- このとき以下の性質が成り立つ
- **性質 07:** $B[?]B, C[?]C$ の中にそれぞれ C, B は入らない
- **性質 08:** $A[?]B, C[?]A$ の中にそれぞれ C, B は入らない
- **性質 09:** $A[?]A$ の中に B, C は入らない
- **性質 10:** $A[?]B, C[?]A$ の中にそれぞれ B, C は入らない
- つまり $A[?]A, A[?]B, C[?]A$ の中は A が入り B は $B[?]B, B[?]C$ の中のみに入る また C は $C[?]C, B[?]C$ の中のみに入る

3

最適解の構造

- ③ Large が 1 つあるとき
- [#4] $Y \leq q + t$ かつ $Z \leq r + t$ のとき
- $A[?]A, A[?]B, A[?]C$ は A のみ入る
- B は $B[?]B, B[?]C$ の中のみに入る
- C は $C[?]C, B[?]C$ の中のみに入る
- 先に B, C を入れて後で A を残りに入れることを考える
- すると次の **Subproblem2** の形に帰着できる

3 Subproblem2

- **Subproblem2**

- 3つの書店 D, E, F がある それぞれ本を販売している
- 金貨 Y 枚と銀貨 Z 枚を持っている (B の個数と C の個数)
- 以下のように本を買う
- 書店 D で本 i を金貨 d_i 枚で買う (B[?]B の ? に B のみを入れる)
- 書店 E で本 i を銀貨 e_i 枚で買う (C[?]C の ? に C のみを入れる)
- 書店 F で本 i を金貨と銀貨合わせて f_i 枚で買う (B[?]C の ? に B か C を入れる)
- 買い方に対して **罰金** を以下のように定める 最小化せよ

3

Subproblem2

- **Subproblem2**
- 買い方に対して**罰金**を以下のように定める 最小化せよ
- $((\text{書店 D で買えなかった本の数}) + (\text{書店 E で買えなかった本の数})) * 2 + (\text{書店 F で買えなかった本の数}) * 1$
- つまり A を ? に入れることによるコストの増加分を最小化したい

3

Subproblem2

- **Subproblem2**

- 変数 W を $W = 0$ で初期化する
- D, E で本を買うたびに $+2$ して F で買うたびに $+1$ する
- 操作後の W を最大化せよ
- という形になる
- 買えなかった分に関する式の最小化を
- 買えた分に関する式の最大化に言い換えるイメージ
- こちらのほうが見やすいので以降こちらを採用

3 Subproblem2 の解法

- **Subproblem2** を考える
- 書店 D と書店 E で買う本の数を $O(N^2)$ 通り全探索する
- 書店 F でどれだけ買えるかを知りたい
- D と E で使った分から F でどれだけ使えるか分かる
- あとはどれだけ買えるかを二分探索する $O(\log N)$
- よって $O(N^2 \log N)$ など解けました

3

場合分けに戻ってくる

- ④ Large がないとき
- ここにも場合分けと性質の列挙があります
- Small の個数で場合分けしていきます
- [#1] Small が 3 つあるとき
- $X + Y + Z < p + q + r$ より個数の総和の条件に矛盾
- [#2] Small が 2 つあるとき
- B, C が Small とする $B < q, C < r$ より $A > p + s + t + u$
- $A > p + s + t + u \geq p + s + u$ となり矛盾

3

場合分けに戻ってくる

- ④ Large がないとき
- [#3] Small が 1 つあるとき
- C が Small とする $X \geq p, Y \geq q, Z < r$ である
- A, B は Large ではないので総和の条件を考えると
- $p + u \leq X \leq p + s + u, q + t \leq Y \leq q + s + t, Z < r$ となる
- また $p + q + s + t + u \leq X + Y$

3

場合分けに戻ってくる

- ④ Large がないとき
- [#3] Small が 1 つあるとき
- このとき以下の性質が成り立つ
- 性質 11: $A[?]A$, $B[?]B$, $A[?]B$ の中に C は入らない
- 性質 12: $A[?]A$ の中に B は入らない
- 性質 13: $B[?]B$ の中に A は入らない
- 性質 14: $C[?]A$ の中に B は入らない
- 性質 15: $B[?]C$ の中に A は入らない
- 性質 16: $B[?]C$ の中に C は入らない
- 性質 17: $C[?]A$ の中に C は入らない

- **性質 11:**

- $A[?]A, B[?]B, A[?]B$ の中に C は入らない

- **証明 11:**

- $Z < r$ よりそれらに入っている C の個数の分 $C[?]C$ の中に A, B が入っている
- これらで swap すれば良い

- **性質 12:**

- $A[?]A$ の中に B は入らない

- **証明 12:**

- $A[?]A$ の中に B が入っているとする
- $A[?]B$ や $B[?]B$ の中には B のみ入っている
- そうでないなら **性質 11** より A が入っているのでこれらを swap すれば良い
- これと $p + u \leq X$ より $A[?]A$ の中に入っている B の個数の分の A が $C[?]C$ か $B[?]C$ にある

- **性質 12:**

- $A[?]A$ の中に B は入らない

- **証明 12:**

- $A[?]A$ の中に B が入っているとする
- これと $p + u \leq X$ より $A[?]A$ の中に入っている B の個数の分の A が $C[?]C$ か $B[?]C$ にある
- これらを swap すれば良い
- $C[A]C \rightarrow C[AB]C$ となることがあるがこれによるコストの上昇は $+1$ 以下で $A[B]A \rightarrow A[]A$ でコストは 2 減るので正当

- **性質 13:**
- $B[?]B$ の中に A は入らない
- **証明 13:**
- A と B を入れ替えることで **証明 12** と同様の形になる

- **性質 14:**
- $C[?]A$ の中に B は入らない
- **証明 14:**
- $C[?]A$ の中に B が入っているとする
- $A[?]B, B[?]C$ に A はないとしていい あるなら swap 可能
- $p + u \leq X$ より $C[?]A$ の中の B の個数分の A が $C[?]C$ に入っている
- これらを swap すれば良い

- **性質 15:**
- $B[?]C$ の中に A は入らない
- **証明 15:**
- A と B を入れ替えることで **証明 14** と同様の形になる

- **性質 16:**

- $B[?]C$ の中に C は入らない

- **証明 16:**

- $B[?]C$ の中に C があるとする
- $C[?]C$ の中に B があるなら swap していいのでないとする
- $C[?]C$ の中には A か C しかない
- $C[?]A$ の中に C があるなら C が Small ($= C < r$) より $C[?]C$ の中に A が入っていてこれと $C[?]C$ の中の A を swap すれば良い
- よって $C[?]A$ の中は C のみ

- **性質 16:**

- $B[?]C$ の中に C は入らない

- **証明 16:**

- これとここまでの性質より B は $B[?]B$, $A[?]B$, $B[?]C$ にしかない
- 結局 $C[?]C$ の中の A と $A[?]B$ の中の B は両方 $B[?]C$ の中の C の個数分取れる
- この A, B, C を $A \rightarrow B \rightarrow C$ のサイクルに沿って swap すれば結局 $C[?]C$ の中の A の個数が減ってその分 C が増える
- よってコストは増えないので示された

- **性質 17:**
- $C[?]A$ の中に C は入らない
- **証明 17:**
- A と B を入れ替えることで **証明 16** と同様の形になる

3

まとめ

- ④ Large がないとき
- [#3] Small が 1 つあるとき
- 性質 11: $A[?]A, B[?]B, A[?]B$ の中に C は入らない
- 性質 12: $A[?]A$ の中に B は入らない
- 性質 13: $B[?]B$ の中に A は入らない
- 性質 14: $C[?]A$ の中に B は入らない
- 性質 15: $B[?]C$ の中に A は入らない
- 性質 16: $B[?]C$ の中に C は入らない
- 性質 17: $C[?]A$ の中に C は入らない

3

最適解の構造

- ④ Large が無いとき
- [#3] Small が 1 つあるとき
- $A[?]A$, $A[?]C$ は A のみ入る
- $B[?]B$, $B[?]C$ は B のみ入る
- C は $C[?]C$ の中のみに入る
- 決まってないのは $A[?]B$ に A , B を何個入れるか
- これは ② と同様に **Subproblem1** の形に帰着できる

3

最適解の構造

- ④ Large がないとき
- [#4] Small がないとき
- まず $A[?]A, B[?]B, C[?]C$ に A, B, C を全部入れる
- うまく $A[?]B, B[?]C, C[?]A$ に入れると $A[]B, B[]C, C[]A$ のみに行ける
- これで明らかな下界を達成してる
- これは？の個数について帰納法で証明できる
- 帰納法をせずとも余裕のある方から貪欲に入れてけば良い
- よってこのケースは簡単

3

まとめ

- 以上より場合分けが全部できました
- 大変な場合分けでした

3

まとめ

- 以上より場合分けが全部できました
- ① Large が 3 つあるとき: 矛盾
- ② Large が 2 つあるとき: **Subproblem1** に帰着
- ③ Large が 1 つあるとき: **Subproblem2** に帰着
- ④ Large がないとき:
 - [#1] Small が 3 つあるとき: 矛盾
 - [#2] Small が 2 つあるとき: 矛盾
 - [#3] Small が 1 つあるとき: **Subproblem1** に帰着
 - [#4] Small がないとき: 簡潔に解ける

3

結論

- よってこの問題は **Subproblem1** と **Subproblem2** に帰着できた
- **Subproblem1** はクエリあたり $O(N^2)$ で解けた
- **Subproblem2** はクエリあたり $O(N^2 \log N)$ で解けた
- よって全体で $O(QN^2 \log N)$ で解けました
- イメージとして **Subproblem1** は 1 つ Small があって、他の文字が入ってくる部分のコストを小さくする
- **Subproblem2** は 1 つ Large があって、それが他にはみ出る部分のコストを小さくする

Subtask 4

$N \leq 5\,000$

4

今後の流れ

- この問題は **Subproblem1** と **Subproblem2** に帰着できる
- 各問題を高速化したりクエリに対応するようにするのが目標
 - **Subproblem1**
 - 数列 c が与えられる ($C[?]C$ の ? の個数に対応)
 - c の正の要素を 1 つ選んで減らす操作を Z 回行う ($C[?]C$ の ? に C を入れること)
 - この後の c について **罰金** を以下のように定める 最小化せよ
 - $c_i \geq 1$ なる i の個数を W とする (C 以外が入る $C[?]C$ の数)
 - c の部分和で a が作れるなら $2 * W$
 - 作れないなら $2 * W + 1$ (ペナルティが 2 となる分を足す)
 - $a \leq \text{sum}(c_i) - Z$ は成り立っています

4

今後の流れ

- この問題は **Subproblem1** と **Subproblem2** に帰着できる
- 各問題を高速化したりクエリに対応するようにするのが目標
 - **Subproblem2**
 - 3つの書店 D, E, F がある それぞれ本を販売している
 - 金貨 Y 枚と銀貨 Z 枚を持っている (B の個数と C の個数)
 - 変数 W を $W = 0$ で初期化し以下のように本を買う
 - 書店 D で本 i を金貨 d_i 枚で買う (B[?]B の ? に B のみを入れる)
 - 書店 E で本 i を銀貨 e_i 枚で買う (C[?]C の ? に C のみを入れる)
 - 書店 F で本 i を金貨と銀貨合わせて f_i 枚で買う (B[?]C の ? に B か C を入れる)
 -
 - D, E で本を買うたびに +2 して F で買うたびに +1 する
 - 操作後の W を最大化せよ

4

小課題 3 との違い

- N は小さいままだが Q は大きくなった
- 小課題 3 の解法をクエリに対応するようにしたい
- **Subproblem1** と **Subproblem2** でクエリにすると何が変わるかを考えよう
- まずどの形の文字列における ? の個数が重要かを考える

4 Subproblem1 をクエリに対応

- **Subproblem1** でクエリにすると何が変わるかを考えよう
- まずどの形の文字列における ? の個数が重要かを考える
- さっきは $C[?]C$ における ? の個数の列を使った
- X, Y, Z の値によっては $A[?]A, B[?]B$ におけるものが重要になりうる
- いずれの場合でもどれか 1 つの列が重要で、列の中身は端の文字が同じなら同じ つまり 3 通りのどれかが大事

4 Subproblem1 をクエリに対応

- **Subproblem1** でクエリにすると何が変わるかを考えよう
 - **Subproblem1**
 - 数列 c が与えられる ($C[?]C$ の ? の個数に対応)
 - c の正の要素を 1 つ選んで減らす操作を Z 回行う ($C[?]C$ の ? に C を入れること)
 - この後の c について **罰金** を以下のように定める 最小化せよ
 - $c_i \geq 1$ なる i の個数を W とする (C 以外が入る $C[?]C$ の数)
 - c の部分和で a が作れるなら $2 * W$
 - 作れないなら $2 * W + 1$ (ペナルティが 2 となる分を足す)
 - $a \leq \text{sum}(c_i) - Z$ は成り立っています

4 Subproblem1 をクエリに対応

- **Subproblem1** でクエリにすると何が変わるかを考えよう
- どの列を使うかでクエリを分ける $C[?]C$ の列を使うとする
- すると同じ列 c について以下のような形のクエリを処理することになる
- c の大きい方から w 個取り出しその部分和で $[l, r]$ を作ることができるか判定せよ
- クエリでは (w, l, r) が与えられます

4 Subproblem1 をクエリに対応

- **Subproblem1** でクエリにすると何が変わるかを考えよう
- c の大きい方から w 個取り出しその部分和で $[l, r]$ を作ることができるか判定せよ
- クエリでは (w, l, r) が与えられます
- 正確にはクエリにより変わる z から w を計算する必要がある
- これは w が小さい順に見ていけば部分和問題の DP テーブルを進めつつクエリに答えられる

4 Subproblem1 をクエリに対応

- **Subproblem1** でクエリにすると何が変わるかを考えよう
- c の大きい方から w 個取り出しその部分和で $[l, r]$ を作ることができるか判定せよ
- これは w が小さい順に見ていけば部分和问题の DP テーブルを進めつつクエリに答えられる
- DP[0] を用意 $\rightarrow w = 0$ のクエリを処理 $\rightarrow$
- DP[1] を用意 $\rightarrow w = 1$ のクエリを処理 $\rightarrow \dots$ というイメージ
- 全部で $O(N^2 + Q \log Q)$ 丁寧に sort すると $O(N^2 + Q)$

4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
 - **Subproblem2**
 - 3つの書店 D, E, F がある それぞれ本を販売している
 - 金貨 Y 枚と銀貨 Z 枚を持っている (B の個数と C の個数)
 - 変数 W を $W = 0$ で初期化し以下のように本を買う
 - 書店 D で本 i を金貨 d_i 枚で買う (B[?]B の ? に B のみを入れる)
 - 書店 E で本 i を銀貨 e_i 枚で買う (C[?]C の ? に C のみを入れる)
 - 書店 F で本 i を金貨と銀貨合わせて f_i 枚で買う (B[?]C の ? に B か C を入れる)
 -
 - D, E で本を買うたびに +2 して F で買うたびに +1 する
 - 操作後の W を最大化せよ

4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- 先ほどと同様に文字列の？に関する個数の列のうち、どれが欲しいかはクエリによって変わらう
- しかし列の中身が変わらないので、どの列を使うかでクエリを場合分けしておくとい
- $B[?]B$, $B[?]C$, $C[?]C$ の？に関する個数の列が重要とする

4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- クエリでは (y, z) が与えられる (金貨と銀貨の枚数)
- 書店の情報はクエリによらず同一
- あらかじめ全ての (y, z) について列挙することを考える
- まず d, e, f (本の値段) は sort してあるとしていい
- d は先頭から足していって y 以下となる最後のところで打ち切っていい それ以降の金貨で買える本は買えないため
- e についても同様

4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- あらかじめ全ての (y, z) について列挙することを考える
- このとき金貨と銀貨を同一視して良い
- 書店 D と E (それぞれ金貨、銀貨だけで買うやつ) から買う戦略を考えると示せる
- つまり $d = (2, 3, 3)$, $e = (1, 4, 10)$, $f = (1, 5)$, $(y, z) = (4, 5)$ なら
- $d = (2)$, $e = (1, 4)$ と打ち切っていい
- また d と e は $(1, 2, 4)$ とまとめて、通貨は 9 枚あるとしていい

4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- このとき金貨と銀貨を同一視して良い
- 書店 D と E (それぞれ金貨、銀貨だけで買うやつ) から買うことを考えると示せる
- D と E で買うと W は +2 で F では +1
- よってこの問題は価値が 1 か 2 かのナップサック問題となる
- 価値が 1 か 2 という設定を使いたい

4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- この問題は価値が 1 か 2 かのナップサック問題となる
- 価値が 1 なら → 安い方から貪欲
- 2 が増えるとどうなるか？ → 価値を 1 種類にしたい
- 価値 1 のものを 2 つ買えば価値 2 とみなせる
- 価値が 1 のものを買う個数の偶奇を固定し、価値が 1 のものを安い順に 2 つずつ merge して価値が 2 のものを作る

4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- 価値が 1 のものを買う個数の偶奇を固定し、価値が 1 のものを安い順に 2 つずつ merge して価値が 2 のものを作る
- $d = (2, 3, 3)$, $e = (1, 4, 10)$, $f = (1, 5)$, $(y, z) = (4, 5)$ なら
- $d = (2)$, $e = (1, 4)$ と打ち切る
- また d と e は $(1, 2, 4)$ とまとめて、通貨は 9 枚あるとしていい
- f から奇数買う $\rightarrow (1, 2, 4)$ から $9 - 1 = 8$ 枚で買う
- f から偶数買う $\rightarrow (1, 2, 4, 6 (= 1 + 5))$ から 9 枚で買う

4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- 価値が 1 のものを買う個数の偶奇を固定し、価値が 1 のものを安い順に 2 つずつ merge して価値が 2 のものを作る
- f から奇数買う $\rightarrow (1, 2, 4)$ から $9 - 1 = 8$ 枚で買う
- f から偶数買う $\rightarrow (1, 2, 4, 6 (= 1 + 5))$ から 9 枚で買う
- どれだけ買えるかは二分探索！
- 数列を小さい順に足していったらある値を超えるところを調べる

4 Subproblem2 をクエリに対応

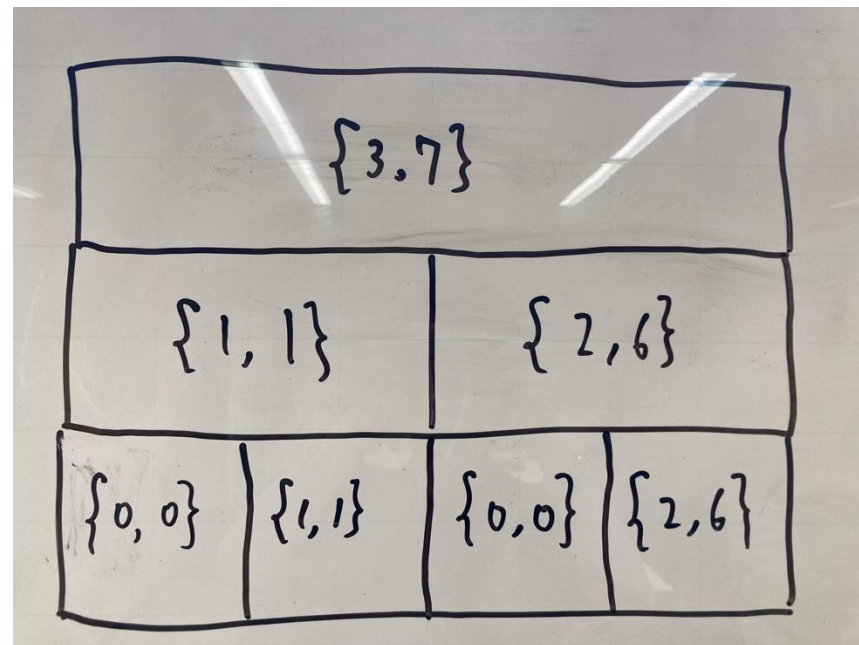
- **Subproblem2** でクエリにすると何が変わるかを考えよう
- これを全ての (y, z) について探索したい
- f から偶数買う場合を計算できれば同じことを 2 回すれば良い
- y を固定して z を増やしていく
- y が固定 $\rightarrow d$ を打ち切るところは同じ
- z が増えていく $\rightarrow e$ を打ち切るところが後ろになっていく
- つまり「前から足していってある値を超えるのはいつか」の二分探索の対象となる数列が増えていく

4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- これを全ての (y, z) について探索したい
- 以下を処理したい
- ① 数列にある値を追加
- ② 小さい順に足していってある値を超えるのはいつか計算
- ただ値は小さい
- どの値が数列に何回登場したかを管理できないか
- 「**個数と総和を管理する Segment Tree**」を使う
- Segment Tree のノードに {個数, 総和} の pair を置く

4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- 「**個数と総和を管理する Segment Tree**」を使う
- Segment Tree のノードに区間の {個数, 総和} の pair を置く
- (1, 3, 3) なら右図のようになる
- ① 値の追加
- これはいつもと同じ
- $O(\log N)$



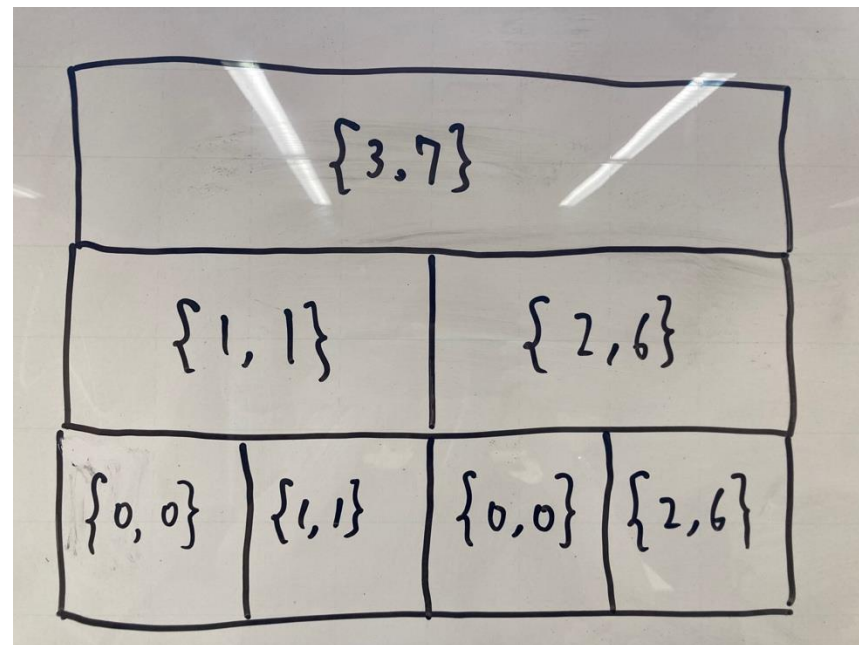
4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- ②小さい順に足していったある値を超えるのはいつか計算
- これを「どの値まで足して良いか」
- に変換する
- $(1, 3, 3)$ と 5 なら 2 まで足していい
- 1 から M まで全て足せるような
- M を求めたい

{3, 7}			
{1, 1}		{2, 6}	
{0, 0}	{1, 1}	{0, 0}	{2, 6}

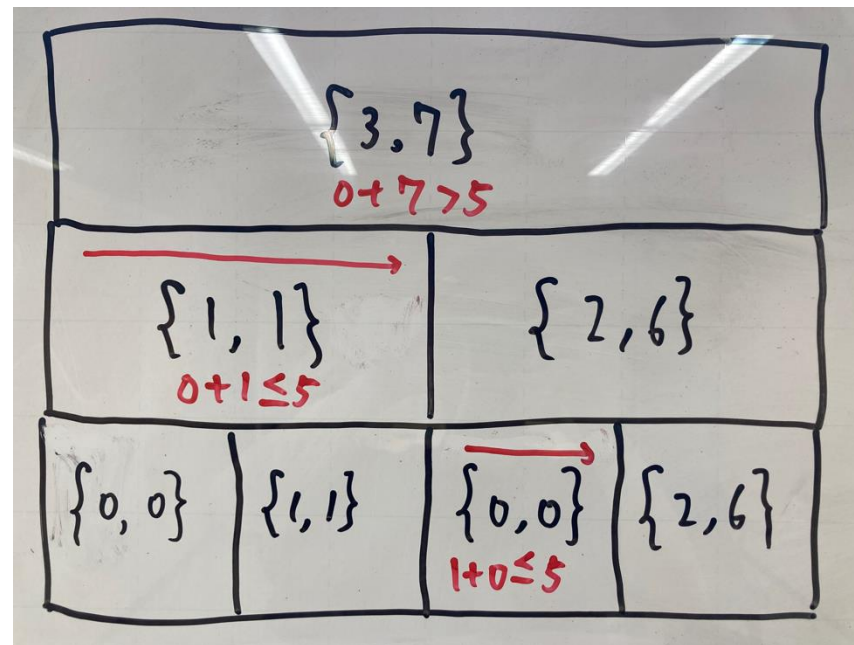
4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- ②小さい順に足していったらある値を超えるのはいつか計算
- 1 から M まで全て足せるような
- M を求めたい
- Segment Tree 上の二分探索！
- M で二分探索する $\rightarrow O(\log^2 N)$



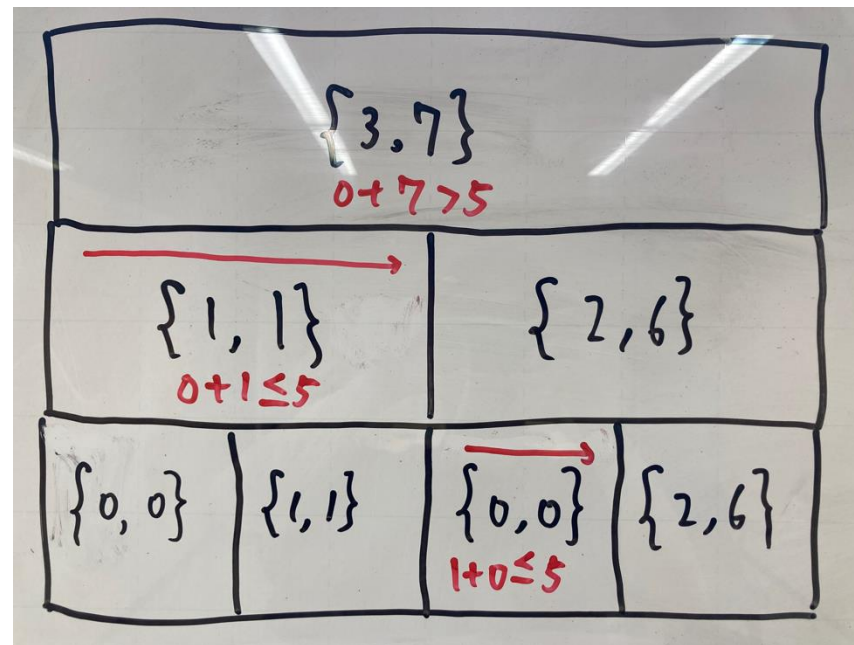
4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- Segment Tree 上の二分探索
- なんと $O(\log N)$ にもできる
- 図のイメージで答えを上位の bit から
- 決めつつ下のノードへ向かう
- インターネットに色々あります



4 Subproblem2 をクエリに対応

- **Subproblem2** でクエリにすると何が変わるかを考えよう
- y を固定したときの (y, z) について $O(N \log N)$ で計算できた
- y を動かすと $O(N^2 \log N)$ で全ての
- (y, z) について計算できる
- よって $O(N^2 \log N + Q)$ で解けた



- **Subproblem1:** 部分和問題の DP を c が大きい順に埋めつつクエリを処理していく
- **Subproblem2:** 書店 F で買う個数の偶奇を固定して考えると貪欲アルゴリズムになる Segment Tree を使う
- **Subproblem1:** $O(N^2 + Q)$ や $O(N^2 + Q \log Q)$
- **Subproblem2:** $O(N^2 \log N + Q)$ や $O(N^2 \log^2 N + Q)$
- よって全体では $O(N^2 \log N + Q)$
- セグ木の二分探索で $\log$ がもう 1 つつく $O(N^2 \log^2 N + Q)$ はかなり TLE になりそう

Subtask 5

$$Q \leq 10$$

5

小課題 3 との違い

- Q は小さいままだが N は大きくなった
- 小課題 3 の解法を高速化したい
- **Subproblem1** と **Subproblem2** で高速化できそうなところを探そう

5 Subproblem1 の高速化

- **Subproblem1** で高速化できるところを考えよう
 - **Subproblem1**
 - 数列 c が与えられる ($C[?]C$ の ? の個数に対応)
 - c の正の要素を 1 つ選んで減らす操作を Z 回行う ($C[?]C$ の ? に C を入れること)
 - この後の c について **罰金** を以下のように定める 最小化せよ
 - $c_i \geq 1$ なる i の個数を W とする (C 以外が入る $C[?]C$ の数)
 - c の部分和で a が作れるなら $2 * W$
 - 作れないなら $2 * W + 1$ (ペナルティが 2 となる分を足す)
 - $a \leq \text{sum}(c_i) - Z$ は成り立っています

5 Subproblem1 の高速化

- **Subproblem1** で高速化できるところを考えよう
- 部分和問題は $DP[i][j] := i$ 番目まで見て総和が j を作れるかとする DP で $O(N^2)$ で解けるのでした
- なんとか高速化できないか
- 小手先の改善を考える
- 同じ値をまとめて遷移しよう
- $DP[i][j] := i$ までの値を処理して総和が j を作れるかとする

5 Subproblem1 の高速化

- **Subproblem1** で高速化できるところを考えよう
- $DP[i][j] := i$ までの値を処理して総和が j を作れるかとする
- $DP[i]$ から $DP[i + 1]$ を作る
- $i + 1$ なる値がないならそのまま
- n 個あるとする
- $DP[i + 1][j]$ が True となる条件は $DP[i][j], DP[i][j - (i + 1)], DP[i][j - 2(i + 1)], \dots, DP[i][j - n(i + 1)]$ のどれかが True
- これは $i + 1$ 個飛ばしの累積和で高速化できる

5 Subproblem1 の高速化

- **Subproblem1** で高速化できるところを考えよう
- よって $O((\text{値の種類数}) * N)$ になった
- 値の種類数は $O(N)$ なので定数倍しか早くならない

5 Subproblem1 の高速化

- **Subproblem1** で高速化できるところを考えよう
- よって $O((\text{値の種類数}) * N)$ になった
- 値の種類数は $O(N)$ なので定数倍しか早くならない
- ということではなく計算量が $O(N\sqrt{N})$ に改善している
- 値の総和は $O(N)$ なら値の種類数は $O(\sqrt{N})$ になっている
- 非自明に見えるが、実は $1 + 2 + \dots + \sqrt{N}$ が $N/2$ ぐらいなので成立している

5 Subproblem2 の高速化

- **Subproblem2** で高速化できるところを考えよう
 - **Subproblem2**
 - 3つの書店 D, E, F がある それぞれ本を販売している
 - 金貨 Y 枚と銀貨 Z 枚を持っている (B の個数と C の個数)
 - 変数 W を $W = 0$ で初期化し以下のように本を買う
 - 書店 D で本 i を金貨 d_i 枚で買う (B[?]B の ? に B のみを入れる)
 - 書店 E で本 i を銀貨 e_i 枚で買う (C[?]C の ? に C のみを入れる)
 - 書店 F で本 i を金貨と銀貨合わせて f_i 枚で買う (B[?]C の ? に B か C を入れる)
 -
 - D, E で本を買うたびに +2 して F で買うたびに +1 する
 - 操作後の W を最大化せよ

5 Subproblem2 の高速化

- **Subproblem2** で高速化できるところを考えよう
- 書店 D と書店 E で買う本の数を $O(N^2)$ 通り全探索する
- 書店 F でどれだけ買えるかを知りたい
- D と E で使った分から F でどれだけ使えるか分かる
- あとはどれだけ買えるかを二分探索する $O(\log N)$
- よって $O(N^2 \log N)$ などでは解けていた
- D と E で買う本の合計 M を全探索できないか

5 Subproblem2 の高速化

- **Subproblem2** で高速化できるところを考えよう
- D と E で買う本の合計 M を全探索できないか
- D と E で買ってない中で安い方を買うことを M 回繰り返す貪欲が最適
- D と E で合計 M 冊買うのに必要な金額は d, e を merge して sort しておけば二分探索で $O(\log N)$
- 残りで F で買える分も二分探索で $O(\log N)$
- つまり $O(N \log N)$ で解ける

5

まとめ

- **Subproblem1:** 部分和問題の DP を更新するとき同じ値を一緒に更新するとなんと計算量が改善します
- **Subproblem2:** 書店 D, E で買う個数の合計を固定して考えると必要な金額の計算は F と同様の貪欲アルゴリズムになります
- **Subproblem1:** $O(N\sqrt{N})$
- **Subproblem2:** $O(N\log N)$
- よって全体では $O(QN\sqrt{N})$

Subtask 6

S に C が無い, $Z = 0$

6

ふりかえり

- 小課題 3 のところでやったのでここではおさらい
- 以下の戦略が最適
- $A + (? \text{ が何個か}) + A$ の形について短い順に貪欲に A を入れる
- $B + (? \text{ が何個か}) + B$ の形について短い順に貪欲に B を入れる
- あとは適当に入れる
- $A[?]A$ の $?$ の個数を並べた列と $B[?]B$ の個数を並べた列を sort して累積和 + 二分探索するとクエリあたり $O(\log N)$ で解ける
- 全体で $O(N + Q \log N)$

6

ふりかえり

- 別方針
- あらかじめ $\text{ans}[i] := ?$ に A を i 個入れる時の答え を用意する
- $\text{ans}[0]$ は簡単 全部に B を入れる
- $\text{ans}[i]$ から $\text{ans}[i + 1]$ を作りたい
- さっきの戦略を基に考えると $O(N)$ で $\text{ans}[i]$ が列挙できる
- 全体では $O(N + Q)$

Subtask 7

$$Z = 0$$

- 場合分けや証明、帰着させた後の問題が少し楽になります
- **Subproblem1:**
 - ? に C を入れることがないので部分和问题そのものになります
 - 部分和である区間のどれかが作れますかというもの
- $O(N\sqrt{N} + Q)$ など解けます

- 場合分けや証明、帰着させた後の問題が少し楽になります
- **Subproblem2:**
- 少しいい形になります $B[?]B$ と $B[?]C$ に B を入れて残りに A を入れる形 (もしくは A と B が逆の形) になりこれは価値が 1 と 2 のナップサック問題のようになります
- 価値が 1 の個数の偶奇を決めれば貪欲になります
- 小課題 4 でやったように価値が 1 のもの 2 つをまとめる

- 場合分けや証明、帰着させた後の問題が少し楽になります
- **Subproblem2:**
 - 小課題 4 でやったように価値が 1 のもの 2 つをまとめる
 - というか $Z = 0$ なので小課題 4 の解法と同じ
 - 小課題 4 では y を固定して z を増やしたが z を (0 に)固定して y を増やすイメージ
- $O(N \log N + Q)$ など解けます
- 今回は Segment Tree 上の二分探索で $\log$ が余計に 1 つつけて $O(N \log^2 N + Q)$ にしても間に合うと思います

- **Subproblem1:** 普通の部分和問題の形になります
- **Subproblem2:** 小課題 4 の前準備と同じことをすればいいです
 $Z = 0$ なので計算量から N が 1 つ落ちます
- $\log$ が 1 つ余分についても余裕だと思います

- **Subproblem1:** $O(N\sqrt{N} + Q)$
- **Subproblem2:** $O(N\log N + Q)$ や $O(N\log^2 N + Q)$

- よって全体では $O(N\sqrt{N} + Q)$

Subtask 8

追加制約なし

8 小課題 4 と小課題 5 の復習

- 小課題 4 では解法をクエリに対応できるようにした
- 小課題 5 ではクエリあたりを高速にした
- これの両方ができれば小課題 8 が解けそう
- 本当にできるのか

8 Subproblem1 でアイデアを合体

- **Subproblem1** で高速化とクエリ対応の両方をしよう
 - **Subproblem1**
 - 数列 c が与えられる ($C[?]C$ の ? の個数に対応)
 - c の正の要素を 1 つ選んで減らす操作を Z 回行う ($C[?]C$ の ? に C を入れること)
 - この後の c について **罰金** を以下のように定める 最小化せよ
 - $c_i \geq 1$ なる i の個数を W とする (C 以外が入る $C[?]C$ の数)
 - c の部分和で a が作れるなら $2 * W$
 - 作れないなら $2 * W + 1$ (ペナルティが 2 となる分を足す)
 - $a \leq \text{sum}(c_i) - Z$ は成り立っています

8

Subproblem1 アイデアを合体

- **Subproblem1** で高速化とクエリ対応の両方をしよう
- 高速化のアイデアは部分和問題の DP を i までの値を処理して総和が j を作れるかとするものだった
- クエリ対応のアイデアは DP テーブルの途中の状態でクエリに答えて、DP テーブルを更新してを繰り返すものだった
- この 2 つを合体しよう！

8 Subproblem1 でアイデアを合体

- **Subproblem1** で高速化とクエリ対応の両方をしよう
- クエリごとにどの DP テーブルを使うかを決める
- c_i を大きい方から M まで全部足して Z 以下となる最小の M が DP テーブルの 1 つめのキーとなる
- この M を求めるのは簡単
- 小課題 4 で使った Segment Tree を使ってもいいし c の中身は変わらないので累積和 + 二分探索でも良い

8

Subproblem1 でアイデアを合体

- **Subproblem1** で高速化とクエリ対応の両方をしよう
- あとは $DP[i][j] := i$ 以上の c_i の部分和で j を作れるか
- から $i - 1$ を追加で何個か追加してある範囲ができるかになる
- もはや文章では意味不明
- $c = (1, 3, 3, 4)$ で $Z = 5$ とかなら $1, 3$ まで消せる
- $DP[4]$ (4 以上の要素のみでできる部分和の情報) と 3 を使うか使わないかからある範囲が作れるかを知りたい

8 Subproblem1 でアイデアを合体

- **Subproblem1** で高速化とクエリ対応の両方をしよう
- $c = (1, 3, 3, 4)$ で $Z = 5$ とかなら 1, 3 まで消せる
- $DP[4]$ (4 以上の要素のみでできる部分和の情報) と 3 を何個使うか (と z) からある範囲が作れるかを知りたい
- とりあえずある数 a が作れるか (範囲の長さ 1) とすると $DP[4][[a, a + z]]$ と $DP[4][[a - 3, a - 3 + z]]$ のどちらかが True を確認
- 範囲にしても $DP[4][[\min(a), \max(a) + z]]$ と $DP[4][[\min(a) - 3, \max(a) - 3 + z]]$ のどちらかが True を確認すれば良い

8

Subproblem1 でアイデアを合体

- **Subproblem1** で高速化とクエリ対応の両方をしよう
- $c = (1, 3, 3, 4)$ で $Z = 5$ とかなら 1, 3 まで消せる
- DP[4] (4 以上の要素のみでできる部分和の情報) と 3 を何個使うか (と z) からある範囲が作れるかを知りたい
- DP[4][[min(a), max(a) + z]] と DP[4][[min(a) - 3, max(a) - 3 + z]] のどちらかが True を確認すれば良い
- ただ確認する範囲の数がこのままでは $O(N)$ になって TLE
- びえん

8

Subproblem1 アイデアを合体

- **Subproblem1** で高速化とクエリ対応の両方をしよう
- $DP'[i][j] := DP[i][j - k(i - 1)]$ が True となる最小の k とする
- そのような k がなければ $+\infty$ にする
- $DP[4][[\min(a), \max(a) + z]]$ と $DP[4][[\min(a) - 3, \max(a) - 3 + z]]$ のどちらかが True を確認するというのは
- $DP'[4][[\min(a), \max(a) + z]]$ の $\min \leq k$ になる

8

Subproblem1 アイデアを合体

- **Subproblem1** で高速化とクエリ対応の両方をしよう
- $DP'[i][j] := DP[i][j - k(i - 1)]$ が True となる最小の k とする
- そのような k がなければ $+\infty$ にする
- DP' の計算は DP 配列をにらめば $j = 0, 1, \dots$ の順に $O(N)$ で列挙できる
- 計算すべき i は $O(\sqrt{N})$ 種類なので全体で $O(N\sqrt{N})$
- 区間の min を取るのは Segment Tree でクエリあたり $O(\log N)$

8 Subproblem1 でアイデアを合体

- **Subproblem1** で高速化とクエリ対応の両方をしよう
- DP' の計算は DP 配列をにらめば i あたり $O(N)$
- 計算すべき i は $O(\sqrt{N})$ 種類なので全体で $O(N\sqrt{N})$
- 区間の min を取るのは Segment Tree でクエリあたり $O(\log N)$
- 全体では $O(N\sqrt{N} + Q\log N)$

8 Subproblem2 でアイデアを合体

- **Subproblem2** で高速化とクエリ対応の両方をしよう
 - **Subproblem2**
 - 3つの書店 D, E, F がある それぞれ本を販売している
 - 金貨 Y 枚と銀貨 Z 枚を持っている (B の個数と C の個数)
 - 変数 W を $W = 0$ で初期化し以下のように本を買う
 - 書店 D で本 i を金貨 d_i 枚で買う (B[?]B の ? に B のみを入れる)
 - 書店 E で本 i を銀貨 e_i 枚で買う (C[?]C の ? に C のみを入れる)
 - 書店 F で本 i を金貨と銀貨合わせて f_i 枚で買う (B[?]C の ? に B か C を入れる)
 -
 - D, E で本を買うたびに +2 して F で買うたびに +1 する
 - 操作後の W を最大化せよ

8 Subproblem2 でアイデアを合体

- **Subproblem2** で高速化とクエリ対応の両方をしよう
- 高速化のアイデアは D と E で買う本の合計を全探索するものだった
- クエリ対応のアイデアは F で買う本の偶奇を場合分けして 2 つの本をまとめるものだった
- この 2 つを合体しよう！

8

Subproblem2 アイデアを合体

- **Subproblem2** で高速化とクエリ対応の両方をしよう
- まずクエリについて d や e を金貨と銀貨で買える分で打ち切る
- これ以降はどうせ買えないのでないとしていい
- F で買う偶奇を固定すると結局 d の前何項と e の前何項と f を merge して sort して先頭から足していった値を超えるのがいつか調べるものになる

8

Subproblem2 でアイデアを合体

- **Subproblem2** で高速化とクエリ対応の両方をしよう
- d の前何項と e の前何項と f を merge して sort して先頭から足していってある値を超えるのがいつか調べるものになる
- 小課題 4 と同じように値の分布を管理してどの値までは全部足せるかを調べたい
- d, e, f それぞれについては簡単に計算できてこれらを足し合わせれば良い
- $d = (3, 6), e = (2, 2, 3, 3), f = (1, 8)$ で 2 までならそれぞれ {0 個, 総和 0}, {2 個, 総和 4}, {1 個, 総和 1} という感じ

8

Subproblem2 でアイデアを合体

- **Subproblem2** で高速化とクエリ対応の両方をしよう
- d, e, f それぞれについては簡単に計算できてこれらを足し合わせれば良い
- $d = (3, 6), e = (2, 2, 3, 3), f = (1, 8)$ で 2 までならそれぞれ {0 個, 総和 0}, {2 個, 総和 4}, {1 個, 総和 1} という感じ
- 足し合わせれば {3 個, 総和 5} になる
- この総和がある値以下になる最大の M が分かれば M 以下の個数がわかる
- $M + 1$ がいくつ足せるかは M 以下の総和から分かる

8 Subproblem2 でアイデアを合体

- **Subproblem2** で高速化とクエリ対応の両方をしよう
- d, e の打ち切りを考慮する必要があるが、これは d, e の累積和をとっておけば簡単にできる
- $d = (3, 6)$ で 2 までなら {0 個, 総和 0} などの情報を用意するのは `lower_bound` を使えば $O(\log N)$
- この操作は $O(\log N)$ 回あるのでクエリあたり $O(\log^2 N)$
- 前もって累積和を d, e, f について取得すれば操作が $O(1)$
- クエリあたり $O(\log N)$ になる

8 Subproblem2 でアイデアを合体

- **Subproblem2** で高速化とクエリ対応の両方をしよう
- この操作は $O(\log N)$ 回あるのでクエリあたり $O(\log^2 N)$
- 前準備でクエリあたり $O(\log N)$ になる
- F で買う個数の偶奇を固定したものがこれで解けた
- 2 回これをすればいい
- 全体で $O((N + Q \log N) \log N)$ や $O((N + Q) \log N)$ で解ける

- **Subproblem1:** $O(N\sqrt{N} + Q)$
- **Subproblem2:** $O((N + Q\log N)\log N)$ や $O((N + Q)\log N)$
- よって全体では $O(N\sqrt{N} + Q\log N)$ や $O(N\sqrt{N} + Q\log^2 N)$
- おそらく **Subproblem2** の前準備をサボって $O(N\sqrt{N} + Q\log^2 N)$ にしても間に合うのではないか
- ところで実装はいかがでしょうか

- 実装はとても大変そう
- ここに辿り着いた人はみんな辛いので遠くに行きたい気持ちになると思う
- 九州の写真を流そう

8

九州

- きれいだ
- 甕島大橋
- 鹿児島島の西にある



8

九州

- きれいだ 現実に戻る
- 関門大橋
- 地下を徒歩で歩ける



8

まとめ

- 実装はとても大変だと思う
- 実装力も大事！
- 大変な問題を捨てる決断力も大事！

Step 9

得点分布

9

得点分布

- 理想:

点数	1	2	3	4	5	6	7	8	人数	累計
100	0	0	0	0	0	0	0	0	27	27

- 選抜という意味では差がつかないな

得点分布

- 現実:

[illegible]

Multi-Communication 解説

ynymxiaolongbao

暇な人向け

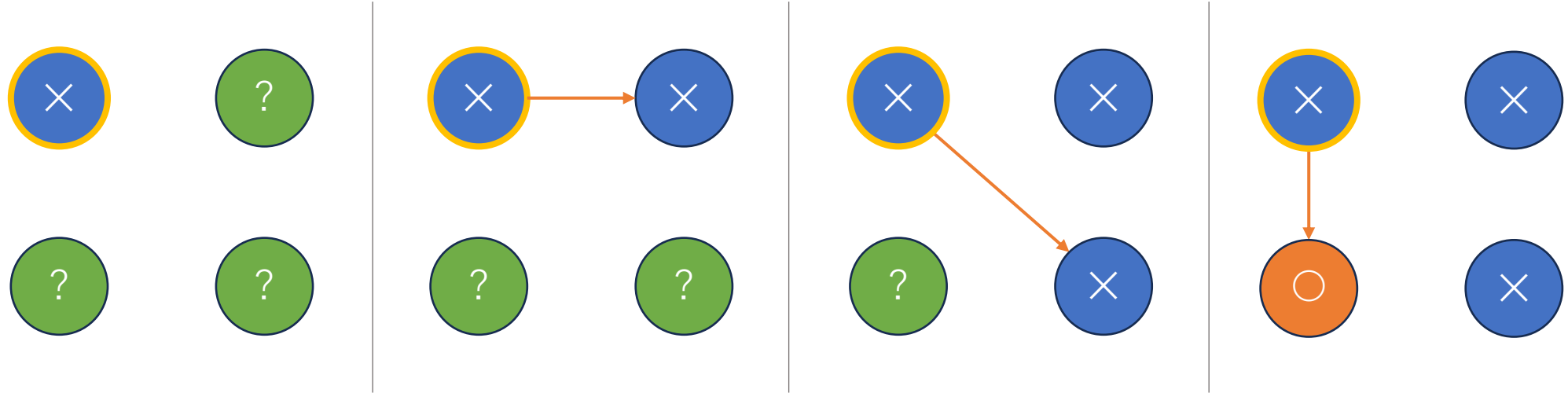
- 秘密の非負整数 x, y があり、Aliceは $(x, \lfloor \frac{y}{2} \rfloor)$ Bobは $(\lfloor \frac{x}{2} \rfloor, y)$ を伝えられる。この枠組みは二人とも知っている。
- 以下をすべて満たすような (x, y) ($x + y \leq 3$) をすべて挙げよ
- $x + y \leq 3$ であることを、二人とも知っている
- $x + y \leq 3$ であることを、二人とも知っていることを、二人とも知っている
- $x + y \leq 3$ であることを、二人とも知っていることを、二人とも知っていることを、二人とも知っている
- ...

問題概要

問題概要

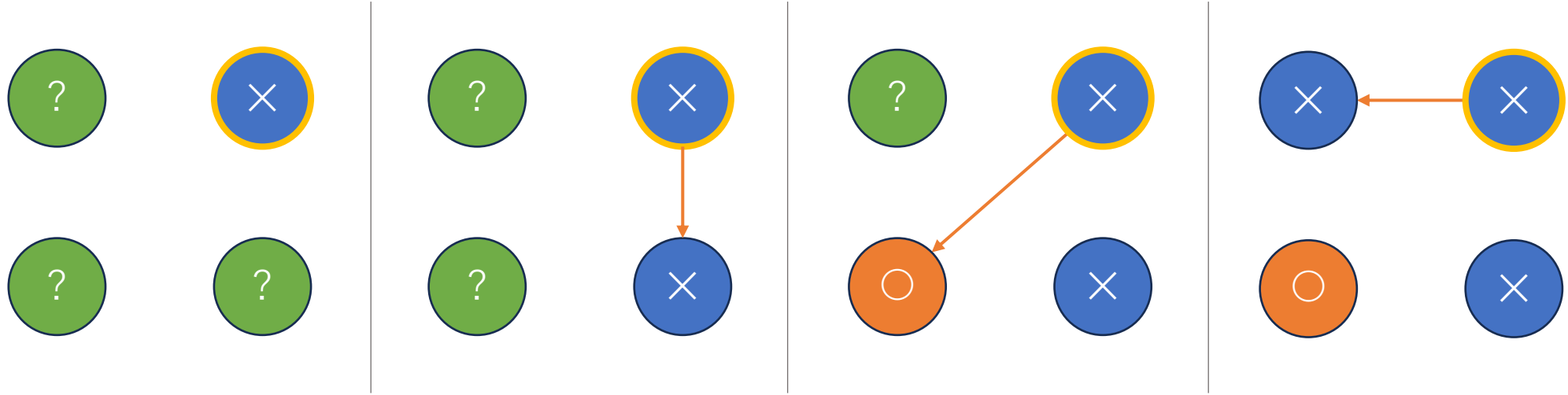
- N 人の人がいて、その中から一人、親が選ばれる。
 - 最初、各人は、自分が親かどうかだけ知っている。
 - L ターン行う。各ターンで、各人は、ボードに○か×を書き、別の人一人を指定してその人のボードを見ることができる。
 - 最後に、各人が、誰が親か答える。
-
- できるだけ小さいターン数 L で目標を達成せよ。

9点($N = 4, L = 3$)



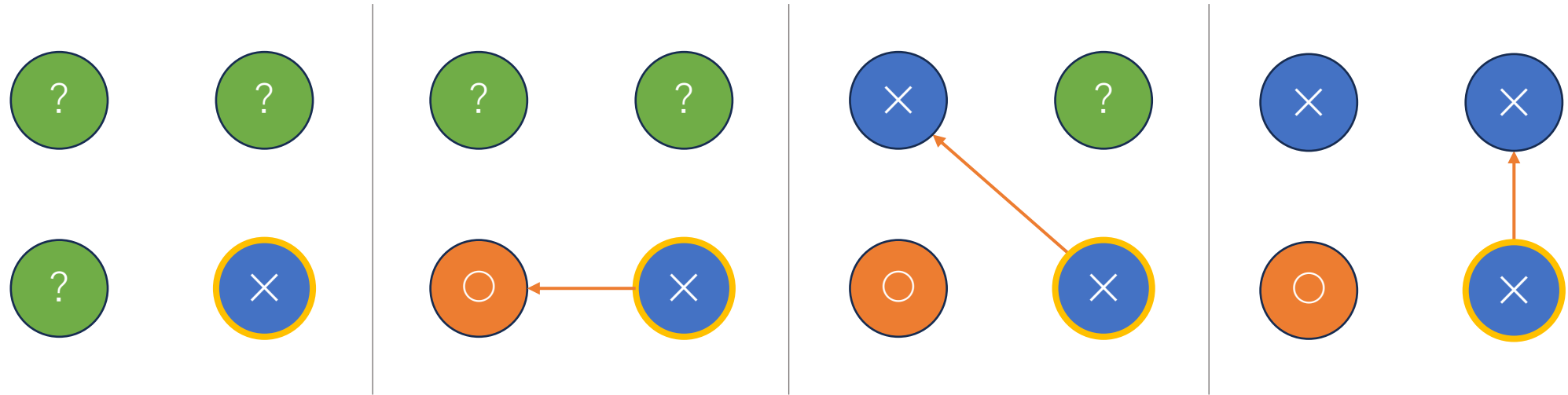
- ボードに書く文字 … 親は○, 子は×を書く
- 見る人 … 自分以外の $N-1$ 人を順番に見る
- 自分含む N 人全員の○×が分かる ○を書いていた人が親

9点($N = 4, L = 3$)



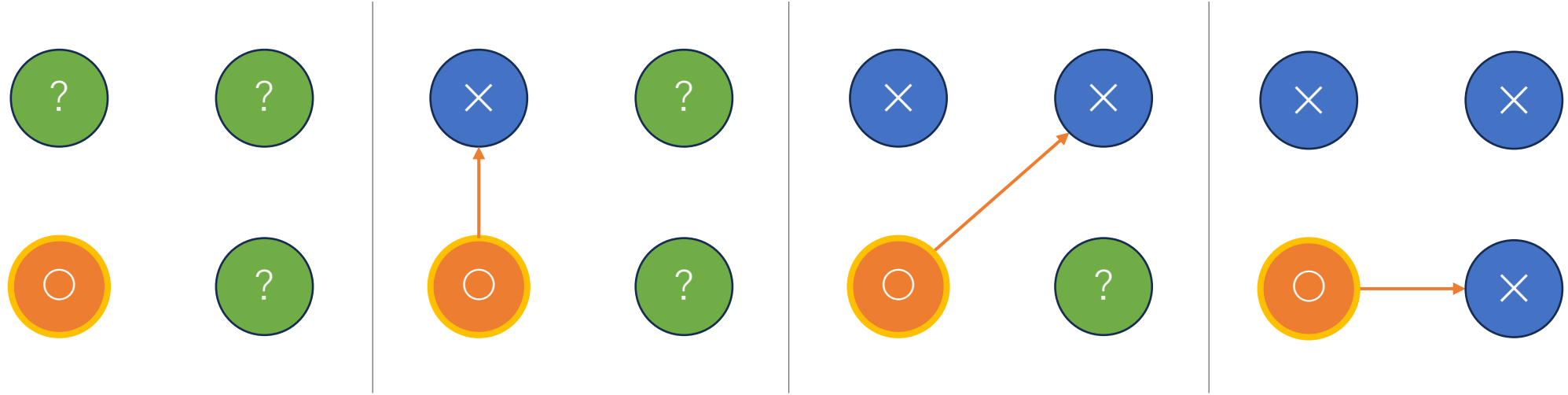
- ボードに書く文字 … 親は○，子は×を書く
- 見る人 … 自分以外の $N-1$ 人を順番に見る
- 自分含む N 人全員の○×が分かる ○を書いていた人が親

9点($N = 4, L = 3$)



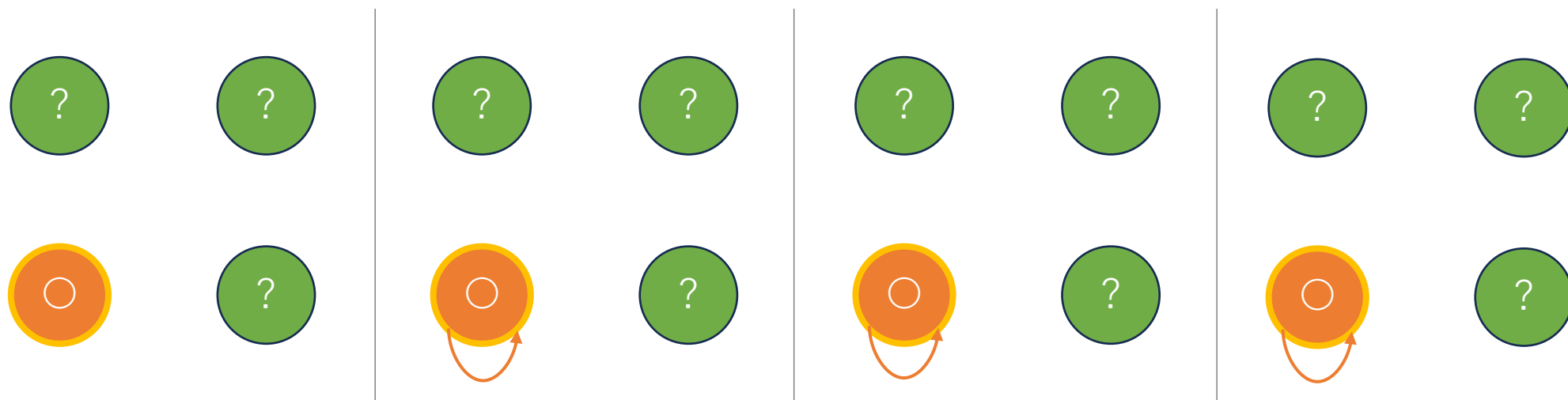
- ボードに書く文字 … 親は○，子は×を書く
- 見る人 … 自分以外の $N-1$ 人を順番に見る
- 自分含む N 人全員の○×が分かる ○を書いていた人が親

9点($N = 4, L = 3$)



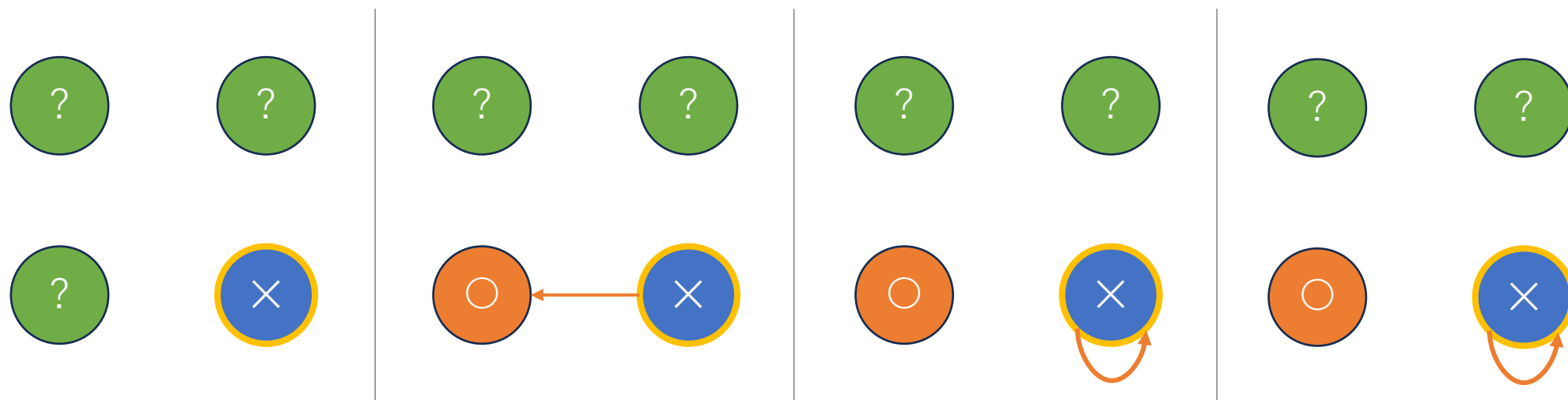
- ボードに書く文字 … 親は○, 子は×を書く
- 見る人 … 自分以外の $N-1$ 人を順番に見る
- 自分含む N 人全員の○×が分かる ○を書いていた人が親

9点($N = 4, L = 3$) 補足



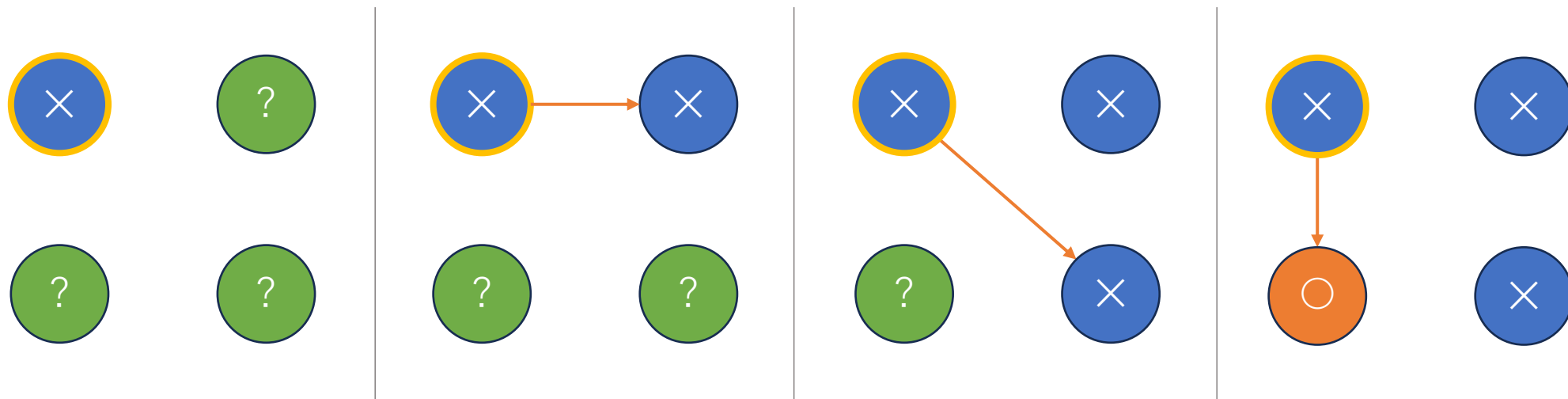
- $N-1$ 人全員を見るのではなく、「○を見つけたらその後は適当」でもよい
- この小課題では不要だが、後の小課題で実装が楽になる

9点($N = 4, L = 3$) 補足



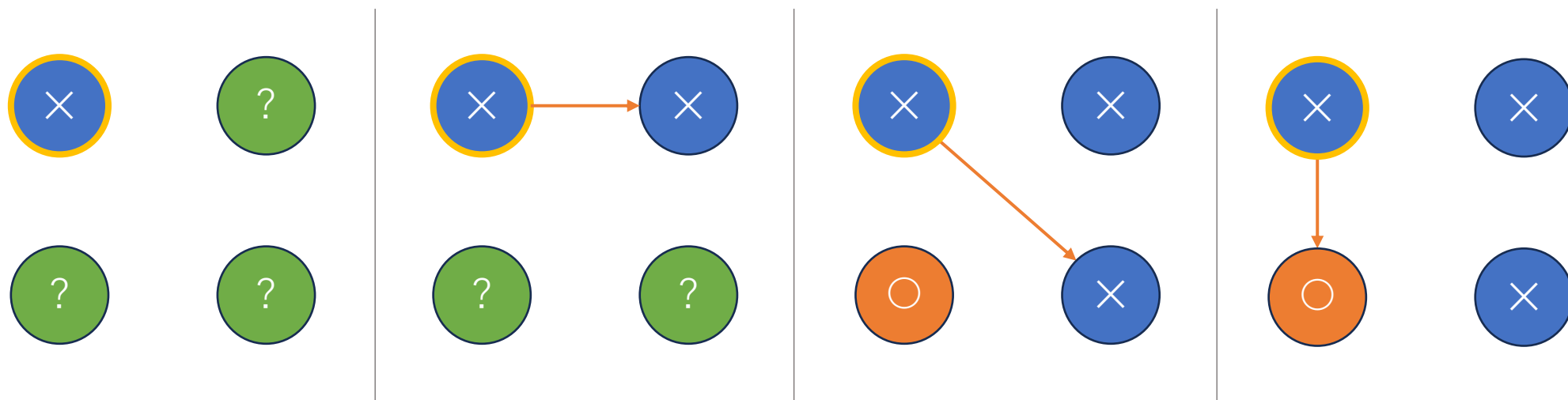
- $N-1$ 人全員を見るのではなく、「○を見つけたらその後は適当」でもよい
- この小課題では不要だが、後の小課題で実装が楽になる

16点($N = 4, L = 2$)



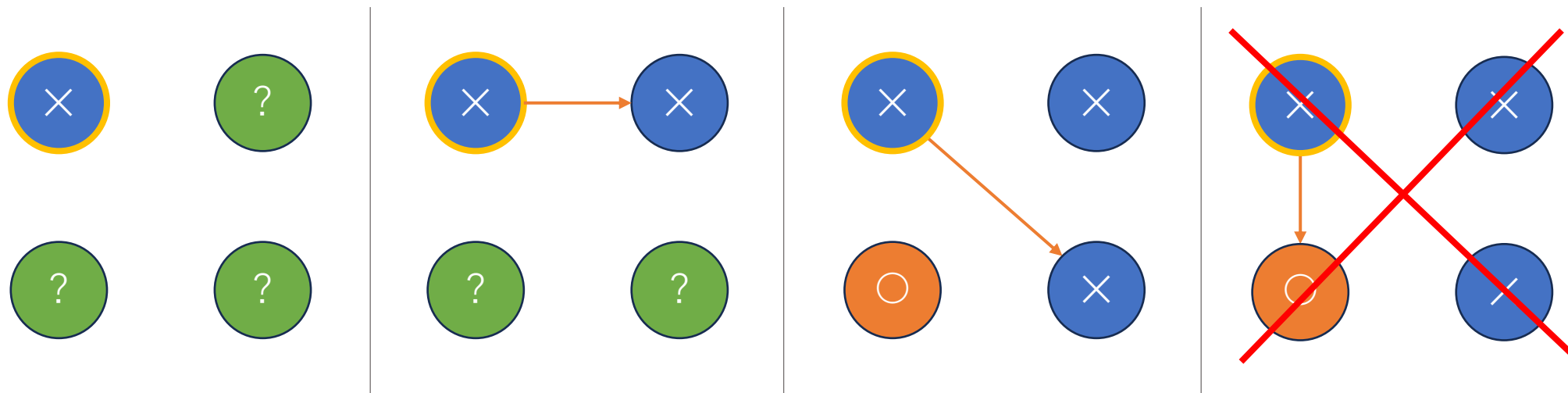
- $L = N - 1$ のときの戦略をしてみる

16点($N = 4, L = 2$)



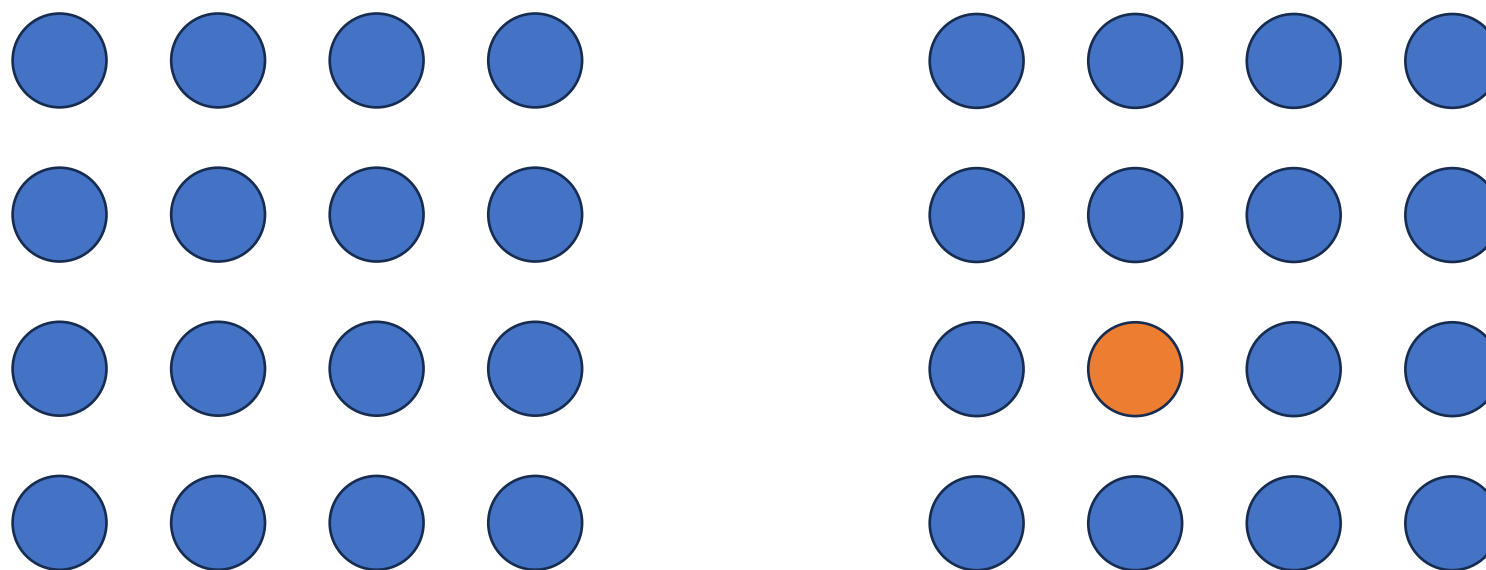
- $L = N - 1$ のときの戦略をしてみる
- 最後から2ターン目の時点で，消去法で全員の○×が分かる

16点($N = 4, L = 2$)



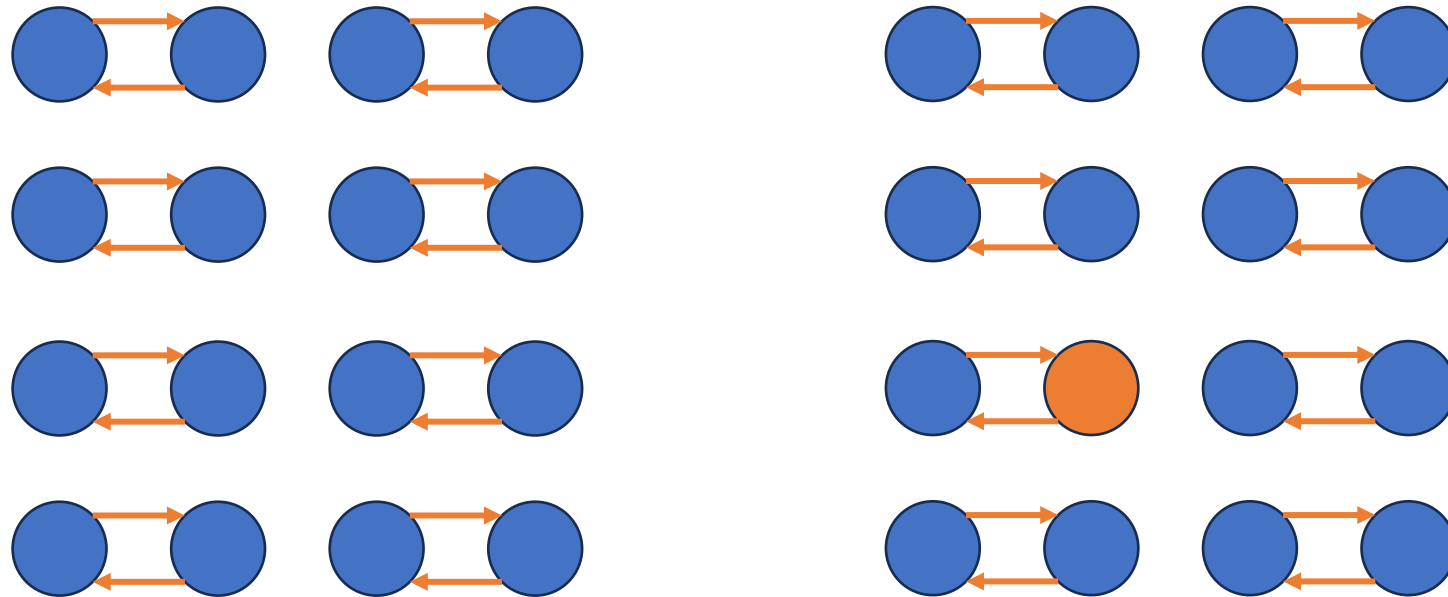
- $L = N - 1$ のときの戦略をしてみる
- 最後から2ターン目の時点で、消去法で全員の○×が分かる
- 最後のターンは無くても親（○）が誰か答えられる

40点($N = 32$, $L = 20$)



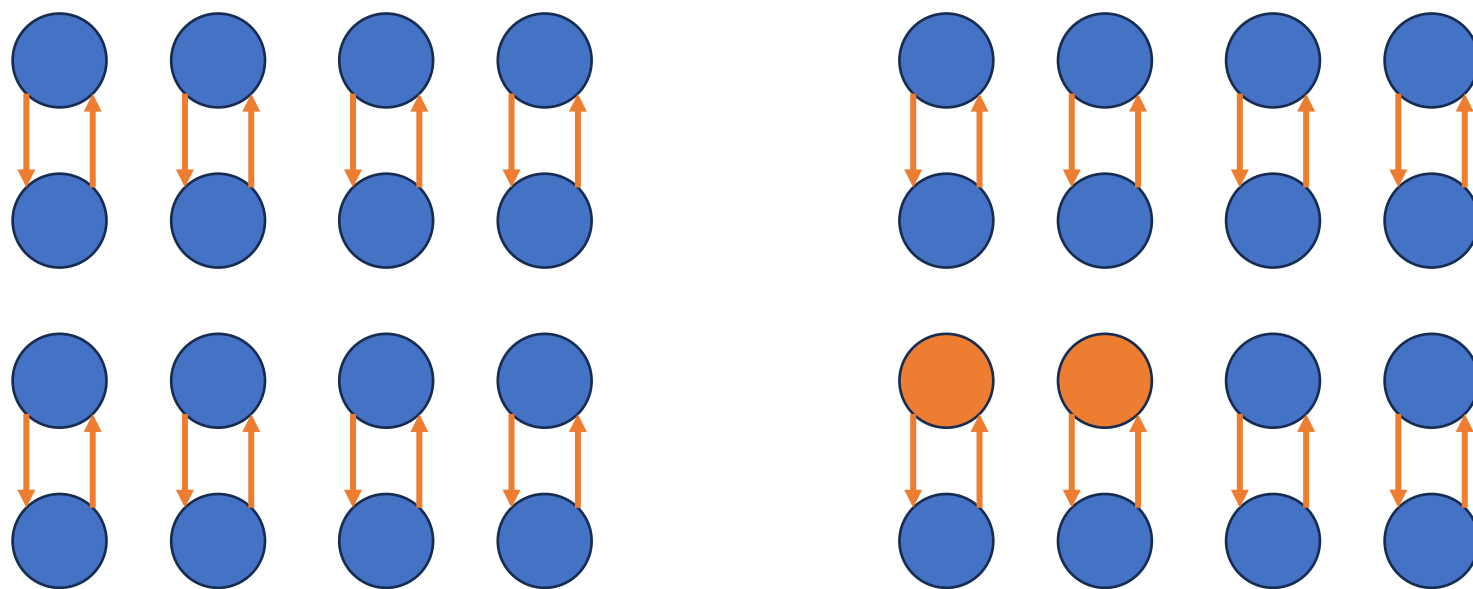
- N 人を、 $N/2$ 人ずつのグループに分ける
- 全員が、親がどちらのグループか分かるようにしたい

40点($N = 32$, $L = 20$)



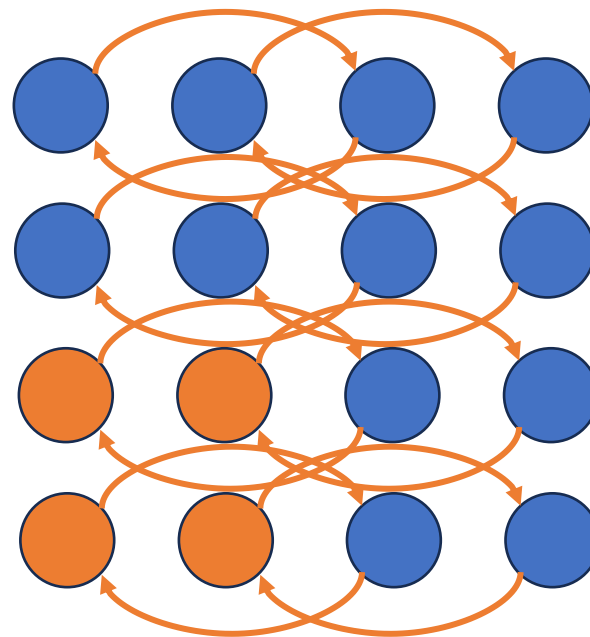
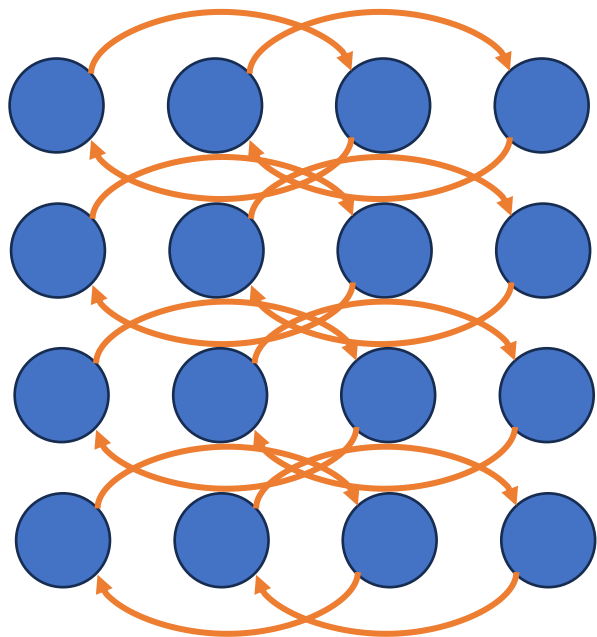
- 1, 2, 4, 8, 16人と、親が自分のグループにいることを知っている人を増やしていく

40点($N = 32, L = 20$)



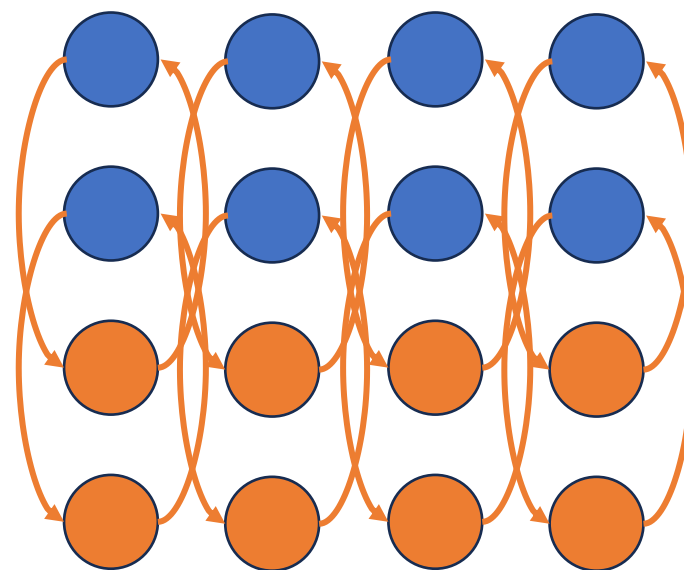
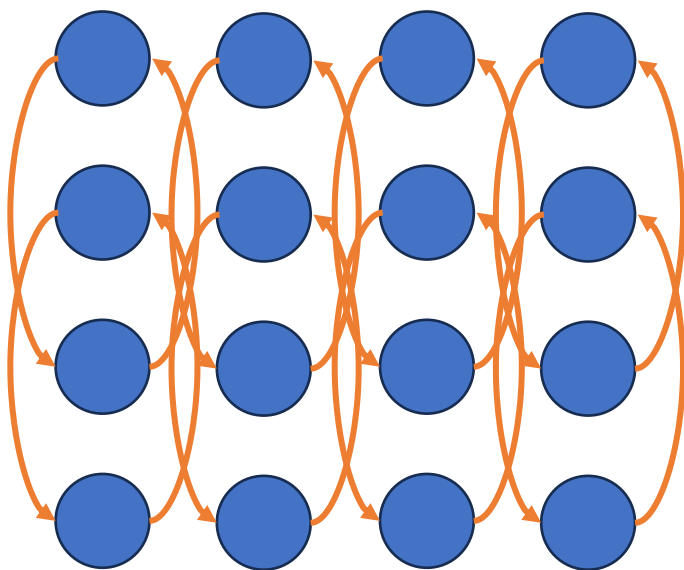
- 1, 2, 4, 8, 16人と、親が自分のグループにいることを知っている人を増やしていく

40点($N = 32, L = 20$)



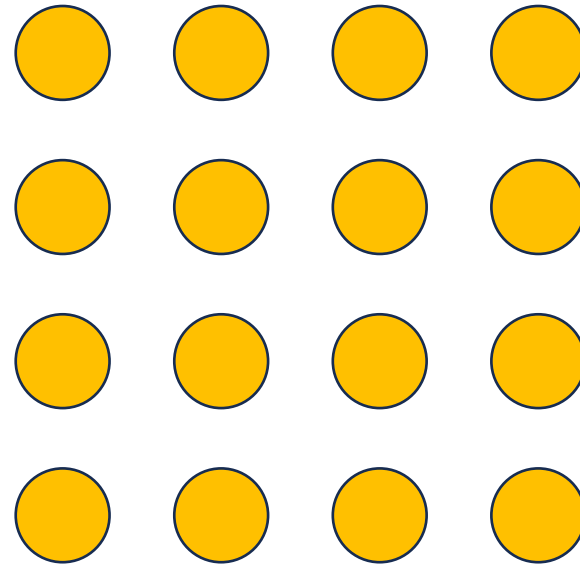
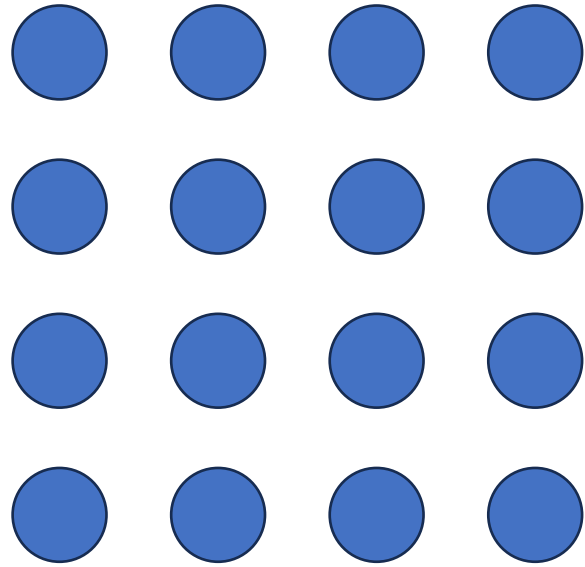
- 1, 2, 4, 8, 16人と、親が自分のグループにいることを知っている人を増やしていく

40点($N = 32, L = 20$)



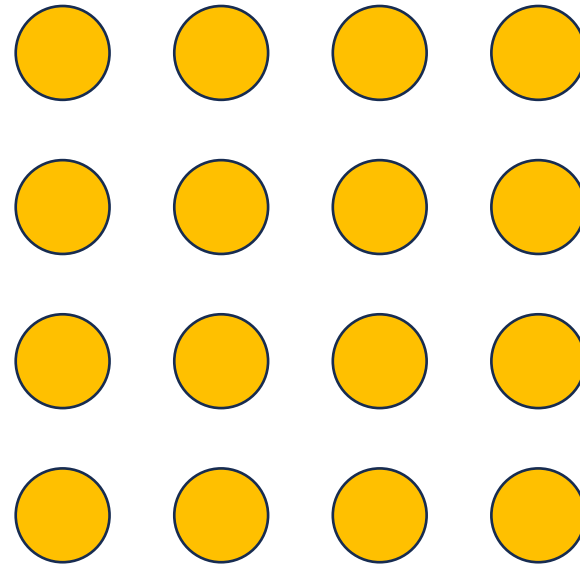
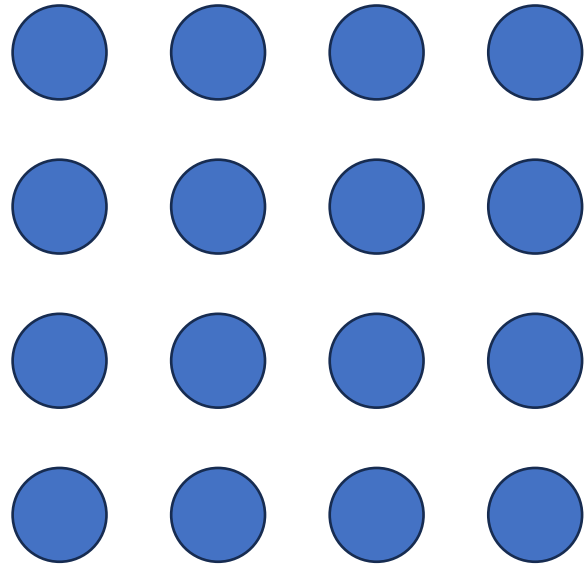
- 1, 2, 4, 8, 16人と、親が自分のグループにいることを知っている人を増やしていく

40点($N = 32, L = 20$)



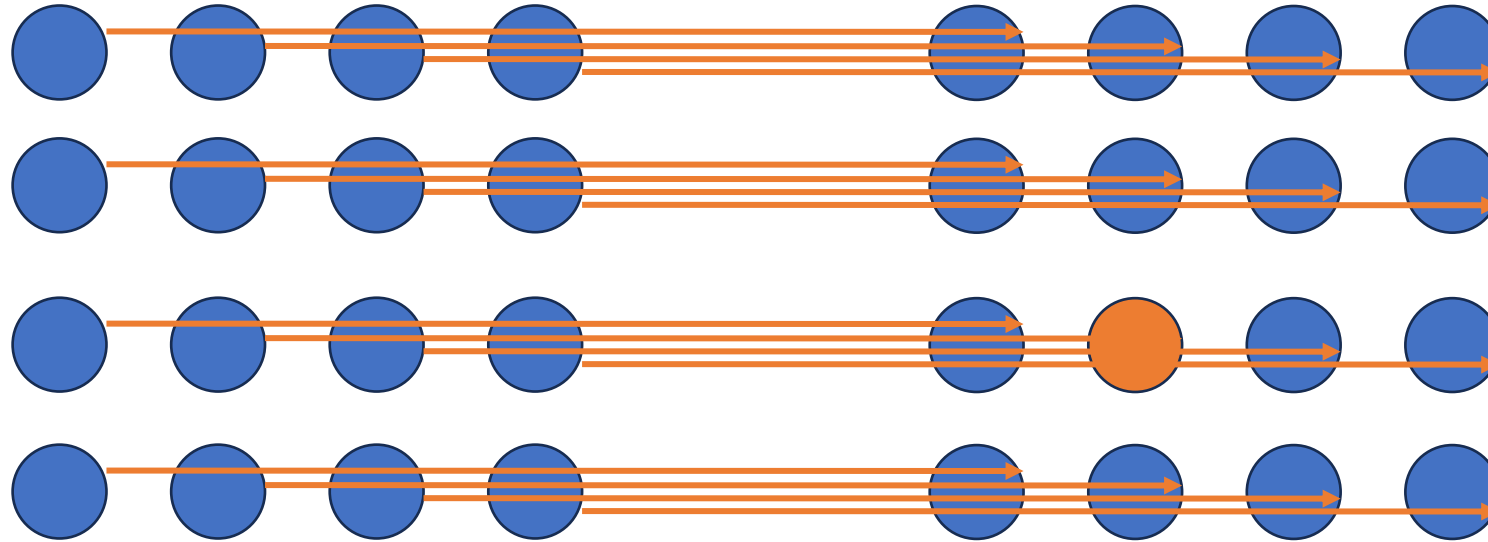
- 1, 2, 4, 8, 16人と、親が自分のグループにいることを知っている人を増やしていく
- $\log N - 1 (= 4)$ 回で、どちらに親がいるかが全員に共有された

40点($N = 32, L = 20$)



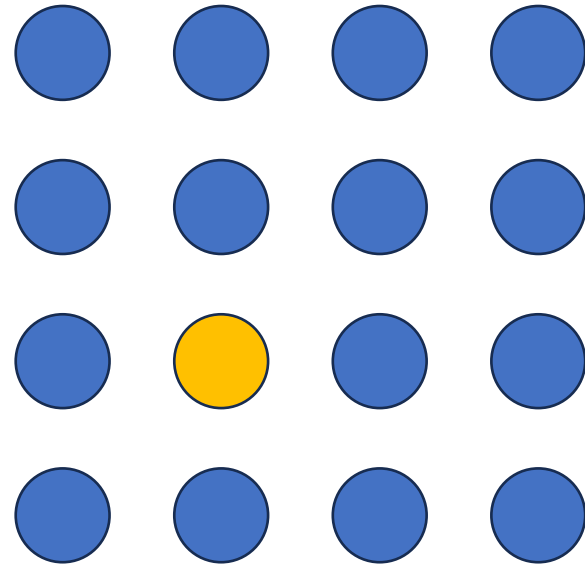
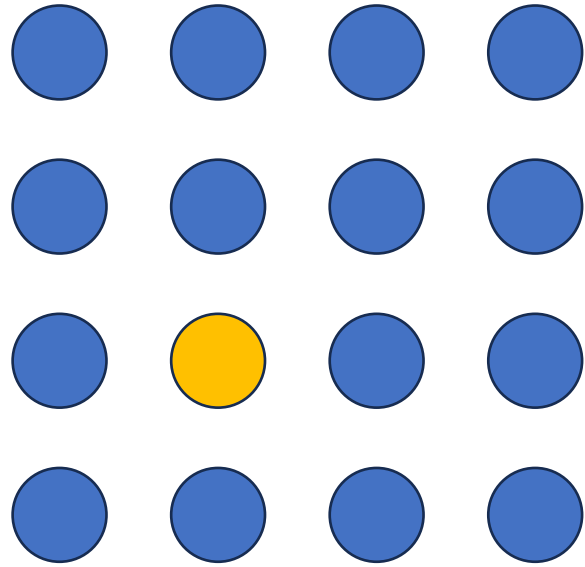
- 二進数で表したときのある桁の値(0か1か)でグループ分けし、どちらに親がいるか（親のその桁の値が0か1か）を全員で共有
- 各桁についてこれを行い、 $\log N * (\log N - 1) = 20$ ターン

58点($N = 32, L = 14$)



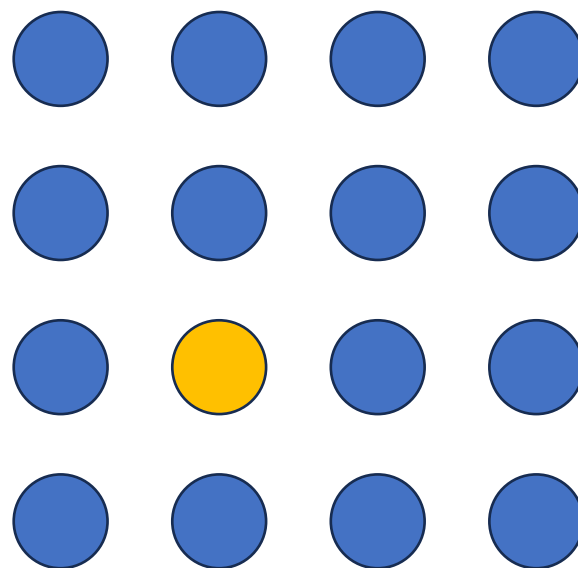
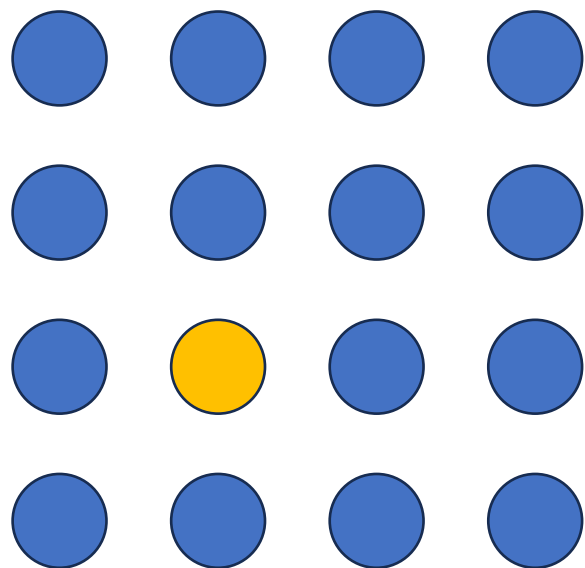
- 親の最上桁の値(0か1か)が分かった後、各人は自分の番号の最上桁を反転したら親の番号と同じになるか見る

58点($N = 32, L = 14$)



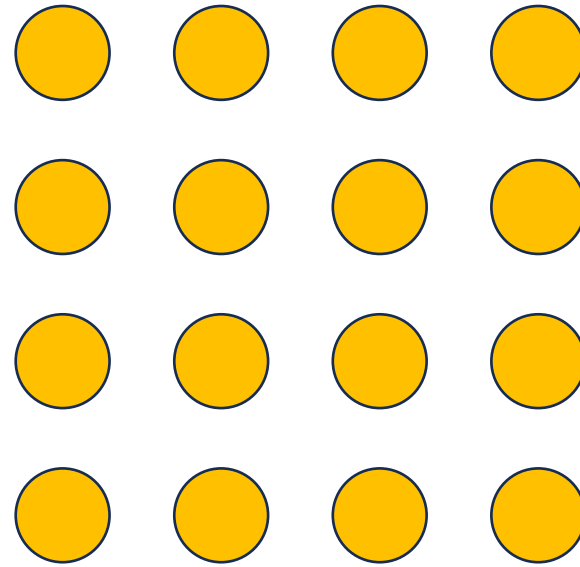
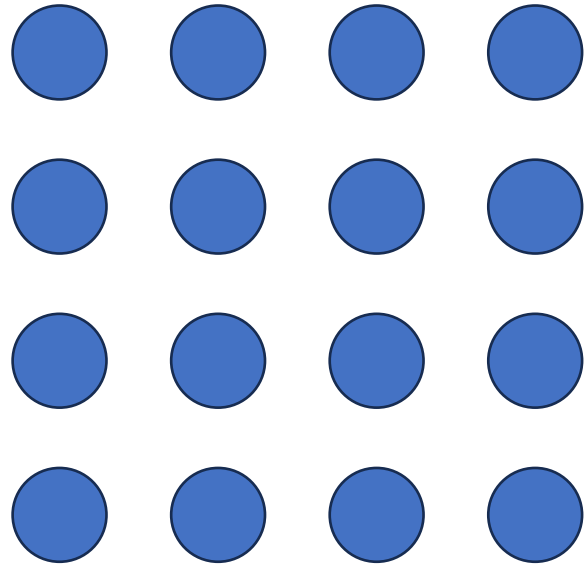
- 親の最上桁の値(0か1か)が分かった後、各人は自分の番号の最上桁を反転したら親の番号と同じになるか見る
- サイズが半分になった同じ問題二つに

58点($N = 32, L = 14$)



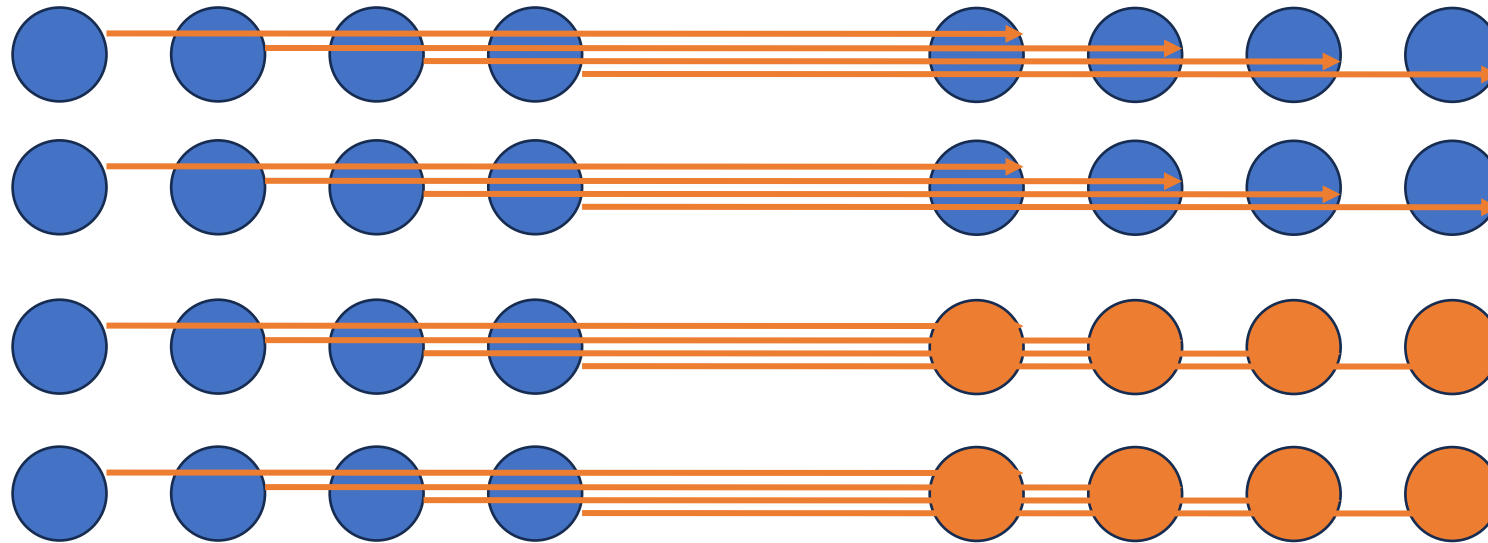
- $N = 2$ になるまで再帰
- $\log N + \dots + 3 + 2 = (\log N + 2) * (\log N - 1) / 2$ 回

76点($N = 32$, $L = 8$)



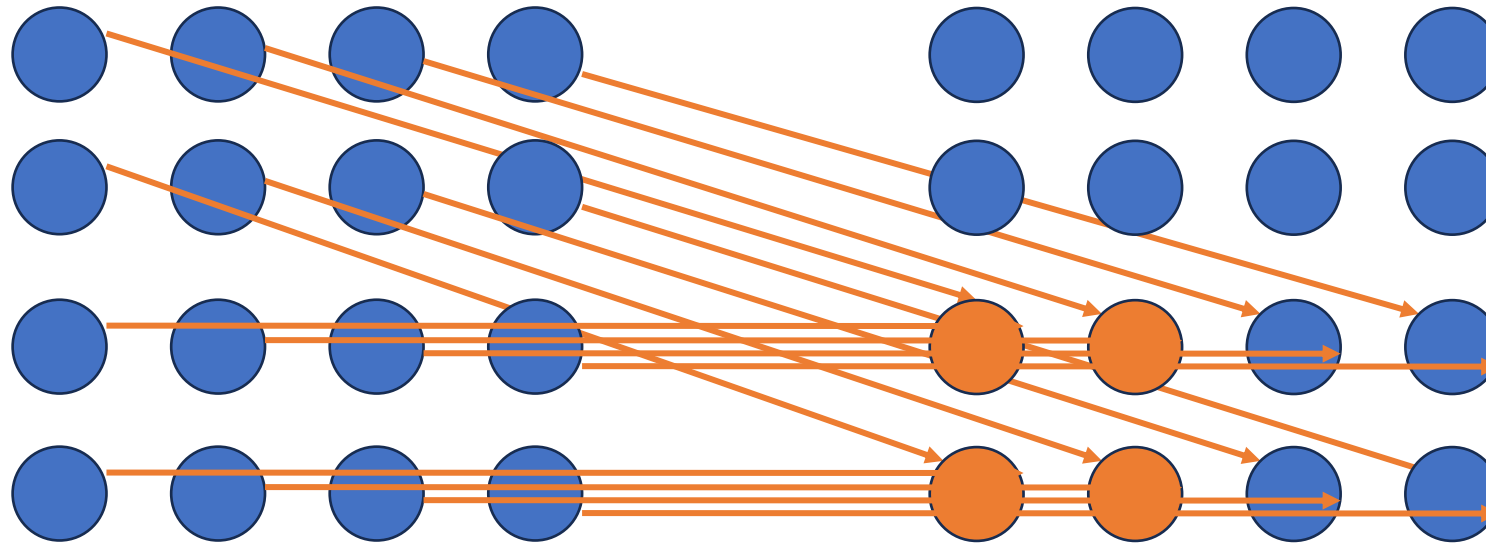
- 二つのグループに分かれた後、親がいるグループのサイズを16, 8, 4, 2, 1 と半分に絞って戻していく
- 見る時は、必ず親がいる方のグループを見るようにする

76点($N = 32$, $L = 8$)



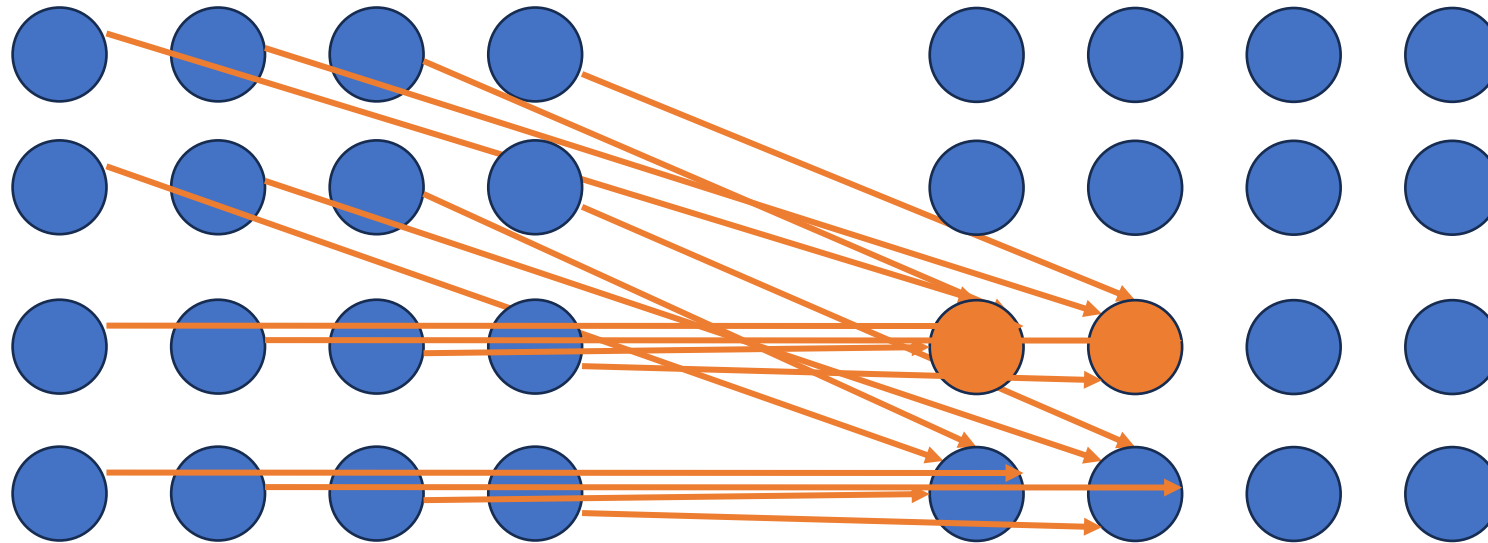
- 二つのグループに分かれた後、親がいるグループのサイズを16, 8, 4, 2, 1 と半分に絞って戻していく
- 見る時は、必ず親がいる方のグループを見るようにする

76点($N = 32$, $L = 8$)



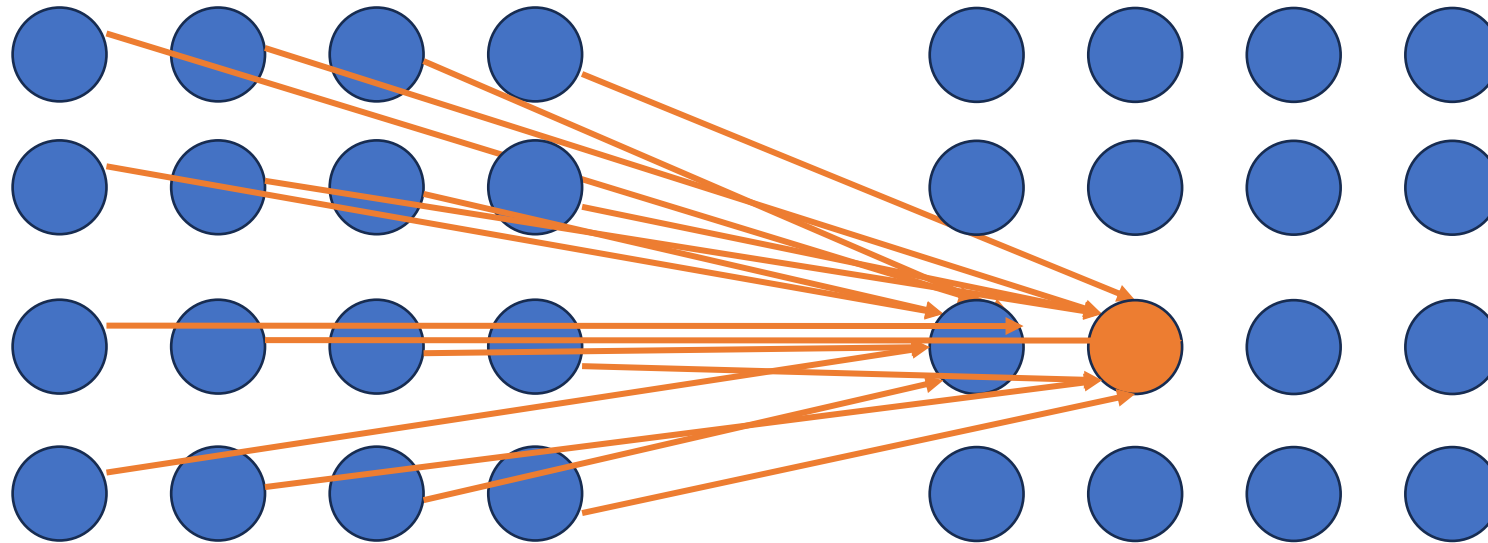
- 二つのグループに分かれた後、親がいるグループのサイズを16, 8, 4, 2, 1 と半分に絞って戻していく
- 見る時は、必ず親がいる方のグループを見るようにする

76点($N = 32$, $L = 8$)



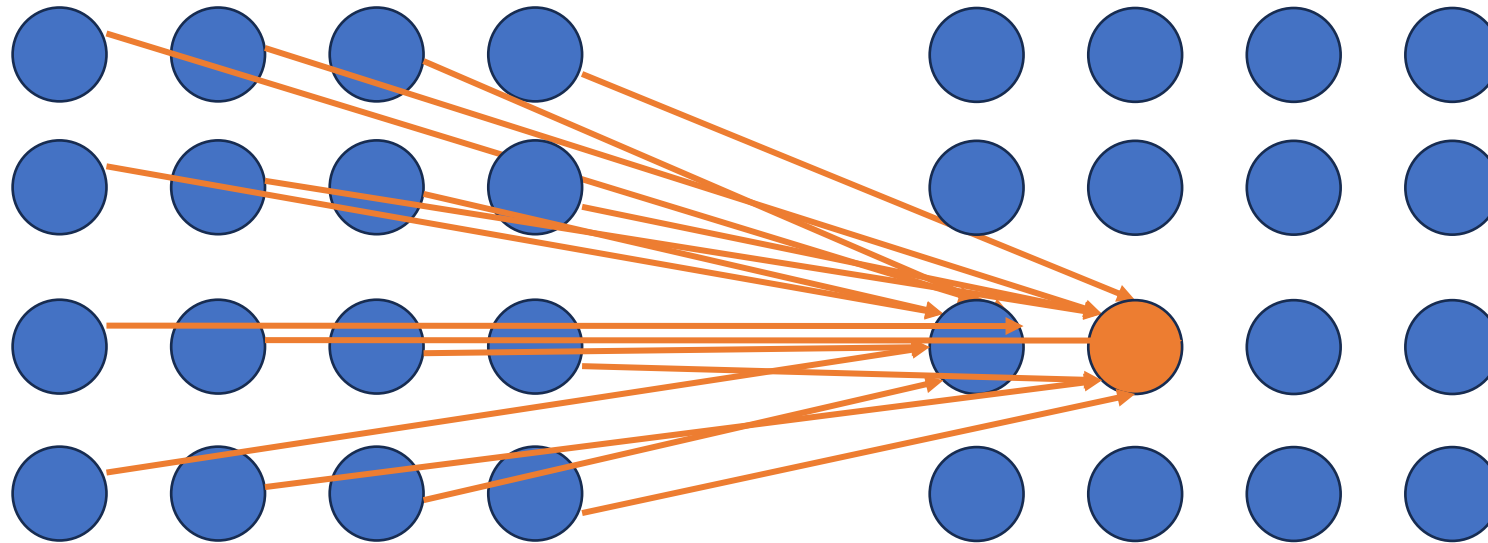
- 二つのグループに分かれた後、親がいるグループのサイズを16, 8, 4, 2, 1 と半分に絞って戻していく
- 見る時は、必ず親がいる方のグループを見るようにする

76点($N = 32$, $L = 8$)



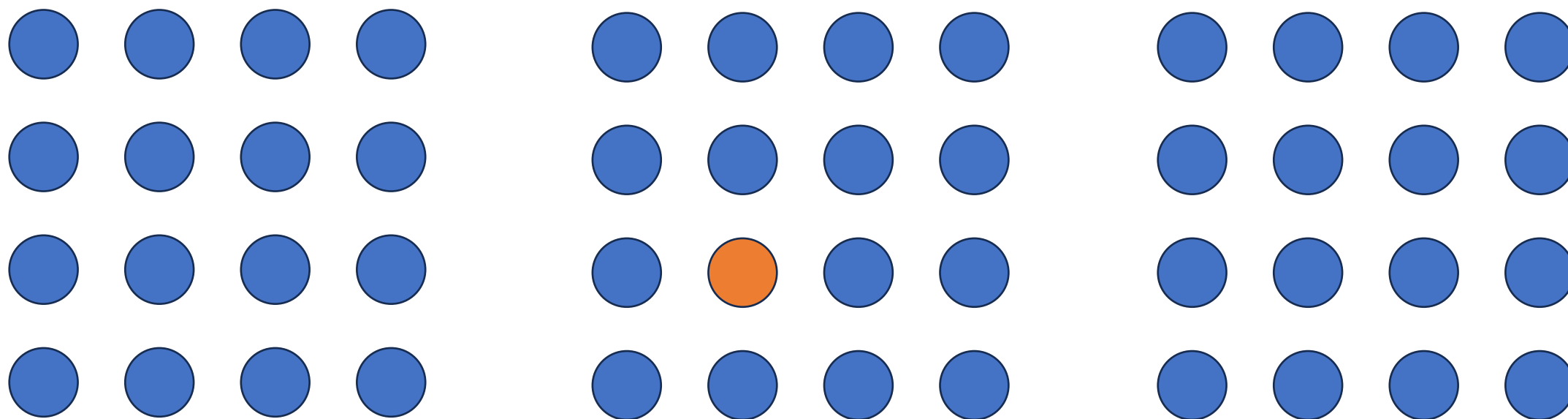
- 二つのグループに分かれた後、親がいるグループのサイズを16, 8, 4, 2, 1 と半分に絞って戻していく
- 見る時は、必ず親がいる方のグループを見るようにする

76点($N = 32, L = 8$)



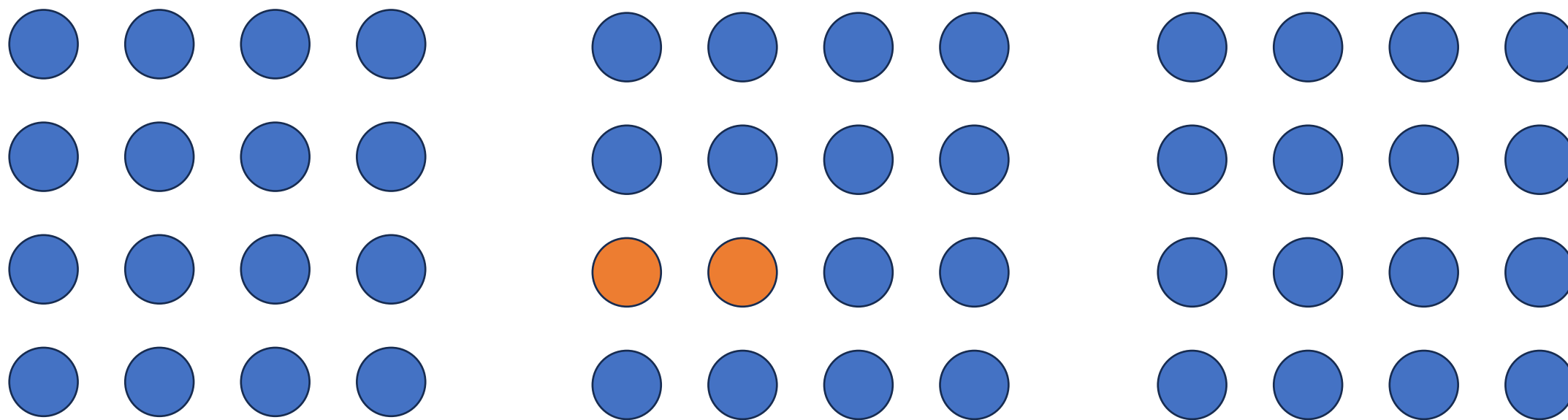
- $2(\log N - 1) = 8$ 回
- 右の16人が見る先は省略しているが、同様

100点($N = 48$, $L = 9$)



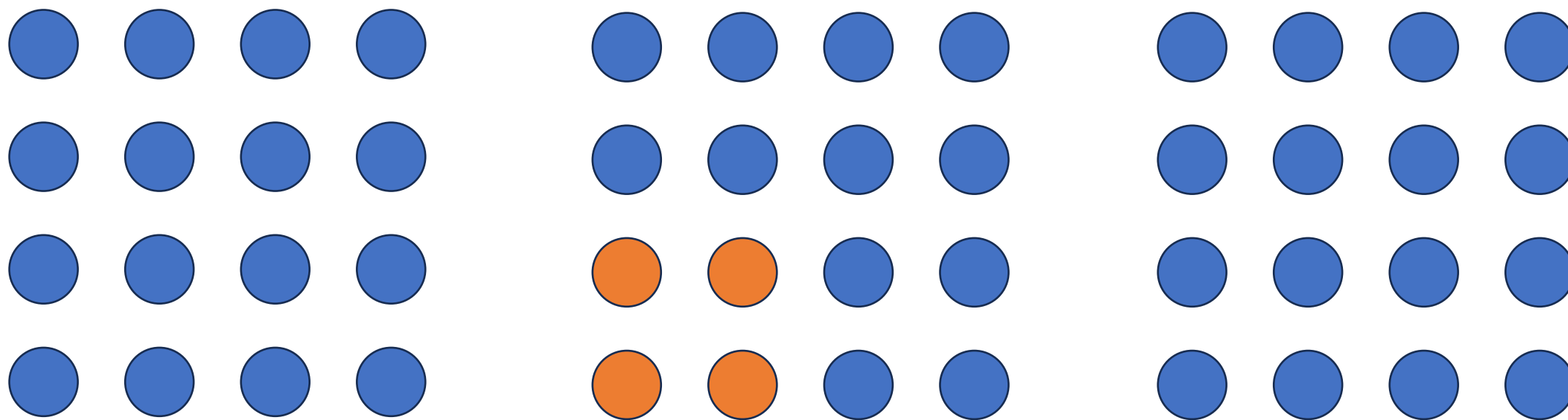
- 3グループに分かれ、自分のグループに親がいるかをまず把握

100点($N = 48$, $L = 9$)



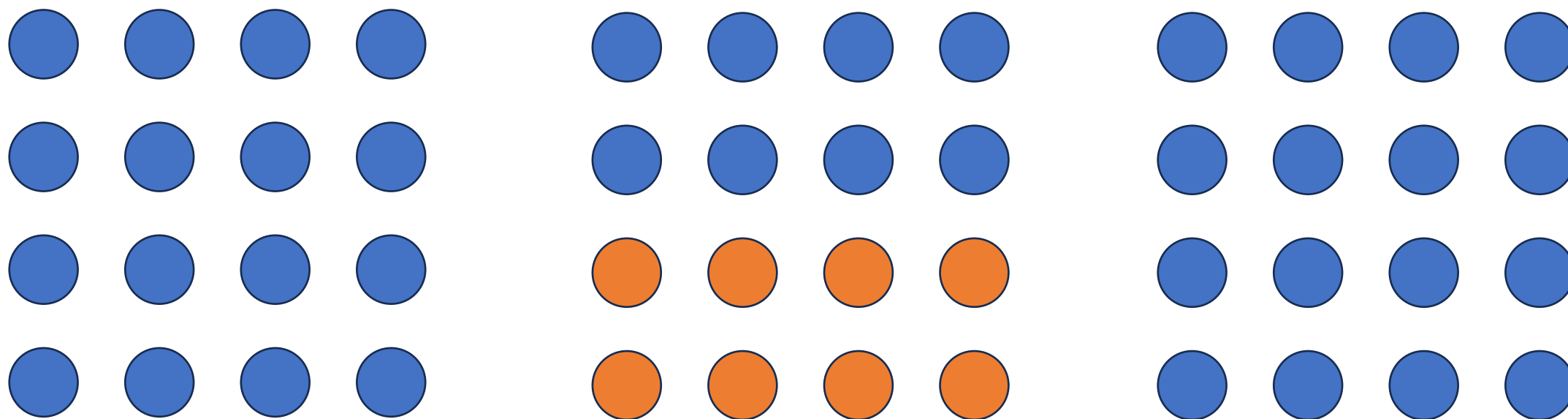
- 3グループに分かれ、自分のグループに親がいるかをまず把握

100点($N = 48$, $L = 9$)



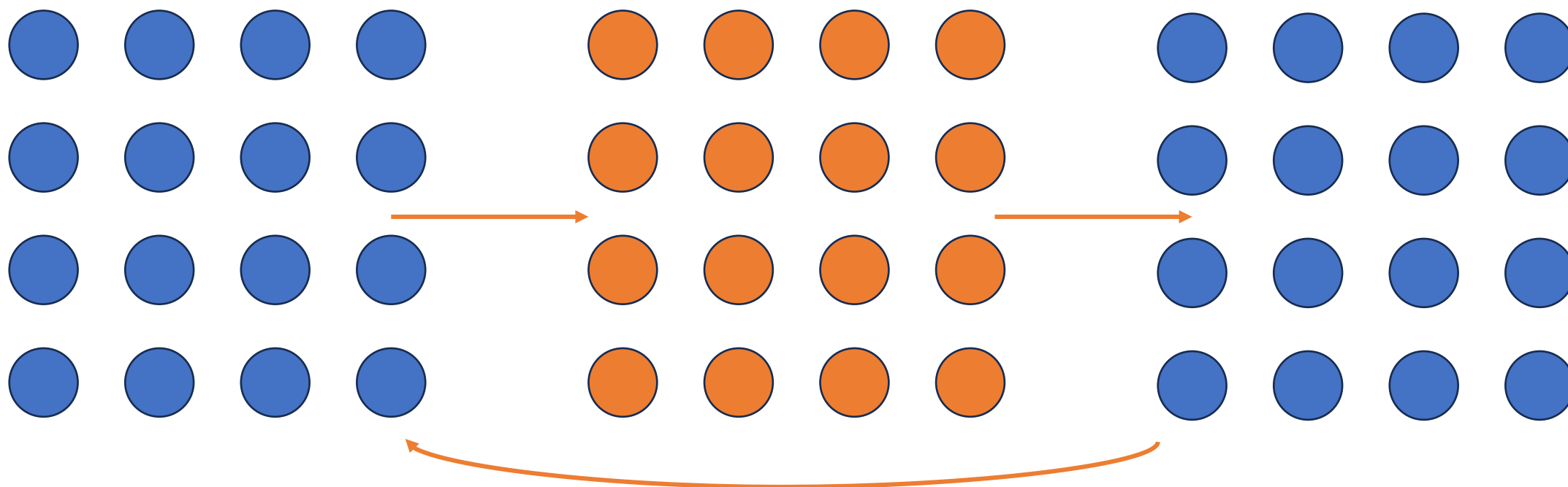
- 3グループに分かれ、自分のグループに親がいるかをまず把握

100点($N = 48$, $L = 9$)



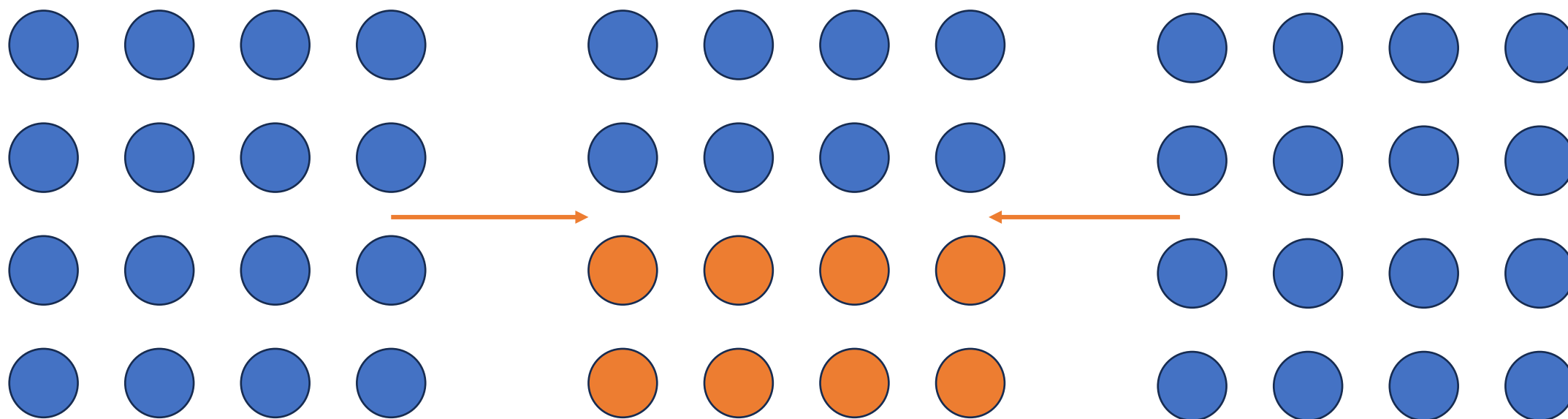
- 3グループに分かれ、自分のグループに親がいるかをまず把握

100点($N = 48$, $L = 9$)



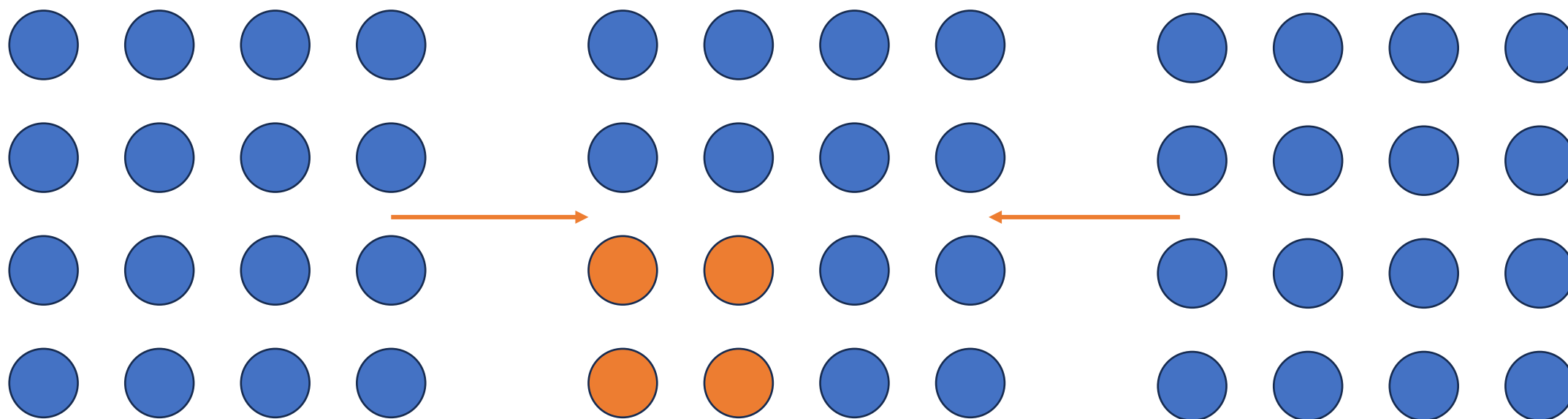
- 別のグループを一つ選んで、そのグループに親がいるかを見る
- 3グループのうち2つ見たので、消去法で親がいるグループが分かる

100点($N = 48, L = 9$)



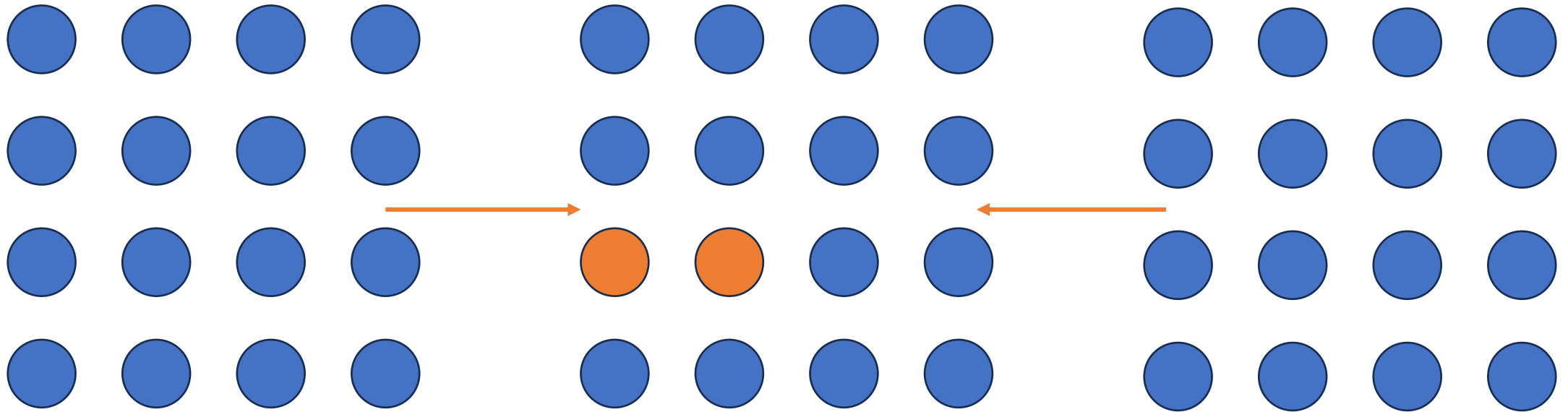
- 親がいるグループが特定できた後は前の小課題と同様
- $N = 3/2 M$ として、 $2(\log M - 1) + 1$ 回

100点($N = 48, L = 9$)



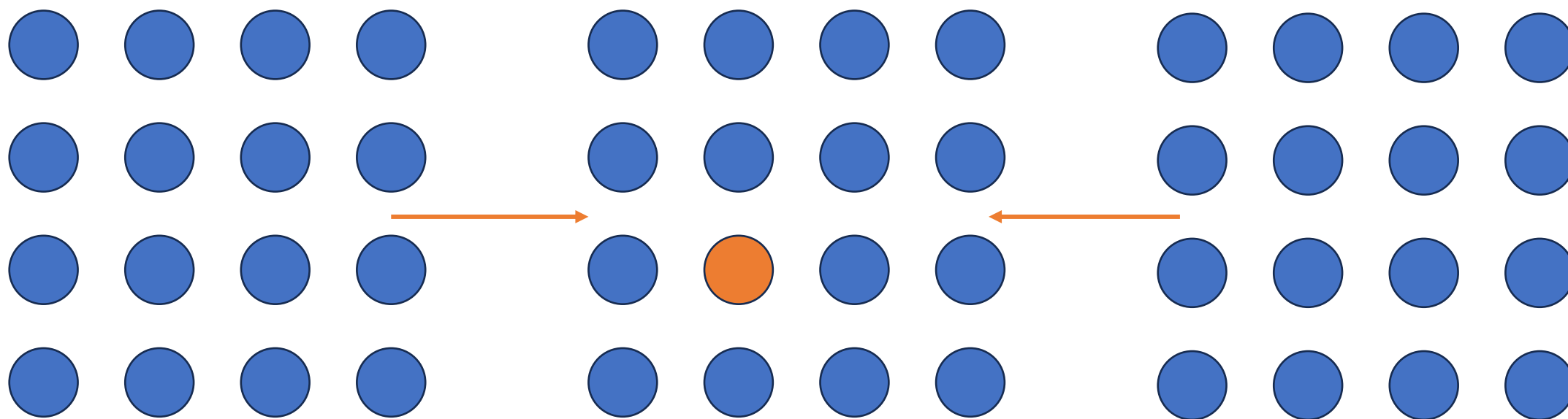
- 親がいるグループが特定できた後は前の小課題と同様
- $N = 3/2 M$ として、 $2(\log M - 1) + 1$ 回

100点($N = 48, L = 9$)



- 親がいるグループが特定できた後は前の小課題と同様
- $N = 3/2 M$ として、 $2(\log M - 1) + 1$ 回

100点($N = 48, L = 9$)



- 親がいるグループが特定できた後は前の小課題と同様
- $N = 3/2 M$ として、 $2(\log M - 1) + 1$ 回

最適性の証明

- L が偶数のとき, $N = 2^{\frac{L}{2}+1}$ が最大
- L が奇数のとき, $N = 1.5 * 2^{\frac{L-1}{2}+1}$ が最大
- L ターンを前 a 回と後 b 回に分ける ($2^a < N$)
- 各参加者が、前 a 回で「影響を受けた可能性がある（後述）」
参加者は 2^a 人 残りの $N - 2^a$ 人は区別することができない
- $N - 2^a$ 人の中に親が入ることがある（仮にはじめに全員が×を読んだときの挙動を考えればよい）
- 後 b 回で絞れるのは 2^b 人まで（得られる情報は毎回1bit）
- $N \leq 2^a + 2^b$

最適性の証明

- ある時刻で「参加者 x が参加者 y から影響を受けた可能性がある」とは
- 参加者 y がボードに書いた文字を、参加者 x がこれまでに見た
- 参加者 y がボードに書いた文字を、見た参加者がその後ボードに書いた文字を、参加者 x がこれまでに見た
- 参加者 y がボードに書いた文字を、見た参加者がその後ボードに書いた文字を、見た参加者がその後ボードに書いた文字を、参加者 x がこれまでに見た
- ...
- 自分が影響を受けた参加者の誰も初期状態で知らない情報は、この時点では知ることができない

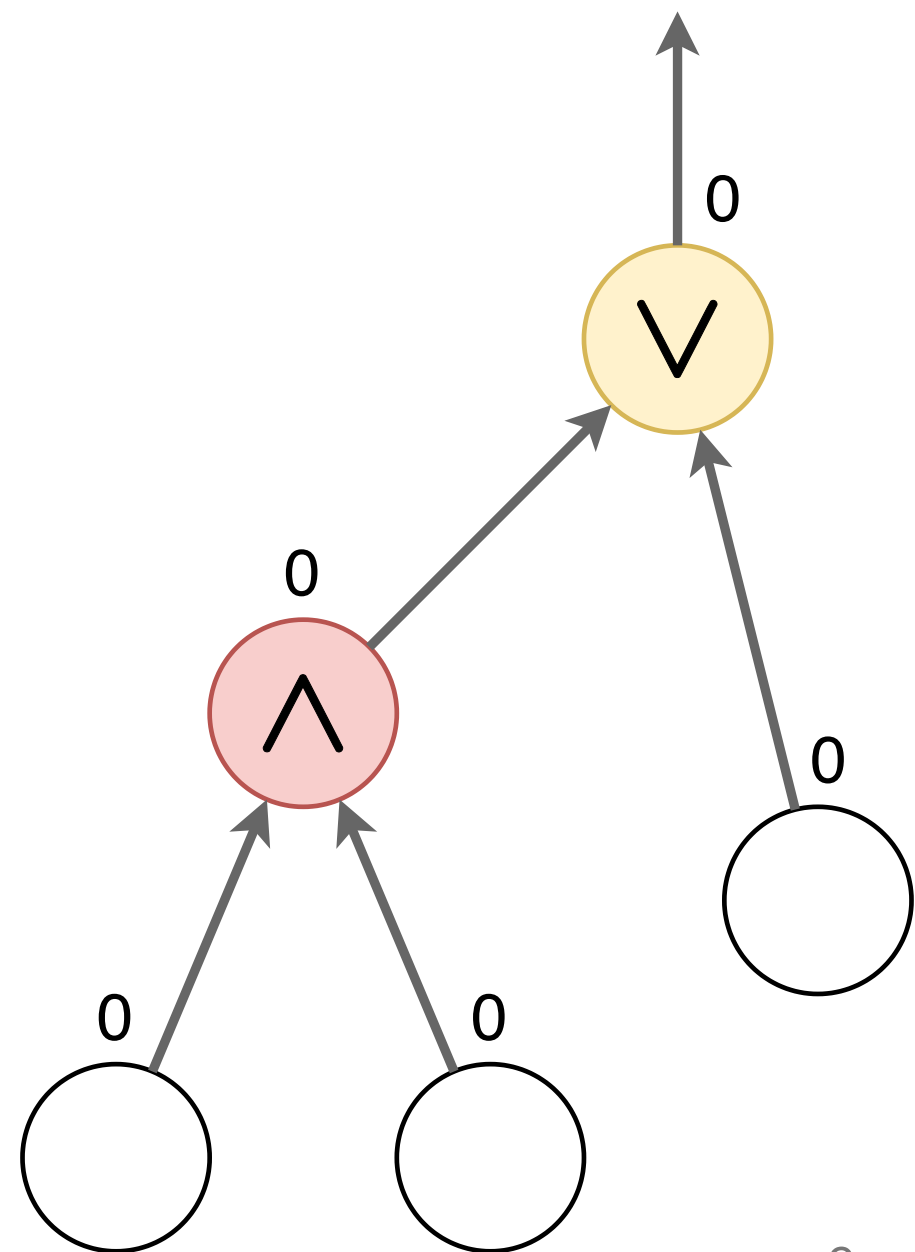
JOI/JOIG 2024/2025 春季トレーニング

Day 4 – 電子回路 2 (Circuit 2) 解説

解説担当: 山縣 龍人 (tatyam)

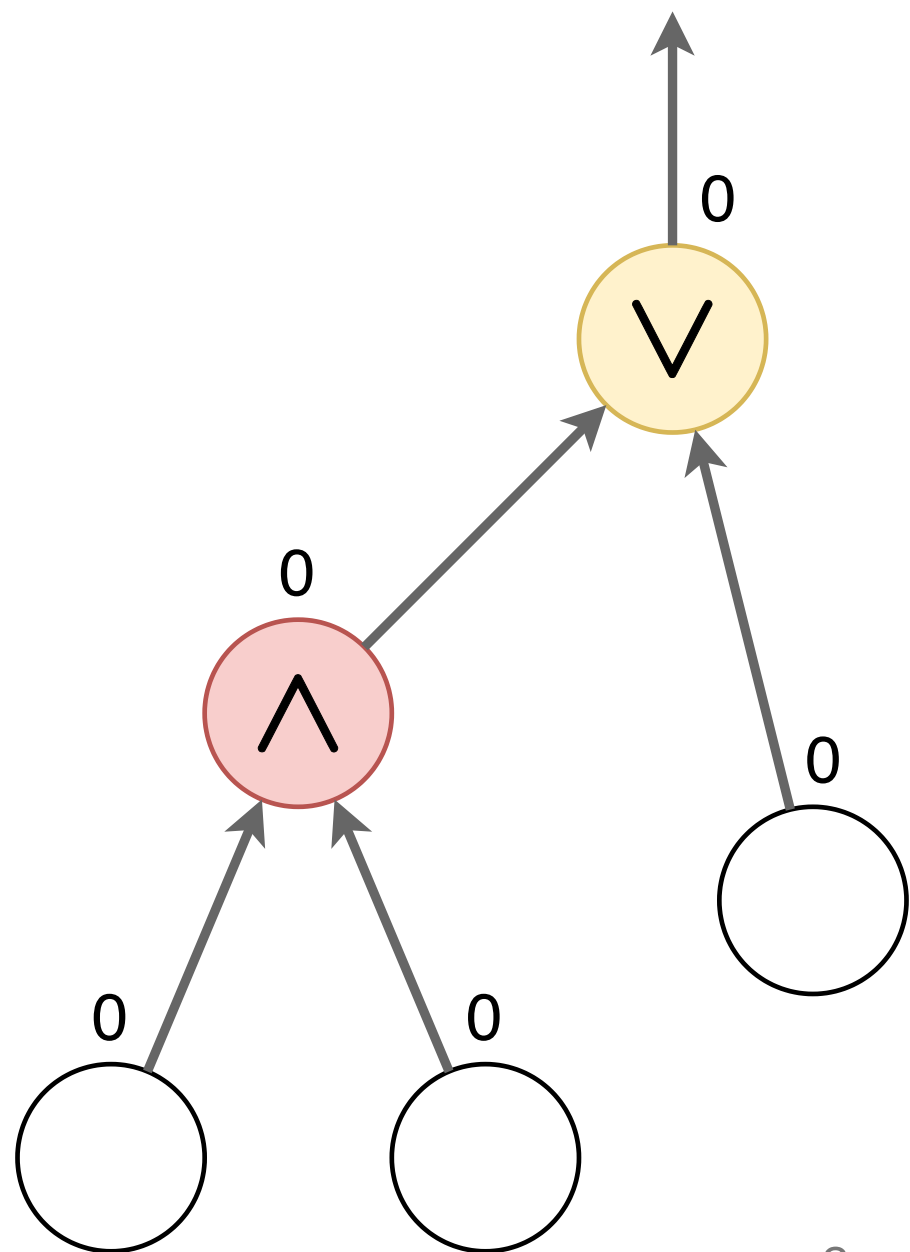
問題概要

- $N + 1$ 個の葉からなる根付きの二分木 (電子回路) がある
- 葉は 0 を出力する
- 葉以外の頂点は AND か OR のどちらかで、それぞれ子の出力の AND / OR を出力する
- どれが AND でどれが OR かは与えられないので、特定したい
- **OR は $R (\leq 120)$ 個以下 !**



問題概要

- あなたは各辺の上に NOT を好きなだけ設置し，そのときの出力を聞くことができる
- 1000 回以下の質問で，どの頂点に OR が設置されているかを特定せよ
- $N \leq 8000$



小課題 (JOI)

1. (1 点) $N = 1$.
2. (4 点) $N \leq 1000$, $R = 1$.
3. (5 点) $N \leq 1000$.

	$R \leq 70$	$R \leq 120$
パス	小課題 4 : 17 点	小課題 5 : 8 点
完全二分木	小課題 6 : 23 点	小課題 7 : 8 点
一般の二分木	小課題 8 : 27 点	小課題 9 : 7 点

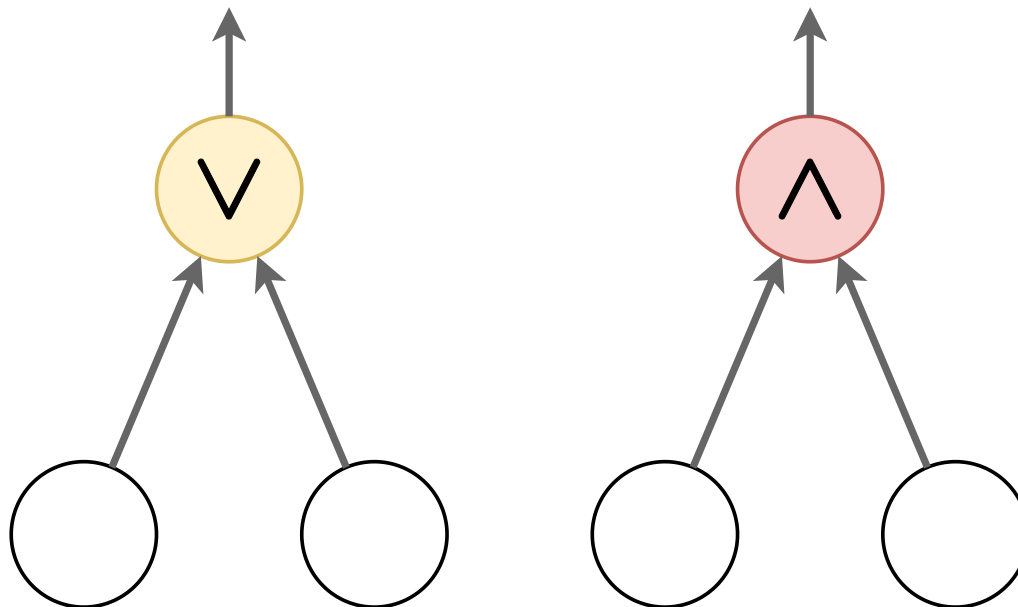
小課題 (JOIG)

1. (1 点) $N = 1$.
2. (8 点) $N \leq 1000$, $R = 1$.
3. (10 点) $N \leq 1000$.

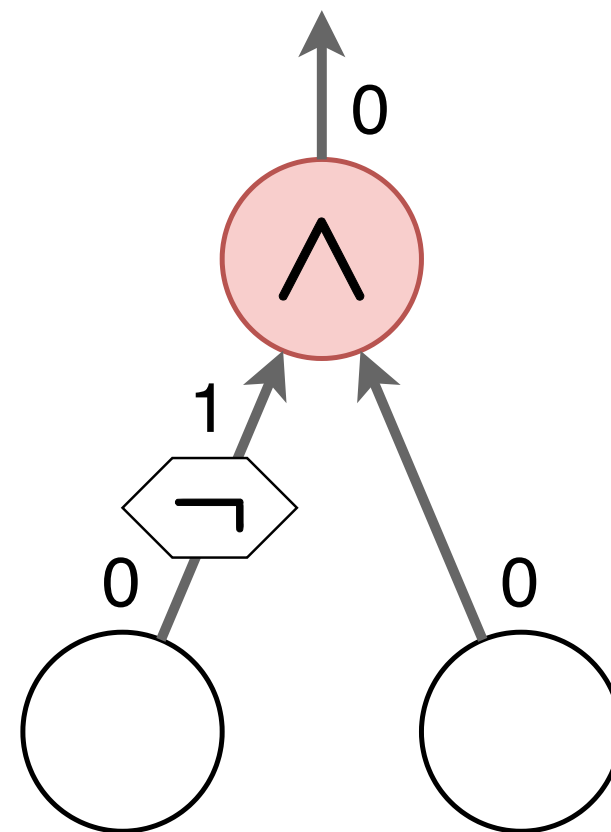
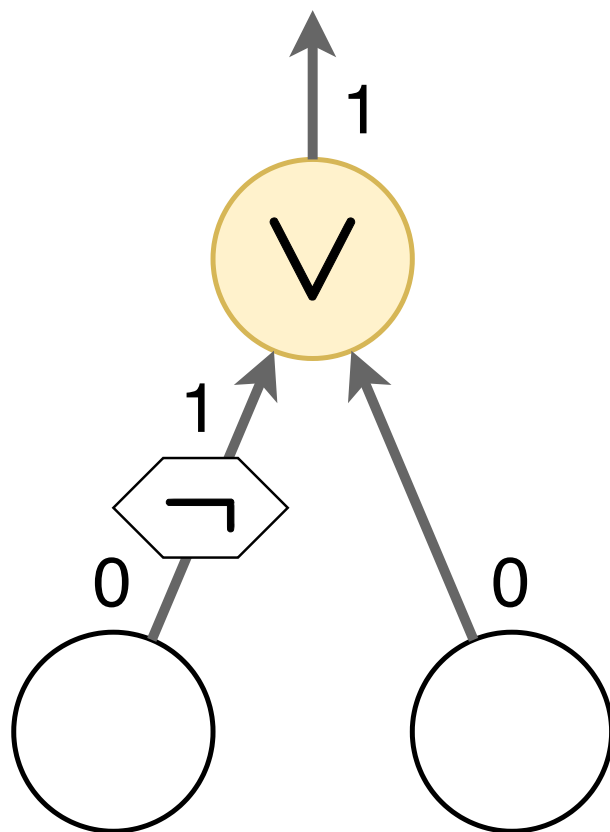
	$R \leq 70$	$R \leq 120$
パス	小課題 4 : 21 点	小課題 5 : 9 点
完全二分木	小課題 6 : 25 点	小課題 7 : 9 点
一般の二分木	小課題 8 : 14 点	小課題 9 : 3 点

小課題 1: $N = 1$

回路は以下のどちらか



小課題 1: $N = 1$



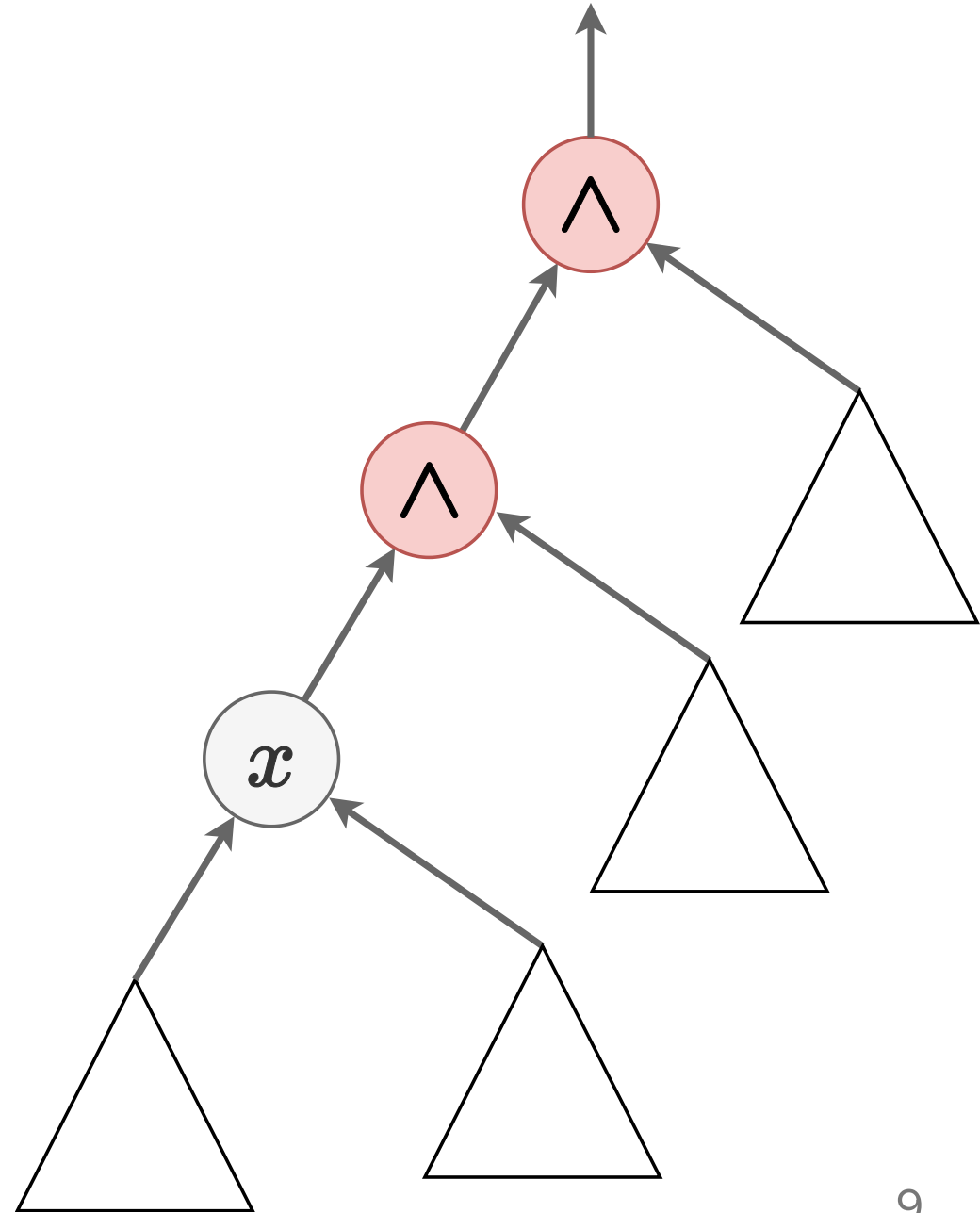
→ 010 を質問すれば区別可能

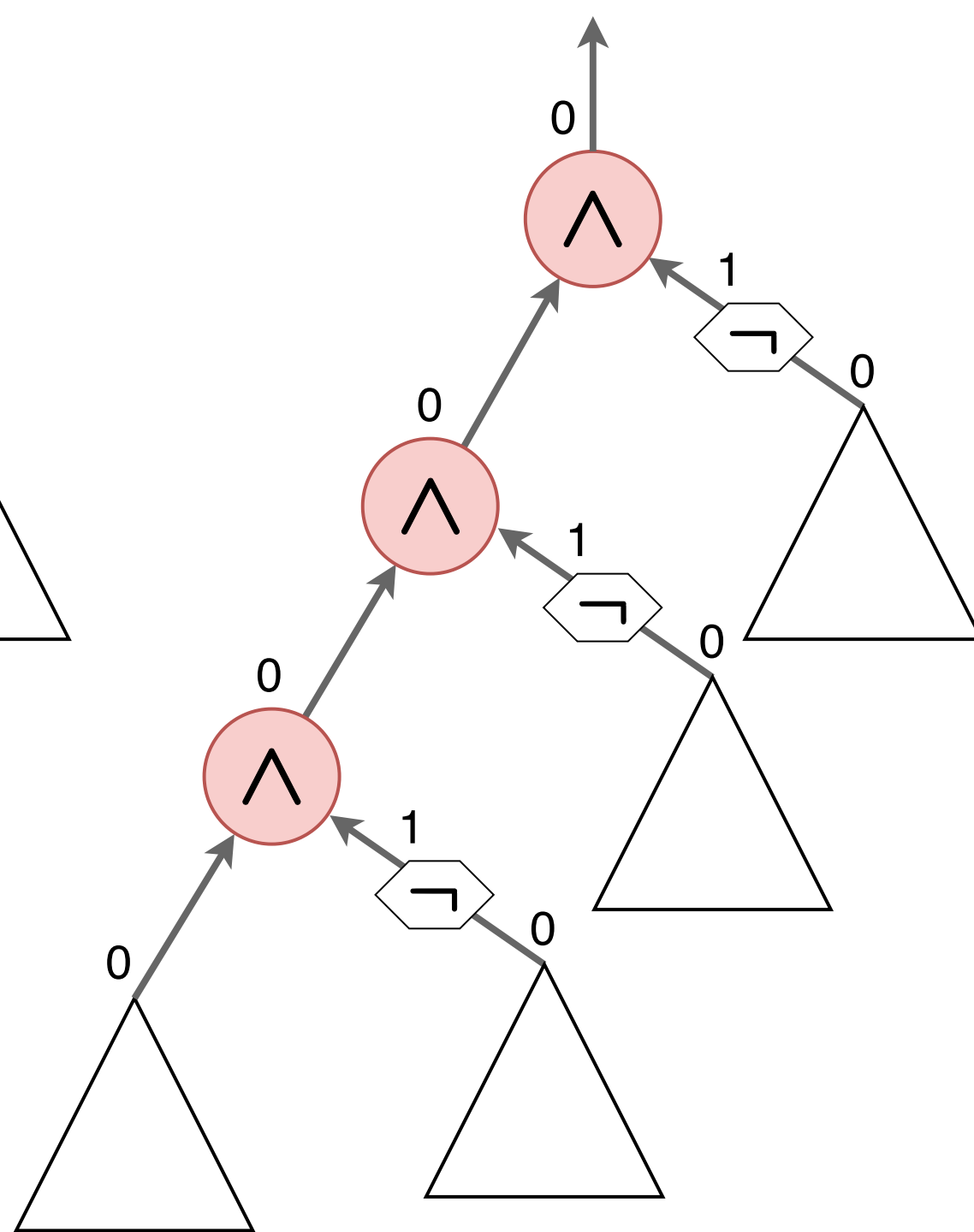
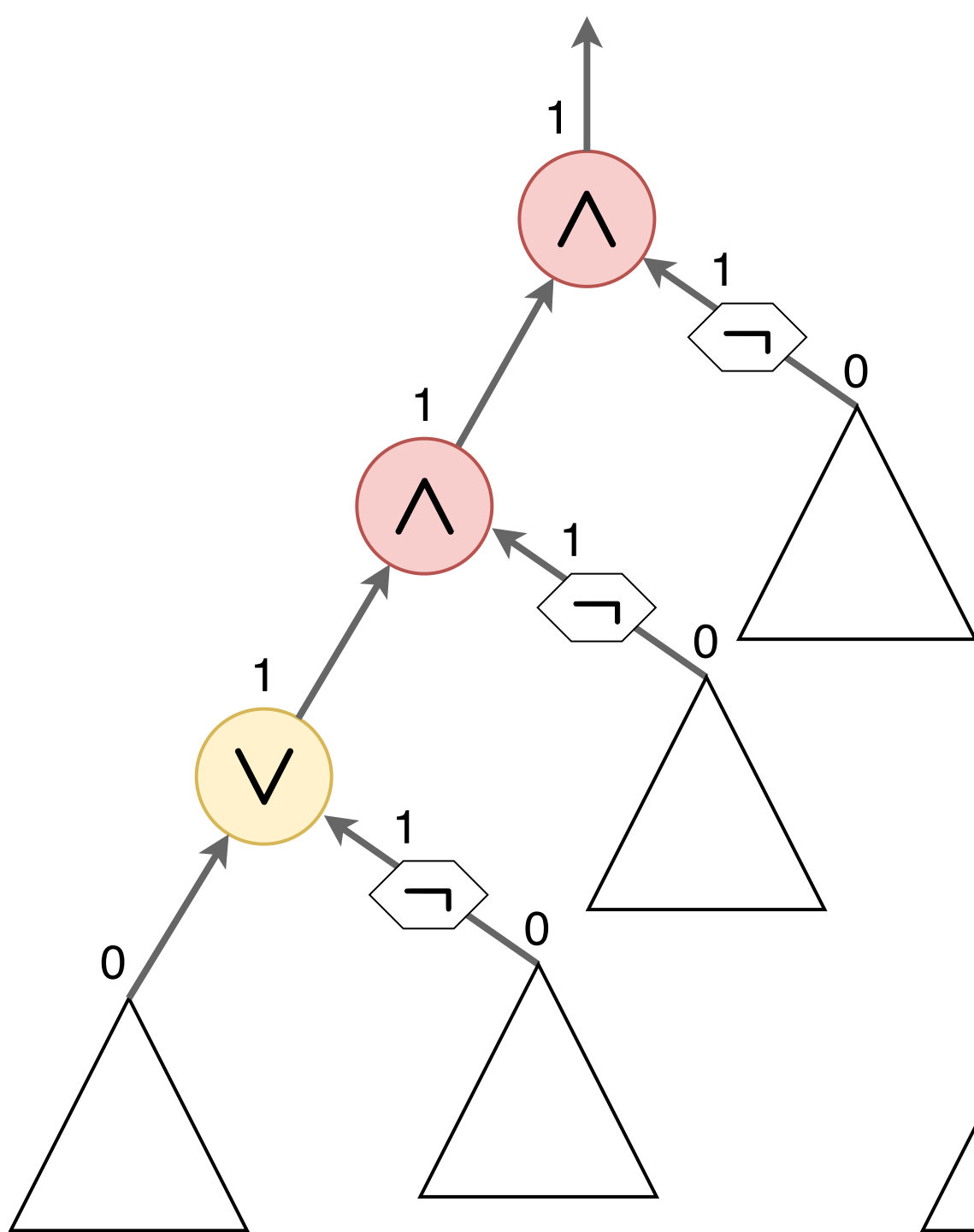
小課題 2: $N = 1000$, $R = 1$

- 各頂点について, OR かどうかを 1 クエリで特定したい

小課題 2: $N = 1000$, $R = 1$

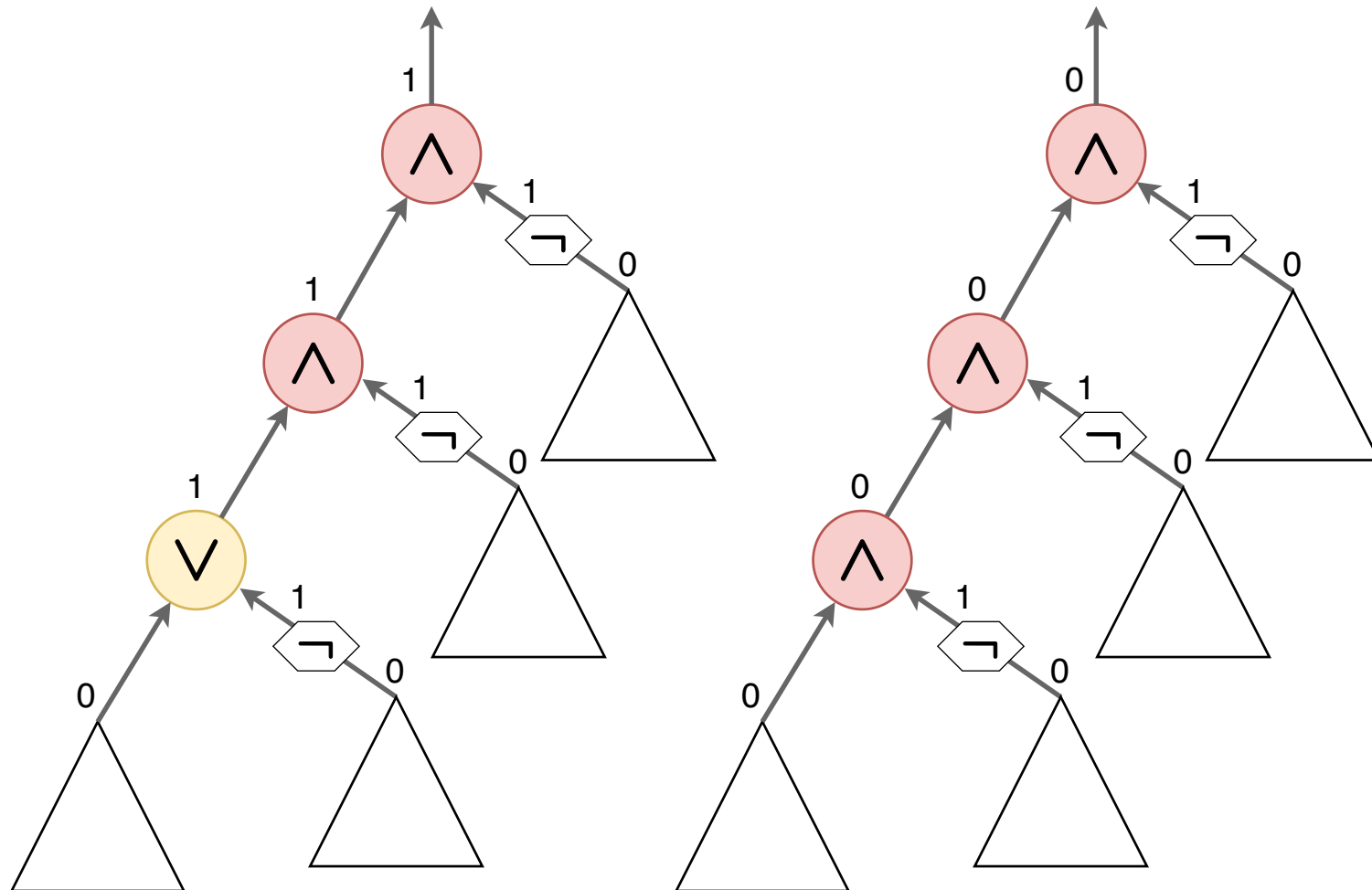
頂点 x が OR であるか聞くには...?





小課題 2: $N = 1000$, $R = 1$

- 根 $-x$ のパスについて, 片側から 1 を入力すると区別できる

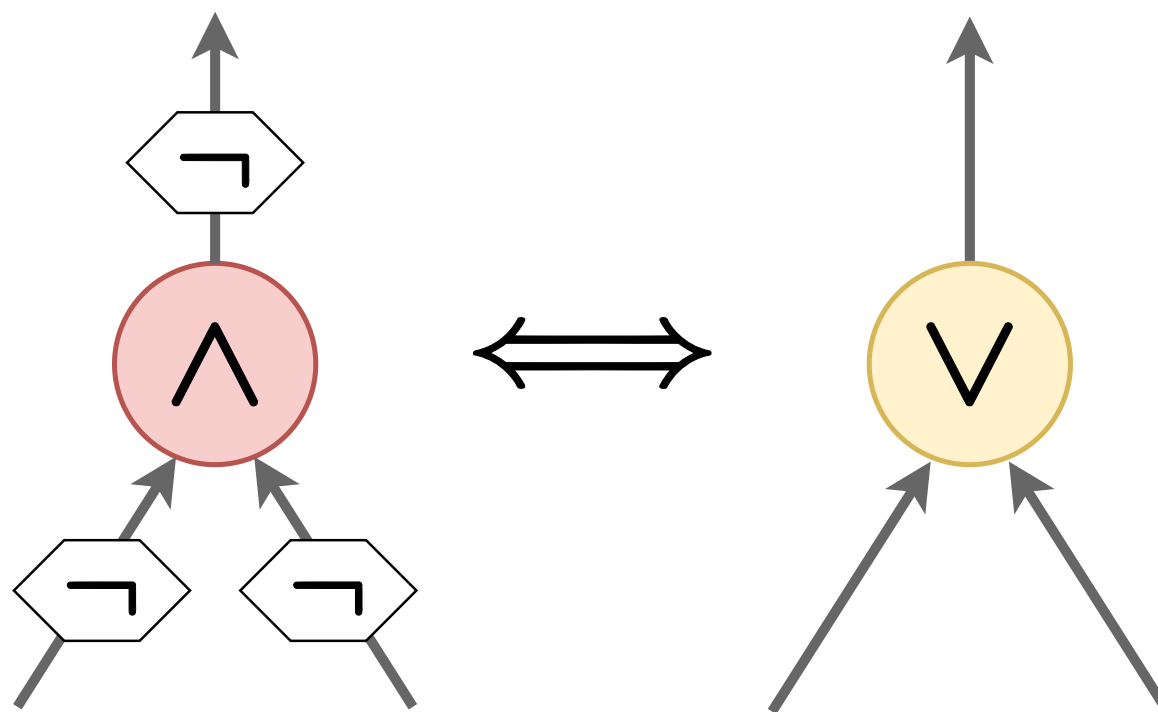


小課題 2: $N = 1000$, $R = 1$

- 根 $-x$ のパスについて, 片側から 1 を入力すると, **そのパス上に OR があるかが分かる**
- 上から順に, その頂点が OR であるかを聞く

小課題 3: $N = 1000$

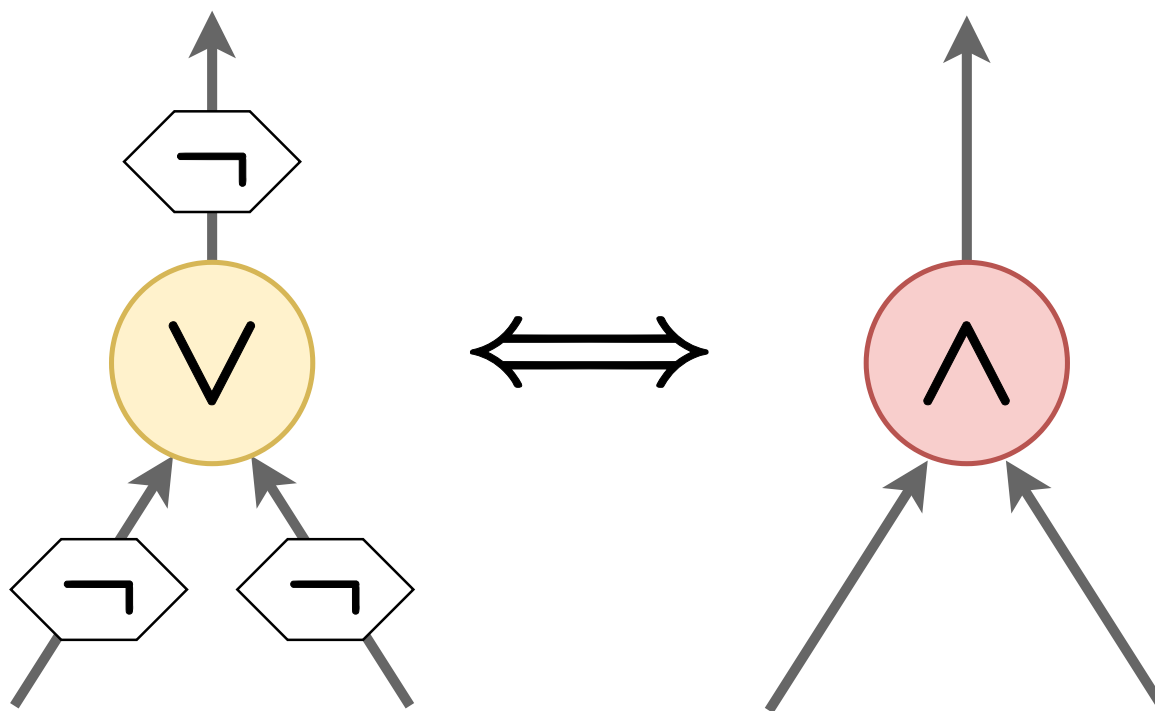
重要な考察



- AND に NOT を 3 個設置すると OR になる

小課題 3: $N = 1000$

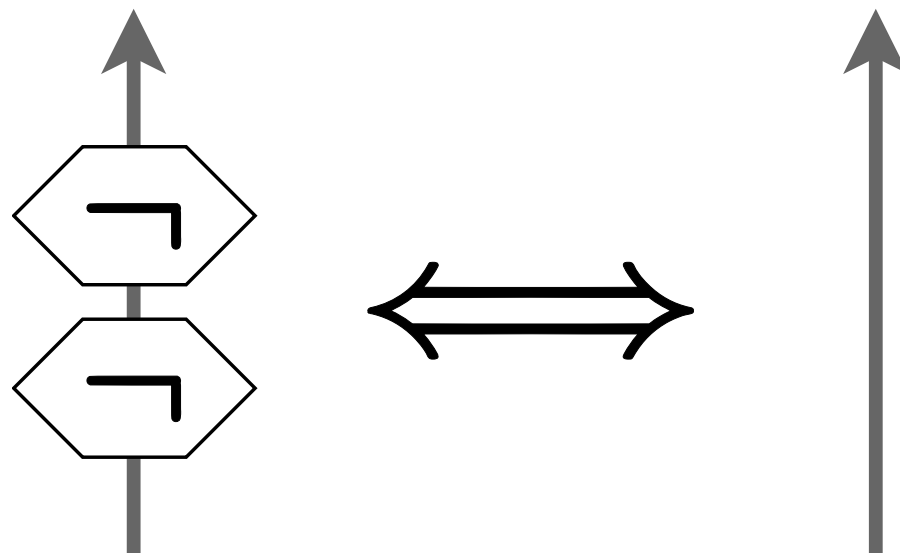
重要な考察



- OR に NOT を 3 個設置すると AND になる

小課題 3: $N = 1000$

重要な考察



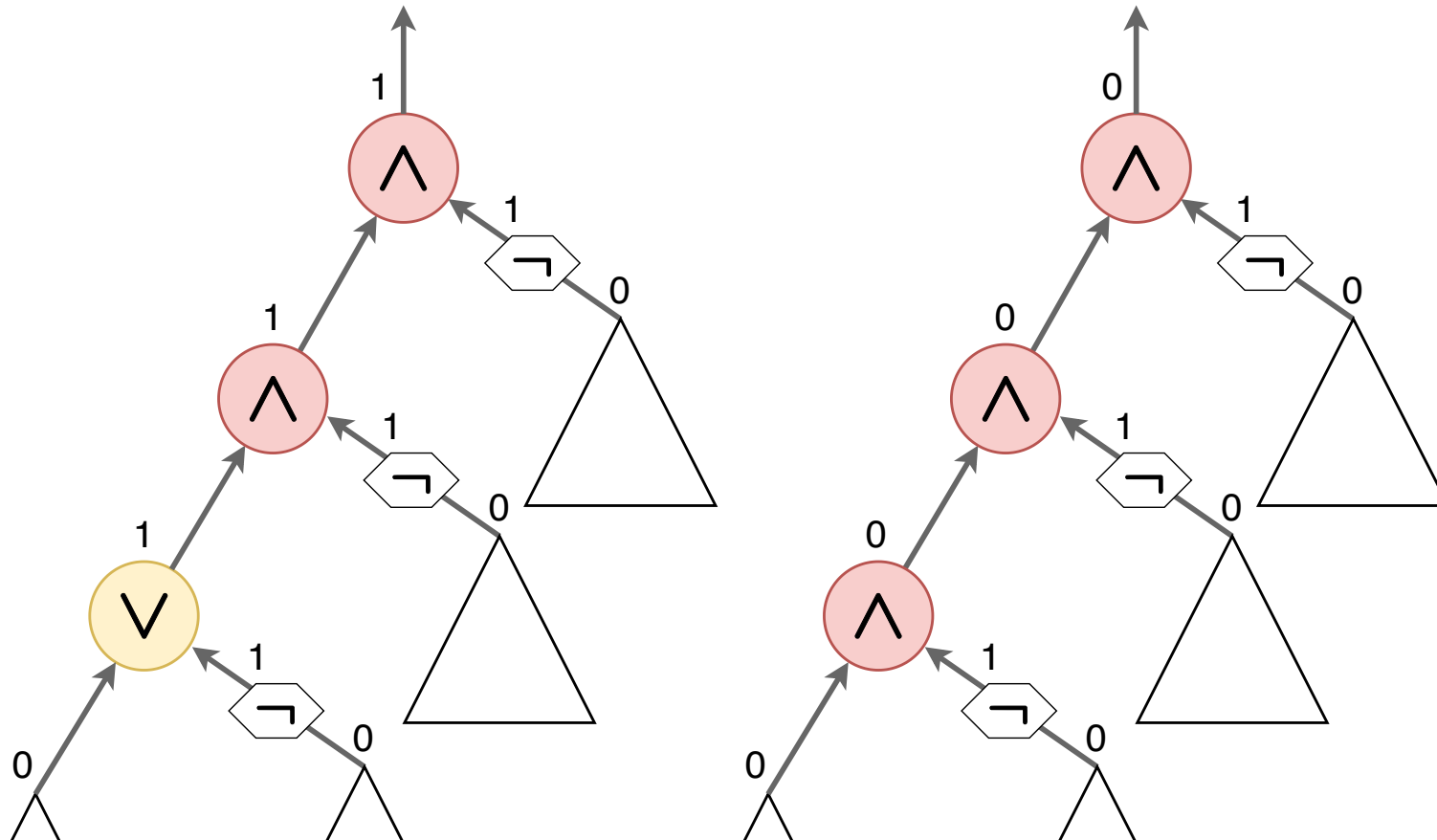
- 2 個の NOT を対消滅させることができる

小課題 3: $N = 1000$

- 小課題 2 と同様に, 上から順に頂点が OR であるかを聞く
- OR だったら, **NOT を 3 個設置して AND であると見なす**

小課題 4: パス, $R \leq 70$

- 根 $-x$ のパスについて, 片側から 1 を入力すると, **そのパス上に OR があるかが分かる**



小課題 4: パス, $R \leq 70$

- 根 $-x$ のパスについて, 片側から 1 を入力すると, **そのパス上に OR があるかが分かる**

→ インタラクティブの基本, **二分探索!**

小課題 4: パス, $R \leq 70$

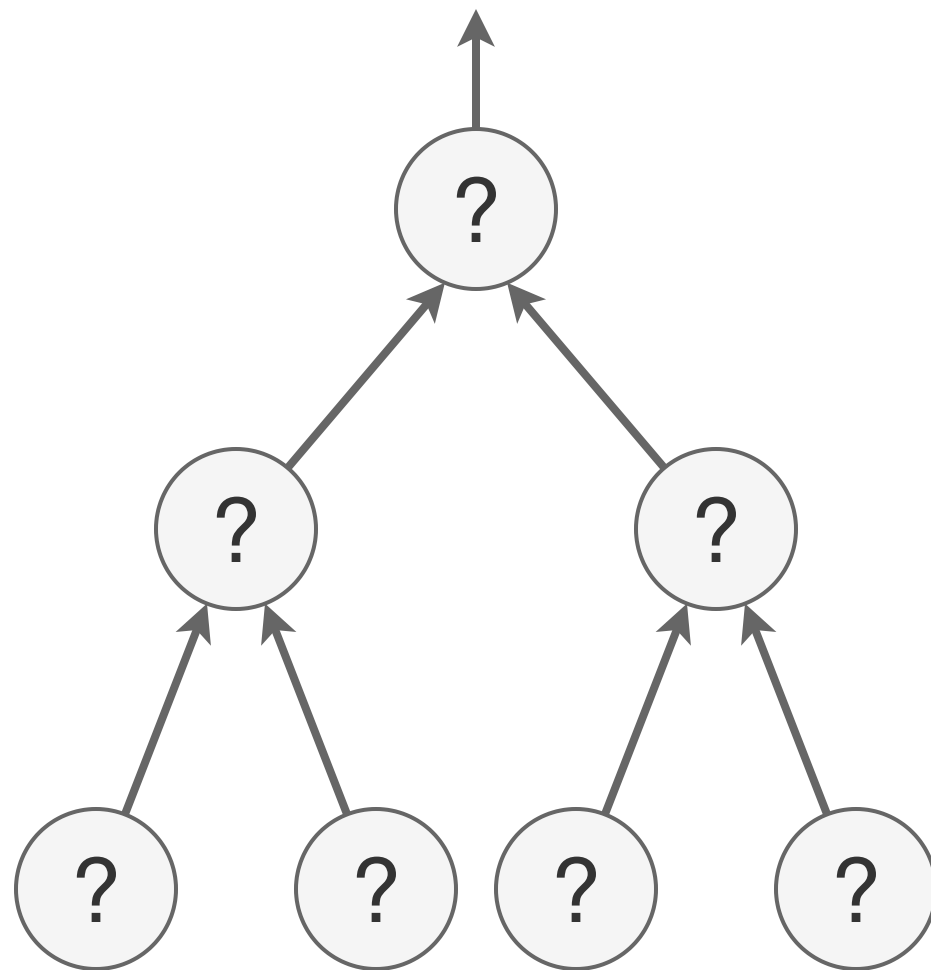
- 根 $-x$ のパスについて, 片側から 1 を入力すると, **そのパス上に OR があるかが分かる**
- 二分探索により, 14 回の質問で**最も根に近い OR の位置**が分かる (工夫すると 13 回)

小課題 4: パス, $R \leq 70$

- 根 $-x$ のパスについて, 片側から 1 を入力すると, **そのパス上に OR があるかが分かる**
- 二分探索により, 14 回の質問で**最も根に近い OR の位置**が分かる (工夫すると 13 回)
- OR の位置が分かったら, NOT を 3 個設置して AND であると見なす

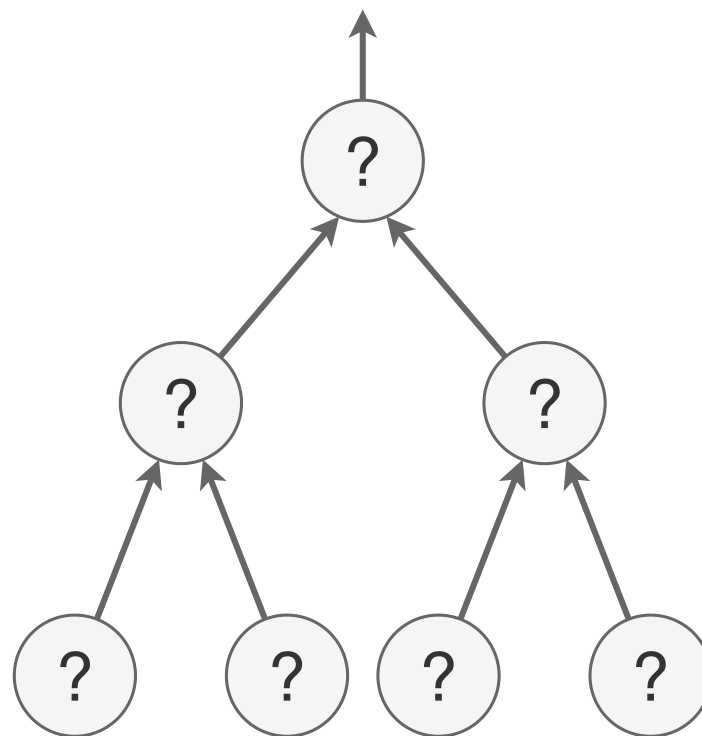
→ $14 \times 70 = 980$ 回の質問で解けた

小課題 6 : 完全二分木, $R \leq 70$



Q. どうやって二分探索する？

小課題 6 : 完全二分木, $R \leq 70$

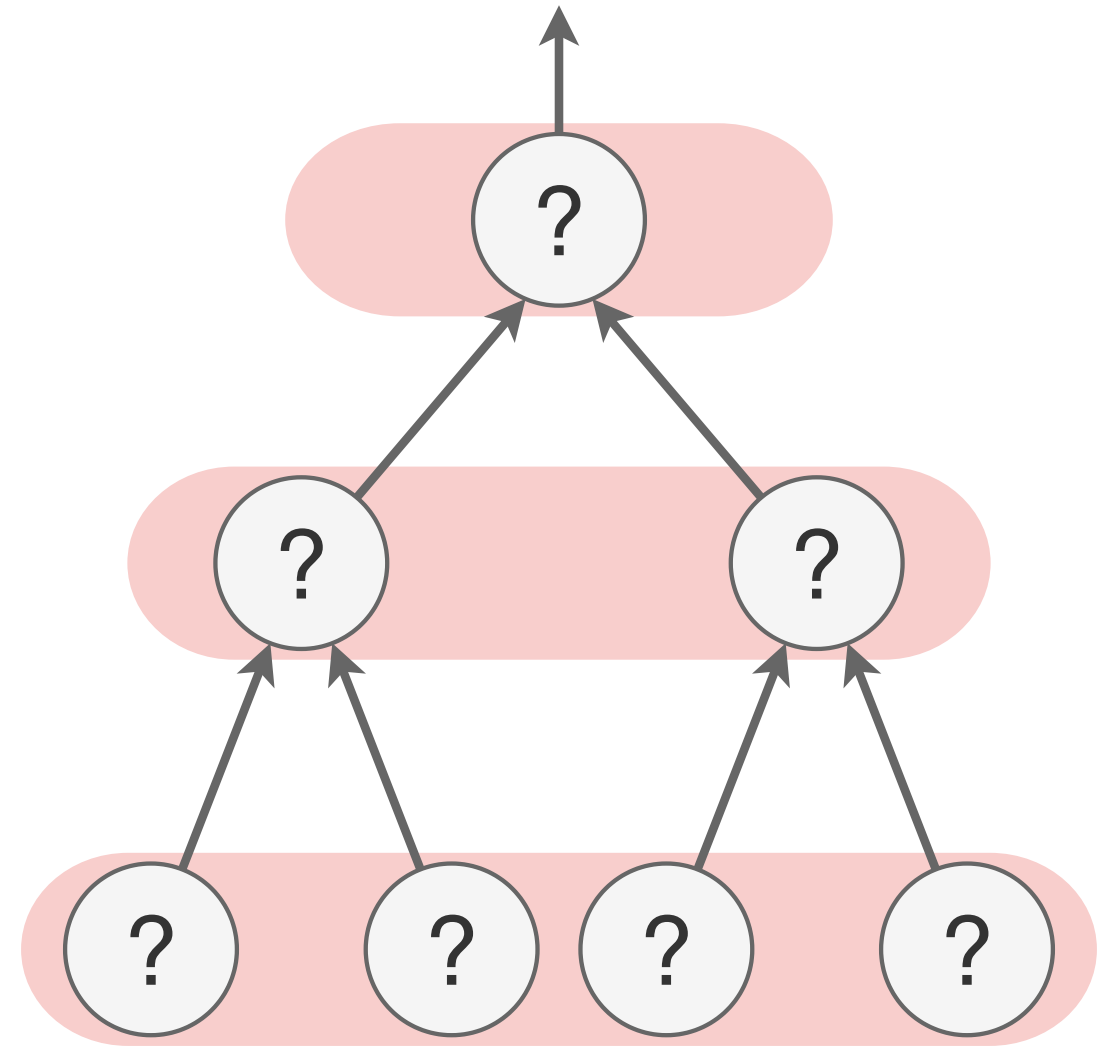


Q. どうやって二分探索する？

ヒント: 上から順に決める

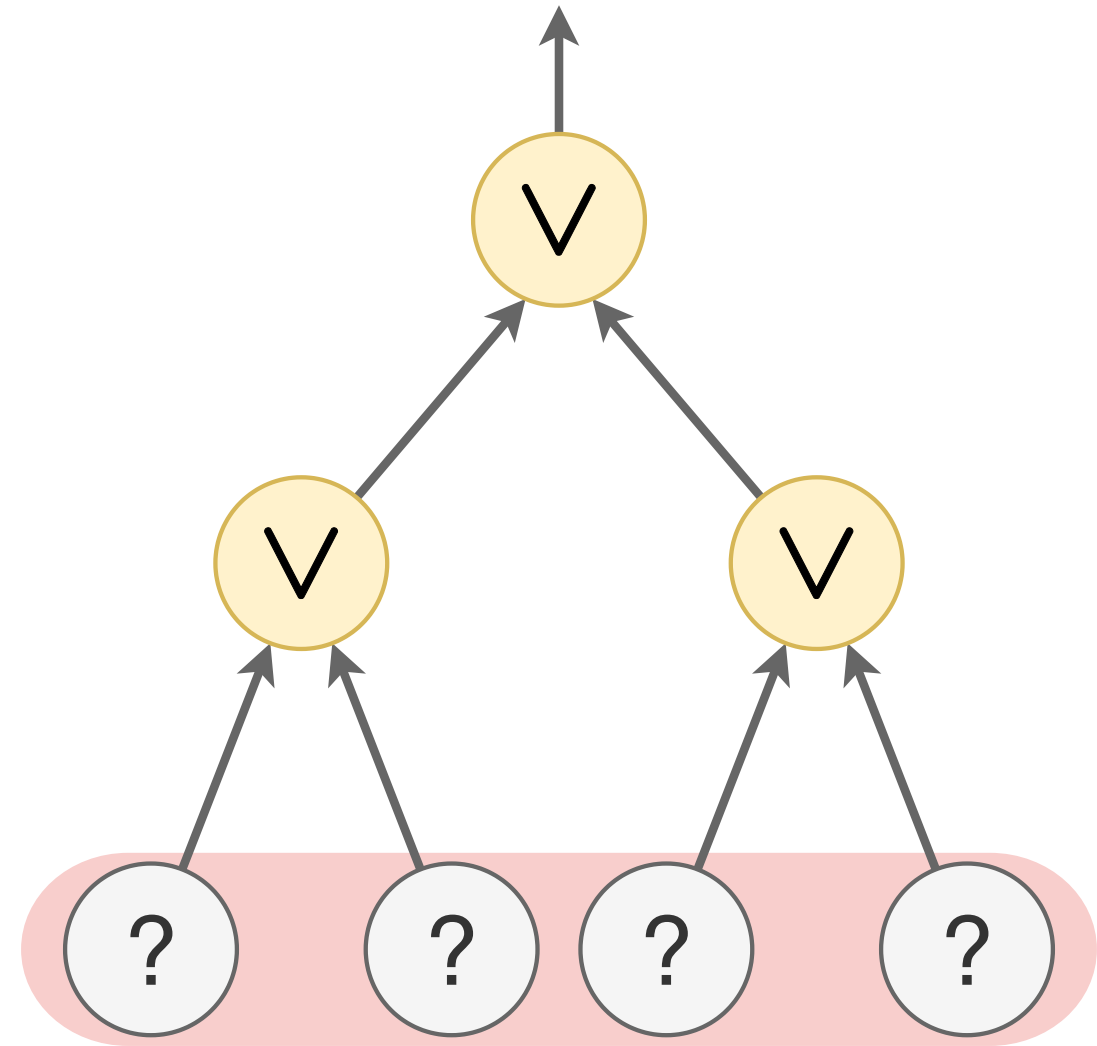
小課題 6 : 完全二分木, $R \leq 70$

上から順に決める



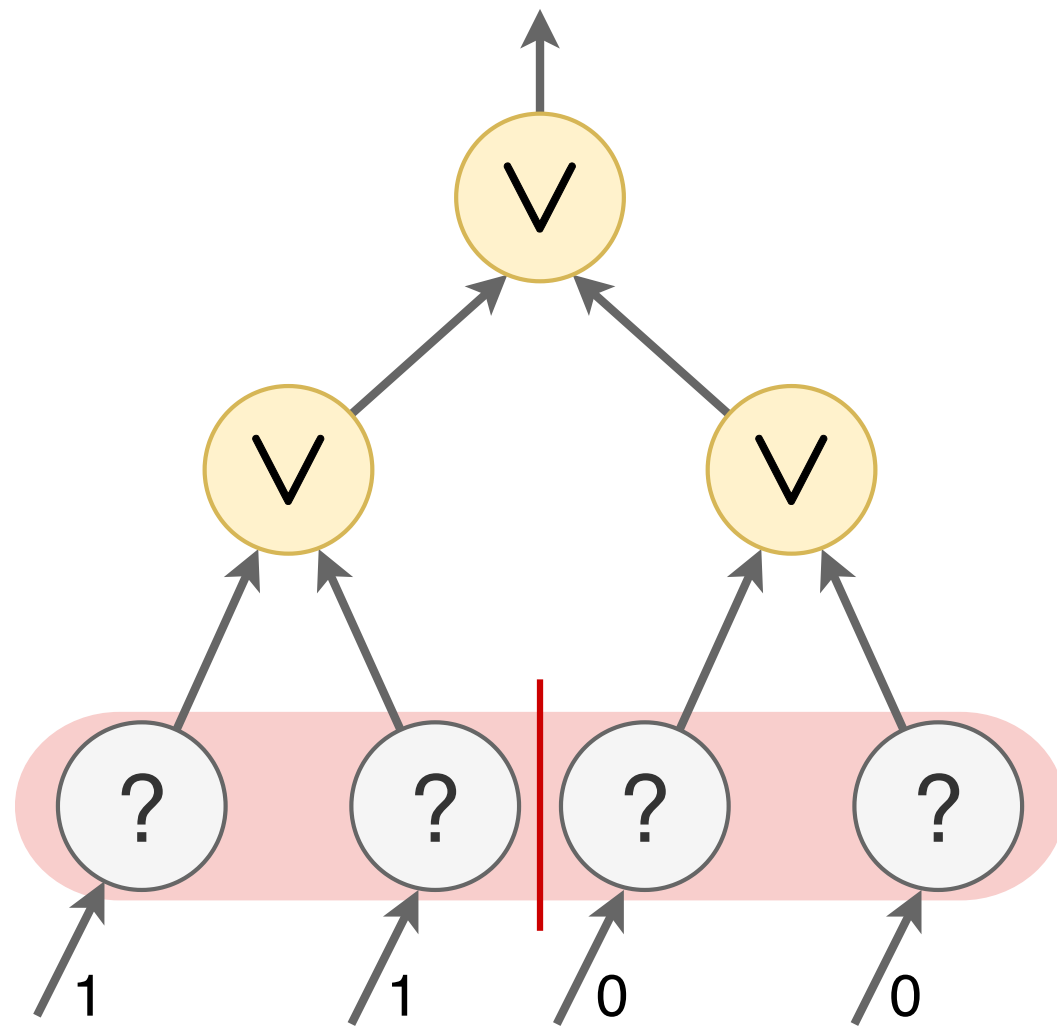
小課題 6 : 完全二分木, $R \leq 70$

Q. 上が全部 OR であるとき,
同じ高さにある頂点の集合を
二分探索するには?



小課題 6: 完全二分木, $R \leq 70$

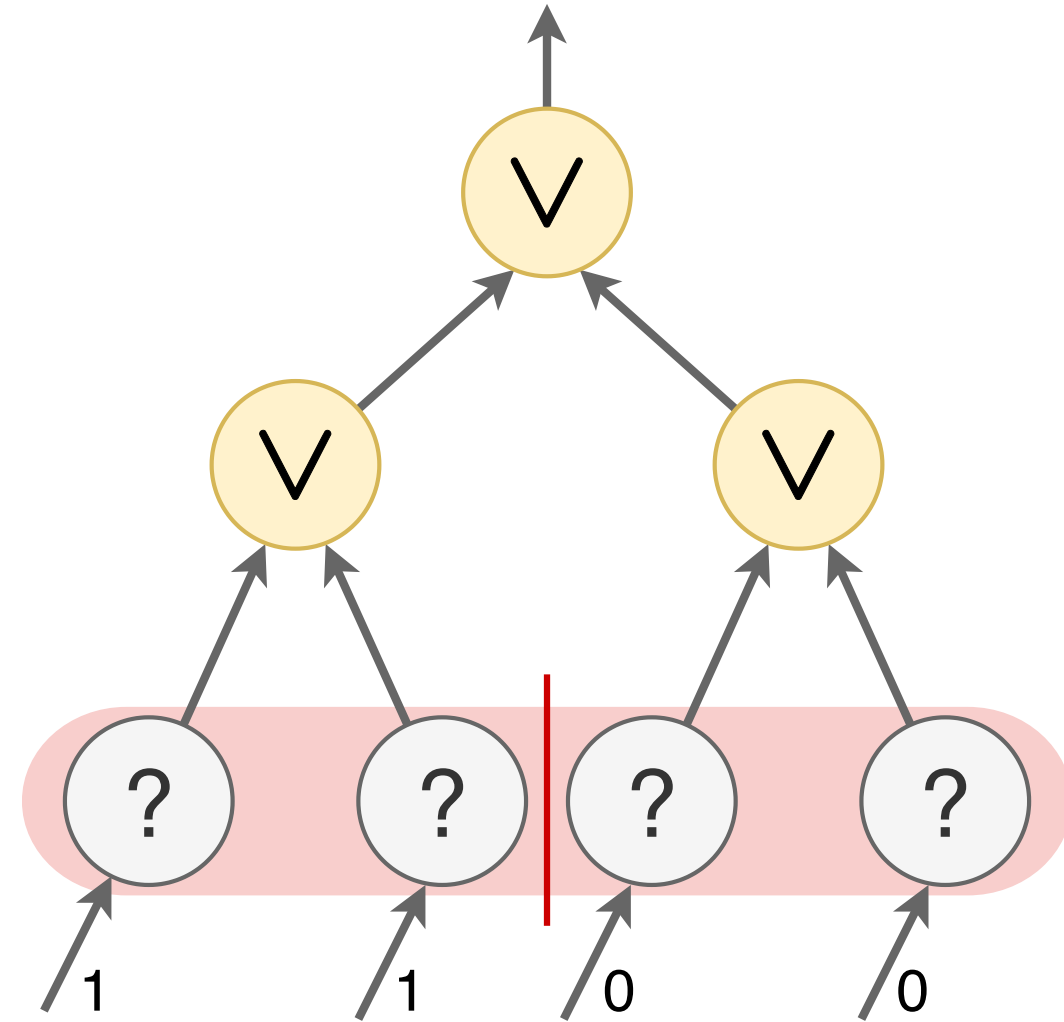
- あるところまで 1, あるところから 0 を片側に
入力すると,
**1 を入力した頂点の中に OR が
あるかどうか分かる**



小課題 6: 完全二分木, $R \leq 70$

- あるところまで 1, あるところから 0 を片側に入力すると,
1 を入力した頂点の中に OR があるかどうか分かる

→ 二分探索により, 13 回の質問で
最も左にある OR の位置が分かる
(工夫すると 12 回)



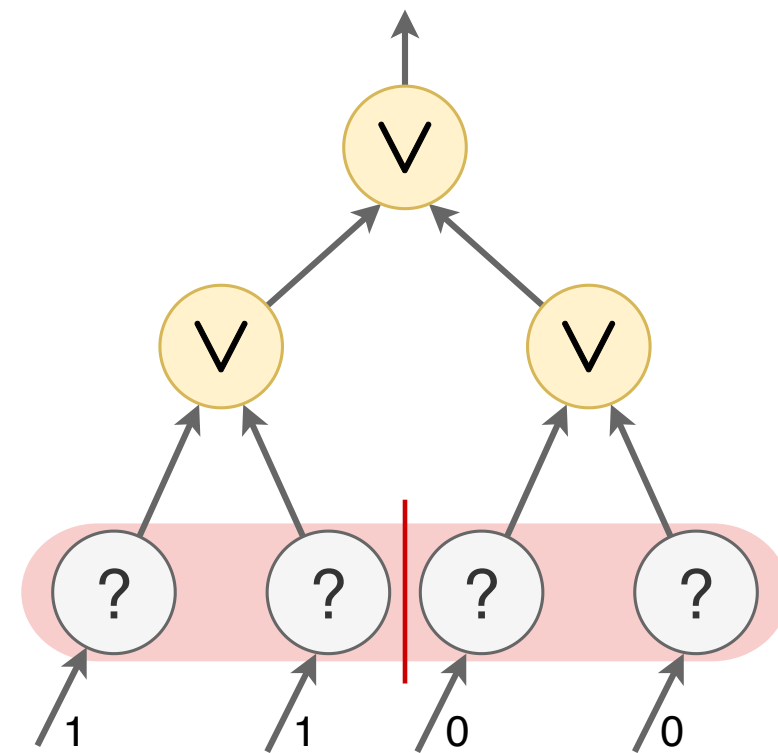
小課題 6 : 完全二分木, $R \leq 70$

- あるところまで 1, あるところから 0 を片側に入力すると, **1 を入力した頂点の中に OR があるかどうか**が分かる

→ 二分探索により, 13 回の質問で**最も左にある OR の位置**が分かる (工夫すると 12 回)

- 発見した OR は AND に変更し, 1 段終わったらすべて OR に変更する

$13 + 13 \times 70 = 923$ 回の質問で解けた



小課題 8: 一般の二分木, $R \leq 70$

これまでのまとめ

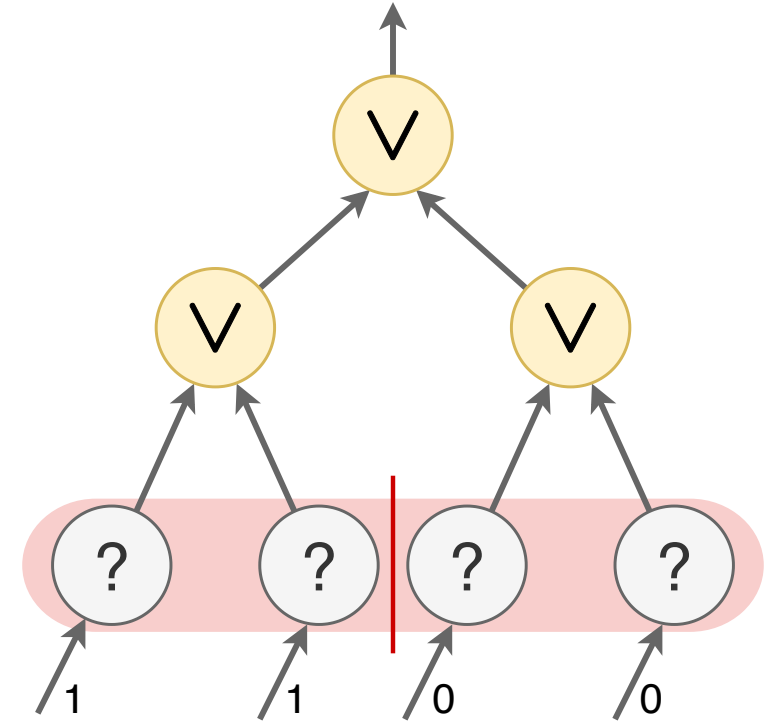
- 根からのパス
- 同じ高さの頂点の集合

は二分探索できる

Q. 一般の二分木の場合はどのように二分探索する？

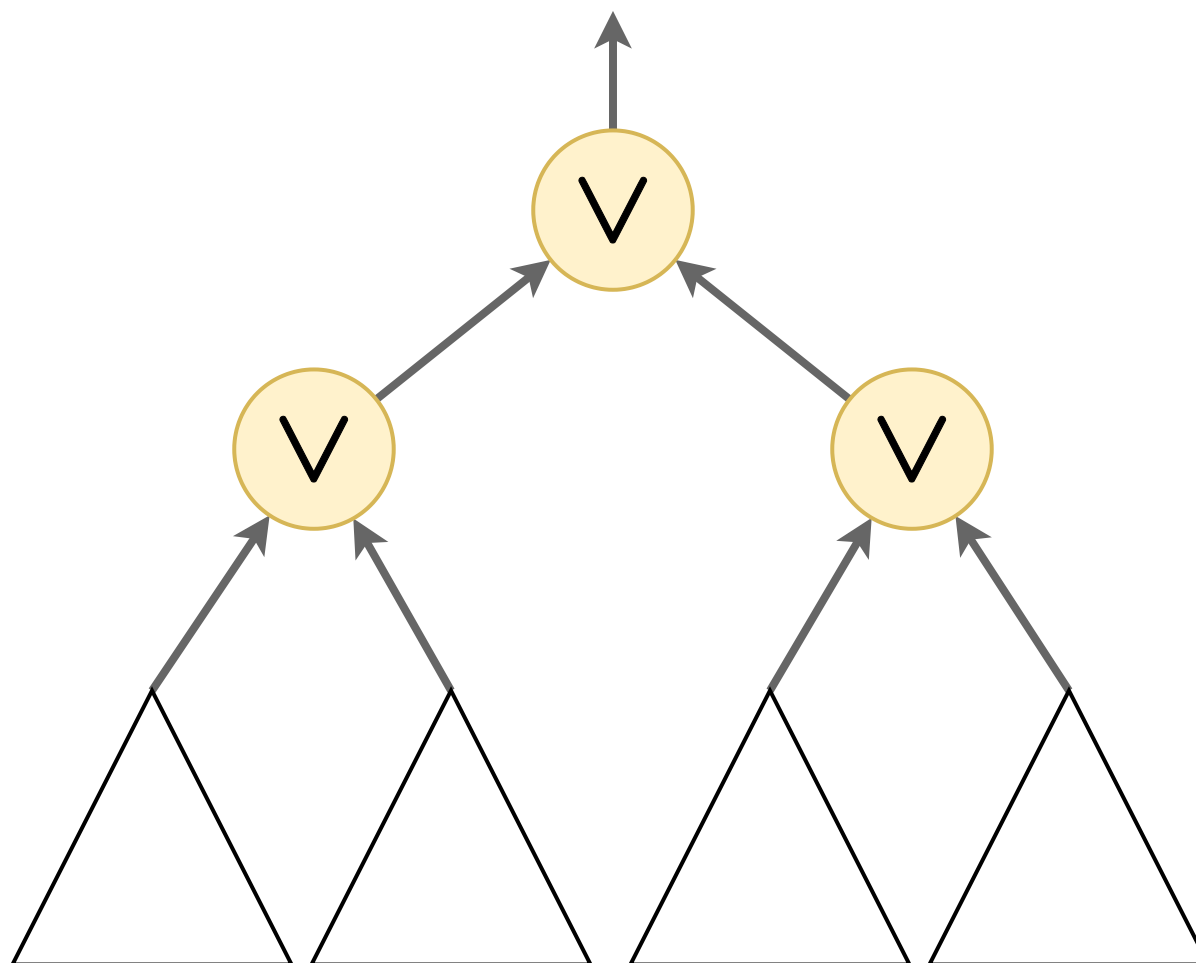
小課題 8: 一般の二分木, $R \leq 70$

なぜ同じ高さの頂点が二分探索できたのか？

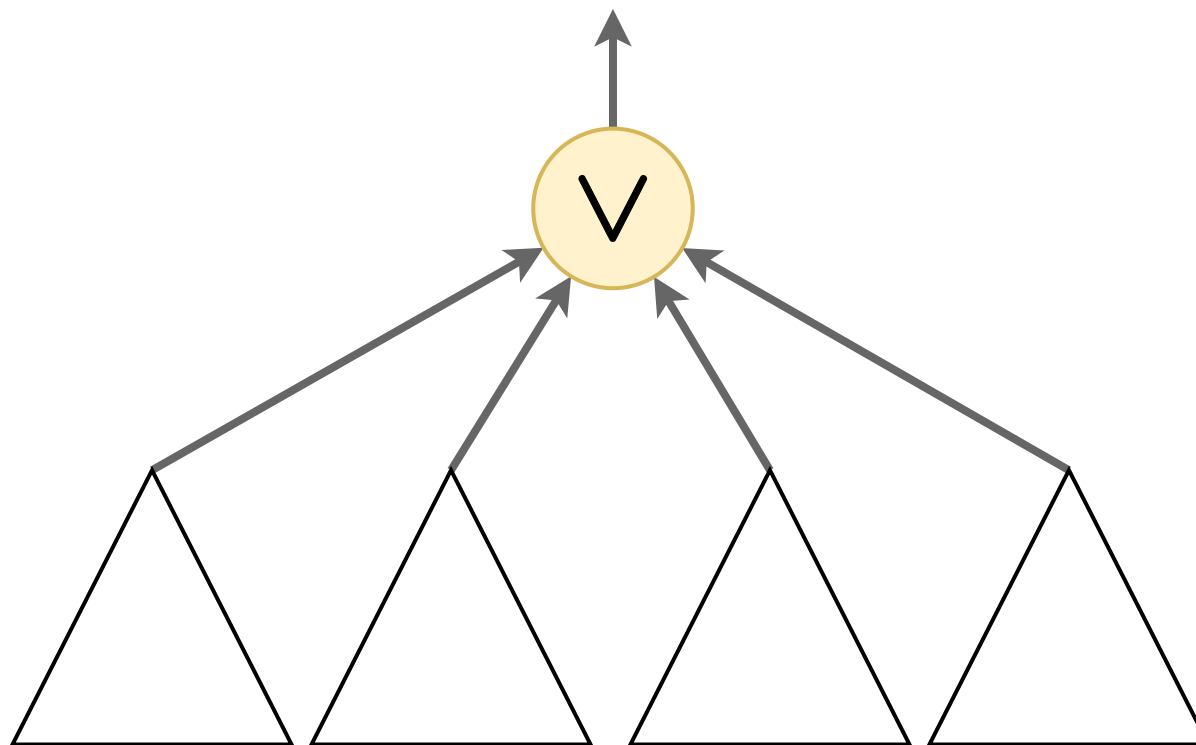


小課題 8 : 一般の二分木, $R \leq 70$

上が全部 OR であるとき...

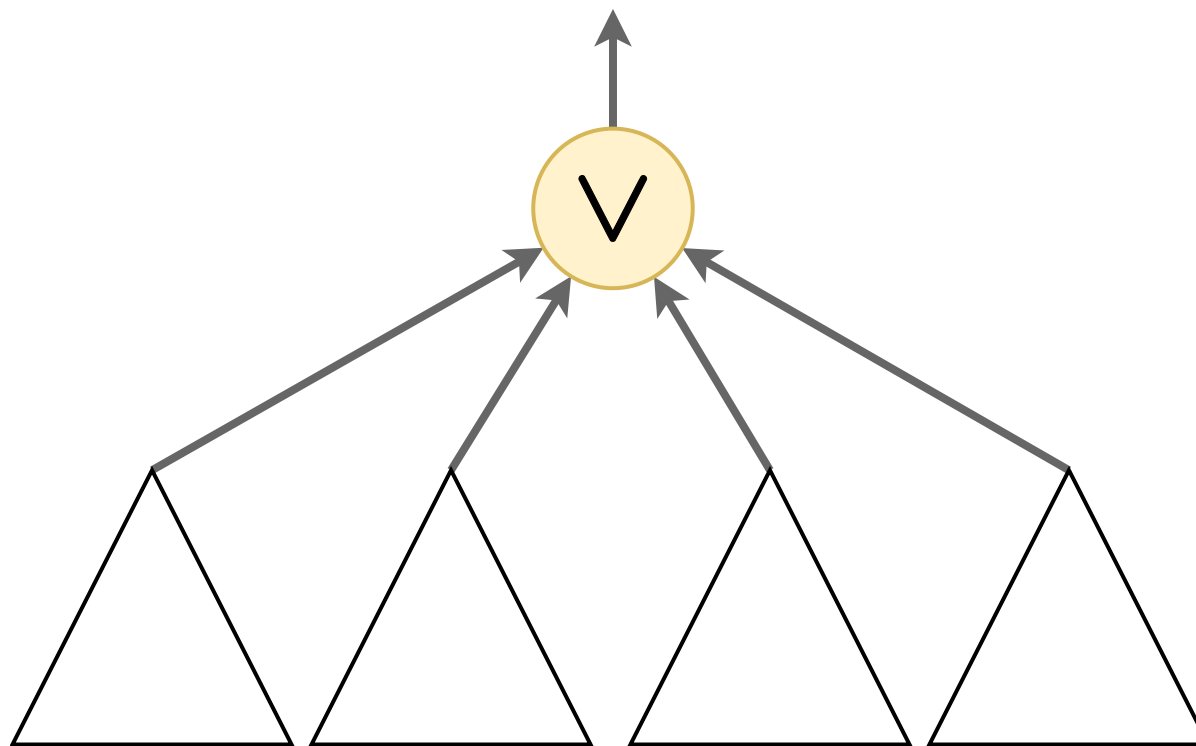


小課題 8 : 一般の二分木, $R \leq 70$



すべての部分木の OR が出力される！

小課題 8: 一般の二分木, $R \leq 70$



すべての部分木の OR が出力される... ?

小課題 8 : 一般の二分木, $R \leq 70$

もしかして : Space Thief

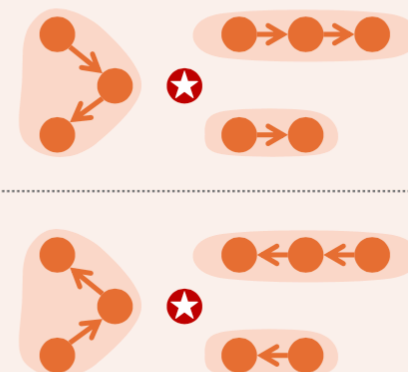
木の重心で分ける

41 / 68

木の重心を中心としたスターグラフ
だと思って問題を解く

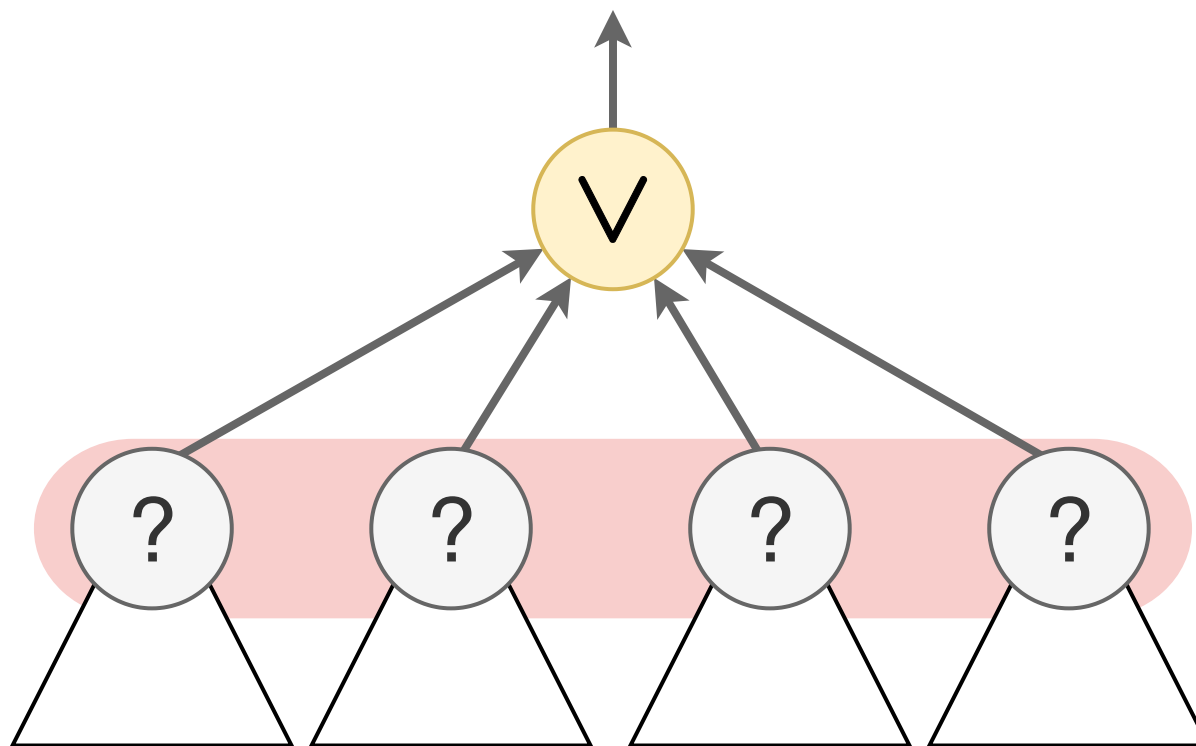
→ もし No しか得られなかった場
合 A, B は同じ “連結成分”

アイデア: このとき各 ~~連結成分~~
部分木
に対して同じ問題を並列に解く!



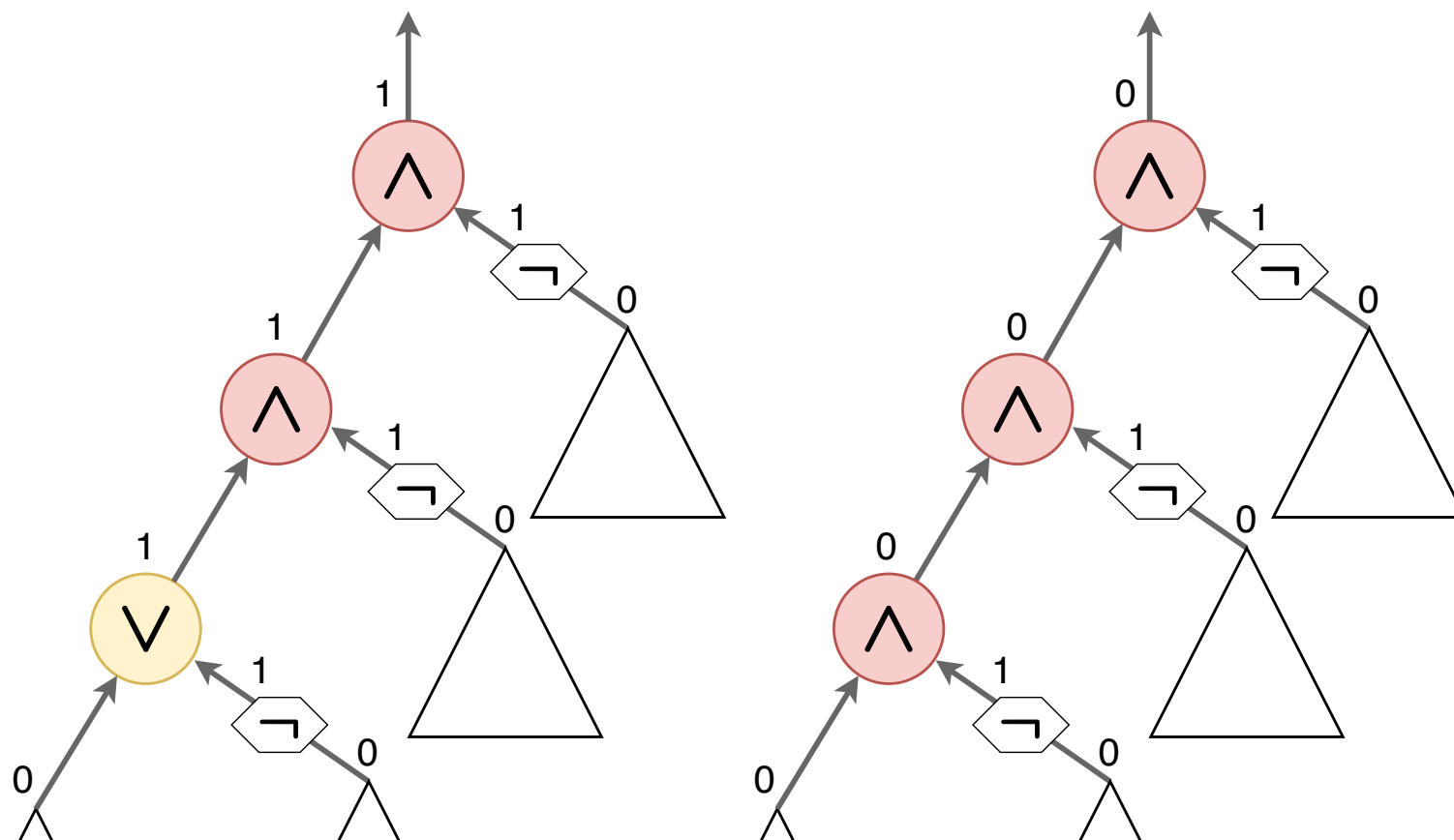
小課題 8: 一般の二分木, $R \leq 70$

- 各部分木で根のみを聞くと, 同じ高さの頂点を聞くのと同じ



小課題 8: 一般の二分木, $R \leq 70$

- 各部分木で根のみを聞くと, 同じ高さの頂点を聞くのと同じ
- ところで, パスの場合は二分探索できた



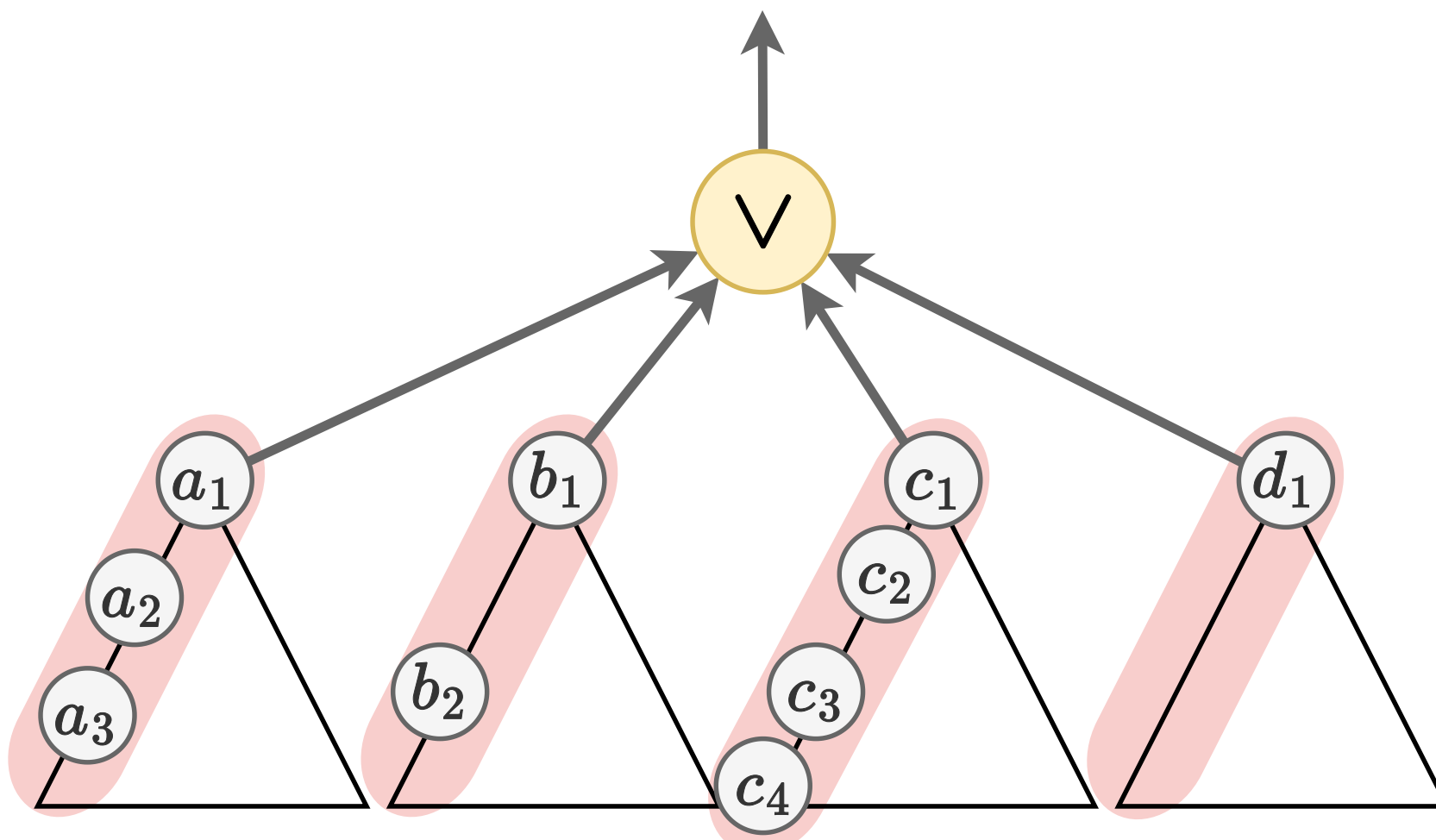
小課題 8: 一般の二分木, $R \leq 70$

- 各部分木で根のみを聞くと, 同じ高さの頂点を聞くのと同じ
- ところで, パスの場合は二分探索できた

→ **各部分木で根からのパスを聞こう!**

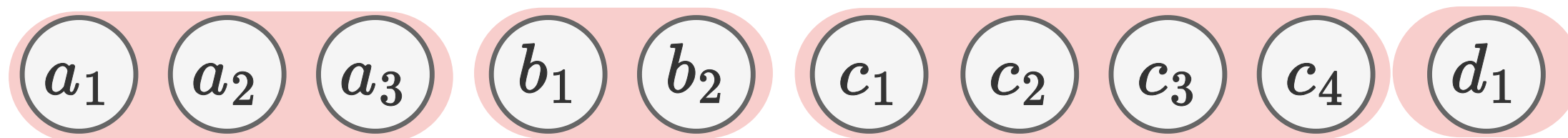
小課題 8: 一般の二分木, $R \leq 70$

- 各部分木で根からのパスを聞く



小課題 8 : 一般の二分木, $R \leq 70$

- 各部分木で根からのパスを聞く



パスをつなげる / トポロジカルソート すると,
この頂点の列を二分探索できる !

小課題 8: 一般の二分木, $R \leq 70$

まとめ

以下を繰り返す

1. 各部分木について, **根からのパスを 1 つ選ぶ**
2. パスをつなげて二分探索し, パス上の OR をすべて見つける
3. パス上の AND を OR に変更する
4. パスを削除し, いくつかの部分木に分解する

→ どのようなパスを選ぶのが良いか？

小課題 8: 一般の二分木, $R \leq 70$

Q. 木をパスに分解して, 高さを $O(\log N)$ に
する方法といえは?

小課題 8 : 一般の二分木, $R \leq 70$

Q. 木をパスに分解して, 高さを $O(\log N)$ にする方法といえは？

A. **HL 分解** !

小課題 8 : 一般の二分木, $R \leq 70$

以下を繰り返す :

1. 各部分木について, 根からのパスを 1 つ選ぶ
 - **HL 分解 : 左右の子のうち部分木の頂点数が多い方に降りていくようなパスを選ぶ**
2. パスをつなげて二分探索し, パス上の OR をすべて見つける
 - 二分探索で 14 回 (工夫すると 13 回)
3. パス上の AND を OR に変更する
4. パスを削除し, いくつかの部分木に分解する

以下を繰り返す：

1. 各部分木について，根からのパスを 1 つ選ぶ
 - **左右の子のうち部分木の頂点数が多い方に降りていくようなパスを選ぶ**
2. パスをつなげて二分探索し，パス上の OR をすべて見つける
 - 二分探索で 14 回 (工夫すると 13 回)
3. パス上の AND を OR に変更する
4. パスを削除し，いくつかの部分木に分解する

繰り返しは $\text{floor}(\log_2(N + 1)) \leq 12$ 回以下 なので，
 $12 + 14 \times 70 = 992$ 回の質問で解けた！

小課題 5: パス, $R \leq 120$

質問回数を削減したい！

小課題 4: パス, $R \leq 70$

- 根 $-x$ のパスについて, 片側から 1 を入力すると, **そのパス上に OR があるかが分かる**
- 二分探索により, 14 回の質問で**最も根に近い OR の位置**が分かる (工夫すると 13 回)
- OR の位置が分かったら, NOT を 3 個設置して AND であると見なす

→ $14 \times 70 = 980$ 回の質問で解けた

どこに無駄があった？

小課題 5: パス, $R \leq 120$

確率的に考えよう！

区別しなければならない答えは $\sum_{r=0}^{120} \binom{8000}{r} \approx 2^{894.1}$ 通り

これが一様ランダムに選ばれるとしたとき,
二分探索で, 最初に「前半分の中に OR がありますか？」を聞くと
確率 3.1×10^{-37} で NO になる

→ ほとんど情報を得られていない！

小課題 5: パス, $R \leq 120$

確率が $1/2$ に近くなるように二分探索しよう！

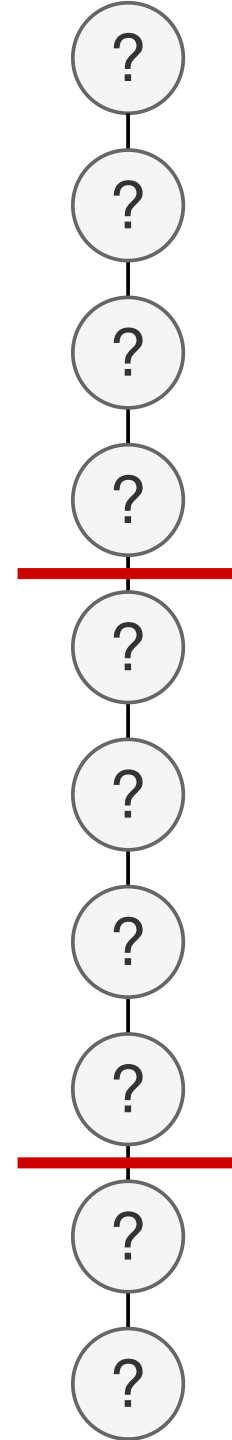
小課題 5: パス, $R \leq 120$

例

パスを 64 頂点ずつに分割し, 上から順に
二分探索で OR を探す

パスが $8000/64 = 125$ 個に分割され, 1 回の
二分探索に 7 回かかるので,

$125 + 7 \times 120 = 965$ 回の質問で解けた!



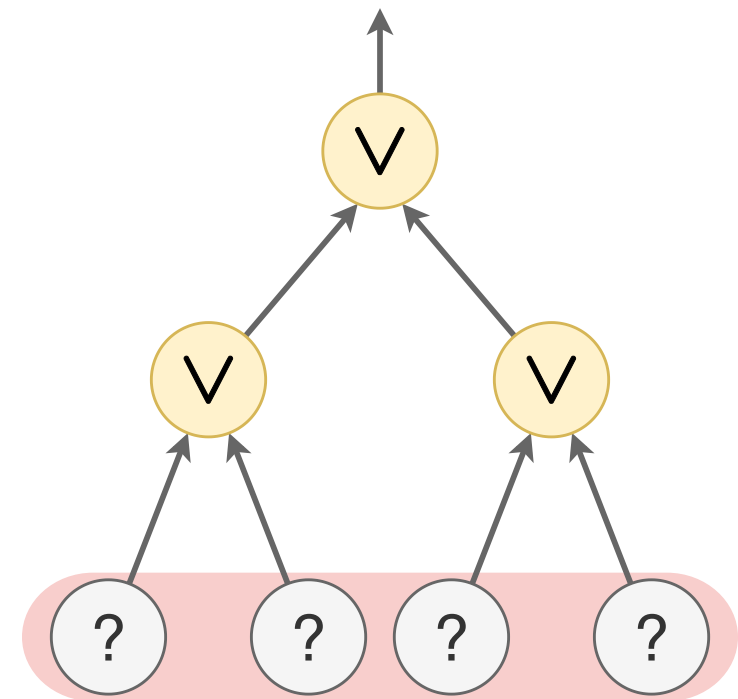
小課題 7: 完全二分木, $R \leq 120$

高さごとに分割

パスを 64 頂点ずつに分割し,
上から順に二分探索で OR を探す

パスが 131 個に分割され,
1 回の二分探索に 7 回かかるので,

$131 + 7 \times 120 = 971$ 回の質問で解けた！



小課題 9 : 一般の二分木, $R \leq 120$

HL 分解し, light-edge の深さごとに分割

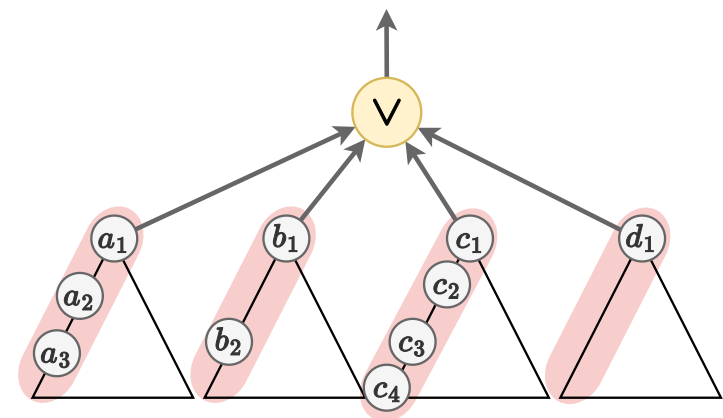
パスを 64 頂点ずつに分割し,

上から順に二分探索で OR を探す

パスが $12 + 125 = 137$ 個に分割され,

1 回の二分探索に 7 回かかるので,

$137 + 7 \times 120 = 977$ 回の質問で解けた !



小課題 9 : 一般の二分木, $R \leq 120$

HL 分解し, light-edge の深さごとに分割

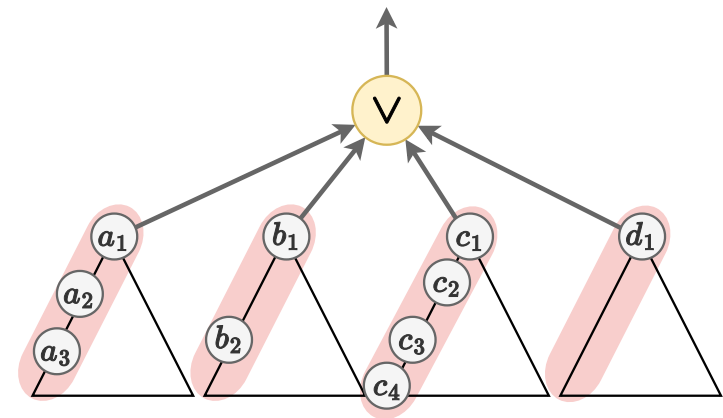
パスを 64 頂点ずつに分割し,

上から順に二分探索で OR を探す

パスが $12 + 125 = 137$ 個に分割され,

1 回の二分探索に 7 回かかるので,

$137 + 7 \times 120 = 977$ 回の質問で解けた !



小課題 9 : 一般の二分木, $R \leq 120$

オーダー解析

パスを w 頂点ずつに分割すると,
パスが $\log_2(N + 1) + N/w$ 個できる.

1 回の二分探索に $1 + \log_2 w$ 回かかるから,
質問が $(1 + \log_2 w)R + \log_2(N + 1) + N/w$ 回

$w = \ln 2 \cdot N/R$ とすると最小になり,
 $\log_2(N + 1) + (1.91 + \log_2(N/R))R$ 回で解ける !

統計情報 (JOI)

得点	小課題									人数	累積
100	1	2	3	4	5	6	7	8	9	2	2
77	1	2	3	4	-	6	-	8	-	1	3
66	1	2	3	4	5	6	7	-	-	1	4
57	1	-	-	4	5	6	7	-	-	1	5
50	1	2	3	4	-	6	-	-	-	2	7
27	1	2	3	4	-	-	-	-	-	7	14
26	1	2	-	4	5	-	-	-	-	1	15
10	1	2	3	-	-	-	-	-	-	5	20
1	1	-	-	-	-	-	-	-	-	2	22
0	-	-	-	-	-	-	-	-	-	3	25

統計情報 (JOIG)

得点	小課題										人数	累積
40	1	2	3	4	-	-	-	-	-		1	1
1	1	-	-	-	-	-	-	-	-		5	6
0	-	-	-	-	-	-	-	-	-		7	13



Mígratíon Plan

Edítoríal : blackyukí

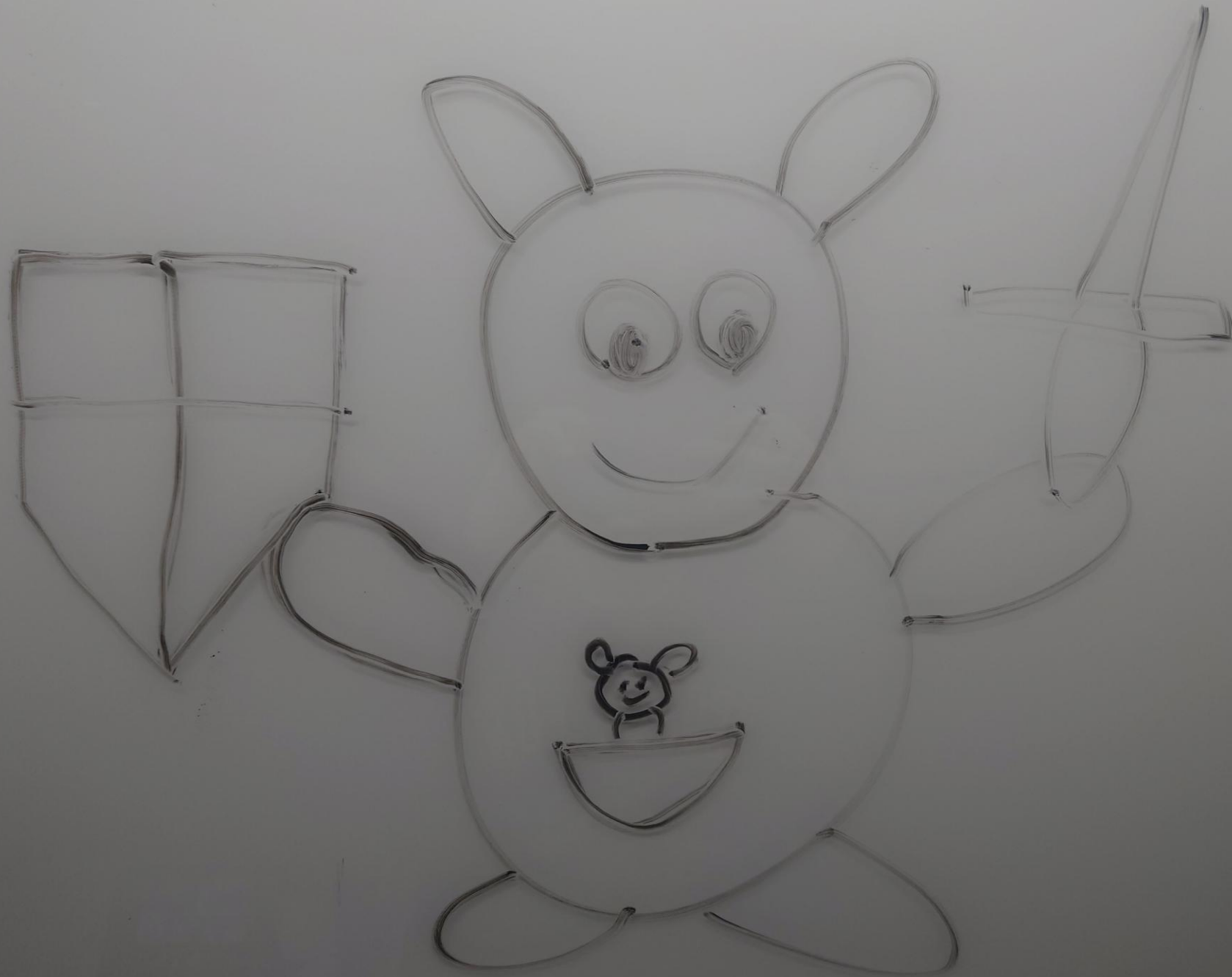
問題概要

- 根付き木が与えられる
- 3種類のクエリを処理
 - 深さ X の頂点に住んでいるビーバー全員が、今いる頂点の祖先である深さ Y の頂点に移動
 - 頂点 X に住むビーバーの数が増える
 - 頂点 X に住むビーバーの数を答える



勇者ビ太郎

絵: yuto1115



小課題 2

$N \leq 20$

小課題2

- N が小さい
- 愚直シミュレーション
- クエリ2,3 は $O(1)$ で処理できる
- r 回登った祖先を前計算でメモしておく、クエリ1は $O(N)$ で処理できる

小課題 1

$D=1$

小課題1

- クエリ1で $O(N)$ かけていると間に合わない
- ビーバーが住んでいない町を地下に埋める
- クエリ1で1つの町を処理する度にその町は消滅
- 地下に埋まった町が復活するのは、
クエリあたり高々1回

→全体 $O(N+Q)$ に抑えられる

小課題 3

$D \leq 20$

小課題3

- ・ 小課題2の実装を、小課題1のアイデアで高速化
- ・ ビーバーが住んでいない町を地下に埋めると
全体計算量が $O((N+Q)D)$ に抑えられる

小課題3

…なぜか？

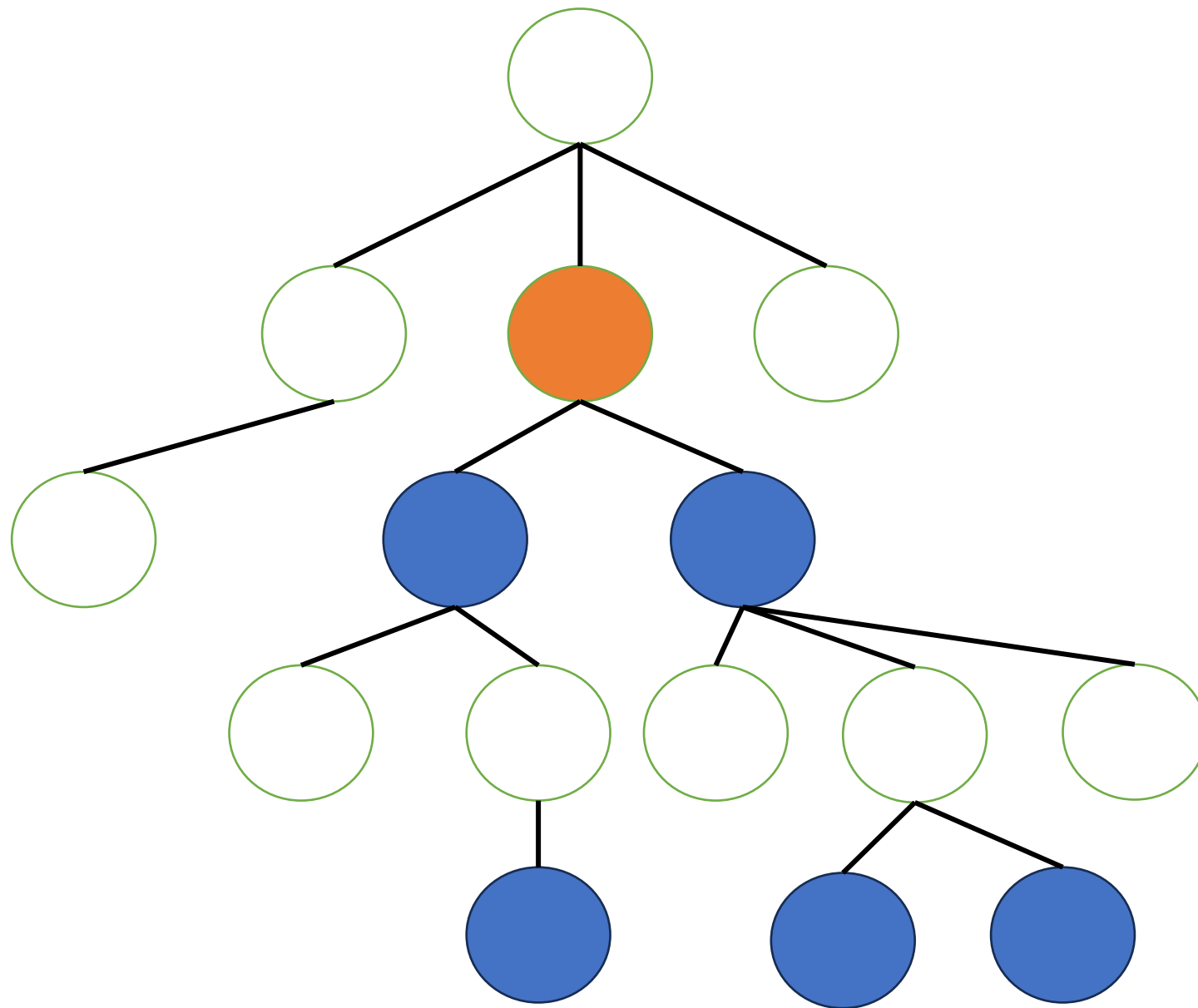
- 1匹のビーバーが移動できる回数は高々 D 回
- クエリ1で1つの町を処理する度にその町は消滅
- 地下に埋まった町が復活するのは、
全部で高々 $((N + Q)D)$ 回 ← 高々 $(N+Q)$ 種類のビーバー
→ 全体 $O((N+Q)D)$

小課題 4

加算クエリがない
&
回答クエリが 5 個以下

小課題 4

- これまではクエリ 1 で時間を使って、クエリ 2,3 は楽をする方針だった
- 小課題 4 は逆にクエリ 1 を高速に処理してクエリ 3 は時間を使っていいよ、と言ってそう
- 現在町 X に住んでいるビーバーがどこ由来か？
ということを考える



小課題 4

- 結局、重要な情報は、
「元々深さ X にいたビーバーの現在の深さ」
- 具体的な町の番号は関係ない
- 深さ X の町、という抽象的な頂点を作り、
クエリ 1 が飛んでくる度にマージすれば良い

→ 計算量は $O(D \log D + Q_3 N + Q)$

小課題 5

回答クエリが 5 個以下

小課題 5

- 深さ X の抽象的な頂点に加え、
「時刻 T に追加された深さ X の頂点」というものを考える
- 同様にマージしていくことができる

小課題 6

加算クエリがない

小課題 6

- 回答クエリも高速に処理する必要がある
- 「頂点 x の部分木の中で、
深さが $d_1, d_2, \dots$ である頂点の総和」

小課題 6

- 頂点 x の部分木 $\rightarrow$ オイラーツアーで区間
- 深さが $d_1, d_2, \dots$
 - $\rightarrow$ 深さのマージ過程を表す木における部分木
 - $\rightarrow$ オイラーツアー区間

小課題 6

- 各頂点は座標 (x,y) を持っている。
- 回答クエリでは $l_1 \leq x \leq r_1$ かつ $l_2 \leq y \leq r_2$ を満たす点の総和を求めたい。

→長方形領域の総和を求める問題に帰着される

小課題 6

- 回答クエリは最後にまとめてオフライン処理
→ BIT + 平面走査で $O((Q+N)\log(Q+N))$

小課題 7

追加の制約なし

小課題5と小課題6の解法を組み合わせます

AC !

実装は…？

結構
大変です

- 木を2つ構築する
- それぞれオイラーツアー順に並べる
- マージテクをする
- 区間を計算する
- 最後に平面走査
- 加算クエリの実処理も難しい

別解

- 小課題5の解法のボトルネックを思い出す
- 回答クエリで該当する頂点を全部見ていた（最悪 N 回）

→ 回答クエリで現在頂点 A に集まっているビーバーたちは
今後同一視できるのでは？

→ set など管理して見る度に erase してまとめてやればいい

実装は…？

40行ぐらい！！

得点分布



100点

55点

42点

38点

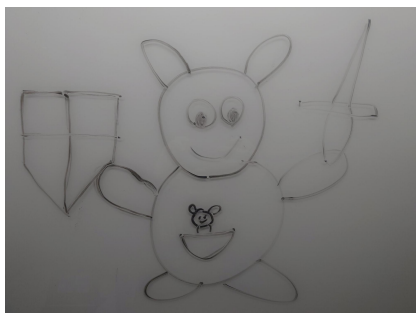
30点

27点

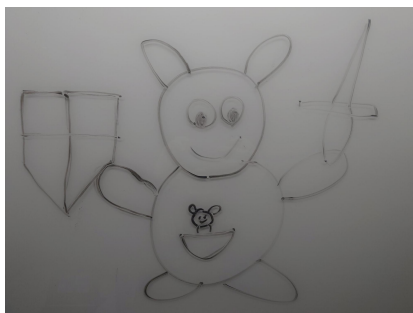
25点

20点

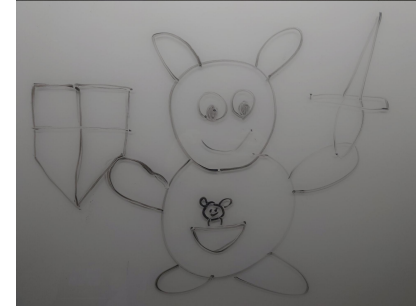
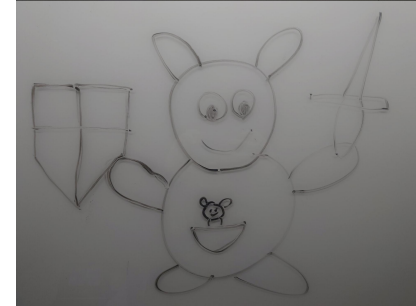
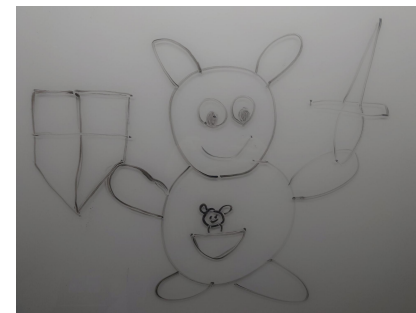
12点



8点



4点



0点

JOI/JOIG 2024/2025 春季トレーニング

Day 4 「ういろう」

解説担当：西本将樹（maspy）

問題概要

- 変数 x が $x = 0$ で初期化されている.
- $i = 1, 2, \dots, N$ の順に, $a = A[i]$ として次のどちらかを行う.
 - $x := x + a$.
 - $x \geq a$ のとき, $x := x - a$ として 1 点を獲得.
- 最大何点獲得できますか?
- これを Q 個の区間で解いてください.

小課題 1 ($N \leq 20, Q \leq 20, A \leq 100$)

- 操作列は高々 2^N 通り. これらを全探索する.
- 実装方針の例：
 - 2^N 通りすべてに対してシミュレーション.
 - 1手ずつ dfs.
- 計算量： $O(Q \times N \cdot 2^N)$ や $O(Q + N \cdot 2^N)$.
 - $\sum_{L,R} (R - L) 2^{R-L} = O(N \cdot 2^N)$
- 小課題 1：JOI 3 点, JOIG 7 点.

小課題 2 ($N \leq 300, Q \leq 20, A \leq 100$)

- 愚直なシミュレーションにおいて、必要な情報だけ残す.
- 操作列の履歴は必要ない, x と獲得スコアだけが問題.
- **dp[i] : ある時点で $x = i$ である場合の獲得スコア最大値**
というテーブルを 1 手ずつ更新していく.
 - $\text{dp}[i][j]$: i 枚終了時点で $x = j$ である場合の獲得スコア最大値.
というテーブルを作っても構いません. 時間・空間が増えるし扱うべき添字が増えるので, 個人的には推奨しません.
- 計算量 : $O(Q \times N^2 A)$.
- 小課題 1 + 2 : JOI 8 点, JOIG 18 点.

小課題 3 ($N \leq 5000, Q \leq 20, A \leq 100$)

- 愚直なシミュレーションにおいて、必要な情報だけ残す.
- 操作列の履歴は必要ない, x と獲得スコアだけが問題.
- x の範囲: $[0, NA]$. 獲得スコアの範囲: $[0, N]$.

小課題 3 ($N \leq 5000, Q \leq 20, A \leq 100$)

- 愚直なシミュレーションにおいて、必要な情報だけ残す.
- 操作列の履歴は必要ない, x と獲得スコアだけが問題.
- x の範囲: $[0, NA]$. 獲得スコアの範囲: $[0, N]$
- $\text{dp}[i]$: ある時点で獲得スコア i である場合の x としてありうる最大値
- 計算量: $O(Q \times N^2)$.
- 小課題 1 + 2 + 3: JOI 15 点, JOIG 33 点.

小課題 4 ($N \leq 2 \times 10^5, Q \leq 20, A \leq 100$)

- 全探索的な方法（DP を含む）では難しいと思います.
- 適切な貪欲法を考えます.
- $i = 1, 2, 3, \dots$ の順に進めながら, 「その時点での最適な状態」を持ちます.

小課題 4 ($N \leq 2 \times 10^5, Q \leq 20, A \leq 100$)

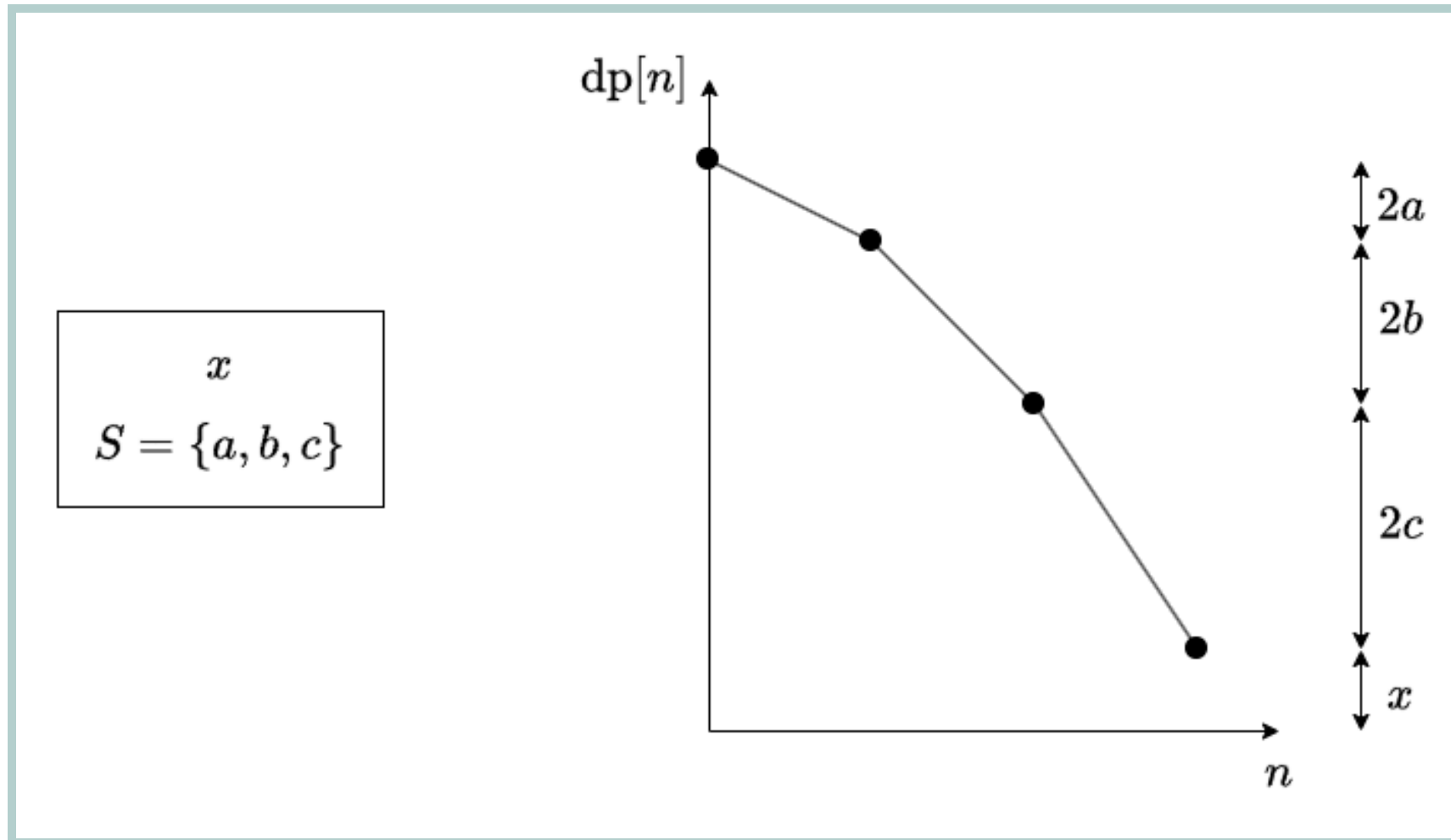
- 状態として
 - x の値
 - 多重集合 S : 今まで減算で使った値の多重集合
- 「最適な状態」とはそれ以降何が来ても最適解が得られる状態のこととします.
- ただしその際に, 「今まで減算で使った値をやっぱり加算にします」という操作を許します.

小課題 4 ($N \leq 2 \times 10^5, Q \leq 20, A \leq 100$)

- $a = A[i]$ として, 次の貪欲アルゴリズムが正当です.
 - $a \leq x$ ならば $x := x - a, S := S \cup \{a\}$.
 - $a > x$ かつ $b = \max S > a$ ならば $x := x + 2 \times b - a, S := S - \{b\} \cup \{a\}$.
 - $a > x$ かつ $b \leq a$ ならば $x := x + a$.
- $\max S$ を取り出すために priority queue を使って実装すると $O(N \log N)$ 時間.

小課題 4 ($N \leq 2 \times 10^5, Q \leq 20, A \leq 100$)

- なお、この貪欲法は小課題 3 で説明した dp テーブルと対応があります。



小課題 4 ($N \leq 2 \times 10^5, Q \leq 20, A \leq 100$)

- 小課題 3 の dp 更新は次の形.
 - $g(x) = \max(f(x) + a, f(x - 1) - a).$
 - ただし $g(x) < 0$ になりそうなら補正.
- 前者の操作は max plus convolution.
 - $g(x) = \max_{y+z=x} (f(y) + h(z))$
 - $h(x) := \begin{cases} a & (x = 0) \\ -a & (x = 1) \\ -\infty & (\text{otherwise}) \end{cases}$

小課題 4 ($N \leq 2 \times 10^5, Q \leq 20, A \leq 100$)

- dp テーブルは常に上凸になる.
- 上凸関数の max plus convolution は傾き集合のマージ (この場合ある傾きをひとつ挿入).
- 「dp テーブルは実は上凸です」という問題では, その理由が max plus convolution で説明できることが多い.
- 凸関数の扱いが上手いと, 小課題 3 の dp 高速化として直接小課題 4 の解法を得ることもできる.
- 小課題 1 + 2 + 3 + 4 : JOI 30 点, JOIG 57 点

小課題 5 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 2$)

- ここから Q が増えます.
- 小課題 5 は $A \leq 2$ で, やや特殊な解法が可能です.

小課題 5 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 2$)

- 小課題 4 の貪欲を考えると, priority queue の中身としては, 2 の個数 y だけを持てばよいです.
- 状態は, (x, y) ($0 \leq x < 4$) という 2 つの整数組で書けて, 各要素はこれに対する関数です.
- **関数合成をセグメント木にのせましょう.**
- 合成として現れる関数を $O(1)$ サイズのデータとして書ければ OK.

小課題 5 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 2$)

- 関数は $A[i] = 1, 2$ に応じて次の形です.
- $A[i] = 1$:
 - $1 \leq x < 4$ ならば $(x, y) \mapsto (x - 1, y)$.
 - $x = 0$ かつ $y \geq 1$ ならば $(x, y) \mapsto (3, y - 1)$.
 - $x = 0$ かつ $y = 0$ ならば $(x, y) \mapsto (1, 0)$.
- $A[i] = 2$:
 - $0 \leq x \leq 1$ ならば $(x, y) \mapsto (x + 2, y)$.
 - $x \geq 2$ ならば $(x, y) \mapsto (x - 2, y + 1)$

小課題 5 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 2$)

- $z := x + 4y$ とするとかなり簡単になります.
- $A[i] = 1$:
 - $1 \leq z$ ならば $z \mapsto z - 1$.
 - $z = 0$ ならば $z \mapsto 1$.
- $A[i] = 2$:
 - $z \mapsto z + 2$.

小課題 5 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 2$)

- 結局, こんな感じの関数になります.



小課題 5 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 2$)

- これをセグメント木にのせると解けます. $O(N + Q \log N)$ 時間.
- $x = 0, 1, 2, 3$ それぞれの場合の関数を持つことも考えられますが, 上の方法よりは実装が複雑になると思います.
- 小課題 1 + 2 + 3 + 4 + 5 : JOI 51 点, JOIG 78 点.

満点 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 100$)

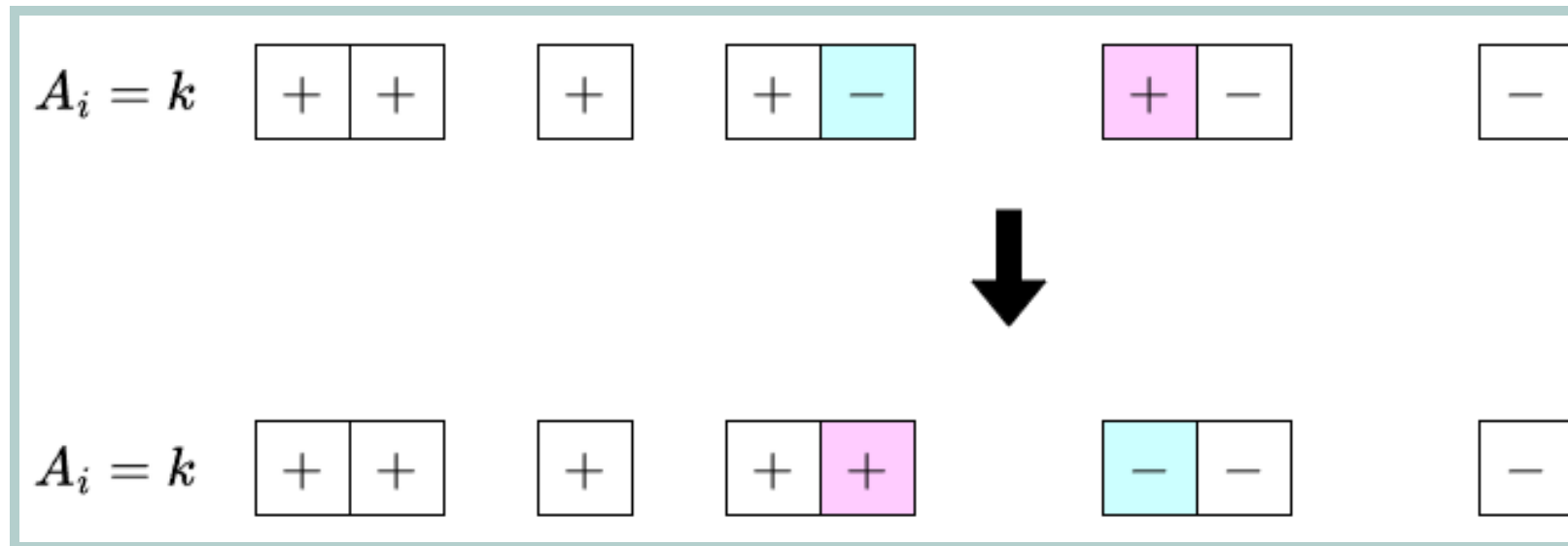
- 小課題 6, 7 の制約に大きな違いはないので, まとめて解説します.
- 小課題 4 の貪欲法では途中の状態のパターンが多すぎて難しいと思います.
- 全く別の貪欲法を考えます.
- A の種類数が少ないことに注目します.
- $A = 1, 2, \dots, 100$ の順に何かを計算するという 100 ステップの計算で答えを求めることを考えます.

満点 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 100$)

- k を固定したとき, $A[i] = k$ であるような i 全体に対する戦略を決められそうか?
- 次を満たす p が存在するとしてよいです.
 - $i < p$ ならば $A[i] = k$ を加算で使う.
 - $p \leq i$ ならば $A[i] = k$ を減算で使う.
- 証明は簡単な exchange argument.

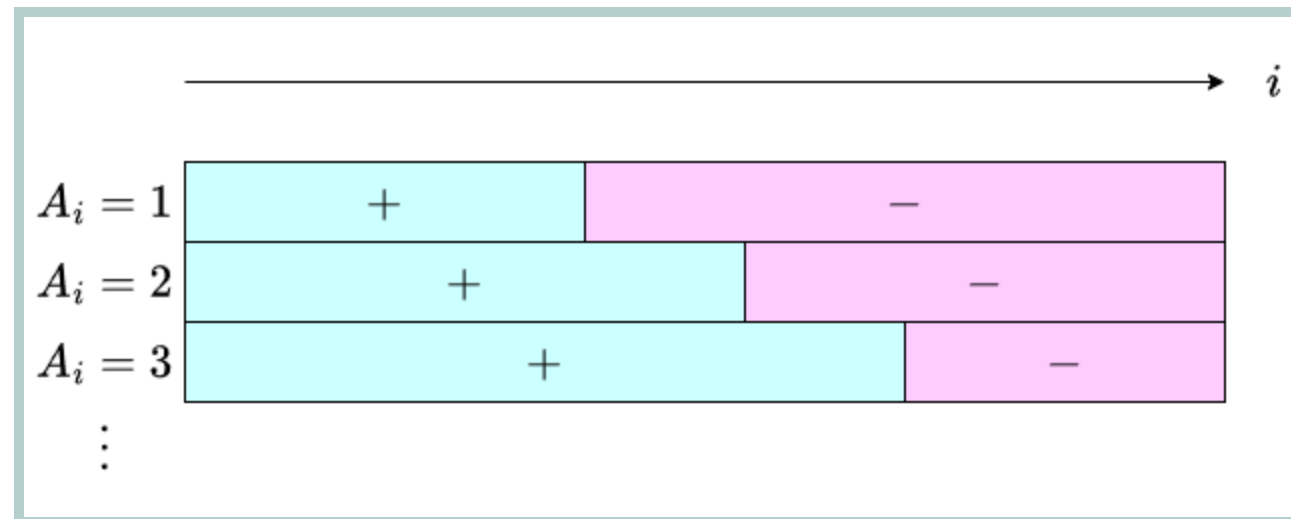
満点 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 100$)

- **exchange argument** : (性質) を満たさない解があったとき, こういう入れ替えを行っても解は悪くならない. これを可能な限り繰り返したら (性質) を満たす解に到達する. よって最適解であって (性質) を満たすものが存在する.



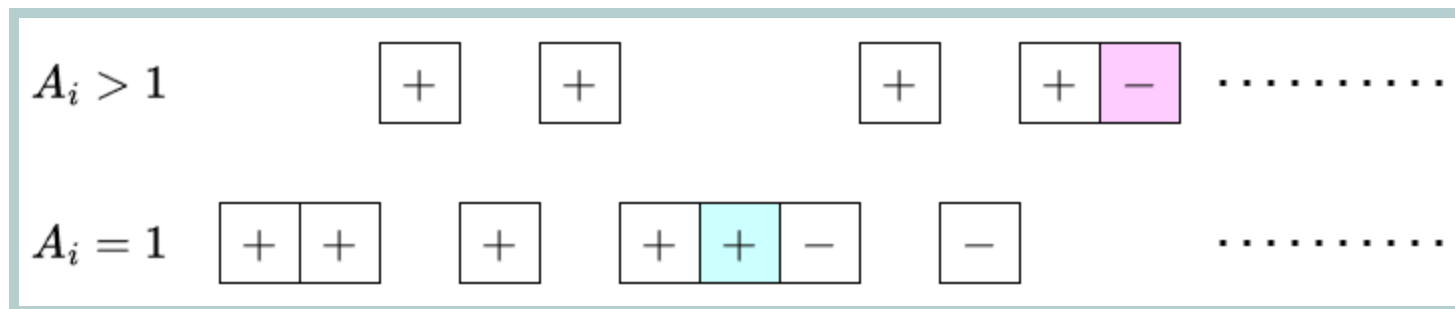
満点 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 100$)

- 上と全く同様に、最適解に次を仮定できることが分かります：
 - $a > b, A[i] = a, A[j] = b$ のとき,
 $A[j]$ を加算で使うならば $A[i]$ も加算で使う.
- したがって、**境界の位置は単調になります**. これらの境界を $k = 1, 2, \dots, 100$ の順にすべて求めるというのが解法の概要です.



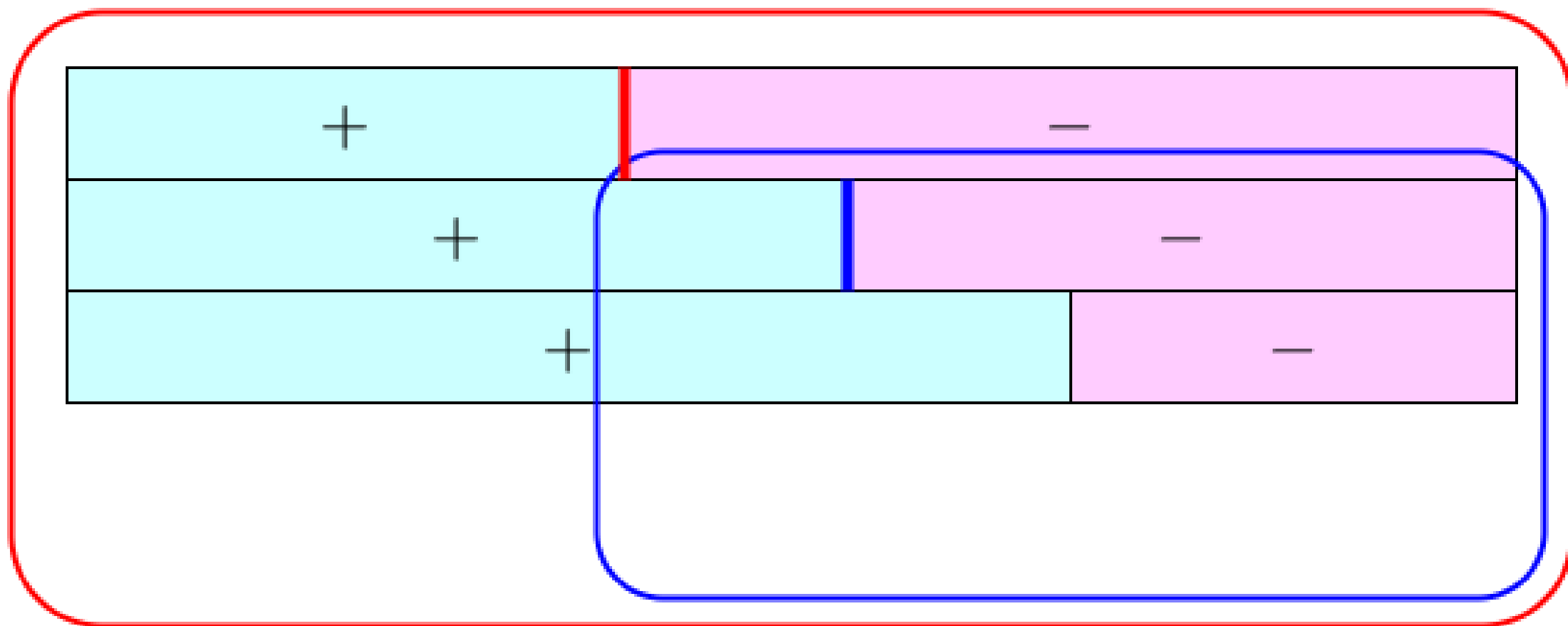
満点 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 100$)

- 境界を $k = 1, 2, 3, \dots$ の順に決めていきます.
- $k = 1$ の場合, 次が証明可能です:
 - $A[i] = 1$ を加算で使う回数は最小化してよい. つまり,
 $A[i] > 1$ がすべて加算である場合に 1 の最小加算回数が n 回
ならば, 最適解であって 1 の加算回数 n 回のものが存在.
- やっぱり exchange argument で証明可能です.
(最小値の加算と最小値以外の最初の減算の入れ替え)



満点 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 100$)

- 解法をまとめると次のようになります.
- $A[i] = 1$ である i で最後に加算するものの場所 p を求める.
 - ある場所以降をすべて減算にすると矛盾するような p の最大値.
- 場所 p までは ($A[i] \geq 2$ を含めて) すべて加算で確定します.
場所 p 以降は $A[i] = 1$ についてはすべて減算で確定します.
- 場所 p 以降で, $A[i] \geq 2$ に対する戦略を決める問題を解きます.
 $A[i] = 2$ であるような場所の境界を同様に決めて, 再帰的に処理します.



満点 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 100$)

- 各段階では, 「 k 未満はすべて減算, $k + 1$ 以上はすべて加算」という前提で, k をいくつ減算できるかという問題を解きます.
- 各 k に対してそれを行うためのデータ構造をひとつ持ちます.
- 必要な計算を整理すると, 静的な列に対する range minimum query があればよいことが分かります.
- 例えばセグメント木で $O(AN + QA \log N)$ 時間で解けます.
- 小課題 1 + 2 + 3 + 4 + 5 + 6 : JOI 80 点, JOIG 92 点.
- 小課題 1 + 2 + 3 + 4 + 5 + 6 + 7 : JOI 100 点, JOIG 100 点.

満点 ($N \leq 2 \times 10^5, Q \leq 2 \times 10^5, A \leq 100$)

どうしても TLE してしまう場合、以下のような手段が考えられます.

- すべてのクエリを並列に処理すれば、同時に持つべきセグメント木は 1 つでよい.
- 静的な列に対する offline range minimum query をセグメント木よりも高速に行う.
 - 例えば, j を増やしながらか $ans[i] = \min\{a_i, \dots, a_j\}$ を管理すれば UnionFind で処理できる.
- いずれも C++ で満点をとるには不要だとは思いますが.

統計情報 (JOI)

得点	人数	累積
100	2	2
80	1	3
51	2	5
30, 36	6	11
3, 8, 15	12	23
0	4	27

統計情報 (JOIG)

得点	人数	累積
33	5	5
18	1	6
7	5	11
0	2	13